

基于时间序列预测与线性规划的微网购电优化策略

摘 要

在现代微网负荷调控中, 小区需要用电而形成整体负载, 小区自身有光伏发电, 但不能时刻满足小区供电, 于是需要向外网购电。负载与电价会实时波动, 为了供给负载并节省购电费用, 储能设备与微网购电的调控成为了关键问题。依据某小区一年负载、光伏发电与电价的实时监测情况, 本文主要研究回应用电、发电情况的规律、制定小区购电费用最小的微网购电方案。通过线性规划、MSTL 时间序列分解等进行求解。

针对问题一: 以全天购电费最小为目标, 根据题目电力收支平衡、首末储电量相同等作为约束条件, 采用线性规划求解。结果显示, 最终策略整体体现出在电费低谷多买、电费高峰不买特征。全天购电费为 35126.95 元。

针对问题二: 分为预测与优化两部分。对于负载预测, 根据早晚高峰, 工作日与周末, 全年缓慢升降的用电规律, 用 MSTL 把负载分解为趋势、日周期、周周期与残差四部分。对于光伏预测, 由于太阳周年运动与日内太阳高度变化, 定义季节晴空强度 $B_{d,t}^{PV}$; 由于云层、雾霾等天气因素, 定义日透射率 $\kappa_d \in [0, 1]$ 。由于相邻天气过程通常具有一定持续性, 使用 $AR(p)$ 对 κ 做日尺度滚动预测。所有分解得到的残差仍具有显著自相关性, 则用 $AR(p)$ 模型把残差进一步分解为短期自相关部分和白噪声。对于紧急购电价格高而导致的风险不对称, 我们对扣除光伏发电的净负荷高于预测的正向偏差提取历史加权分位数, 作为应对供电低于负载时的“备用”, 并利用报童模型根据缺电与多电的边际代价之比动态调整分位水平, 使备用量自适应地向代价更高的缺电侧倾斜。优化部分框架与第一问相同, 目标函数变为紧急购电费、窗口跨入次日时按正常电价计费的虚拟购电费。结果显示符合与光伏功率预测准确, 全年紧急购电费仅占 3.37%。

针对问题三: 首先通过线性插值处理预报以 1h 为单位与模型以 10min 为单位的匹配问题。随后基于问题二, 把购电调整分解为向上与向下两部分: $P_{k,t}^{buy} = P_{k-1,t}^{buy} + U_{k,t}^+ - U_{k,t}^-$, 分别按 $1.5\lambda_t U_{k,t}^+ \Delta t$ 与 $0.5\lambda_t U_{k,t}^- \Delta t$ 分别计入追加和违约费用。最终结果显示, 经过预报, 向下调整总电费占总调整电费的 78.99%。对于引入其他时刻的预报, 我们构造了光伏功率的动态非对称风险带。由于预测偏高易触发高价的紧急购电, 风险带的危险侧容许宽度更窄。随后, 我们通过预测值偏离带状区域的越界量筛选出需要增加额外预报的候选时点, 再把时点代入预测模型, 根据最后的全年购电费是否降低等来判断是否引入候选时点。最终结果建议不添加预报时点。

针对问题四: 由于电价也具有日周期与周周期特征, 我们沿用 MSTL 结合 AR 模型, 构建电价预测模型。随后通过实时更新的电价做轻量级指数加权残差修正以实现滚动更新。优化框架沿用前两问设计, 只改变目标函数中的电价为预测电价, 最后用实际电价计算总购电费用。对于引入预报时点, 将电价与光伏两类同时纳入联合风险, 将局部可提供信息价值最大的时点纳入候选时点。最终建议不添加预报时点。

关键字: 微网购电 线性规划 MSTL 时间序列分解 $AR(p)$ 自回归 报童模型

一、 问题重述

1.1 问题背景

随着分布式能源的快速发展，配备储能设备的微网可通过“低充高放”有效降低整体购电费用。然而，小区负载、光伏发电和外部电价均随时间波动，且储能设备存在容量、功率与效率等物理限制。如何在满足用电需求的前提下合理安排购电与充放电策略，并应对光伏预报更新与电价波动带来的调控复杂性，是实现微网经济运行的关键。

1.2 问题提出

某微网由小区负载、光伏发电、储能设备及外部电网购电组成。题目附件提供了全年的负载与光伏实际功率、多时点的光伏功率预报、波动电价，以及储能设备的物理参数（包括最大容量、充放电功率限制、初始电量、安全运行区间及充放电效率等）。现需结合上述信息建立数学模型，分析以下问题：

问题一：为了在电价和小区负载每天相同、光伏发电功率预测数据已知的情形下制定微网当天的计划购电策略，需要满足以下条件，给出计划购电量与储能设备充放电方案：

1. 保证微网提供的电能不低于小区负载
2. 满足储能设备最大容量、最大充放电功率、安全电量区间和充放电效率等约束
3. 使储能设备在 0:00 和 24:00 的储电量相同
4. 以全天购电费用最小为目标

问题二：在每天电价相同，而小区负载和光伏发电实际功率随时间变化的情形下，制定微网当天计划购电策略，需要综合考虑以下因素，从而给出指定日期的计划购电量、储能设备充放电量和紧急购电量：

1. 微网提供的电能不可低于小区负载
2. 若实际供电低于小区负载，需向外网紧急购电，紧急购电电价为交易时刻电价的 5 倍
3. 除紧急购电费用外，其他时间段的购电费用按计划购电量计算

最后使用构建的模型给出完整购电策略，并保存相应结果。

问题三：在每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点光伏发电功率预报的条件下，微网可根据 0:00 预报制定当天计划购电策略，也可根据其他时刻的预报调整购电策略。调整购电策略会带来两类费用后果：若计划购电量高于调整购电量，高出部分按交易时刻电价的 50% 承担违约费用；若调整购电量高于计划购电量，高出

部分按交易时刻电价的 1.5 倍计费。需要以当天总购电费用最小为目标，给出计划购电策略与调整购电策略，并分析是否需要引入其他时刻的预报。

问题四：针对外部电网电价实时波动的实际情况，与二三问相同，需要对每日电价进行预测，在波动电价下重新计算问题二和问题三。

二、 问题分析

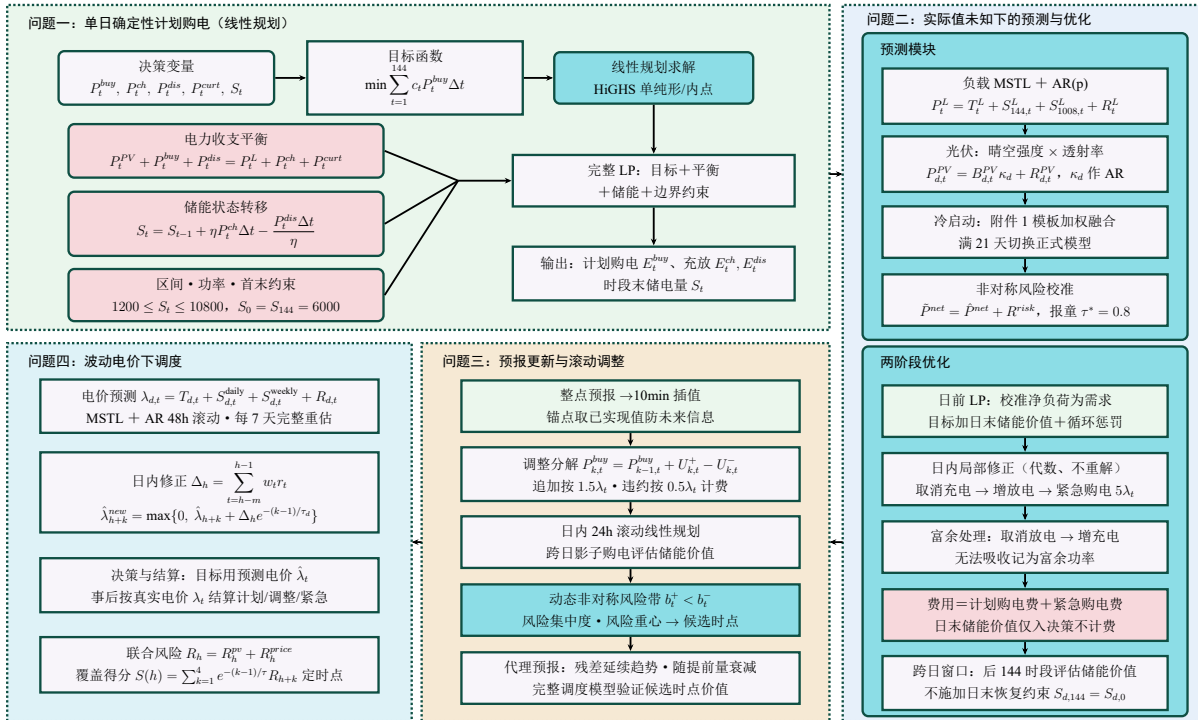


图 1 问题求解流程图

2.1 问题一分析

问题 1 要求在给定电价、小区负载和光伏发电预测功率的条件下，制定微网当天的计划购电与储能设备充放电方案，属于单日调度优化问题。首先，确定决策变量为一天 144 个 10 分钟时段的购电量 E_t^{buy} 、储能设备充放电量 E_t^{ch} 与 E_t^{dis} ，以及各时段末储电量 S_t 。接着，以全天购电费用最小为目标： $\min \sum_{t=1}^{144} c_t E_t^{buy}$ 。随后建立约束条件：电力收支平衡约束、供电可靠性约束、储能设备充放电功率约束、储电量状态转移与区间约束、首末储电量约束。由于目标函数与各类约束均为线性关系，利用线性规划模型即可求解得到计划购电策略与储能设备充放电策略。

2.2 问题二分析

本问的难点在于每日 0:00 无法获知当日真实的小区负载与光伏发电功率，需先预测再优化，且预测与实际的偏差会在运行过程中逐步暴露。

预测环节中，负载侧基于小区用电早晚高峰、工作日与周末差异、全年缓慢升降三类稳定规律，通过 MSTL 分解为趋势、日周期、周周期与残差四部分，若残差存在显著自相关性则补充 AR (p) 模型，将其进一步分解为短期自相关部分与白噪声；光伏侧将发电特性拆分为由太阳运行决定的季节晴空强度与由天气决定的日透射率，对日透射率序列采用 AR (p) 开展日尺度滚动预测，乘性结构无法解释的日内短时波动残差参照负载残差的逻辑处理，同时施加非负与夜间归零的物理约束。针对初始阶段无历史数据导致模型无法启动的问题，以附件 1 的典型负载与光伏曲线为先验模板，与逐步积累的近期实际数据加权混合，随时间推移逐步降低先验模板的参考比重，待历史数据足以支撑周周期与晴空包络估计后，完全切换为正式预测模型。此外考虑到预测偏差的代价不对称——预测偏少会触发 5 倍电价的紧急购电、代价远高于预测偏多，故按历史净负荷正向误差的分位数构造风险备用量叠加到预测值上，为缺电侧预留余量；缺口实际出现时调用储能还是紧急购电，由日内优化按成本决策。

优化沿用第一问框架，仅调整日内滚动目标函数：0:00 后计划购电费已锁定，只需最小化紧急购电费、跨日按正常电价计的虚拟购电费，并附加极小循环惩罚以抑制同时充放电。

2.3 问题三分析

首先处理预报粒度与模型时段的匹配问题，附件 3 给出的是每小时的预报，而模型以 10 分钟为时段。于是我们将整点预报线性插值到 10 分钟时段。除此之外，问题三的优化结构相对问题二变化不大，只需要把购电调整量，分解为向上与向下两部分即可： $P_{k,t}^{\text{buy}} = P_{k-1,t}^{\text{buy}} + U_{k,t}^+ - U_{k,t}^-$ ，分别按 $1.5\lambda_t U_{k,t}^+ \Delta t$ 与 $0.5\lambda_t U_{k,t}^- \Delta t$ 分别计入追加和违约费用。为判断是否有必要引入其他时刻的预报，我们构造了光伏发电功率的动态非对称风险带。由于预测偏高易触发高价的紧急购电，风险带的危险侧容许宽度更窄。随后，我们通过计算预测值偏离带状区域的越界量来量化各时段的“风险集中度”，并据此筛选出需要增加额外预报的候选时点。

2.4 问题四分析

在电价是波动的情况下，由于电价同样具有显著的日周期与周周期特征，我们沿用问题二与问题三中的 MSTL 结合 AR 模型，构建电价的 48 小时滚动预测模块。在优化部分，问题四完全继承前两问的建模，包括动态非对称风险机制、储能设备物理约束等。仅在决策时将目标函数中的结算电价替换为预测电价，最后计算购电费时切换成真实电价。在判断是否引入其他预报时点时，由于问题三的信息差主要来自光伏发电功率，而在问题四波动电价的情况下，电价预测误差会导致产生独立于光伏的经济损失，所以我们定义包含光伏和电价的联合经济风险，作为其他预报时点的判定。

三、 模型假设

1. 附件 1 给出的电价、小区负载与光伏发电预测功率在对应 10 分钟时段内保持恒定；
2. 微网只允许向外部电网购电，不允许向外部电网反送电；
3. 弃电仅来源于光伏发电的富余部分，不丢弃外网购电与储能设备放电；
4. 储能设备充、放电效率恒定为 0.9，不随储电量、温度与设备老化而变化，且忽略自放电与待机损耗；
5. 外部电网供电能力充足，紧急购电与购电方案调整均可在对应时段瞬时完成，不考虑最小调整粒度与供应上限；
6. 忽略微网内部线路与变压器损耗，功率平衡在并网点处成立；
7. 不考虑设备故障、检修与寿命衰减对运行策略的影响；
8. 负荷与光伏预测误差的统计特性在短期内保持平稳，未来时段的误差分布可由近期同日型历史样本近似表征；

四、 符号说明

符号	说明	单位
d, t, h, k	天索引、10 分钟时段索引、小时索引、预报发布阶段	—
$P_{d,t}^L, P_{d,t}^{PV}, P_{d,t}^{net}$	第 d 天第 t 时段的实际负荷、光伏、净负荷功率	kW
$\hat{P}_{d,t}^L, \hat{P}_{d,t}^{PV}, \hat{P}_{d,t}^{net}$	对应上述物理量的预测值	kW
$e_{d,t}^{net}$	净负荷预测误差（实际值减预测值）	kW
$E_{d,t}^{buy}, E_{d,t}^{ch}, E_{d,t}^{dis}$	第 d 天第 t 时段的计划购电量、充电量、放电量	kWh
$E_{d,t}^{emg}, E_{d,t}^{sur}$	第 d 天第 t 时段的紧急购电量、富余电量	kWh
$\lambda_t, \hat{\lambda}_t$	第 t 时段的真实波动电价与预测电价	元/kWh
S_t, S_{min}, S_{max}	第 t 时段末储电量及其安全运行下限、上限	kWh
P_{max}, η_c, η_d	储能设备最大充放电功率、充电效率、放电效率	kW, —, —
b_t^+, b_t^-	动态非对称风险带的上侧（危险）与下侧（安全）容许宽度	kW
Δ_t	光伏预测危险高估越界量	kW
$B_{d,h}$	基线方案下第 d 天第 h 小时的购电量	kWh
$G_{d,t}^{PV}$	季节晴空强度（近似晴空下当天各时段本应发的电功率）	kW
$R_{d,t}^{risk}, \rho$	时段风险备用量、风险强度系数	kW, —
$\alpha_{d,t}, \beta_{d,t}, \tau_{d,t}$	动态非对称风险权重（正/负向）、最优分位水平	—, —, —
c_t^+, c_t^-	单位正/负向误差的边际补救代价	元/kWh
C_{total}	全年累计总购电费用	元

五、 问题一的模型建立与求解

问题一要求在电价、小区负载与光伏发电预测功率均已知的确定性条件下，制定微网单日（144 个 10 分钟时段）的计划购电策略与储能设备充放电方案，使全天购电费用最小。由于目标函数与全部约束均可表示为决策变量的线性表达式，我们将其建立为线性规划（LP）模型，并调用线性规划求解器求得全局最优解^[1]。

5.1 单日购电与储能调度优化模型

5.1.1 决策变量与参数

取第 t 个 10 分钟时段（ $t = 1, 2, \dots, 144$ ，时段长度 $\Delta t = 1/6 \text{ h}$ ）的外网购电功率 P_t^{buy} 、储能设备充电功率 P_t^{ch} 、储能设备放电功率 P_t^{dis} 、富余功率 P_t^{curt} 以及时段末储电量 S_t 为决策变量，对应电量由 $E_t = P_t \Delta t$ 换算。已知参数为：电价 c_t 、小区负载功率 P_t^{L} 、光伏预测功率 P_t^{PV} 、充放电效率 $\eta = 0.9$ 、最大充放电功率 $P_{\max} = 5000 \text{ kW}$ 、安全电量区间 $[S_{\min}, S_{\max}] = [1200, 10800] \text{ kWh}$ 、初始储电量 $S_0 = 6000 \text{ kWh}$ 。

5.1.2 目标函数

以全天购电费用最小为目标：

$$\min \sum_{t=1}^{144} c_t P_t^{\text{buy}} \Delta t. \quad (1)$$

5.1.3 约束条件

- (1) 电力收支平衡：光伏发电、外网购电与储能设备放电之和，等于小区负载、储能设备充电与弃电之和： $P_t^{\text{PV}} + P_t^{\text{buy}} + P_t^{\text{dis}} = P_t^{\text{L}} + P_t^{\text{ch}} + P_t^{\text{curt}}, \quad \forall t.$
- (2) 弃电仅来自光伏富余，不得超过该时段光伏预测功率： $0 \leq P_t^{\text{curt}} \leq P_t^{\text{PV}}, \quad \forall t.$
- (3) 储能设备电量状态转移：充电按效率 η 折算存入，放电按 $1/\eta$ 折算消耗内部电量： $S_t = S_{t-1} + \eta P_t^{\text{ch}} \Delta t - \frac{P_t^{\text{dis}} \Delta t}{\eta}, \quad \forall t.$
- (4) 安全电量区间，避免过度充电或过度放电： $S_{\min} \leq S_t \leq S_{\max}, \quad \forall t.$
- (5) 充放电功率上限： $0 \leq P_t^{\text{ch}} \leq P_{\max}, \quad 0 \leq P_t^{\text{dis}} \leq P_{\max}, \quad \forall t.$
- (6) 外网购电非负： $P_t^{\text{buy}} \geq 0, \quad \forall t.$
- (7) 单日首末储电量相同，并取附录 1 给出的初始电量： $S_0 = S_{144} = 6000.$

目标函数与所有约束均为决策变量的线性表达式，构成一个线性规划问题，可由单纯形法或内点法求得全局最优解。

5.2 模型求解与结果分析

调用 SciPy 优化模块中的 `linprog` 函数（HiGHS 求解器）对上述线性规划进行求解。下面从功率平衡、购电策略、储能调度与功率去向四个角度展示并验证求解结果。

```
1 c = np.zeros(4 * T); c[0::4] = pri * dt # 目标系数：仅购电按电价计费
2 bounds = [b for t in range(T) for b in ((0, None), (0, Pmax), (0, Pmax), (0, pv[t]))]
3 # 购电/充电/放电/弃电的上下界
4 res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
5 # HiGHS 自动选用单纯形法或内点法求全局最优
6 pbuy, pch, pdis, pw = res.x.reshape(T, 4).T # 拆回购电/充电/放电/弃电四条功率曲线
```

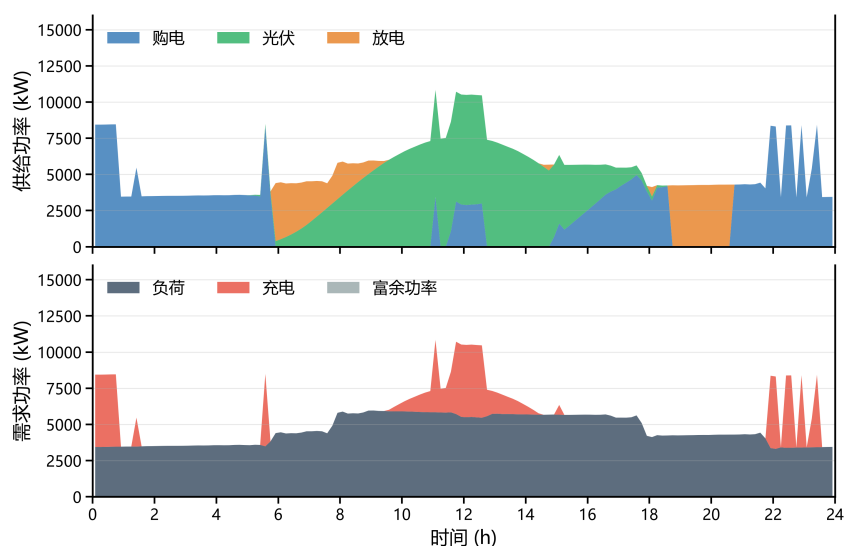


图2 问题一微网功率平衡：供给侧与需求侧逐时段对照

图2将功率平衡约束(1)的供给侧（购电、光伏、放电）与需求侧（负载、充电、富余功率）逐时段堆叠对照。午间光伏出力高于负载时，富余部分可被储能设备充电吸收，夜间与早晚高峰时段则由购电与放电补足负载。

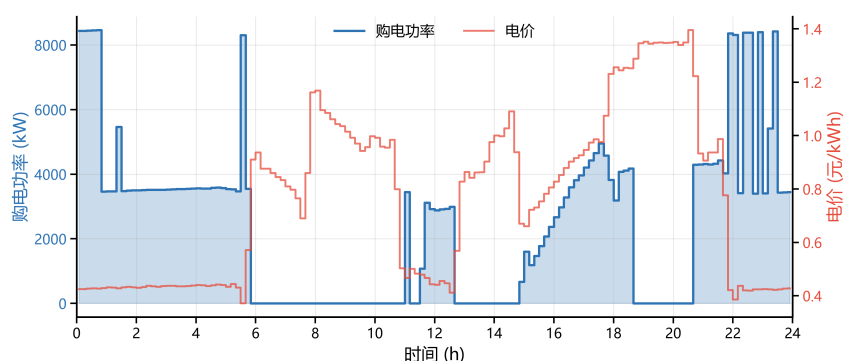


图3 问题一计划购电功率与电价分时对照

由图3可以看出，最优购电策略呈现明显的“低谷多买、高峰不买”特征：购电集中于凌晨与夜间电价低谷，在满足小区负载的同时以满功率5000 kW为储能设备充电；早高峰与晚高峰购电功率降为0；午间除11:40–12:40因储能设备尚未充满且电价较低而少量购电充电外，其余时段购电为0。

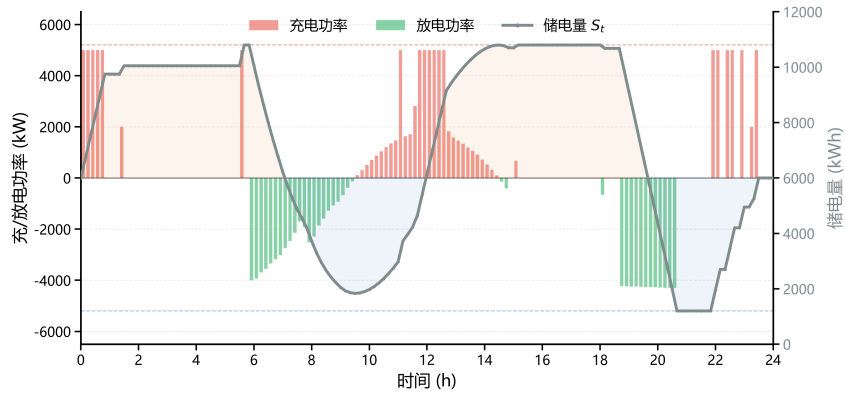


图 4 问题一储能设备充放电功率与储电量轨迹

由图 4，储电量 S_t 在 1200–10800 kWh 上下限之间波动，并于 24:00 回落至 6000 kWh，满足约束 (7)；储电量在午间与午后多次触及上限、在晚高峰触及下限，表明储能设备容量在本题电价结构下被充分利用；充电主要集中于夜间、中午，分别对应夜间电价低集中购电、中午光伏发电富余。

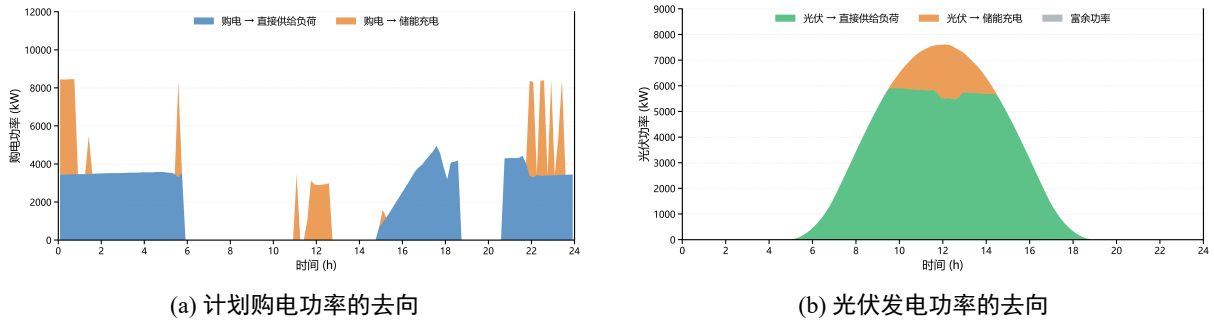


图 5 问题一计划购电与光伏发电的功率去向

图 5 从能量去向角度进一步验证调度结构：全天购电 59483 kWh 中 75.6% 直接供给负荷、24.4% 用于储能设备充电；全天光伏 55483 kWh 中 88.7% 直接供给负荷、11.3% 存入储能设备、富余功率为 0。购电、光伏充电合计 20741 kWh。具体求解结果如下：

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	486.4029	14:00-14:10	0.0000
16:00-16:10	394.9315	18:00-18:10	636.9826	20:00-20:10	0.0000
全天购电量		59482.6990	全天购电费		35126.9500

表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4500.0000	0.0000	4:00-8:00	833.3333	6365.8412
8:00-12:00	4787.9643	1702.9970	12:00-16:00	5286.0352	91.1014
16:00-20:00	0.0000	5780.1319	20:00-24:00	5333.3333	2859.8681
0:00 储电量		6000.0000	24:00 储电量		6000.0000

六、 问题二的模型建立和求解

问题二须在实际值未知时确定全天购电策略，预测偏差可能触发高价紧急购电。本问建模分为预测与优化两部分：每天 0:00 基于历史数据预测未来 48 h 的负载与光伏发电功率，经非对称风险校准得到校准净负荷；随后确定日前购电计划，并在日内通过局部修正消化实时偏差。其中，前 144 个 10 分钟时段用于当天日前计划，后 144 个时段用于跨越午夜评估储能设备的未来价值。

6.1 预测模型

预测模型分为四个部分，负载预测、光伏发电功率预测、冷启动处理办法、非对称风险校准。

6.1.1 负载预测

负载同时具有日内生活规律、工作日与周末差异及全年缓慢升降三类结构。由于每 10 min 一个观测点，一天与一周分别对应 144 与 1008 个时段，故采用 MSTL 将负载分解为趋势、日周期、周周期与残差四部分：

$$P_t^L = T_t^L + S_{144,t}^L + S_{1008,t}^L + R_t^L, \quad (2)$$

其中 t 为 10 min 时段序号 ($t = 1, 2, \dots, 144$)， T_t^L 为缓慢变化的趋势项， $S_{144,t}^L$ 与 $S_{1008,t}^L$ 分别为周期 144 与 1008 的季节项，对应日周期与周周期， R_t^L 为剔除确定性结构后的残差。MSTL 以局部加权回归依次提取各季节分量与趋势分量，对邻近观测赋予较高权重^[2]：

$$\min_{\beta} \sum_i w_i(t) (y_i - x_i^T \beta)^2, \quad (3)$$

其中 y_i 为第 i 个观测点的负载功率， x_i 为该点局部回归的设计向量， β 为待估的局部回归系数， $w_i(t)$ 为第 i 个观测在目标时刻 t 处的权重，距 t 越近权重越大。本题仅含一年数据，年周期只有一个完整样本，无法稳定估计，故不另设年季节项，而由趋势项 T_t^L 承担全年升降。若分解残差仍存在显著自相关，则建立 AR(p) 模型：

$$R_t^L = c + \sum_{i=1}^p \phi_i R_{t-i}^L + \varepsilon_t, \quad (4)$$

其中 c 为常数项， ϕ_i 为第 i 阶自回归系数， ε_t 为白噪声。该式将残差分解为可预测的短期自相关部分与白噪声，阶数 p 从候选集合 $\{1, 2, 3, 6, 12, 18, 36\}$ 中按滚动预测误差选取。最终负载预测为各分量预测之和：

$$\hat{P}_{t+h}^L = \hat{T}_{t+h}^L + \hat{S}_{144,t+h}^L + \hat{S}_{1008,t+h}^L + \hat{R}_{t+h}^L, \quad (5)$$

其中 h 为预测步长（以 10 min 时段计），带帽符号表示对应分量的预测值；趋势项以末端局部趋势外推，季节项按相同相位延拓，残差项由自回归模型递推。为控制计算量，我们采用“低频完整分解、高频轻量更新”策略：由于日、周用电规律数日内几乎不变，两次完整拟合间直接复用其周期模板；针对逐日漂移的整体水平，提取最近 3 天数据扣除周期模板得到去季节化序列，拟合直线外推趋势（斜率限幅于 ± 5 kW/时段以防发散）；并每日更新残差自回归的递推起点。该机制使 MSTL 计算量降至原来的 $\frac{1}{7}$ ，在保证精度的同时显著提升求解效率。

6.1.2 光伏发电功率预测

光伏功率的日内形状与季节幅值由太阳位置决定，具有确定性；当日发电水平则主要受云量等天气因素衰减。据此将光伏功率表示为季节晴空强度与日透射率的乘积加残差：

$$P_{d,t}^{\text{PV}} = G_{d,t}^{\text{PV}} \kappa_d + R_{d,t}^{\text{PV}}, \quad 0 \leq \kappa_d \leq 1, \quad (6)$$

其中 d 为日期序号， t 为时段序号， $G_{d,t}^{\text{PV}}$ 为季节晴空强度，表示近似晴空条件下，当天第 t 时段光伏系统能够转换的最大电功率。 κ_d 为日透射率，表示当天实际发电量占晴空潜在发电量的比例； $R_{d,t}^{\text{PV}}$ 为乘性结构未能解释的短时残差。晴空强度由滚动历史窗口内同一时刻光伏功率的高分位数估计并平滑得到：

$$\tilde{G}_{d,t}^{\text{PV}} = Q_\alpha(\{P_{j,t}^{\text{PV}} : j \in W_d\}), \quad \alpha = 0.93, \quad (7)$$

其中 $Q_\alpha(\cdot)$ 表示取 α 分位数（ $\alpha = 0.93$ 为分位水平）， $P_{j,t}^{\text{PV}}$ 为历史第 j 天第 t 时段的实际光伏功率，窗口 W_d 取最近 60 天且仅含 $j < d$ 的历史日以保证不引入未来信息， $\tilde{G}_{d,t}^{\text{PV}}$ 为平滑前的晴空强度估计，沿日内时间轴平滑后记为 $G_{d,t}^{\text{PV}}$ 。日透射率定义为

$$\tilde{\kappa}_d = \frac{\sum_{t=1}^{144} P_{d,t}^{\text{PV}} \Delta t}{\sum_{t=1}^{144} G_{d,t}^{\text{PV}} \Delta t}, \quad \kappa_d = \min\{1, \max\{0, \tilde{\kappa}_d\}\}, \quad (8)$$

其中 $\Delta t = \frac{1}{6} \text{ h}$ 为时段长度，分子与分母分别为第 d 天的实际发电量与晴空潜在发电量， $\tilde{\kappa}_d$ 为未截断的透射率估计， κ_d 为截断到 $[0, 1]$ 后的日透射率。相邻天气过程具有持续性，故对日透射率序列建立 AR(p) 模型进行日滚动预测^[3]：

$$\kappa_d = c + \sum_{i=1}^p \phi_i \kappa_{d-i} + u_d, \quad (9)$$

其中 c 为常数项， ϕ_i 为自回归系数， u_d 为白噪声。由该式得到预测值 $\hat{\kappa}_d$ ，阶数 p 从候选集合 $\{1, 2, 3, 5, 7\}$ 中选取。结构残差 $R_{d,t}^{\text{PV}} = P_{d,t}^{\text{PV}} - G_{d,t}^{\text{PV}} \kappa_d$ 若自相关显著，同样以自回

归模型修正（阶数从 $\{1, 2, 3, 6, 12\}$ 中选取）。最终光伏预测重构为

$$\hat{P}_{d,t}^{\text{PV}} = G_{d,t}^{\text{PV}} \hat{\kappa}_d + \hat{R}_{d,t}^{\text{PV}}, \quad (10)$$

其中 $\hat{\kappa}_d$ 为日透射率预测值， $\hat{R}_{d,t}^{\text{PV}}$ 为残差自回归修正项。并施加物理约束：预测值非负，且对历史上稳定无发电的夜间时段直接置零。

6.1.3 冷启动与一月份预热

2025 年 1 月 1 日 0:00 尚无历史实际数据，上述模型无法启动。我们以附件 1 的典型负载与光伏曲线作为先验模板，与已积累的近期同一时刻实际值加权融合，使得随着时间推移就越少参考附件 1 的数据，加权融合表达式如下：

$$\hat{P}_{d,t} = w_0(d) P_t^{\text{att1}} + \sum_{j=1}^{d-1} w_j(d) P_{d-j,t}, \quad w_0(d) \downarrow 0, \quad (11)$$

其中 P_t^{att1} 为附件 1 典型曲线在第 t 时段的取值， $P_{d-j,t}$ 为第 $d-j$ 天第 t 时段的实际值， $w_0(d)$ 与 $w_j(d)$ 分别为先验模板与历史实际值的权重；近期同一时刻实际值按指数衰减加权，先验权重 $w_0(d)$ 随 d 增大单调递减至 0，历史满一周后进一步混入上周曲线数据。历史不足 21 天时以该融合曲线作为预测；达到 21 天后切换至 MSTL 与季节晴空强度的正式模型。于是我们规定，下文针对全年的所有求解结果均为从 2 月 1 日到年末的求解结果。

6.1.4 非对称风险校准

定义净负荷及其预测与误差：

$$P_{d,t}^{\text{net}} = P_{d,t}^{\text{L}} - P_{d,t}^{\text{PV}}, \quad \hat{P}_{d,t}^{\text{net}} = \hat{P}_{d,t}^{\text{L}} - \hat{P}_{d,t}^{\text{PV}}, \quad (12)$$

其中 $P_{d,t}^{\text{net}}$ 与 $\hat{P}_{d,t}^{\text{net}}$ 分别为第 d 天第 t 时段的实际净负荷与预测净负荷。相应预测误差为

$$e_{d,t}^{\text{net}} = \left(P_{d,t}^{\text{L}} - \hat{P}_{d,t}^{\text{L}} \right) - \left(P_{d,t}^{\text{PV}} - \hat{P}_{d,t}^{\text{PV}} \right), \quad (13)$$

其中 $e_{d,t}^{\text{net}}$ 为净负荷预测误差，即负载预测误差与光伏发电功率预测误差之差。当 $e_{d,t}^{\text{net}} > 0$ 时实际净负荷高于预测，对应负载低估或光伏高估，易触发 5 倍电价的紧急购电；当 $e_{d,t}^{\text{net}} < 0$ 时通常仅形成富余，代价较轻，故误差风险具有显著非对称性。

为量化该非对称性，比较单位正向误差的两种补救方式。其一为立即紧急购电，单位正向误差的最小边际补救成本为

$$c_{d,t}^+ = \min \left\{ 5\lambda_t, \min_{s \in \mathcal{F}_{d,t}} \frac{\lambda_s}{\eta_c \eta_d} \right\}, \quad (14)$$

其中 $c_{d,t}^+$ 为单位正向误差的最小边际补救成本，取立即紧急购电与最优回补两种方案中较便宜的一种；若 $\mathcal{F}_{d,t} = \emptyset$ 则取 $c_{d,t}^+ = 5\lambda_t$ 。对于负向误差（实际净负荷低于预测），若

多余电量无法被储能设备吸收，其单位边际损失即为原计划购电成本 $c_{d,t}^- = \lambda_t$ 。对历史正、负向误差样本分别以指数加权平均，得到单位正向误差边际补救成本与单位负向误差边际损失的滚动估计 $\bar{c}_{d,t}^+$ 与 $\bar{c}_{d,t}^-$ 。由于非对称损失中起决定作用的是两侧权重的相对比例，不失一般性令 $\beta_{d,t} = 1$ ，并定义动态权重

$$\alpha_{d,t} = \text{clip} \left(\frac{\bar{c}_{d,t}^+}{\max \{ \bar{c}_{d,t}^-, \varepsilon_c \}}, \alpha_{\min}, \alpha_{\max} \right), \quad (15)$$

其中 $\varepsilon_c = 10^{-4}$ ， $\text{clip}(\cdot)$ 将权重限制在 $[1, 8]$ 内。历史样本不足 5 个时取初值 $\alpha_{d,t} = 4$ 。其合理性可由报童模型解释：相比原计划按 λ_t 购电，缺电 1 kWh 需按 5 倍电价紧急购电，额外增加的边际代价 $C_u = 4\lambda_t$ ；多电 1 kWh 的边际代价 $C_o = \lambda_t$ 。最优分位水平 $\tau^* = C_u / (C_u + C_o) = 0.8$ 。在风险校准公式 $\tau_{d,t} = \alpha_{d,t} / (\alpha_{d,t} + \beta_{d,t})$ 中，令 $\beta_{d,t} = 1$ ，为使分位水平匹配 0.8，解得初值 $\alpha_{d,t} = 4^{[4]}$ 。

在实际运行中， $\alpha_{d,t}$ 由公式 (15) 动态计算得出，进而得到时变的分位水平 $\tau_{d,t} = \alpha_{d,t} / (\alpha_{d,t} + 1)$ 。为了防止极端误差导致 $\tau_{d,t}$ 过大或过小（即过度保守或过度激进），我们将其截断在 $[0.55, 0.88]$ 之间。对最近 28 天同日型历史样本以 0.9 指数衰减加权，取正向净负荷误差的加权 $\tau_{d,t}$ 分位数，再乘以风险强度系数 ρ ，构造时段风险备用量：

$$R_{d,t}^{\text{risk}} = \rho Q_{\tau}(\{\max(e_{j,t}^{\text{net}}, 0) : j < d\}), \quad (16)$$

其中 $Q_{\tau}(\cdot)$ 为加权 τ 分位数。 ρ 反映了系统对预测误差的风险规避程度，由于全年总费用关于 ρ 非凸且非光滑，我们采用“理论指导 + 滚动回测”确定其值。由于前述报童模型推导的 $\tau^* = 0.8$ ，我们在 $[0.70, 0.90]$ 内以 0.05 为步长进行滚动回测，由图6，全年总费用随 ρ 呈“U 型”变化，并在 $\rho = 0.75$ 处达到全局最小，故最终取 $\rho = 0.75$ 。

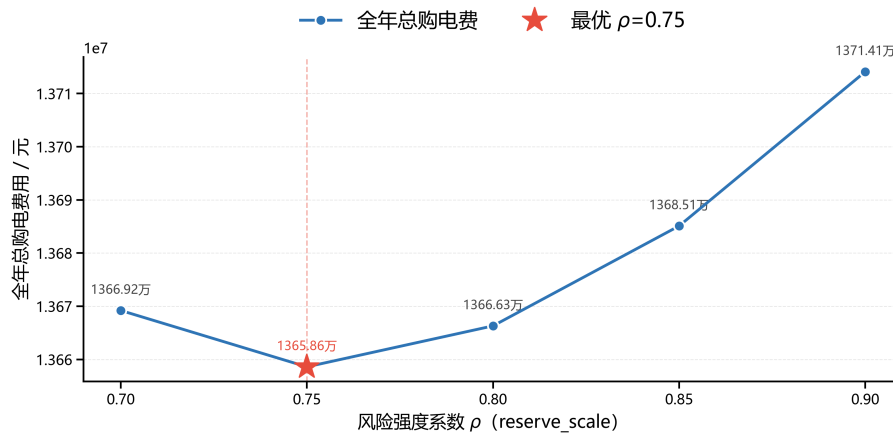


图 6 全年总购电费用随风险强度系数的变化

风险校准后的净负荷预测为： $\tilde{P}_{d,t}^{\text{net}} = \hat{P}_{d,t}^{\text{net}} + R_{d,t}^{\text{risk}}$ 。

6.2 优化模型

6.2.1 日前计划购电

每天 0:00 以风险校准净负荷为需求、以当天 0:00 实际储能状态为初值，求解一次全天线性规划。与第一问相比，问题二的优化模型主要有以下不同：

- (1) **目标函数**：在计划购电费的基础上，增加日末储能机会价值与循环惩罚两项：

$$\min J_d = \sum_{t=1}^{144} \lambda_t P_{d,t}^{\text{buy}} \Delta t + \mu (S_{d,0} - S_{d,144}) + \varepsilon \sum_{t=1}^{144} (P_{d,t}^{\text{ch,p}} + P_{d,t}^{\text{dis,p}}) \Delta t, \quad (17)$$

其中 $\mu = \eta_d \lambda_{\min}$ 为日末储能机会价值（ $\lambda_{\min}$ 为全天最低正常电价），表示日末每保留 1 kWh 电量，次日至少能按放电效率折算出的保守收益，用于防止优化器在日末过度放电； $\varepsilon > 0$ 为极小的循环惩罚系数，用于软性抑制无意义的同时充放电。这两项仅影响决策，不计入实际费用。

- (2) **功率平衡**：需求侧由第一问的已知负载与光伏，替换为风险校准后的预测净负荷 $\tilde{P}_{d,t}^{\text{net}}$ ：

$$P_{d,t}^{\text{buy}} + P_{d,t}^{\text{dis,p}} = \tilde{P}_{d,t}^{\text{net}} + P_{d,t}^{\text{ch,p}} + P_{d,t}^{\text{sur,p}}. \quad (18)$$

- (3) **日内局部修正**：计划购电确定后，真实负载与光伏发电功率逐时到达。由于前序时段的预测误差会导致当前真实储电量偏离计划轨迹。因此，设当前时段为 t ，首先将日前计划充放电功率投影到当前真实储电量与功率上限所允许的可行范围内，得到实际可执行的充、放电功率 $P_{d,t}^{\text{ch}}$ 与 $P_{d,t}^{\text{dis}}$ 。在此基础上，计算当前时段的实际供需偏差：

$$\Delta P_{d,t} = P_{d,t}^{\text{L}} - (P_{d,t}^{\text{buy,*}} + P_{d,t}^{\text{PV}} + P_{d,t}^{\text{dis}} - P_{d,t}^{\text{ch}}), \quad (19)$$

若 $\Delta P_{d,t} > 0$ ，表示当前供电不足。为最小化经济损失，按代价递增顺序进行代数修正：首先取消当前时段不必要的计划充电；其次，在最大放电功率与储电量下限允许的范围内增加储能设备放电，代价为消耗储能；若上述调整后仍存在缺口，则不足部分记为紧急购电 $P_{d,t}^{\text{em}}$ ，并按 $5\lambda_t$ 的高价计费。

若 $\Delta P_{d,t} < 0$ ，表示当前供给富余。同样按代价递增顺序处理：先取消不必要的计划放电，再于充电功率与储电量上限允许范围内增加充电，将多余电能储存起来留待后用；仍无法吸收的部分记为系统富余功率 $P_{d,t}^{\text{sur}}$ 。

通过上述机制，日内阶段无需重复求解全天或 24 h 优化，仅进行当前时段的代数判断与状态更新，从而在严格保持计划购电不变的同时，将大部分预测误差消化在低成本的储能调整中，显著降低了计算量。

其余约束条件与第一问完全一致：

$$\begin{cases} S_{d,t}^p = S_{d,t-1}^p + \left(\eta_c P_{d,t}^{\text{ch,p}} - \frac{P_{d,t}^{\text{dis,p}}}{\eta_d} \right) \Delta t & (\text{储能状态方程}), \\ 1200 \leq S_{d,t}^p \leq 10800 & (\text{储电量上下限}), \\ 0 \leq P_{d,t}^{\text{ch,p}}, P_{d,t}^{\text{dis,p}} \leq 5000 & (\text{充放电功率上限}), \\ P_{d,t}^{\text{buy}}, P_{d,t}^{\text{sur,p}} \geq 0 & (\text{非负约束}), \\ S_{d,0}^p = S_{d,0}^{\text{actual}} & (\text{跨日状态传递}), \end{cases} \quad (20)$$

特别地，不施加日末恢复约束 $S_{d,144} = S_{d,0}$ 。储能设备留电量由电价、净负荷与未来紧急购电风险共同决定。

6.2.2 全年购电费用

全天计划购电费用与紧急购电费用分别为

$$C_d^{\text{plan}} = \sum_{t=1}^{144} \lambda_t P_{d,t}^{\text{buy,*}} \Delta t, \quad C_d^{\text{em}} = \sum_{t=1}^{144} 5 \lambda_t P_{d,t}^{\text{em}} \Delta t, \quad (21)$$

当天实际总费用为 $C_d = C_d^{\text{plan}} + C_d^{\text{em}}$ ，全年总费用为各日之和。日末储能机会价值项仅用于优化决策，不计入费用统计。

6.3 结果分析

6.3.1 负荷与光伏发电功率预测结果

由图 7，负载预测曲线紧密跟随实际日均负载的周周期波动与季节性升降。由图8，光伏预测方面，由于引入了季节晴空强度与日透射率分解，预测曲线很好地刻画了光伏出力夏季与初秋较高的特征，有效滤除了天气引起的剧烈日内波动。

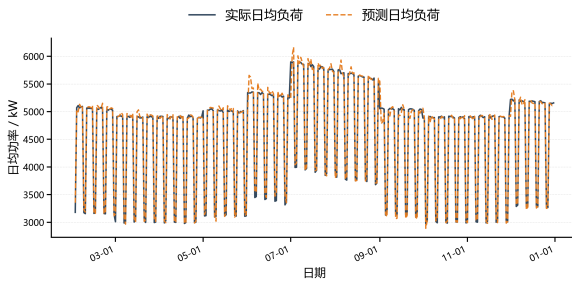


图 7 全年负荷日均预测结果对比

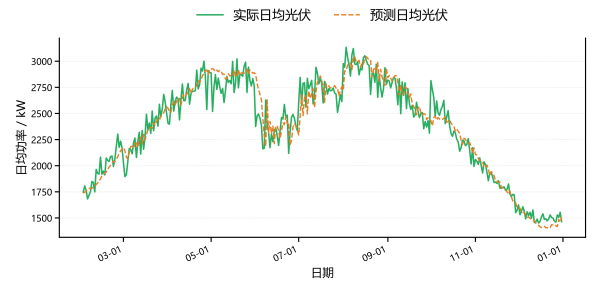


图 8 全年光伏日均预测结果对比

以 9 月 23 日为例，图 9 展示了该日负载与光伏的逐时预测对比。该日光伏发电预测功率在约 10:00-14:00 略低于实际功率，这是模型中考虑非对称风险导致的，因为

如果光伏预测功率比实际光伏功率高了，那么就更有可能导致紧急购电，体现了模型的有效性。

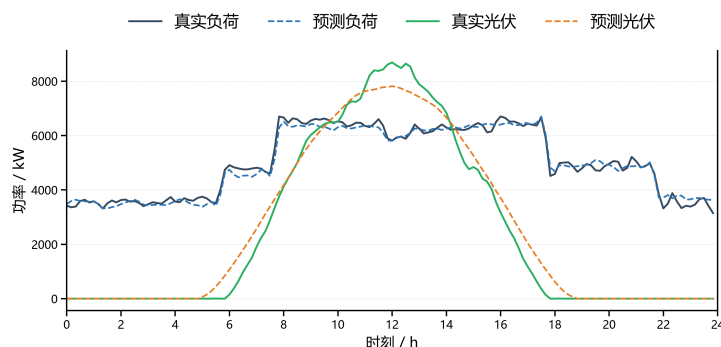


图9 9月23日负荷与光伏预测对比

6.3.2 经济性与调度结果分析

由图10，储能设备全年日末电量主要在1200–4000 kWh区间波动，仅在少数光伏发电功率极大或负荷极低的日期触及10800 kWh上限。这表明模型并未盲目追求日末满充，而是根据次日预测与电价结构动态调整留电量。由图11，储能设备在夜间低谷电价时段快速充电至上限附近，在白天光伏充足且电价较高时段保持高位或缓慢放电，并在晚间高峰集中放电以替代高价购电，体现了模型在无日末恢复约束下，根据电价信号自主优化储能时序价值的合理性。

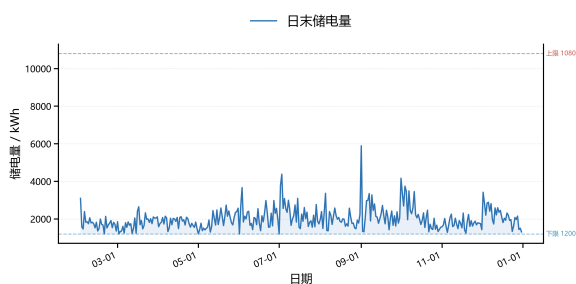


图10 2月起储能设备日末电量轨迹

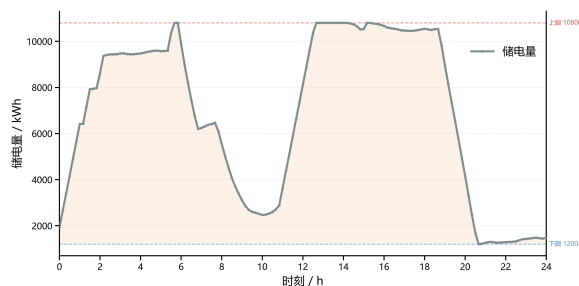


图11 3月20日储能设备电量变化

图12和图13分别展示了月度购电费用构成与紧急购电量。全年累计总费用为13,658,561.24元，其中计划购电费占绝对主导，紧急购电费仅为460,577.71元，占比约3.37%。6月和10月由于光伏预测误差较大或负荷波动剧烈，紧急购电量相对较高，但整体控制在较低水平。全年334个正式运行日中，有190天未发生任何紧急购电，验证了非对称风险校准机制在抑制高价缺电方面的显著效果。

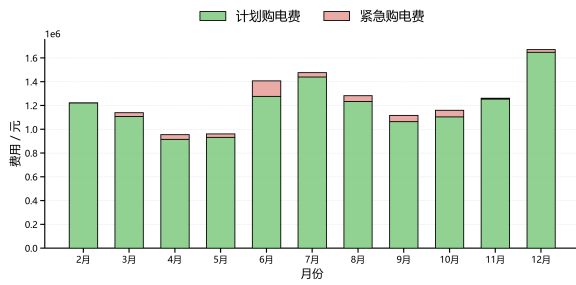


图 12 2月起月度购电费用构成

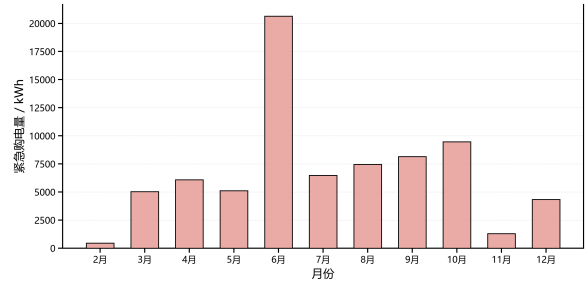


图 13 2月起月度紧急购电量

6.3.3 风险校准机制结果分析

由图 14，动态权重 α 在 1 至 8 之间动态跳变，反映了不同时段缺电与多电边际代价的非对称性变化；由图 15，风险备用电量在光伏波动剧烈的上午时段显著抬升，而在动态权重较低的午间时段相应回落。两图结合表明，模型能够根据实时的储能状态与电价结构动态调整风险偏好，避免了固定权重导致的过度保守或防御不足。

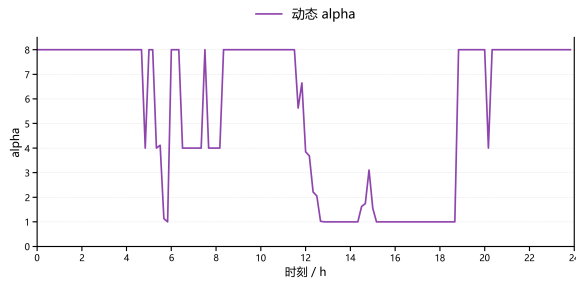


图 14 3月20日动态非对称风险权重

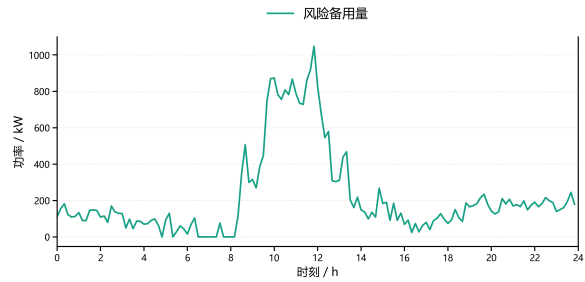


图 15 3月20日风险备用电量

6.3.4 指定日期结果汇总

表 3 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
2025 年 3 月 20 日					
10:00-10:10	0.0000	12:00-12:10	656.0758	14:00-14:10	0.0000
16:00-16:10	502.3997	18:00-18:10	674.1534	20:00-20:10	0.0000
全天购电量		69043.4708	全天购电费		41776.7472
2025 年 6 月 21 日					
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	154.2361	18:00-18:10	0.0000	20:00-20:10	0.0000
全天购电量		36310.5429	全天购电费		20844.2500
2025 年 9 月 23 日					
10:00-10:10	0.0000	12:00-12:10	570.7559	14:00-14:10	0.0000
16:00-16:10	486.5106	18:00-18:10	766.6558	20:00-20:10	23.1711
全天购电量		70629.1734	全天购电费		43670.4344
2025 年 12 月 21 日					
10:00-10:10	0.0000	12:00-12:10	1021.8394	14:00-14:10	0.0000
16:00-16:10	847.7241	18:00-18:10	730.7123	20:00-20:10	0.0000
全天购电量		97795.9882	全天购电费		63237.9143

表 4 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
2025 年 3 月 20 日					
0:00-4:00	8441.5942	35.5391	4:00-8:00	1789.8129	4979.2959
8:00-12:00	6280.4742	2777.3772	12:00-16:00	3293.0820	371.8762
16:00-20:00	140.6032	5973.8617	20:00-24:00	372.8946	2722.5539
0:00 储电量		1916.5719	24:00 储电量		1469.2937
2025 年 6 月 21 日					
0:00-4:00	2097.4756	22.7004	4:00-8:00	1651.4257	1719.9540
8:00-12:00	7809.9808	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	146.0190	5531.4312	20:00-24:00	1063.0907	2485.8966
0:00 储电量		2333.2888	24:00 储电量		2980.0567
2025 年 9 月 23 日					
0:00-4:00	9135.3645	0.0000	4:00-8:00	1349.7415	6253.8799
8:00-12:00	6282.3682	1896.6157	12:00-16:00	4079.9338	555.1772
16:00-20:00	37.6072	6085.6330	20:00-24:00	978.6923	2359.5057
0:00 储电量		1422.3258	24:00 储电量		2043.2054
2025 年 12 月 21 日					
0:00-4:00	8076.9325	107.9823	4:00-8:00	2389.3734	1566.3029
8:00-12:00	5880.7629	7179.2478	12:00-16:00	6752.1884	2220.1094
16:00-20:00	0.0000	5474.9855	20:00-24:00	716.3914	2842.5258
0:00 储电量		2314.7156	24:00 储电量		2203.0731

表 5 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
20:30-20:40	267.3429	无紧急购电	0.0000	20:20-20:40	488.3076	7:50-8:00	0.3403

七、 问题三的模型的建立和求解

7.1 6 小时预报购电调整模型

首先处理预报粒度与模型时段的匹配。附件 3 给出的是整点(1 小时)预报,而我们的调度模型以 10 分钟为一个时段(每天 144 个时段)。设第 k 个发布时点($k \in \{0, 6, 12, 18\}$)发布的未来 24 个整点光伏预报为 $\hat{P}_{k,h}^{PV}$ ($h = 1, 2, \dots, 24$)。我们采用分段线性插值,将其转换为 10 分钟时段的预报值 $\hat{P}_{k,t}^{PV}$ ($t = 1, 2, \dots, 144$)。插值时,当前发布时刻的锚点不使用未来真实值(0:00 取 0, 其余时点取上一 10 分钟区间已实现的实际光伏),以避免未来信息泄露。

其次,当新预报到达时,我们需要重新优化尚未执行的购电计划。设第 $k-1$ 次更新前已确认的购电计划为 $P_{k-1,t}^{buy}$, 第 k 次更新后的新计划为 $P_{k,t}^{buy}$ 。我们将购电调整量分解为向上追加和向下削减两部分:

$$P_{k,t}^{buy} = P_{k-1,t}^{buy} + U_{k,t}^+ - U_{k,t}^-, \quad U_{k,t}^+ \geq 0, U_{k,t}^- \geq 0, \quad (22)$$

其中 $U_{k,t}^+$ 为向上调整量, $U_{k,t}^-$ 为向下调整量。根据题目规则, 向上追加的购电按同期正

常电价的 1.5 倍结算，向下削减的购电需承担同期正常电价 50% 的违约费用。因此，第 k 次调整带来的增量费用为：

$$\Delta C_k = \sum_t \left(1.5\lambda_t U_{k,t}^+ + 0.5\lambda_t U_{k,t}^- \right) \Delta t, \quad (23)$$

其中 λ_t 为第 t 时段的正常电价， $\Delta t = 1/6$ h。

7.2 日内 24 小时滚动线性规划模型

在获得新预报后，我们以当前实际储能状态为初值，重新求解未来 24 小时的线性规划。由于 18:00 的预报窗口会跨越午夜，我们将决策窗口分为“当天剩余时段”和“跨入次日的辅助时段”。当天剩余时段的购电通过上述调整变量修改，跨日部分则作为“影子购电”按正常电价计入目标，仅用于评估日末储能的机会价值，不写入当天正式方案。

滚动优化的目标函数为最小化调整费用、跨日影子购电费、紧急购电费以及极小的储能循环惩罚：

$$\min J_k = \Delta C_k + \sum_{t \in \mathcal{H}^{\text{next}}} \lambda_t \bar{P}_t^{\text{buy}} \Delta t + \sum_t 5\lambda_t P_t^{\text{em}} \Delta t + \varepsilon \sum_t (P_t^{\text{ch}} + P_t^{\text{dis}}) \Delta t, \quad (24)$$

其中 $\mathcal{H}^{\text{next}}$ 为跨入次日的辅助时段集合， $\bar{P}_t^{\text{buy}}$ 为影子购电功率， P_t^{em} 为紧急购电功率， ε 为极小的循环惩罚系数^[5]。

功率平衡与储能约束与问题二保持一致。对于当天剩余时段，功率平衡为：

$$P_{k,t}^{\text{buy}} + P_t^{\text{dis}} + \hat{P}_{k,t}^{\text{PV}} = \hat{P}_t^{\text{L}} + P_t^{\text{ch}} + P_t^{\text{sur}}, \quad (25)$$

其中 $\hat{P}_t^{\text{L}}$ 为沿用问题二的负荷预测， P_t^{sur} 为富余功率。储能状态方程、电量上下限（1200–10800 kWh）与充放电功率上限（5000 kW）均与问题二相同。

7.3 模型求解结果与分析

7.3.1 全年滚动优化结果概览

由图 16 和图 17 可以看出，总费用在夏季和冬季较高，这与冬季光伏出力弱、夏季空调负荷高有关。值得注意的是，图 17 中调整净费用在多数月份为负值，这说明日内滚动优化通过向下调整，成功释放了 0:00 初始计划中的保守余量，形成了净节约。

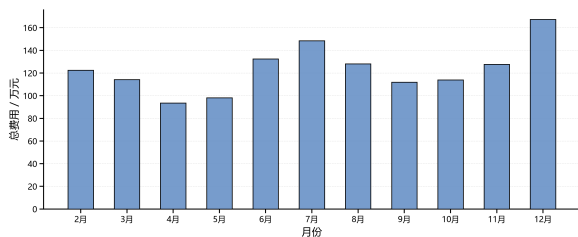


图 16 完整滚动方案的月度总购电费用

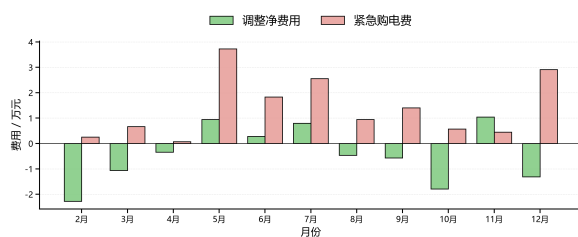


图 17 月度调整净费用与紧急购电费用

7.3.2 典型日期购电策略调整分析

为直观展示滚动优化的过程，我们选取 2025 年 3 月 20 日和 6 月 21 日两个典型日期，对比 0:00 初始计划与最终执行购电策略。

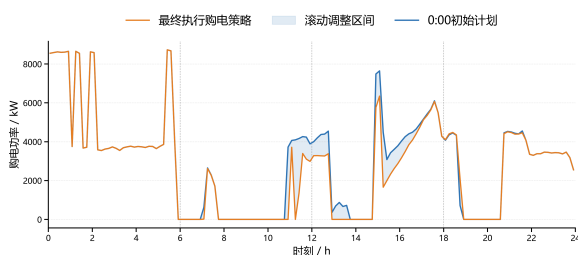


图 18 3-20 初始计划与最终购电策略

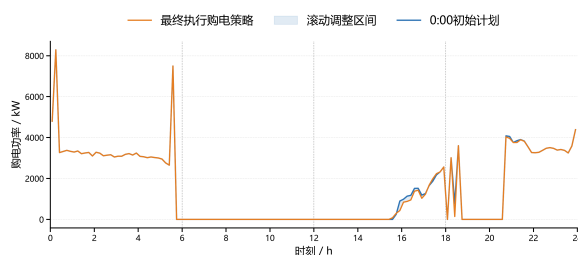


图 19 6-21 初始计划与最终购电策略

由图 18 和图 19 可以看出：初始计划购电功率在凌晨时段普遍较高，这是因为此时光伏出力极低，系统主要依赖外网购电。随着预报到来，调整购电普遍低于初始计划，说明新预报帮助系统识别并削减了过度购电。对比两图还可发现，夏季（6 月）日间的购电需求显著低于春季（3 月），这是因为夏季光伏出力更强。

7.3.3 不同预报时点的边际价值

为了量化 6:00、12:00、18:00 预报的具体作用，我们固定负荷预测、风险参数和储能参数，仅改变允许使用的日内预报更新时间。

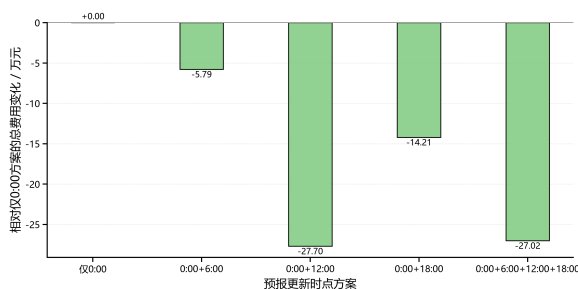


图 20 不同预报更新时点的经济性增益

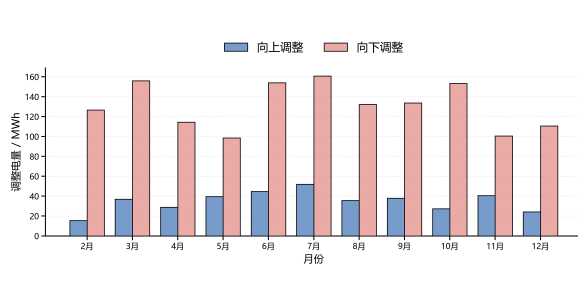


图 21 月度向上与向下调整电量

由图 20，加入 12:00 预报后总费用下降最多，说明 12:00 预报具有最高的经济边际价值。这是因为 12:00 临近光伏峰值，能直接修正下午的光伏出力判断。而加入 18:00 预报虽然对总费用的改善不如 12:00，但能显著降低紧急购电风险，体现了跨日模型预测控制对晚间储能配置的前瞻作用。

由图 21，每月的向下调整电量远大于向上调整电量，全年向下调整电量约为向上调整电量的 3.76 倍。这一数量关系进一步印证了我们的策略：初始购电计划总体偏保守，随着日内更接近实际运行时点的预报到达，模型多数时候都在合理地削减 0:00 时安排的部分购电，从而避免高价紧急购电，进而实现了整体费用的优化。

7.3.4 指定日期结果汇总

表 6 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
2025 年 3 月 20 日					
10:00-10:10	0.00	12:00-12:10	497.64	14:00-14:10	0.00
16:00-16:10	478.60	18:00-18:10	713.23	20:00-20:10	0.00
全天购电量		66,521.00	全天购电费		45,800.15
2025 年 6 月 21 日					
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	72.21	18:00-18:10	426.73	20:00-20:10	0.00
全天购电量		35,951.90	全天购电费		20,695.31
2025 年 9 月 23 日					
10:00-10:10	0.00	12:00-12:10	378.50	14:00-14:10	0.00
16:00-16:10	485.75	18:00-18:10	815.24	20:00-20:10	0.00
全天购电量		69,198.04	全天购电费		45,339.54
2025 年 12 月 21 日					
10:00-10:10	0.00	12:00-12:10	832.75	14:00-14:10	0.00
16:00-16:10	837.17	18:00-18:10	746.50	20:00-20:10	0.00
全天购电量		95,715.63	全天购电费		65,318.21

表 7 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
2025 年 3 月 20 日					
0:00-4:00	8,560.04	35.54	4:00-8:00	1,549.06	6,032.08
8:00-12:00	4,090.28	2,665.05	12:00-16:00	6,461.45	322.32
16:00-20:00	236.06	5,584.66	20:00-24:00	351.33	2,908.51
0:00 储电量		1,804.78	24:00 储电量		1,430.23
2025 年 6 月 21 日					
0:00-4:00	2,111.42	22.70	4:00-8:00	1,697.81	1,719.95
8:00-12:00	7,941.78	0.00	12:00-16:00	0.00	0.00
16:00-20:00	135.48	5,615.93	20:00-24:00	1,019.48	2,485.90
0:00 储电量		2,160.38	24:00 储电量		2,837.44
2025 年 9 月 23 日					
0:00-4:00	9,135.37	0.00	4:00-8:00	1,382.37	6,550.19
8:00-12:00	5,969.35	1,896.62	12:00-16:00	4,859.00	411.46
16:00-20:00	124.82	5,654.53	20:00-24:00	833.28	2,920.80
0:00 储电量		1,422.32	24:00 储电量		2,125.43
2025 年 12 月 21 日					
0:00-4:00	8,381.06	107.98	4:00-8:00	2,395.54	1,611.48
8:00-12:00	5,153.96	7,766.48	12:00-16:00	8,007.80	2,021.02
16:00-20:00	4.16	5,610.91	20:00-24:00	656.58	2,842.53
0:00 储电量		2,041.00	24:00 储电量		2,001.96

表 8 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:30-10:00	112.32	无紧急购电	0.00	20:20-20:30	6.38	7:50-8:00	17.77
20:30-20:40	99.08					10:30-10:50	327.15

7.4 通过动态非对称风险带筛选额外预报时间点

关于是否有必要引入其他时刻的预报，首先，并不是预报越多越好，因为新增预报会带来额外的合同调整成本。因此，我们需要先识别哪些时段最缺预报，再通过完整调度模型验证其价值。

我们构造了一个关于光伏功率的动态非对称风险带，来量化现有预报的可靠性。定义光伏预测误差 $e_t = \hat{P}_t^{\text{PV}} - P_{\text{act},t}^{\text{PV}}$ 。当 $e_t > 0$ （预测偏高）时，系统会倾向于少买电，一旦实际光伏不足，极易触发 5 倍电价的紧急购电，风险极高；而当 $e_t < 0$ （预测偏低）时，系统只是多买了一点普通电，代价轻微。因此，风险带必须是非对称的：危险方向（上侧）的容许宽度更窄，安全方向（下侧）的容许宽度更宽。

风险带的上下界定义为：

$$P_{\text{act},t}^{\text{PV}} - b_t^- \leq \hat{P}_t^{\text{PV}} \leq P_{\text{act},t}^{\text{PV}} + b_t^+, \quad (26)$$

其中 b_t^+ 和 b_t^- 分别为上侧和下侧的容许宽度。为了确定这两个宽度，我们分两步进行。

第一步是确定总带宽的基础尺度 w_h 。中午光伏出力大，预测误差的天然量级就比凌晨大，因此不能取固定值。我们对每个目标小时 h ，取历史同时刻预测误差绝对值的 85% 分位数作为原始尺度，并引入一个与光伏幅值相关的下限（防止中午带宽过窄）以及绝对最小带宽，最后对相邻小时进行加权平滑，得到基础尺度 w_h 。

第二步是根据系统当前的“边际补救代价”将总带宽 $2w_h$ 分配给上下两侧。当预测偏高（危险方向）时，如果储能设备电量充足，可以通过放电弥补缺口，代价较低；如果储能见底，只能按 5 倍电价紧急购电，代价极高。我们据此计算危险方向和安全方向的单位边际代价 c_t^+ 与 c_t^- 。显然，代价越高的方向，容许宽度应该越窄。因此，上下侧带宽按边际代价反比分配：

$$b_t^+ = \frac{2c_t^-}{c_t^+ + c_t^-} w_h, \quad b_t^- = \frac{2c_t^+}{c_t^+ + c_t^-} w_h, \quad (27)$$

由于危险方向的边际代价 c_t^+ 通常大于安全方向的 c_t^- ，该式自然保证了 $b_t^+ < b_t^-$ 。当储能设备电量较低、可放电能力弱时， c_t^+ 会逼近 5 倍紧急购电代价，风险带在危险方向会自动收窄。

在确定了风险带后，我们定义每个时段的“危险高估越界量”为

$$\Delta_t = \max \left(\hat{P}_t^{\text{PV}} - (P_{\text{act},t}^{\text{PV}} + b_t^+), 0 \right). \quad (28)$$

为了找出最需要补充预报的时段，我们将白天划分为三个区间 $I_1 = [0, 6)$ 、 $I_2 = [6, 12)$ 、 $I_3 = [12, 18)$ ，并按区间汇总危险高估的越界总量，得到“风险集中度”：

$$E_k = \sum_{t \in I_k} \Delta_t, \quad k \in \{1, 2, 3\}, \quad (29)$$

风险集中度越大的区间，说明现有预报在该时段越不精确。为了精准定位，我们进一步计算该区间的“风险重心”时刻：

$$t_k^* = \frac{\sum_{t \in I_k} t \cdot \Delta_t}{E_k}, \quad (30)$$

风险重心 t_k^* 反映了该区间内预报偏差最严重、信息最匮乏的核心时刻。我们取 t_k^* 对应的整点时刻，作为候选新增预报时点。

对于预报数据的构造，我们以最近一次官方预测为基准，利用截至当前的实际观测残差进行指数衰减加权修正，以模拟信息更新。该过程严格仅使用已知历史数据，杜绝未来信息泄露。

7.5 额外预报时间求解结果

7.5.1 候选时点筛选

由表9，将白天划分为三个区间统计危险高估风险，6:00–12:00 区间贡献了约 61.51% 的危险高估风险，风险重心位于 10:00 附近；下午 12:00–18:00 区间贡献了约 37.41% 的风险，风险重心位于 15:00 附近。因此，筛选出 10:00 和 15:00 作为候选新增预报时点。

表 9 风险带筛选得到的候选新增预报时点

区间	风险重心	候选时点	危险高估总量/kW	风险占比/%	采用
0:00–6:00	6.000	5:00	853.03	1.08	否
6:00–12:00	11.195	10:00	48,376.77	61.51	是
12:00–18:00	15.635	15:00	29,424.00	37.41	是

7.5.2 候选预测时点与原方案对比分析

由图 22，同时加入 10:00 和 15:00 的增强方案在危险高估总量上下降约 19.3%，保守低估总量下降约 53.5%，RMSE 和 MAE 分别下降 13.7% 和 8.9%。图 23 以 3 月 20 日

为例展示了新增预报前后的风险带变化：原方案（上图）在 10:00–13:00 时段存在多个危险高估点，预报值显著高于实际光伏发电功率；增加预报后（下图），生效预报曲线更紧密地贴合实际光伏轨迹，危险高估点的数量和幅度均明显减小。

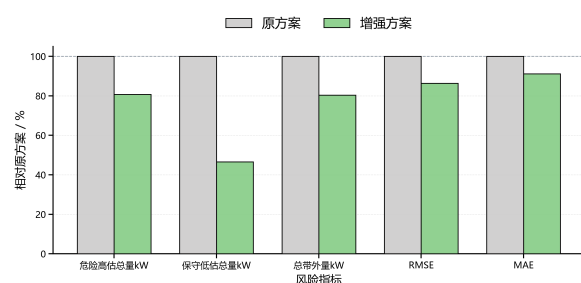


图 22 增强方案与原方案的风险指标

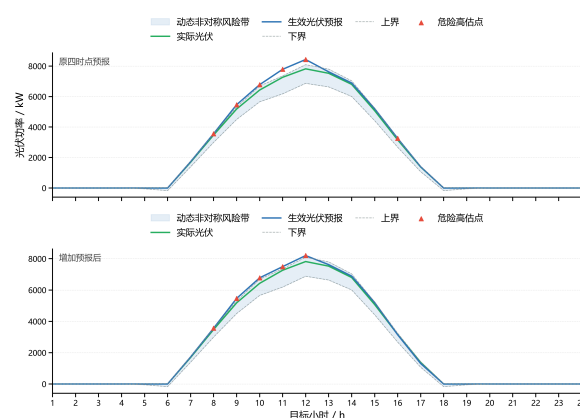


图 23 3-20 动态非对称风险带前后对比

预测精度的提高并不必然导致系统总费用的降低。图 24和图 25 展示了新增预报时点对总费用和紧急购电量的影响。由图 24，所有新增预报方案均使全年总费用上升，这是因为新预报虽然减少了高价紧急购电，但也触发了更多次合同向上或向下调整，新增的调整成本超过了紧急购电的节省。由图 25，“原方案 +10:00”使紧急购电量从 28.67 MWh 降至 26.83 MWh（下降约 6.4%），表现出明确的可靠性价值；而“原方案 +15:00”的紧急购电量反而微升至 28.94 MWh，说明下午新增预报主要改善了保守低估问题，却未稳定降低供电不足风险，甚至因调整策略波动引入了新的风险。

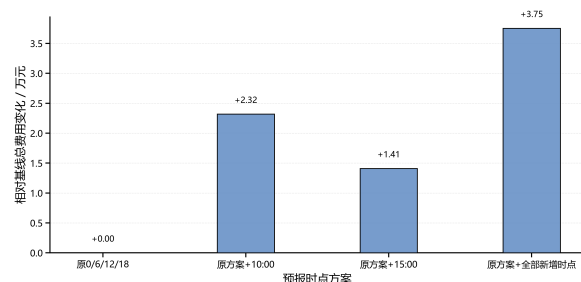


图 24 新增预报时点对总费用的影响

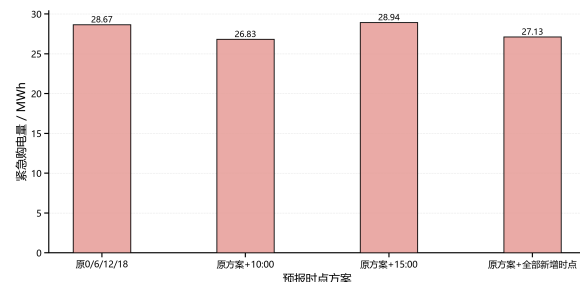


图 25 新增预报对紧急购电量的影响

7.5.3 决策建议

- (1) 以经济性为首要目标：由于所有新增时点方案均使总费用增加，推荐维持方案。
- (2) 以可靠性为首要目标：若运营者愿意支付约 0.17% 的额外费用以换取 6.4% 的紧急购电削减，可选择性增加 10:00 预报。

八、 问题四的模型建立和求解

8.1 波动电价预测模型

问题四引入了附件 4 给出的波动电价，使得计划购电与紧急购电的结算价格随时间动态变化。这要求模型在决策时能够对未来电价走势进行合理预。

电价序列与负荷类似，具有显著的日周期与周周期特征。我们沿用问题二中已验证有效的多季节趋势分解（MSTL）结合自回归（AR）模型，构建电价的 48 小时滚动预测模块。设第 d 天第 t 时段的真实电价为 $\lambda_{d,t}$ ，将其分解为：

$$\lambda_{d,t} = T_{d,t} + S_{d,t}^{\text{daily}} + S_{d,t}^{\text{weekly}} + R_{d,t}, \quad (31)$$

其中 $T_{d,t}$ 为趋势分量， $S_{d,t}^{\text{daily}}$ 与 $S_{d,t}^{\text{weekly}}$ 分别为日周期与周周期分量， $R_{d,t}$ 为残差分量。趋势项以末端局部趋势外推，季节项按相同相位的近期周期成分延拓，残差项由自回归模型递推。MSTL 同样采用“低频完整分解、高频轻量更新”的加速策略，每 7 天完整重估一次季节结构，中间日期仅利用最近 3 天真实数据快速校正趋势与残差。

在日内滚动优化阶段（6:00、12:00、18:00），我们利用发布时刻前已实现的真实电价，对 0:00 的基础预测进行轻量级指数加权残差修正。设发布时点为 h ，最近 m 个时段的真实电价残差为 $r_t = \lambda_{d,t} - \hat{\lambda}_{d,t}$ ，则修正偏差为：

$$\Delta_h = \sum_{t=h-m}^{h-1} w_t r_t, \quad w_t \propto \exp(-\kappa(h-t)), \quad (32)$$

随后对未来第 k 个时段的原预测进行衰减修正： $\hat{\lambda}_{h+k}^{\text{new}} = \max\{0, \hat{\lambda}_{h+k} + \Delta_h e^{-(k-1)/\tau_d}\}$ 。这种修正仅反映近期价格偏差的延续趋势，且随提前量增加逐渐衰减，严格避免未来信息泄露。

8.2 日前与日内滚动优化

在优化框架上，问题四完全继承前两问的设计。负荷预测、光伏预报的 10 分钟插值、动态非对称风险备用机制、储能设备物理约束、日内实际执行的局部修正规则均保持不变。

唯一的变化在于目标函数中的结算电价。在 0:00 制定日前计划以及 6:00、12:00、18:00 进行日内滚动模型预测控制时，目标函数中的正常购电电价 λ_t 被替换为决策时刻可获得的预测电价 $\hat{\lambda}_t$ 。例如，0:00 日前优化的目标函数写为：

$$\min C_0 = \sum_{t=1}^{144} \hat{\lambda}_t P_t^{\text{buy}} \Delta t + \mu(E_0 - E_{144}) + \varepsilon_{\text{cyc}} \sum_{t=1}^{144} (P_t^{\text{ch}} + P_t^{\text{dis}}) \Delta t, \quad (33)$$

而在事后结算时，计划购电、向上/向下调整购电以及 5 倍电价的紧急购电，均统一按附件 4 的真实交易电价 λ_t 计算实际发生费用。

8.3 波动电价下判断引入预报时点的变化

问题三在固定电价下讨论新增预报，其风险几乎全部来自光伏预测偏高，且集中在白天光伏出力时段，因此采用划分白天三区间结合风险集中度筛选风险重心是合理的。问题四引入波动电价后，这一前提不再成立，筛选方法必须相应升级，原因有二。其一，信息价值的来源变多了：即使光伏预报完全准确，电价预测误差仍会使系统在高价时段多购电、在低价时段少购电，从而产生独立于光伏的经济损失。其二，电价预测误差导致的潜在购电损失与该小时购电量成正比，夜间与傍晚购电量最大，其可能导致的经济损失甚至超过正午的光伏风险（见图 28），风险由“白天单峰”变为“全天多峰”，问题三只划分白天三区间的做法会系统性地漏掉夜间候选时点。

为此，本文把两类风险统一折算为另外的风险尺度后逐时相加。对目标小时 h ，定义联合经济风险

$$R_h = R_h^{\text{pv}} + R_h^{\text{price}}, \quad (34)$$

其中光伏部分沿用问题三的动态非对称风险带，取全年危险高估越界量与危险方向边际代价之积：

$$R_h^{\text{pv}} = \sum_d \Delta_{d,h} c_{d,h}^+, \quad (35)$$

电价部分取电价预测绝对误差与该小时基线购电量之积：

$$R_h^{\text{price}} = \sum_d \left| \hat{\lambda}_{d,h} - \lambda_{d,h} \right| B_{d,h}, \quad (36)$$

其中 $\hat{\lambda}_{d,h}$ 为按原 0/6/12/18 方案在该时刻最后可得的电价预测， $B_{d,h}$ 为基线方案下该小时的购电量。

在联合风险之上，筛选逻辑由“区间重心”改为“未来风险覆盖得分”。其直观含义是：在 h 时发布一份新预报，对 h 之前已经发生的风险毫无帮助，只能改善其后若干小时；且提前时间越长，预报越不准、来不及调整，故其价值应按指数衰减。据此定义

$$S(h) = \sum_{k=1}^K e^{-(k-1)/\tau} R_{h+k}, \quad K = 4, \tau = 1.5, \quad (37)$$

即发布时间 h 的信息价值等于其未来 4 小时联合风险的衰减加权和。于是只要寻找 $S(h)$ 的局部峰值，也就是局部信息价值最大的点，即可选出候选预报时点。

8.4 模型求解结果与分析

8.4.1 在波动电价的情况下求解问题二

由图 26，0:00 预测电价准确捕捉了全天电价的波动趋势与峰值。由图 27，全年 10 分钟级预测点紧密分布于 $y = x$ 线附近，验证了电价预测模型在全年的稳健性。

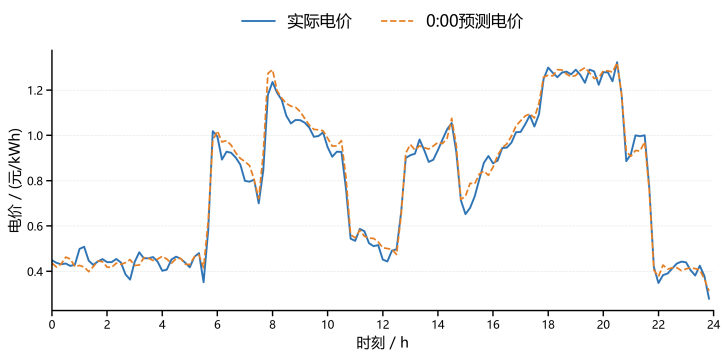


图 26 3 月 20 日波动电价预测对比

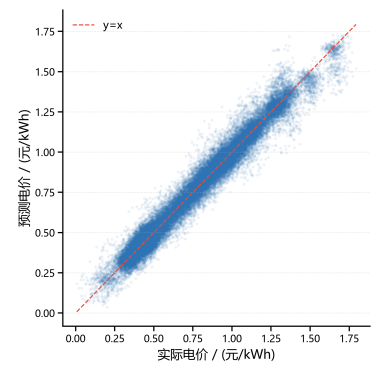


图 27 电价预测一致性

以下为题目需要的求解结果：

表 10 微网在指定时间段的计划购电量及全天的计划购电量和计划购电费

时间段	购电量	时间段	购电量	时间段	购电量
2025 年 3 月 20 日					
10:00-10:10	0.0000	12:00-12:10	638.2803	14:00-14:10	0.0000
16:00-16:10	500.8639	18:00-18:10	0.0000	20:00-20:10	0.0000
全天购电量		68551.8045	全天购电费		42784.8578
2025 年 6 月 21 日					
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	149.1563	18:00-18:10	466.2119	20:00-20:10	0.0000
全天购电量		36309.5012	全天购电费		17980.7978
2025 年 9 月 23 日					
10:00-10:10	0.0000	12:00-12:10	561.5406	14:00-14:10	0.0000
16:00-16:10	481.6190	18:00-18:10	754.7043	20:00-20:10	18.4904
全天购电量		69633.6071	全天购电费		44680.8268
2025 年 12 月 21 日					
10:00-10:10	0.0000	12:00-12:10	1021.8394	14:00-14:10	0.0000
16:00-16:10	840.7300	18:00-18:10	729.7644	20:00-20:10	0.0000
全天购电量		96170.5844	全天购电费		73574.1110

表 11 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
2025 年 3 月 20 日					
0:00-4:00	8522.2617	21.3927	4:00-8:00	2338.2736	5587.6336
8:00-12:00	6264.1518	2777.3772	12:00-16:00	3487.5944	382.9420
16:00-20:00	105.0096	5994.5064	20:00-24:00	307.3442	2677.2691
0:00 储电量		1850.1980	24:00 储电量		1393.3465
2025 年 6 月 21 日					
0:00-4:00	974.9755	27.4262	4:00-8:00	2777.9842	1719.9540
8:00-12:00	7990.8376	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	0.0000	5593.1132	20:00-24:00	1723.3616	3094.2453
0:00 储电量		2172.1160	24:00 储电量		2698.4048
2025 年 9 月 23 日					
0:00-4:00	9215.9119	4.0545	4:00-8:00	1295.3536	6629.0227
8:00-12:00	6251.4723	1896.6157	12:00-16:00	4543.5089	559.6112
16:00-20:00	21.7983	5787.0591	20:00-24:00	906.3690	2609.8334
0:00 储电量		1387.0786	24:00 储电量		1968.9438
2025 年 12 月 21 日					
0:00-4:00	8180.3084	39.9227	4:00-8:00	576.6251	968.1499
8:00-12:00	5810.0698	7174.1066	12:00-16:00	6300.3095	1729.0767
16:00-20:00	9.8044	5507.3495	20:00-24:00	672.0948	2842.5258
0:00 储电量		3031.9250	24:00 储电量		2136.0702

表 12 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
20:30-20:40	327.8844	无紧急购电	0.0000	20:10-20:40	709.1350	7:50-8:00	10.5353

8.4.2 在波动电价的情况下求解问题三

波动电价购电模型的求解结果 模型与问题三的原理一致，题目需要的结果表格如下：

表 13 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
2025 年 3 月 20 日					
10:00-10:10	0.0000	12:00-12:10	497.6442	14:00-14:10	0.0000
16:00-16:10	478.6013	18:00-18:10	713.2311	20:00-20:10	710.5250
全天购电量		66406.1835	全天购电费		47184.6081
2025 年 6 月 21 日					
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	72.2084	18:00-18:10	426.7328	20:00-20:10	0.0000
全天购电量		36087.1102	全天购电费		18004.2407
2025 年 9 月 23 日					
10:00-10:10	0.0000	12:00-12:10	378.5023	14:00-14:10	0.0000
16:00-16:10	485.7450	18:00-18:10	815.2425	20:00-20:10	0.0000
全天购电量		69171.8395	全天购电费		47408.5195
2025 年 12 月 21 日					
10:00-10:10	0.0000	12:00-12:10	832.7509	14:00-14:10	0.0000
16:00-16:10	837.1685	18:00-18:10	746.5003	20:00-20:10	0.0000
全天购电量		94430.7995	全天购电费		76511.7633

表 14 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
2025 年 3 月 20 日					
0:00-4:00	8557.8613	15.1795	4:00-8:00	1439.3960	6032.0818
8:00-12:00	4090.2844	2665.0543	12:00-16:00	6488.5491	322.3158
16:00-20:00	188.8773	5559.4231	20:00-24:00	347.3083	2917.4786
0:00 储电量		1882.8192	24:00 储电量		1426.6088
2025 年 6 月 21 日					
0:00-4:00	1051.3549	22.7004	4:00-8:00	2700.8890	1719.9540
8:00-12:00	7998.7540	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	0.0000	5602.1887	20:00-24:00	1780.7782	3091.9199
0:00 储电量		2160.3845	24:00 储电量		2742.5797
2025 年 9 月 23 日					
0:00-4:00	9247.1879	0.0000	4:00-8:00	1272.1380	6550.5150
8:00-12:00	5969.3521	1896.6157	12:00-16:00	4858.9999	411.4624
16:00-20:00	124.8186	5210.2654	20:00-24:00	806.2337	3354.8341
0:00 储电量		1421.2628	24:00 储电量		2112.4615
2025 年 12 月 21 日					
0:00-4:00	8343.4023	29.2819	4:00-8:00	514.3008	983.4854
8:00-12:00	5153.9558	7717.6411	12:00-16:00	7687.8839	1761.8901
16:00-20:00	3.9944	5610.7777	20:00-24:00	657.0453	2842.5258
0:00 储电量		2928.5209	24:00 储电量		2002.3763

表 15 微网在指定日期的紧急购电量

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:30-10:00	112.3229	无紧急购电		20:10-20:30	16.6094	7:50-8:00	17.7747
20:30-20:40	90.1097					10:30-10:50	306.9711

引入其他时刻预报的判定 按式 (34) 与式 (37) 的联合风险与覆盖得分，由图 28，综合风险集中于夜间购电高峰、正午光伏波动段以及傍晚光伏发电功率下降叠加晚负荷回升段三类时段；图 29 的覆盖得分曲线在 1:00、11:00、16:00 与 21:00 处取得局部峰值，经非极大值抑制后即为最终自动候选时点。

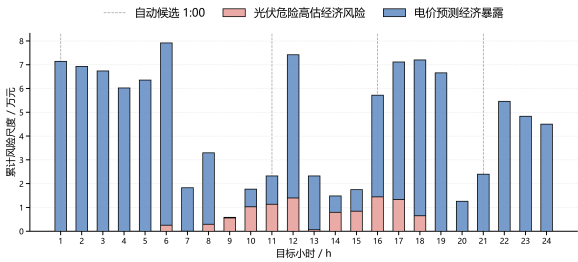


图 28 光伏风险与电价信息价值

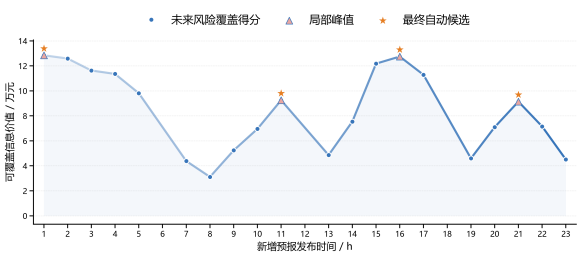


图 29 候选预报时点的信息价值

为验证候选时点的实际调度价值，分别仿真“原方案 +1:00”“原方案 +11:00”“原方案 +16:00”“原方案 +21:00”与“原方案 + 全部自动候选”五种增强方案。由图 30，所有新增方案的全年总费用均高于原 0/6/12/18 方案，其中“+1:00”增加约 8.2 万元，表现最好的单一时点“+21:00”也增加约 0.5 万元；由图 31，所有新增方案均落在经济性

一可靠性 Pareto 图的第一象限，即总费用上升且紧急购电量不降反升。其机制在于：波动电价下新增预报触发了更多合同调整，向上 1.5 倍电价与向下 50% 违约金的调整成本超过了预测精度提升带来的节省，同时频繁重构计划破坏了原方案在 SOC 管理上的稳定性。

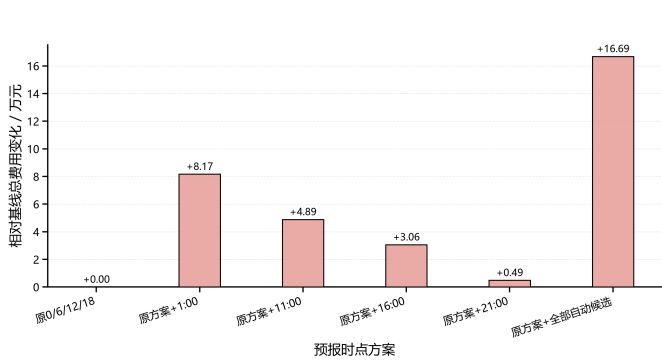


图 30 新增预报时点对总费用的影响

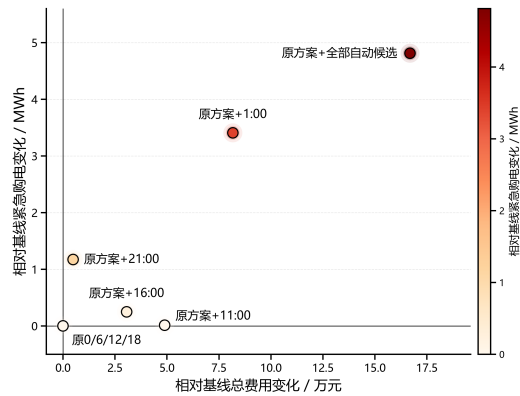


图 31 经济性—可靠性 Pareto 分布

综上，新增预报的信息增益无法覆盖其引发的调整成本，且未改善供电可靠性，故在波动电价背景下维持题目给定的 0:00、6:00、12:00、18:00 四时点预报方案，不增加额外预报时点。

九、 模型的评价

9.1 模型的优点

- **多季节分解贴合物理背景：**MSTL 将负荷与电价分解为趋势、日/周周期与残差，分别以趋势外推、周期延拓与自回归修正，贴合多重周期结构；光伏采用晴空包络乘日透射率的物理结构，分离“太阳决定的曲线形状”与“天气决定的发电强度”。配合低频完整分解、高频轻量更新与冷启动先验混合，兼顾预测精度、小样本稳健性与计算效率。
- **风险带与经济风险驱动的信息价值评估：**动态非对称风险带的带宽由当前储电量下的边际补救代价决定，精准区分可被储能设备吸收的误差与必然触发紧急购电的误差。在新增预报时点筛选上，问题三基于光伏危险高估越界量识别风险重心，问题四进一步升级为光伏与电价预测误差的综合经济风险，统一折算为货币尺度并计算未来风险覆盖得分。结合局部峰值搜索与完整滚动 LP 验证，形成了客观、可复现的“风险识别—候选筛选—经济验证”闭环。

9.2 模型的缺点

- **日透射率缺少细化预测：**由于缺少以日为单位的天气预报，所以没有构建日投射率的预报模型，可能导致日透射率 κ 没有细化预测。
- **风险校准依赖平稳性假设且为点估计：**分位数备用与动态权重假设近期误差分布可以代表未来，且只给出点估计而无概率保证，季节交替或负荷结构突变时备用水平可能短时失准。

9.3 模型的改进方向

- **融合多源信息的非线性预测：**未来可引入数值天气预报与深度学习或集成模型，对光伏与电价进行非线性建模，提升突变场景下的预测精度。
- **升级为概率化优化：**可以机会约束或分布鲁棒优化替代点分位数风险备用，对预测误差分布给出显式概率保证，提升极端天气与电价尖峰场景下决策的稳健性。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试等，详细使用情况见支撑材料。

参考文献

- [1] 郭杉, 郭洋, 刘小恺, 董文娟.. 含光-储-充的微电网日前经济调度方法研究 [J]. 山东电力技术, 2025, 52(3): 1-11, 11.
- [2] BANDARA K, HYNDMAN R J, BERGMEIR C. MSTL: a seasonal-trend decomposition algorithm for time series with multiple seasonal patterns[J]. International Journal of Operational Research, 2025, 52(1): 79-98.
- [3] 黄彦文, 褚德英, 谢颖. 基于船舶运动数据的 AR(p) 模型参数估计和适用性验证 [J]. 船舶工程, 2022, 44(S1): 500-504. DOI: 10.13788/j.cnki.cbgc.2022.S1.094.
- [4] 谢金星. 报童模型的起源与早期发展 [J]. 数学建模及其应用, 2025, 14(1): 106-113. DOI: 10.19943/j.2095-3070.jmmia.2025.01.12.
- [5] 肖浩, 裴玮, 孔力. 基于模型预测控制的微电网多时间尺度协调优化调度 [J]. 电力系统自动化, 2016, 40(18): 7-14+55.

附录 A 支撑材料的文件列表

文件名	功能描述
Q1.py	问题一求解代码
Q2.py	问题二求解代码
Q3.py	问题三优化代码
Q3-2.py	问题三是否需要引入新预测时刻代码
Q4-2.py	波动电价下求解问题二代码
Q4-3.py	波动电价下求解问题三优化代码
Q4-3-2.py	波动电价下问题三引入新预测时刻代码
附件 1.xlsx	原始附件 1 数据
附件 2.xlsx	原始附件 2 数据
附件 3.xlsx	原始附件 3 数据
附件 4.xlsx	原始附件 4 数据
附件 5	结果模板文件夹
result1.xlsx	问题一结果表格
result2.xlsx	问题二结果表格
result3.xlsx	问题三结果表格
result4-2.xlsx	波动电价下重新求解问题二结果表格
result4-3.xlsx	波动电价下重新求解问题三结果表格
AI 工具使用详情.pdf	AI 工具使用详情

附录 B 代码

Q1.py

```

1 import os
2 import pandas as pd
3 import numpy as np
4 from scipy.optimize import linprog
5 import openpyxl
6 import matplotlib
7 matplotlib.use('Agg')
8 import matplotlib.pyplot as plt
9 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
10 OUT_DIR = os.path.join(BASE_DIR, "Q1结果"); os.makedirs(OUT_DIR, exist_ok=True)
11 df=pd.read_excel(os.path.join(BASE_DIR, "附件1.xlsx"))
12 pri=df.iloc[:144, 1].values.astype(float) # lambda_t (元/kWh)
13 load=df.iloc[:144, 2].values.astype(float) # P_L (kW)
14 pv=df.iloc[:144, 3].values.astype(float) # P_PV (kW)
15 T=144
16 dt=1.0/6.0 # 10分钟 = 1/6 小时
17 eta=0.9 # 充放电效率
18 Pmax=5000.0 # 最大充放电功率
19 Smin,Smax,Sben = 1200.0,10800.0,6000.0 # 储能最小/最大电量与初始电量
20 # 决策变量定义
21 n_vars=4*T
22 c=np.zeros(n_vars)
23 c[0::4]=pri * dt

```

```

24 bounds=[bnd for t in range(T) for bnd in ((0, None), (0, Pmax), (0, Pmax), (0, pv[t]))]
25 I = np.eye(n_vars)
26 rse = np.zeros(n_vars)
27 rse[1::4]=eta*dt
28 rse[2::4]=-(1.0/eta)*dt
29 A_eq = np.vstack([I[0::4] - I[1::4] + I[2::4] - I[3::4], rse])
30 b_eq = np.append(load - pv, 0.0)
31 tri = np.triu(np.ones((T, T)))
32 A_max = np.kron(tri, [0, eta * dt, -(1.0 / eta) * dt, 0])
33 A_ub = np.empty((2 * T, n_vars))
34 A_ub[0::2], A_ub[1::2] = A_max, -A_max
35 b_ub = np.empty(2 * T)
36 b_ub[0::2], b_ub[1::2] = Smax - Sben, Sben - Smin
37 # 求解线性规划
38 res=linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
39 if not res.success:
40     raise RuntimeError(f"优化求解失败: {res.message}")
41 print("求解成功! 目标最优购电费用为:", round(res.fun, 2), "元")
42 print("全天计划购电量:", round(float(res.x[0::4].sum() * dt), 2), "kWh")
43 # 解析各变量功率 (kW)
44 pbuy, pch, pdis, pw = res.x.reshape(T, 4).T
45 # 能量转换 (kWh)
46 ebuy, ech, edis=pbuy*dt, pch*dt, pdis*dt
47 # 储能状态轨迹 (kWh)
48 soc = np.concatenate([[Sben], Sben + np.cumsum(eta * pch - pdis / eta) * dt])
49 # 构建"六个功率数据表"
50 def fmt_time(minutes):
51     m = minutes % 1440
52     return f"{m // 60}:{m % 60:02d}" + (" +1" if minutes >= 1440 else "")
53 time_labels = [f"{fmt_time((t + 1) * 10)}-{fmt_time((t + 2) * 10)}" for t in range(T)]
54 # 表1: 指定时间段购电量 + 全天购电量/购电费
55 print("\n表1 指定时间段购电量及全天购电量和购电费")
56 for m in [600, 720, 840, 960, 1080, 1200]: # 10:00 12:00 14:00 16:00 18:00 20:00 起始
57     t = m // 10 - 1
58     lbl = f"{fmt_time(m)}-{fmt_time(m + 10)}"
59     print(f" {lbl} 购电量 = {ebuy[t]:.4f} kWh")
60 print(f"全天购电量 = {ebuy.sum():.4f} kWh")
61 print(f"全天购电费 = {res.fun:.2f} 元")
62 result_path = os.path.join(OUT_DIR, "result1_filled.xlsx")
63 try:
64     wb = openpyxl.load_workbook(result_path)
65 except Exception:
66     wb = openpyxl.Workbook()
67     wb.remove(wb.active)
68 # 写入 "计划购电量" 工作表
69 ws1 = wb["计划购电量"] if "计划购电量" in wb.sheetnames else wb.create_sheet("计划购电量")
70 ws1.cell(row=1, column=1, value="时间段")
71 ws1.cell(row=1, column=2, value="购电量")
72 for t in range(T):
73     ws1.cell(row=t + 2, column=1, value=time_labels[t])
74     ws1.cell(row=t + 2, column=2, value=round(float(ebuy[t]), 4))
75 ws1.cell(row=T + 2, column=1, value="全天购电量")
76 ws1.cell(row=T + 2, column=2, value=round(float(ebuy.sum()), 4))
77 ws1.cell(row=T + 3, column=1, value="全天购电费")
78 ws1.cell(row=T + 3, column=2, value=round(float(res.fun), 2))
79 # 写入 "充放电量" 工作表
80 intervals = [("0:00-4:00", 0, 24), ("4:00-8:00", 24, 48),
81             ("8:00-12:00", 48, 72), ("12:00-16:00", 72, 96), ("16:00-20:00", 96, 120),
82             ("20:00-24:00", 120, 144)]
83 # 表2: 指定时间段充放电量 + 0:00/24:00 储电量
84 print("\n表2 指定时间段充放电量及 0:00 和 24:00 储电量")
85 for name, s_, e_ in intervals:
86     print(f" {name:<12} 充电量 = {ech[s_:e_].sum():.4f} kWh 放电量 = {edis[s_:e_].sum():.4f} kWh")
87 print(f" 0:00 储电量 = {soc[0]:.4f} kWh")
88 print(f" 24:00 储电量 = {soc[-1]:.4f} kWh")
89 if "充放电量" in wb.sheetnames:
90     wb.remove(wb["充放电量"])
91 ws2 = wb.create_sheet("充放电量")
92 ws2.cell(row=1, column=1, value="时间段")
93 ws2.cell(row=1, column=2, value="充电量")
94 ws2.cell(row=1, column=3, value="放电量")
95 ws2.cell(row=1, column=4, value="时刻")
96 ws2.cell(row=1, column=5, value="储电量")
97 for i, (name, start, end) in enumerate(intervals):
98     r=i+2
99     ws2.cell(row=r, column=1, value=name)
100    ws2.cell(row=r, column=2, value=round(float(np.sum(ech[start:end])), 4))
101    ws2.cell(row=r, column=3, value=round(float(np.sum(edis[start:end])), 4))
102    if i==0:
103        ws2.cell(row=r, column=4, value="0:00")
104        ws2.cell(row=r, column=5, value=round(float(soc[0]), 4))

```

```

105     elif i==1:
106         ws2.cell(row=r, column=4, value="24:00")
107         ws2.cell(row=r, column=5, value=round(float(soc[-1]), 4))
108     try:
109         wb.save(result_path)
110         print(f"\n结果已保存至: {result_path}")
111         print(f" 工作表: {wb.sheetnames}")
112     except PermissionError:
113         wb.save(result_path.replace(".xlsx", "_new.xlsx")) # 文件被占用时改存副本
114 # 画图
115 fig_dir = OUT_DIR
116 # 全局样式
117 plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'Arial'], 'axes.
    unicode_minus': False,
118     'font.size': 14, 'axes.labelsize': 16, 'xtick.labelsize': 13, 'ytick.labelsize': 13,
119     'legend.fontsize': 14, 'axes.spines.top': False, 'axes.linewidth': 1.6,
120     'xtick.major.width': 1.5, 'ytick.major.width': 1.5,
121     'xtick.major.size': 6, 'ytick.major.size': 6,})
122 # 配色
123 C_BUY, C_PV, C_DIS= '#2E74B5', '#27AE60', '#E67E22'
124 C_LOAD, C_CH, C_W= '#34495e', '#E74C3C', '#95A5A6'
125 C_SOC, C_PRICE, C_MAX= '#7F8C8D', '#E74C3C', '#C0392B'
126 C_MIN, C_STORE= '#2980B9', '#E67E22'
127 # 时间轴
128 t_mid=(np.arange(T)+0.5)*dt
129 t_soc=np.arange(T+1)*dt
130 # 购电去向分配
131 pv2load = np.minimum(pv, load)
132 rest = load - pv2load
133 dis2load = np.minimum(pdis, rest)
134 rest -= dis2load
135 buy2load = np.minimum(pbuy, rest)
136 buy2store = pbuy - buy2load
137 # 光伏去向分配
138 pv_rem=pv-pv2load
139 pv2store=np.minimum(pv_rem, pch)
140 pv2waste=pv_rem-pv2store
141 from matplotlib.patches import Patch
142 from matplotlib.lines import Line2D
143 # 图1: 微网功率平衡
144 fig,(ax1,ax2)=plt.subplots(2, 1, figsize=(11.5, 7.5), sharex=True,
145     gridspec_kw={'height_ratios': [1, 1], 'hspace': 0.08})
146 ax1.stackplot(t_mid, pbuy, pv, pdis, labels=['购电', '光伏', '放电'], alpha=0.80, colors=[C_BUY,
    C_PV, C_DIS])
147 ax1.set_ylabel('供给功率 (kW)', fontsize=16)
148 ax1.set_xlim(0, 24)
149 ax1.set_ylim(0, 16000)
150 ax1.legend(loc='upper left', ncol=3, fontsize=14, frameon=False)
151 ax1.grid(True, alpha=0.3, axis='y')
152 ax1.tick_params(labelsize=13)
153 ax2.stackplot(t_mid, load, pch, pw,
154     labels=['负荷', '充电', '富余功率'], alpha=0.80,
155     colors=[C_LOAD, C_CH, C_W])
156 ax2.set_xlabel('时间 (h)', fontsize=16)
157 ax2.set_ylabel('需求功率 (kW)', fontsize=16)
158 ax2.set_xlim(0, 24)
159 ax2.set_xticks(range(0, 25, 2))
160 ax2.set_ylim(0, 16000)
161 ax2.legend(loc='upper left', ncol=3, fontsize=14, frameon=False)
162 ax2.grid(True, alpha=0.3, axis='y')
163 ax2.tick_params(labelsize=13)
164 plt.tight_layout()
165 plt.savefig(os.path.join(fig_dir, '第一问微网功率平衡.png'),dpi=300, bbox_inches='tight')
166 plt.close()
167 # 图2: 计划购电功率与电价分时对照
168 fig, ax1 = plt.subplots(figsize=(11.5, 5.2))
169 ax1.fill_between(t_mid, pbuy, step='mid', alpha=0.25, color=C_BUY)
170 ax1.step(t_mid, pbuy, where='mid', color=C_BUY, linewidth=2.2, label='购电功率')
171 ax1.set_xlabel('时间 (h)', fontsize=16)
172 ax1.set_ylabel('购电功率 (kW)', color=C_BUY, fontsize=16)
173 ax1.tick_params(axis='x', labelsize=13)
174 ax1.tick_params(axis='y', labelcolor=C_BUY, labelsize=13)
175 ax1.set_xlim(0, 24)
176 ax1.set_xticks(range(0, 25, 2))
177 ax1.grid(True, alpha=0.3)
178 ax2 = ax1.twinx()
179 ax2.spines['right'].set_visible(True)
180 ax2.step(t_mid, pri, where='mid', color=C_PRICE, linewidth=1.8, alpha=0.75, label='电价')
181 ax2.set_ylabel('电价 (元/kWh)', color=C_PRICE, fontsize=16)
182 ax2.tick_params(axis='y', labelcolor=C_PRICE, labelsize=13)
183 lines1, labels1 = ax1.get_legend_handles_labels()
184 lines2, labels2 = ax2.get_legend_handles_labels()
185 ax1.legend(lines1 + lines2, labels1 + labels2,

```

```

186     loc='upper center', fontsize=14, frameon=False, ncol=2,
187     bbox_to_anchor=(0.5, 1.02))
188 plt.subplots_adjust(left=0.09, right=0.91, top=0.92, bottom=0.16)
189 plt.savefig(os.path.join(fig_dir, '第一问计划购电功率与电价分时对照.png'),
190     dpi=300, bbox_inches='tight')
191 plt.close()
192 # 图3: 储能充放电
193 fig, ax1 = plt.subplots(figsize=(11.5, 5.8))
194 width = dt * 0.7
195 ax1.bar(t_mid, pch, width=width, color=C_CH, alpha=0.55, label='充电功率')
196 ax1.bar(t_mid, -pdiss, width=width, color=C_PV, alpha=0.55, label='放电功率')
197 ax1.axhline(0, color='black', linewidth=0.6)
198 ax1.set_xlabel('时间 (h)', fontsize=16)
199 ax1.set_ylabel('充/放电功率 (kW)', fontsize=16)
200 ax1.set_xlim(0, 24)
201 ax1.set_xticks(range(0, 25, 2))
202 ax1.set_ylim(-6500, 6500)
203 ax1.tick_params(labelsize=13)
204 ax1.grid(True, alpha=0.25, axis='y', linestyle='--')
205 ax2 = ax1.twinx()
206 ax2.spines['right'].set_visible(True)
207 ax2.spines['right'].set_linewidth(1.5)
208 ax2.axhline(Smax, color=C_MAX, linestyle='--', linewidth=1.0, alpha=0.45)
209 ax2.axhline(Smin, color=C_MIN, linestyle='--', linewidth=1.0, alpha=0.45)
210 ax2.axhline(Sben, color=C_SOC, linestyle=':', linewidth=0.8, alpha=0.35)
211 ax2.fill_between(t_soc, soc, Sben, where=soc >= Sben, alpha=0.08, color=C_DIS)
212 ax2.fill_between(t_soc, soc, Sben, where=soc < Sben, alpha=0.08, color=C_BUY)
213 ax2.plot(t_soc, soc, color=C_SOC, linewidth=2.8,
214     marker='o', markersize=2, label='储电量 $S_{t}$', zorder=10)
215 ax2.set_ylabel('储电量 (kWh)', fontsize=16, color=C_SOC)
216 ax2.tick_params(axis='y', labelcolor=C_SOC, labelsize=13)
217 ax2.set_ylim(0, 12000)
218 legend_elements = [Patch(facecolor=C_CH, alpha=0.55, label='充电功率'),
219     Patch(facecolor=C_PV, alpha=0.55, label='放电功率'),
220     Line2D([0], [0], color=C_SOC, linewidth=2.8, marker='o', markersize=3, label='储电量 $S_{t}$')
221     ]
222 ax1.legend(handles=legend_elements, loc='upper center', fontsize=14,
223     frameon=False, ncol=3, bbox_to_anchor=(0.5, 1.02))
224 plt.subplots_adjust(left=0.09, right=0.90, top=0.93, bottom=0.16)
225 plt.savefig(os.path.join(fig_dir, '第一问储能充放电.png'),
226     dpi=300, bbox_inches='tight')
227 plt.close()
228 # 图4: 计划购电的功率去向
229 fig, ax = plt.subplots(figsize=(11.5, 5.8))
230 ax.stackplot(t_mid, buy2load, buy2store,
231     labels=['购电 → 直接供给负荷', '购电 → 储能充电'],
232     alpha=0.75, colors=[C_BUY, C_STORE])
233 ax.set_xlabel('时间 (h)', fontsize=16)
234 ax.set_ylabel('购电功率 (kW)', fontsize=16)
235 ax.set_xlim(0, 24)
236 ax.set_xticks(range(0, 25, 2))
237 ax.set_ylim(0, 12000)
238 ax.tick_params(labelsize=13)
239 ax.grid(True, alpha=0.25, axis='y', linestyle='--')
240 legend_elements = [Patch(facecolor=C_BUY, alpha=0.75, label='购电 → 直接供给负荷'),
241     Patch(facecolor=C_STORE, alpha=0.75, label='购电 → 储能充电'),]
242 ax.legend(handles=legend_elements, loc='upper center', fontsize=14,
243     frameon=False, ncol=2, bbox_to_anchor=(0.5, 1.02))
244 plt.subplots_adjust(left=0.09, right=0.97, top=0.93, bottom=0.16)
245 plt.savefig(os.path.join(fig_dir, '第一问计划购电的功率去向.png'),
246     dpi=300, bbox_inches='tight')
247 plt.close()
248 # 图5: 光伏发电的功率去向
249 fig, ax=plt.subplots(figsize=(11.5, 5.8))
250 ax.stackplot(t_mid, pv2load, pv2store, pv2waste,
251     labels=['光伏 → 直接供给负荷', '光伏 → 储能充电', '富余功率'],
252     alpha=0.75, colors=[C_PV, C_STORE, C_W])
253 ax.set_xlabel('时间 (h)', fontsize=16)
254 ax.set_ylabel('光伏功率 (kW)', fontsize=16)
255 ax.set_xlim(0, 24)
256 ax.set_xticks(range(0, 25, 2))
257 ax.set_ylim(0, 9000)
258 ax.tick_params(labelsize=13)
259 ax.grid(True, alpha=0.25, axis='y', linestyle='--')
260 legend_elements = [Patch(facecolor=C_PV, alpha=0.75, label='光伏 → 直接供给负荷'),
261     Patch(facecolor=C_STORE, alpha=0.75, label='光伏 → 储能充电'),
262     Patch(facecolor=C_W, alpha=0.75, label='富余功率'),]
263 ax.legend(handles=legend_elements, loc='upper center', fontsize=14,
264     frameon=False, ncol=3, bbox_to_anchor=(0.5, 1.02))
265 plt.subplots_adjust(left=0.09, right=0.97, top=0.93, bottom=0.16)
266 plt.savefig(os.path.join(fig_dir, '第一问光伏发电的功率去向.png'),
267     dpi=300, bbox_inches='tight')
268 plt.close()

```

268 print(f'\n问题一 图片已保存到: {fig_dir}')

code/Q1.py

Q2.py

```
1 from __future__ import annotations
2 import logging
3 from dataclasses import dataclass
4 from pathlib import Path
5 from datetime import datetime, timedelta, date
6 from typing import Tuple, Dict
7 import numpy as np
8 from scipy.optimize import linprog
9 from scipy import sparse
10 from statsmodels.tsa.ar_model import AutoReg
11 try:
12     from statsmodels.tsa.seasonal import MSTL
13     HAS_MSTL = True
14 except Exception:
15     HAS_MSTL = False
16 try:
17     from openpyxl import load_workbook
18 except ImportError as e:
19     raise ImportError(
20         "缺少 openpyxl。请在当前 Python 环境执行: pip install openpyxl"
21     ) from e
22 # 绘图
23 import matplotlib
24 matplotlib.use("Agg")
25 import matplotlib.pyplot as plt
26 import matplotlib.dates as mdates
27 plt.rcParams.update({
28     "font.family": "sans-serif",
29     "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial"],
30     "axes.unicode_minus": False, "font.size": 12, "axes.spines.top": False,
31     "axes.linewidth": 1.5, "xtick.major.width": 1.5, "ytick.major.width": 1.5,
32     "xtick.major.size": 6, "ytick.major.size": 6,
33 })
34 # 设置参数
35 class Config:
36     b_dir = str(Path(__file__).resolve().parent)
37     att1 = str(Path(b_dir) / "附件1.xlsx")
38     att2 = str(Path(b_dir) / "附件2.xlsx")
39     _tpl_candidates = [
40         Path(b_dir) / "附件5" / "result2.xlsx",
41         Path(b_dir) / "result2.xlsx",
42     ]
43     template = str(next(p for p in _tpl_candidates if p.exists()))
44     out_dir = str(Path(b_dir) / "Q2结果")
45     out_xlsx = str(Path(out_dir) / "result2_filled.xlsx")
46     fig_dir = str(Path(out_dir) / "第二问图片输出")
47     # 时间离散与储能
48     slots_per_day = 144; dt = 1.0 / 6.0; horizon_slots = 144
49     forecast_slots = 288; eta_c = eta_d = 0.90
50     e_min, e_max = 1200.0, 10800.0
51     p_max, e0 = 5000.0, 6000.0
52     # 预测
53     l_days = 42; pv_days = 60; l_min_days = 21
54     mstl_days = 7 # MSTL 每7天完整重估, 中间仅轻量更新
55     ar_lags_candi = (1, 2, 3, 6, 12, 18, 36)
56     pv_candi, kappa_candi = (1,2,3,6,12), (1,2,3,5,7)
57     clear_quantile, pv_halfwin = 0.93, 2
58     # 动态风险权重
59     rk_hisdays = 28; rk_decay = 0.90; rk_minsamp = 5
60     alpha_cold, alpha_min, alpha_max = 4.0, 1.0, 8.0 # 上界防长期顶格
61     beta_fixed, c_eps = 1.0, 1e-4
62     tau_min, tau_max = 0.55, 0.88 # 上界避免过度保守
63     reserve_scale = 0.75 # 兼顾紧急购电与富余电量
64     # 优化与运行控制
65     eps_cyc, eps_sur, lp_met = 1e-5, 1e-8, "highs"
66     max_days = 365 # 测试可改40, 正式用365
67 CFG = Config()
68 def setup_logger(outdir: Path) -> logging.Logger:
69     outdir.mkdir(parents=True, exist_ok=True)
70     logger = logging.getLogger("Q2")
71     logger.setLevel(logging.INFO)
72     logger.handlers.clear()
73     return logger
74 def excel_serial_to_date(x) -> datetime:
75     return datetime(1899, 12, 30) + timedelta(days=float(x))
```

```

76 def cell_to_date(x):
77     if x is None:
78         return None
79     if isinstance(x, datetime):
80         return x.date()
81     try:
82         from datetime import date as _date
83         if isinstance(x, _date):
84             return x
85     except Exception:
86         pass
87     if isinstance(x, (int, float, np.integer, np.floating)):
88         return excel_serial_to_date(x).date()
89     if isinstance(x, str):
90         txt=x.strip()
91         for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%Y.%m.%d"):
92             try:
93                 return datetime.strptime(txt, fmt).date()
94             except ValueError:
95                 pass
96     raise ValueError(f"无法识别日期单元格: {x!r}")
97 def safe_float_array(x) -> np.ndarray:
98     return np.asarray([[float(v) if v is not None else np.nan for v in row] for row in x], dtype
    =float)
99 def weighted_quantile(values: np.ndarray, weights: np.ndarray, q: float) -> float:
100     values = np.asarray(values, dtype=float)
101     weights = np.asarray(weights, dtype=float)
102     m = np.isfinite(values) & np.isfinite(weights) & (weights > 0)
103     if m.sum() == 0:
104         return 0.0
105     v, w = values[m], weights[m]
106     idx = np.argsort(v)
107     v, w = v[idx], w[idx]
108     cw = np.cumsum(w) / np.sum(w)
109     return float(np.interp(q, cw, v))
110 def moving_average_circular(x: np.ndarray, half: int) -> np.ndarray:
111     if half <= 0:
112         return x.copy()
113     n = len(x)
114     out = np.zeros(n)
115     for i in range(n):
116         lo = max(0, i-half)
117         hi = min(n, i+half+1)
118         out[i] = np.mean(x[lo:hi])
119     return out
120 def choose_ar_forecast(series: np.ndarray, horizon: int, candidates: Tuple[int, ...], fallback:
    float = 0.0) -> np.ndarray:
121     y = np.asarray(series, dtype=float)
122     y = y[np.isfinite(y)]
123     if len(y) < 8 or np.std(y) < 1e-8:
124         return np.full(horizon, float(np.mean(y) if len(y) else fallback))
125     best = None
126     for p in candidates:
127         if len(y) <= p + 5:
128             continue
129         try:
130             model = AutoReg(y, lags=p, trend="ct", old_names=False).fit()
131             if np.isfinite(model.aic) and (best is None or model.aic < best[0]):
132                 best = (model.aic, model)
133         except Exception:
134             continue
135     if best is None:
136         return np.full(horizon, float(np.mean(y[-min(7, len(y)):])))
137     try:
138         pred = np.asarray(best[1].predict(start=len(y), end=len(y)+horizon-1, dynamic=False),
    dtype=float)
139         if np.any(~np.isfinite(pred)):
140             raise ValueError("AR forecast nonfinite")
141         return pred
142     except Exception:
143         return np.full(horizon, float(np.mean(y[-min(7, len(y)):])))
144 #读取数据
145 def load_inputs(cfg: Config, logger):
146     for p in (cfg.att1, cfg.att2, cfg.template):
147         if not Path(p).exists():
148             raise FileNotFoundError(f"未找到文件: {p}")
149     wb1 = load_workbook(cfg.att1, data_only=True, read_only=True)
150     s1 = wb1["Sheet1"]
151     rows1 = list(s1.iter_rows(min_row=2, max_row=145, min_col=2, max_col=4, values_only=True))
152     price = np.array([float(r[0]) for r in rows1], dtype=float)
153     load_prior = np.array([float(r[1]) for r in rows1], dtype=float)
154     pv_prior = np.array([float(r[2]) for r in rows1], dtype=float)
155     wb1.close()
156     wb2 = load_workbook(cfg.att2, data_only=True, read_only=True)

```

```

157 s1 = wb2["小区负载"]
158 sp = wb2["光伏发电实际功率"]
159 load_all = list(s1.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=
160 True))
161 pv_all = list(sp.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=True)
162 )
163 wb2.close()
164 dates = [cell_to_date(r[0]) for r in load_all]
165 load_rows = [r[1:] for r in load_all]
166 pv_rows = [r[1:] for r in pv_all]
167 load = safe_float_array(load_rows)
168 pv_actual = safe_float_array(pv_rows)
169 assert load.shape == (365, 144), load.shape
170 assert pv_actual.shape == (365, 144), pv_actual.shape
171 assert len(price) == 144
172 return dates, price, load_prior, pv_prior, load, pv_actual
173 _LOAD_MSTL_CACHE = {}
174 def load_forecast(day_idx: int, load: np.ndarray, prior: np.ndarray, cfg: Config, logger=None)
175 -> np.ndarray:
176     H = cfg.forecast_slots
177     if day_idx == 0:
178         return np.tile(prior, 2)
179     hist_days = min(day_idx, cfg.l_days)
180     hist_mat = load[day_idx-hist_days:day_idx]
181     # 冷启动: 同一时刻近周日加权, 附件1先验
182     if hist_days < cfg.l_min_days or not HAS_MSTL:
183         k = hist_mat.shape[0]
184         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
185         weights /= weights.sum()
186         shape = np.sum(hist_mat * weights[:, None], axis=0)
187         if hist_days >= 7:
188             shape = 0.55*shape + 0.45*load[day_idx-7]
189             w_prior = max(0.0, 1.0 - hist_days / cfg.l_min_days)
190             shape = (1-w_prior)*shape + w_prior*prior
191         return np.tile(shape, 2)
192     need_refit = (
193         not _LOAD_MSTL_CACHE
194         or day_idx - _LOAD_MSTL_CACHE.get("refit_day", -10**9) >= cfg.mstl_days
195     )
196     if need_refit:
197         y = hist_mat.reshape(-1)
198         try:
199             mstl = MSTL(y, periods=(144, 1008), stl_kwargs={"robust": False}).fit()
200             trend = np.asarray(mstl.trend)
201             seasonal = np.asarray(mstl.seasonal)
202             if seasonal.ndim == 1:
203                 s_daily = seasonal; s_weekly = np.zeros_like(seasonal)
204             else:
205                 s_daily = seasonal[:, 0]; s_weekly = seasonal[:, 1]
206             resid = np.asarray(mstl.resid)
207             daily_tail = s_daily[-min(len(s_daily), 28*144):]
208             weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
209             daily_pattern = np.array([np.nanmean(daily_tail[i::144]) for i in range(144)])
210             weekly_pattern = np.array(
211                 [np.nanmean(weekly_tail[i::1008]) for i in range(1008)])
212             _LOAD_MSTL_CACHE.clear()
213             _LOAD_MSTL_CACHE.update({
214                 "refit_day": day_idx,
215                 "origin_day": day_idx - hist_days,
216                 "daily_pattern": daily_pattern,
217                 "weekly_pattern": weekly_pattern,
218                 "trend_last": float(trend[-1]),
219                 "trend_slope": float(np.clip(
220                     np.polyfit(
221                         np.arange(min(432, len(trend))), dtype=float,
222                         trend[-min(432, len(trend)):],
223                         1
224                     )[0], -5.0, 5.0
225                 )),
226                 "resid_tail": resid[-min(len(resid), 14*144):].copy(),
227             })
228     except Exception as e:
229         if logger:
230             logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
231         k = hist_mat.shape[0]
232         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
233         weights /= weights.sum()
234         shape = np.sum(hist_mat * weights[:, None], axis=0)
235         if hist_days >= 7:
236             shape = 0.5*shape + 0.5*load[day_idx-7]
237         return np.tile(shape, 2)
238 # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
239 daily_pattern = _LOAD_MSTL_CACHE["daily_pattern"]
240 weekly_pattern = _LOAD_MSTL_CACHE["weekly_pattern"]

```

```

238 origin_day = _LOAD_MSTL_CACHE["origin_day"]
239 recent_days = min(hist_days, 3)
240 recent = load[day_idx-recent_days:day_idx].reshape(-1)
241 global_start = (day_idx - recent_days) * 144
242 gidx = global_start + np.arange(len(recent))
243 sd_hist = daily_pattern[gidx % 144]
244 sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
245 deseason = recent - sd_hist - sw_hist
246 m = min(432, len(deseason))
247 x = np.arange(m, dtype=float)
248 coef = np.polyfit(x, deseason[-m:], 1)
249 slope = float(np.clip(coef[0], -5.0, 5.0))
250 level = float(deseason[-1])
251 trend_future = level + slope*np.arange(1, H+1)
252 future_gidx = day_idx*144 + np.arange(H)
253 sd_future = daily_pattern[future_gidx % 144]
254 sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
255 # 残差AR仍每天轻量更新, 不需要重新做MSTL
256 hist_axis = np.arange(len(deseason), dtype=float)
257 fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
258 resid_recent = deseason - fitted_trend_hist
259 ar_future = choose_ar_forecast(resid_recent[-min(len(resid_recent), 3*144):],
260                               H, cfg.ar_lags_candi, 0.0,)
261 pred = trend_future + sd_future + sw_future + ar_future
262 return np.maximum(pred, 0.0)
263 # 光伏: 晴空包络×日透射率+残差AR
264 def build_clear_envelope(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config) -> np.
    ndarray:
265     if day_idx == 0:
266         base = prior.copy()
267         return np.maximum(base, 0.0)
268     w = min(day_idx, cfg.pv_days)
269     hist = pv[day_idx-w:day_idx]
270     q = np.quantile(hist, cfg.clear_quantile, axis=0)
271     q = moving_average_circular(q, cfg.pv_halfwin)
272     prior_w = max(0.0, 1.0 - w/21.0)
273     base = (1-prior_w)*q + prior_w*prior
274     zero_ratio = np.mean(hist <= 1e-8, axis=0)
275     base[zero_ratio > 0.90] = 0.0
276     return np.maximum(base, 0.0)
277 def pv_forecast(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config, logger=None) ->
    Tuple[np.ndarray, Dict]:
278     H = cfg.forecast_slots
279     if day_idx == 0:
280         return np.tile(prior, 2), {"kappa_d": 1.0, "kappa_next": 1.0, "envelope": prior.copy()}
281     B_d = build_clear_envelope(day_idx, pv, prior, cfg)
282     # 下一日包络: 年周期变化慢, 先用当前包络; 用近期日能量变化率做轻微尺度外推
283     recent_energy = np.sum(pv[max(0, day_idx-14):day_idx], axis=1)*cfg.dt
284     scale_next = 1.0
285     if len(recent_energy) >= 7:
286         x = np.arange(len(recent_energy))
287         slope = np.polyfit(x, recent_energy, 1)[0]
288         denom = max(np.mean(recent_energy), 1.0)
289         scale_next = float(np.clip(1.0 + slope/denom, 0.95, 1.05))
290     B_next = B_d * scale_next
291     # 用当前滚动包络近似反演近期日透射率; 只使用过去日期
292     w = min(day_idx, cfg.pv_days)
293     hist = pv[day_idx-w:day_idx]
294     denom = max(np.sum(B_d)*cfg.dt, 1e-6)
295     kappas = np.clip(np.sum(hist, axis=1)*cfg.dt/denom, 0.0, 1.0)
296     kpred = choose_ar_forecast(kappas, 2, cfg.kappa_candi, fallback=float(np.mean(kappas)))
297     kpred = np.clip(kpred, 0.0, 1.0)
298     base_d = B_d*kpred[0]
299     base_next = B_next*kpred[1]
300     # 残差构造: 每个历史日用其日能量对应kappa相对于当前包络重构
301     residual_days = hist - kappas[:, None]*B_d[None, :]
302     resid_series = residual_days.reshape(-1)
303     rpred = choose_ar_forecast(resid_series[-min(len(resid_series), 21*144):], H, cfg.pv_candi,
304                               0.0)
305     pred = np.concatenate([base_d, base_next]) + rpred
306     pred = np.maximum(pred, 0.0)
307     # 夜间稳定无发电时段置零
308     zero_mask = B_d <= 1e-8
309     pred[zero_mask[np.arange(H) % 144]] = 0.0
310     return pred, {"kappa_d": float(kpred[0]), "kappa_next": float(kpred[1]), "envelope": B_d}
311 # 动态alpha/beta与风险备用量
312 def compute_historical_marginal_cost_day(j: int,
313                                         available_day_exclusive: int,
314                                         net_errors: np.ndarray,
315                                         price: np.ndarray,
316                                         soc_hist: np.ndarray,
317                                         planned_hist: np.ndarray,
318                                         load: np.ndarray,

```

```

318                                     pv: np.ndarray,
319                                     cfg: Config) -> Tuple[np.ndarray, np.ndarray]:
320     nT = cfg.slots_per_day
321     cplus = np.full(nT, np.nan, dtype=float)
322     cminus = np.full(nT, np.nan, dtype=float)
323     if j < 0 or j >= available_day_exclusive:
324         return cplus, cminus
325     n_avail = available_day_exclusive * nT
326     plan_f = planned_hist[:available_day_exclusive].reshape(-1)
327     load_f = load[:available_day_exclusive].reshape(-1)
328     pv_f = pv[:available_day_exclusive].reshape(-1)
329     soc_f = soc_hist[:available_day_exclusive].reshape(-1)
330     err_f = net_errors[:available_day_exclusive].reshape(-1)
331     slot_price = np.tile(price, available_day_exclusive)
332     base = j * nT
333     eff_rt = cfg.eta_c * cfg.eta_d
334     for t in range(nT):
335         e = net_errors[j, t]
336         if not np.isfinite(e):
337             continue
338         lam_t = price[t]
339         s0 = soc_hist[j, t] if np.isfinite(soc_hist[j, t]) else cfg.e0
340         g0 = base + t
341         g1 = g0 + 1
342         g2 = min(g0 + cfg.horizon_slots, n_avail - 1)
343         if e > 0:
344             c_em = 5.0 * lam_t
345             c_bat = np.inf
346             if s0 > cfg.e_min + 1e-6 and g1 <= g2:
347                 s1 = slice(g1, g2 + 1)
348                 margin = plan_f[s1] + pv_f[s1] - load_f[s1]
349                 sf = soc_f[s1]
350                 pf = slot_price[s1]
351                 soc_ok = (~np.isfinite(sf)) | (sf < cfg.e_max - 1e-6)
352                 feasible = soc_ok & ((margin > 0.0) | (pf < lam_t))
353                 if np.any(feasible):
354                     c_bat = float(np.min(pf[feasible]) / eff_rt)
355             cplus[t] = min(c_em, c_bat) if np.isfinite(c_bat) else c_em
356         elif e < 0:
357             recovery = 0.0
358             if s0 < cfg.e_max - 1e-6 and g1 <= g2:
359                 s1 = slice(g1, g2 + 1)
360                 ef = err_f[s1]
361                 pf = slot_price[s1]
362                 m = np.isfinite(ef) & (ef > 0.0)
363                 if np.any(m):
364                     recovery = float(np.max(5.0 * pf[m] * eff_rt))
365             cminus[t] = max(0.0, lam_t - recovery)
366     return cplus, cminus
367 def update_risk_cost_cache(day_idx: int,
368                             cplus_cache: np.ndarray,
369                             cminus_cache: np.ndarray,
370                             net_errors: np.ndarray,
371                             price: np.ndarray,
372                             soc_hist: np.ndarray,
373                             planned_hist: np.ndarray,
374                             load: np.ndarray,
375                             pv: np.ndarray,
376                             cfg: Config) -> None:
377     if day_idx <= 0:
378         return
379     j = day_idx - 1
380     cp, cm = compute_historical_marginal_cost_day(
381         j, day_idx, net_errors, price, soc_hist, planned_hist, load, pv, cfg)
382     cplus_cache[j] = cp
383     cminus_cache[j] = cm
384     if day_idx >= 2:
385         j = day_idx - 2
386         cp, cm = compute_historical_marginal_cost_day(
387             j, day_idx, net_errors, price, soc_hist, planned_hist, load, pv, cfg)
388         cplus_cache[j] = cp
389         cminus_cache[j] = cm
390 def dynamic_risk_weights(day_idx: int,
391                             net_errors: np.ndarray,
392                             cplus_cache: np.ndarray,
393                             cminus_cache: np.ndarray,
394                             cfg: Config) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
395     nT = cfg.slots_per_day
396     alpha = np.full(nT, cfg.alpha_cold, dtype=float)
397     beta = np.full(nT, cfg.beta_fixed, dtype=float)
398     reserve = np.zeros(nT, dtype=float)
399     if day_idx <= 0:
400         return alpha, beta, reserve
401     j0 = max(0, day_idx - cfg.rk_hisdays)

```

```

402 hist_days = np.arange(j0, day_idx)
403 if hist_days.size == 0:
404     return alpha, beta, reserve
405 ages = day_idx - hist_days
406 base_w = cfg.rk_decay ** (ages - 1)
407 errs = net_errors[hist_days] # (HIST, 144)
408 cp_all = cplus_cache[hist_days]
409 cm_all = cminus_cache[hist_days]
410 for t in range(nT):
411     et = errs[:, t]
412     # 正向误差历史
413     mp = (et > 0) & np.isfinite(et) & np.isfinite(cp_all[:, t])
414     # 负向误差历史
415     mn = (et < 0) & np.isfinite(et) & np.isfinite(cm_all[:, t])
416     if np.count_nonzero(mp) >= cfg.rk_minsamp and np.count_nonzero(mn) >= cfg.rk_minsamp:
417         cp = float(np.average(cp_all[mp, t], weights=base_w[mp]))
418         cm = float(np.average(cm_all[mn, t], weights=base_w[mn]))
419         raw = cp / max(cfg.c_eps)
420         alpha[t] = min(cfg.alpha_max, max(cfg.alpha_min, raw))
421         tau = alpha[t] / (alpha[t] + beta[t])
422         tau = min(cfg.tau_max, max(cfg.tau_min, tau))
423         if np.count_nonzero(mp) >= cfg.rk_minsamp:
424             reserve[t] = cfg.reserve_scale * weighted_quantile(et[mp], base_w[mp], tau)
425     return alpha, beta, reserve
426 # 7. LP构造
427 @dataclass
428 class LPTemplates:
429     da_Aeq: sparse.csr_matrix
430     da_c: np.ndarray
431     da_bounds: list
432     mu_terminal: float
433 def build_lp_templates(price: np.ndarray, cfg: Config) -> LPTemplates:
434     n = cfg.slots_per_day
435     N = 5 * n
436     ib, ic, idis, isu, ie = 0, n, 2*n, 3*n, 4*n
437     rows, cols, data = [], [], []
438     r = 0
439     # 功率平衡: buy + dis - ch - sur = 风险校准后的预测净负荷
440     for t in range(n):
441         for col, val in ((ib+t, 1.0), (idis+t, 1.0), (ic+t, -1.0), (isu+t, -1.0)):
442             rows.append(r); cols.append(col); data.append(val)
443         r += 1
444     # SOC递推
445     for t in range(n):
446         rows.append(r); cols.append(ie+t); data.append(1.0)
447         if t > 0:
448             rows.append(r); cols.append(ie+t-1); data.append(-1.0)
449             rows.append(r); cols.append(ic+t); data.append(-cfg.eta_c*cfg.dt)
450             rows.append(r); cols.append(idis+t); data.append(cfg.dt/cfg.eta_d)
451         r += 1
452     Aeq = sparse.csr_matrix((data, (rows, cols)), shape=(r, N))
453     mu = cfg.eta_d * float(np.min(price))
454     c = np.zeros(N)
455     c[ib:ib+n] = price * cfg.dt
456     c[ic:ic+n] = cfg.eps_cyc * cfg.dt
457     c[idis:idis+n] = cfg.eps_cyc * cfg.dt
458     c[isu:isu+n] = cfg.eps_sur * cfg.dt
459     c[ie+n-1] = -mu
460     bounds = ((0, None))*n + [(0, cfg.p_max))*n + [(0, cfg.p_max))*n +
461               [(0, None))*n + [(cfg.e_min, cfg.e_max))*n
462     return LPTemplates(Aeq, c, bounds, mu)
463 def solve_day_ahead(price: np.ndarray,
464                     load_fc: np.ndarray,
465                     pv_fc: np.ndarray,
466                     reserve: np.ndarray,
467                     soc0: float,
468                     cfg: Config,
469                     lp: LPTemplates) -> Dict[str, np.ndarray]:
470     """每天0:00求解一次全天计划购电及计划储能LP."""
471     n = cfg.slots_per_day
472     net_req = load_fc[:n] - pv_fc[:n] + reserve
473     b = np.empty(2*n, dtype=float)
474     b[:n] = net_req
475     b[n:] = 0.0
476     b[n] = soc0
477     res = linprog(lp.da_c, A_eq=lp.da_Aeq, b_eq=b, bounds=lp.da_bounds, method=cfg.lp_met)
478     if not res.success:
479         raise RuntimeError(f"日前LP失败: {res.message}")
480     x = res.x
481     ib, ic, idis, isu, ie = 0, n, 2*n, 3*n, 4*n
482     paper_obj = float(res.fun + lp.mu_terminal * soc0)
483     return {"buy": x[ib:ib+n], "ch": x[ic:ic+n], "dis": x[idis:idis+n],
484            "sur": x[isu:isu+n], "E": x[ie:ie+n], "obj": paper_obj}

```

```

485 def operate_one_slot(soc0: float, buy: float, load_actual: float, pv_actual: float,
486                     plan_ch: float, plan_dis: float, cfg: Config) -> Dict[str, float]:
487     dt = cfg.dt
488     # 先把计划动作投影到当前SOC可行范围
489     ch = max(0.0, min(float(plan_ch), cfg.p_max))
490     dis = max(0.0, min(float(plan_dis), cfg.p_max))
491     max_ch_soc = max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt))
492     max_dis_soc = max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt)
493     ch = min(ch, max_ch_soc)
494     dis = min(dis, max_dis_soc)
495     # 基于真实负荷/光伏检查功率平衡
496     gap = float(load_actual - (buy + pv_actual + dis - ch))
497     emg = 0.0
498     sur = 0.0
499     if gap > 1e-10:
500         # 1. 先取消充电, 相当于立即释放可用供电
501         cut_ch = min(ch, gap)
502         ch -= cut_ch
503         gap -= cut_ch
504         # 2. 再增加放电; 紧急购电价格为5倍, 故在可行范围内优先用储能
505         max_dis_total = min(cfg.p_max, max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt))
506         extra_dis = min(gap, max(0.0, max_dis_total - dis))
507         dis += extra_dis
508         gap -= extra_dis
509         # 3. 储能仍不足时才紧急购电
510         emg = max(0.0, gap)
511     elif gap < -1e-10:
512         excess = -gap
513         # 1. 先取消不必要的计划放电
514         cut_dis = min(dis, excess)
515         dis -= cut_dis
516         excess -= cut_dis
517         # 2. 再增加充电
518         max_ch_total = min(cfg.p_max, max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt)))
519         extra_ch = min(excess, max(0.0, max_ch_total - ch))
520         ch += extra_ch
521         excess -= extra_ch
522         # 3. 电池无法继续吸收的部分作为富余功率
523         sur = max(0.0, excess)
524     soc1 = soc0 + (cfg.eta_c * ch - dis / cfg.eta_d) * dt
525     soc1 = min(cfg.e_max, max(cfg.e_min, soc1))
526     return {"ch": ch, "dis": dis, "emg": emg, "sur": sur, "E": soc1}
527 # 结果
528 def fmt_slot(k):
529     mins = k * 10
530     return "24:00" if mins == 1440 else f"{mins//60}:{mins%60:02d}"
531 def fill_result2(template_path: str, out_path: str, dates, price,
532                 plan_buy, ch, dis, soc_start, soc_end, emg, cfg: Config, logger):
533     from copy import copy
534     wb = load_workbook(template_path)
535     sp = wb["计划购电量"]
536     sc = wb["充放电量"]
537     se = wb["紧急购电量"]
538     date_to_idx = {d: i for i, d in enumerate(dates)}
539     first_submit = date(2025, 2, 1)
540     computed_dates = [d for d in dates if d >= first_submit and d in date_to_idx
541                      and np.isfinite(plan_buy[date_to_idx[d]]).all()]
542     # 计划购电量
543     for r in range(2, sp.max_row + 1):
544         raw_date = sp.cell(r, 1).value
545         if raw_date is None:
546             continue
547         try:
548             d = cell_to_date(raw_date)
549         except Exception:
550             continue
551         if d not in date_to_idx:
552             continue
553         i = date_to_idx[d]
554         if not np.isfinite(plan_buy[i]).all():
555             continue
556         vals = (plan_buy[i] * cfg.dt).tolist() # kWh/10min
557         for c, v in enumerate(vals, start=2):
558             sp.cell(r, c).value = None if not np.isfinite(v) else float(v)
559             sp.cell(r, 146).value = float(np.nansum(plan_buy[i]) * cfg.dt) # EP 全天购电量
560             sp.cell(r, 147).value = float(np.nansum(plan_buy[i] * price) * cfg.dt) # EQ 全天购电费
561     # 保存原模板样式作为“样板行”
562     charge_proto = [{"height": sc.row_dimensions[rr].height,
563                     "cells": [sc.cell(rr, cc) for cc in range(1, 7)] for rr in range(2, 8)}]
564     emg_proto = [{"height": se.row_dimensions[rr].height,
565                  "cells": [se.cell(rr, cc) for cc in range(1, 4)] for rr in range(2, 5)}]
566     def copy_cell_style(src, dst):
567         if src.has_style:

```

```

568         dst._style = copy(src._style)
569     if src.number_format:
570         dst.number_format = src.number_format
571     dst.font = copy(src.font)
572     dst.fill = copy(src.fill)
573     dst.border = copy(src.border)
574     dst.alignment = copy(src.alignment)
575     dst.protection = copy(src.protection)
576 # 充放电量
577 if sc.max_row >= 2:
578     sc.delete_rows(2, sc.max_row-1)
579 intervals=[(0,4,"0:00-4:00"),(4,8,"4:00-8:00"),(8,12,"8:00-12:00"),
580            (12,16,"12:00-16:00"),(16,20,"16:00-20:00"),(20,24,"20:00-24:00")]
581 r=2
582 for d in computed_dates:
583     i=date_to_idx[d]
584     for k,(h0,h1,label) in enumerate(intervals):
585         proto=charge_proto[k]
586         sc.row_dimensions[r].height = proto["height"]
587         for cc in range(1,7):
588             copy_cell_style(proto["cells"][cc-1], sc.cell(r,cc))
589         sc.cell(r,1).value = d if k==0 else None
590         sc.cell(r,2).value = label
591         s0=h0*6; s1=h1*6
592         sc.cell(r,3).value=float(np.nansum(ch[i,s0:s1])*cfg.dt)
593         sc.cell(r,4).value=float(np.nansum(dis[i,s0:s1])*cfg.dt)
594         sc.cell(r,5).value = ("0:00" if k==0 else "24:00" if k==1 else None)
595         sc.cell(r,6).value = (float(soc_start[i]) if k==0 else
596                               float(soc_end[i]) if k==1 else None)
597     r += 1
598 # 紧急购电
599 if se.max_row >= 2:
600     se.delete_rows(2, se.max_row-1)
601 r=2
602 for d in computed_dates:
603     i=date_to_idx[d]
604     events=[]; t=0
605     while t<144:
606         if emg[i,t] > 1e-6:
607             st=t; energy=0.0
608             while t<144 and emg[i,t] > 1e-6:
609                 energy += emg[i,t]*cfg.dt
610                 t += 1
611             events.append((f"{{fmt_slot(st)}}-{{fmt_slot(t)}}", float(energy)))
612         else:
613             t += 1
614     if not events:
615         events=[(None,None)]
616     for k,(period,energy) in enumerate(events):
617         proto=emg_proto[min(k, len(emg_proto)-1)]
618         se.row_dimensions[r].height = proto["height"]
619         for cc in range(1,4):
620             copy_cell_style(proto["cells"][cc-1], se.cell(r,cc))
621         se.cell(r,1).value = d if k==0 else None
622         se.cell(r,2).value = period
623         se.cell(r,3).value = energy
624     r += 1
625 # 日期与数值格式统一
626 sc.column_dimensions['A'].width = max(sc.column_dimensions['A'].width or 10, 12)
627 sc.column_dimensions['B'].width = max(sc.column_dimensions['B'].width or 12, 15)
628 se.column_dimensions['A'].width = max(se.column_dimensions['A'].width or 10, 12)
629 se.column_dimensions['B'].width = max(se.column_dimensions['B'].width or 12, 18)
630 for ws in (sc, se):
631     for rr in range(2, ws.max_row+1):
632         if ws.cell(rr,1).value is not None:
633             ws.cell(rr,1).number_format='yyyy/m/d'
634         for cc in ((3,4,6) if ws is sc else (3,)):
635             for rr in range(2, ws.max_row+1):
636                 ws.cell(rr,cc).number_format='0.00'
637 Path(out_path).parent.mkdir(parents=True, exist_ok=True)
638 wb.save(out_path)
639 wb.close()
640 def write_specified_date_results(dates, price, plan_buy, ch, dis, emg,
641                                soc_start, soc_end, cfg:Config, logger):
642     targets=[date(2025,3,20), date(2025,6,21), date(2025,9,23), date(2025,12,21)]
643     date_to_idx={d:i for i,d in enumerate(dates)}
644     out_dir=Path(cfg.out_dir)
645     out_dir.mkdir(parents=True, exist_ok=True)
646     slot_hours=[10,12,14,16,18,20]
647     intvs=[(0,4,'0:00-4:00'),(4,8,'4:00-8:00'),(8,12,'8:00-12:00'),
648            (12,16,'12:00-16:00'),(16,20,'16:00-20:00'),(20,24,'20:00-24:00')]
649     rows1=[]; rows2=[]; rows3=[]
650     for d in targets:
651         # 一次性计算, 供Excel复用
        if d not in date_to_idx: continue

```

```

652     i=date_to_idx[d]
653     if not np.isfinite(plan_buy[i]).all(): continue
654     vals=[float(plan_buy[i,h*6]*cfg.dt) for h in slot_hours]
655     totE=float(np.nansum(plan_buy[i])*cfg.dt)
656     totC=float(np.nansum(plan_buy[i]*price)*cfg.dt)
657     rows1.append([str(d),*vals,totE,totC])
658     for k,(h0,h1,label) in enumerate(intvs):
659         s0,s1=h0*6,h1*6
660         rows2.append([str(d) if k==0 else '', label,
661                       float(np.nansum(ch[i,s0:s1])*cfg.dt),
662                       float(np.nansum(dis[i,s0:s1])*cfg.dt),
663                       float(soc_start[i]) if k==0 else '',
664                       float(soc_end[i]) if k==0 else ''])
665     events=[]; t=0
666     while t<144:
667         if emg[i,t] > 1e-6:
668             st=t; energy=0.0
669             while t<144 and emg[i,t] > 1e-6:
670                 energy += emg[i,t]*cfg.dt; t += 1
671             events.append((f'{fmt_slot(st)}-{fmt_slot(t)}', float(energy)))
672         else:
673             t += 1
674     for k,(period,energy) in enumerate(events if events else [('无紧急购电',0.0))):
675         rows3.append([str(d) if k==0 else '', period, energy])
676 # 表1/表2/表3 Excel
677 from openpyxl import Workbook
678 wb_out = Workbook()
679 sheets = [
680     ('表1计划购电',
681      ['日期','10:00-10:10/kWh','12:00-12:10/kWh','14:00-14:10/kWh',
682       '16:00-16:10/kWh','18:00-18:10/kWh','20:00-20:10/kWh',
683       '全天购电量/kWh','全天购电费/元'], rows1),
684     ('表2充放电',
685      ['日期','时间段','充电量/kWh','放电量/kWh','0:00储电量/kWh',
686       '24:00储电量/kWh'], rows2),
687     ('表3紧急购电', ['日期','紧急购电时间段','紧急购电量/kWh'], rows3),
688 ]
689 for k,(title,hdr,rows) in enumerate(sheets):
690     ws = wb_out.active if k==0 else wb_out.create_sheet(title)
691     ws.title = title
692     ws.append(hdr)
693     for row in rows:
694         ws.append([round(v,4) if isinstance(v,(int,float)) else v for v in row])
695     px = out_dir/'Q2_指定日期表格数据.xlsx'
696     wb_out.save(px)
697 # 可视化与结果摘要
698 C_BUY, C_EM, C_LOAD = '#2E74B5', '#E74C3C', '#34495E' # 蓝/红/深灰蓝
699 C_PV, C_CH, C_DIS = '#27AE60', '#E74C3C', '#E67E22' # 绿/红/橙
700 C_SUR, C_SOC, C_ALPHA = '#95A5A6', '#7F8C8D', '#8E44AD' # 灰/灰/紫
701 C_RES, C_MAX, C_MIN = '#16A085', '#C0392B', '#2980B9' # 青/深红/蓝
702 C_NET, C_SUR_GREEN = '#8E44AD', '#2ECC71' # 紫 / 富余(亮绿,月度组合图)
703 XTICKS_DAY = list(range(0, 25, 2))
704 # 论文图样
705 NO_TITLE = True # 图内不带标题(论文中标题由图注给出)
706 AXIS_FS = 15 # 坐标轴标签字号(黑色)
707 TICK_FS = 12 # 刻度字号
708 LEGEND_FS = 17 # 图例字号
709 SOFT_RED, SOFT_GREEN, SOFT_BLUE = '#E9A6A1', '#8BCF8B', '#7097CA' # figures4papers 柔和色
710 BAR_EDGE = dict(edgecolor='black', linewidth=1.2)
711 def _style_axes(ax):
712     #绘图
713     ax.spines['top'].set_visible(False)
714     ax.grid(True, axis='y', linestyle='--', linewidth=0.7, alpha=0.3)
715     ax.tick_params(labelsize=TICK_FS)
716     def _limit_annot(ax, yv, color, text):
717         ax.axhline(yv, linestyle='--', linewidth=1.2, color=color, alpha=0.55)
718         ax.text(1.005, yv, text, transform=ax.get_yaxis_transform(),
719                fontsize=8.5, color=color, alpha=0.85, va='center', ha='left')
720     def _top_legend(ax, ncol=None, **kw):
721         h, l = ax.get_legend_handles_labels()
722         if not h:
723             return
724         ax.legend(h, l, loc='lower center', frameon=False,
725                  ncol=ncol or max(1, len(h)), fontsize=LEGEND_FS,
726                  bbox_to_anchor=(0.5, 1.02), columnspacing=1.8, **kw)
727     def _save_line_fig(x, ys, labels, title, xlabel, ylabel, path, date_axis=False,
728                       hlines=None, ylim=None, colors=None, line_styles=None,
729                       xticks=None, xlim=None, linewidths=None, hline_annot=False,
730                       fill_under=None):
731         fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
732         for k, (y, lab) in enumerate(zip(ys, labels)):
733             c = colors[k] if colors else None
734             ls = line_styles[k] if line_styles else '-'

```

```

735     lw = linewidths[k] if linewidths else 2.2
736     ax.plot(x, y, linewidth=lw, label=lab, color=c, linestyle=ls)
737 if fill_under is not None:
738     yi, base, fc, fa = fill_under
739     ax.fill_between(x, ys[yi], base, color=fc, alpha=fa, linewidth=0)
740 if hlines:
741     for item in hlines:
742         yv, lab = item[0], item[1]
743         hc = item[2] if len(item) > 2 else '#7F8C8D'
744         if hline_annot:
745             _limit_annot(ax, yv, hc, lab)
746         else:
747             ax.axhline(yv, linestyle='--', linewidth=1.0, color=hc, alpha=0.55, label=lab)
748 if not NO_TITLE:
749     ax.set_title(title, fontsize=13, fontweight='bold', pad=28)
750 ax.set_xlabel(xlabel, fontsize=AXIS_FS, color='black')
751 ax.set_ylabel(ylabel, fontsize=AXIS_FS, color='black')
752 _style_axes(ax)
753 _top_legend(ax)
754 if date_axis:
755     ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=5, maxticks=9))
756     ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
757     fig.autofmt_xdate(rotation=25)
758 if xticks is not None:
759     ax.set_xticks(xticks)
760 if xlim is not None:
761     ax.set_xlim(*xlim)
762 if ylim is not None:
763     ax.set_ylim(*ylim)
764 fig.tight_layout()
765 fig.savefig(path, dpi=300, bbox_inches='tight')
766 plt.close(fig)
767 def _save_bar_fig(xlabels, values, title, xlabel, ylabel, path, color=SOFT_RED):
768     """柱状图: figures4papers 柔和色 + 黑描边, 无标题、大字号。"""
769     fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
770     xx = np.arange(len(values))
771     ax.bar(xx, values, width=0.62, color=color, alpha=0.95, zorder=3, **BAR_EDGE)
772     ax.set_xticks(xx)
773     ax.set_xticklabels(xlabels, fontsize=TICK_FS)
774     if not NO_TITLE:
775         ax.set_title(title, fontsize=13, fontweight='bold', pad=12)
776     ax.set_xlabel(xlabel, fontsize=AXIS_FS, color='black')
777     ax.set_ylabel(ylabel, fontsize=AXIS_FS, color='black')
778     _style_axes(ax)
779     fig.tight_layout()
780     fig.savefig(path, dpi=300, bbox_inches='tight')
781     plt.close(fig)
782 def generate_visualizations(dates, price, load, pv, load_fc_store, pv_fc_store,
783                             plan_buy, ch, dis, emg, sur, soc_start, soc_end,
784                             alpha_store, reserve_store, run_D,
785                             cfg:Config, logger):
786     #生成论文结果
787     fig_dir=Path(cfg.fig_dir); fig_dir.mkdir(parents=True, exist_ok=True)
788     submit_start=date(2025,2,1)
789     valid_days=[i for i in range(run_D)
790                 if dates[i] >= submit_start and np.isfinite(plan_buy[i]).all()]
791     if not valid_days:
792         logger.warning("尚无2月及以后的有效结果, 已跳过可视化。")
793         return
794     xs=[datetime.combine(dates[i], datetime.min.time()) for i in valid_days]
795     # 1) 2月起日末SOC轨迹 (fig4 风格: 蓝粗线+浅填充、右缘限值标注)
796     fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
797     ax.fill_between(xs, soc_end[valid_days], cfg.e_min,
798                    color=C_BUY, alpha=0.10, linewidth=0)
799     ax.plot(xs, soc_end[valid_days], color=C_BUY, linewidth=1.8, label='日末储电量')
800     _limit_annot(ax, cfg.e_min, C_MIN, f'下限 {cfg.e_min:.0f}')
801     _limit_annot(ax, cfg.e_max, C_MAX, f'上限 {cfg.e_max:.0f}')
802     if not NO_TITLE:
803         ax.set_title('2月起储能设备日末电量轨迹', fontsize=13, fontweight='bold', pad=28)
804     ax.set_xlabel('日期', fontsize=AXIS_FS, color='black')
805     ax.set_ylabel('储电量 / kWh', fontsize=AXIS_FS, color='black')
806     _style_axes(ax)
807     _top_legend(ax)
808     ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=5, maxticks=9))
809     ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
810     fig.autofmt_xdate(rotation=25)
811     ax.set_ylim(max(0, cfg.e_min-500), cfg.e_max+500)
812     fig.tight_layout()
813     fig.savefig(fig_dir/"第二问储能设备日末电量轨迹.png", dpi=300, bbox_inches='tight')
814     plt.close(fig)
815     # 全年每日预测RMSE (图18与月度RMSE汇总共用)
816     vd=np.array(valid_days)
817     Lf, Pf = load_fc_store[vd,:144], pv_fc_store[vd,:144]

```

```

818 dl=np.sqrt(np.mean((load[vd]-Lf)**2, axis=1))
819 dp=np.sqrt(np.mean((pv[vd]-Pf)**2, axis=1))
820 dn=np.sqrt(np.mean(((load[vd]-pv[vd])-(Lf-Pf))**2, axis=1))
821 month_of=np.array([dates[i].month for i in valid_days])
822 # 按实际存在月份汇总 (正式运行即2--12月)
823 months_sorted=sorted(set(month_of.tolist()))
824 month_labels=[f"{m}月" for m in months_sorted]
825 plan_cost=[]; emg_cost=[]; emg_energy=[]; sur_energy=[]
826 rmse_load=[]; rmse_pv=[]; rmse_net=[]
827 for m in months_sorted:
828     ids=[i for i in valid_days if dates[i].month==m]
829     plan_cost.append(sum(float(np.sum(plan_buy[i]*price)*cfg.dt) for i in ids))
830     emg_cost.append(sum(float(np.sum(emg[i]*5*price)*cfg.dt) for i in ids))
831     emg_energy.append(sum(float(np.sum(emg[i])*cfg.dt) for i in ids))
832     sur_energy.append(sum(float(np.sum(sur[i])*cfg.dt) for i in ids))
833     sel=month_of==m # 非有限值剔除后再取均值 (与原逻辑等价)
834     rmse_load.append(float(np.mean(dl[sel & np.isfinite(dl)]))
835                       if np.isfinite(dl[sel]).any() else np.nan)
836     rmse_pv.append(float(np.mean(dp[sel & np.isfinite(dp)]))
837                    if np.isfinite(dp[sel]).any() else np.nan)
838     rmse_net.append(float(np.mean(dn[sel & np.isfinite(dn)]))
839                     if np.isfinite(dn[sel]).any() else np.nan)
840 # 2) 月度购电费用: 堆叠展示费用组成
841 fig,ax=plt.subplots(figsize=(10.8,5.4),dpi=300)
842 xx=np.arange(len(months_sorted))
843 ax.bar(xx,plan_cost,width=0.62,color=SOFT_GREEN,alpha=0.95,label="计划购电费",zorder=3,**
844        BAR_EDGE)
845 ax.bar(xx,emg_cost,width=0.62,bottom=plan_cost,color=SOFT_RED,alpha=0.95,label="紧急购电费",
846        zorder=3,**BAR_EDGE)
847 ax.set_xticks(xx); ax.set_xticklabels(month_labels,fontsize=TICK_FS)
848 if not NO_TITLE:
849     ax.set_title("2月起月度购电费用构成",fontsize=13,fontweight='bold',pad=12)
850 ax.set_xlabel("月份",fontsize=AXIS_FS,color='black'); ax.set_ylabel("费用 / 元",fontsize=
851     AXIS_FS,color='black')
852 _style_axes(ax); _top_legend(ax, ncol=2)
853 fig.tight_layout(); fig.savefig(fig_dir/"第二问月度购电费用构成.png",dpi=300,bbox_inches='
854     tight'); plt.close(fig)
855 # 3) 月度紧急购电量 (柔粉红柱)
856 _save_bar_fig(month_labels,emg_energy,"2月起月度紧急购电量","月份","紧急购电量 / kWh",
857               fig_dir/"第二问月度紧急购电量.png",color=SOFT_RED)
858 from openpyxl import Workbook
859 wb_rm = Workbook()
860 ws_rm = wb_rm.active; ws_rm.title = '月度预测RMSE'
861 ws_rm.append(['指标'] + month_labels)
862 ws_rm.append(['负荷RMSE (kW)'] + [round(float(v), 2) for v in rmse_load])
863 ws_rm.append(['光伏RMSE (kW)'] + [round(float(v), 2) for v in rmse_pv])
864 ws_rm.append(['净负荷RMSE (kW)'] + [round(float(v), 2) for v in rmse_net])
865 px_rm = fig_dir.parent / 'Q2_月度预测RMSE.xlsx'
866 wb_rm.save(px_rm)
867 #代表日优先选题目指定日期3月20日; 若尚未计算则选最后一个有效日
868 target=date(2025,3,20)
869 sample_idx=next((i for i in valid_days if dates[i]==target),valid_days[-1])
870 tt=np.arange(144)/6.0
871 compare_days=[]
872 for td in [date(2025,3,20),date(2025,6,21),
873            date(2025,9,23),date(2025,12,21)]:
874     idx=next((i for i in valid_days if dates[i]==td),None)
875     if idx is not None:
876         compare_days.append((idx,
877                               f"第二问{td.month}-{td.day}负荷与光伏预测对比.png",
878                               f"{td} 负荷与光伏预测对比"))
879 for idx,fname,title in compare_days:
880     _save_line_fig(tt,[load[idx],load_fc_store[idx,:144],pv[idx],pv_fc_store[idx,:144]],
881                    ["真实负荷","预测负荷","真实光伏","预测光伏"],title,
882                    "时刻 / h","功率 / kW",fig_dir/fname,
883                    colors=[C_LOAD,C_BUY,C_PV,C_DIS],line_styles=['-','--','-','-'],
884                    linewidths=[2.0,1.8,2.0,1.8],
885                    xticks=XTICKS_DAY, xlim=(0,24))
886 _save_line_fig(tt,[alpha_store[sample_idx]],
887                ["动态 alpha"],
888                f"{dates[sample_idx]} 动态非对称风险权重",
889                "时刻 / h",
890                "alpha",
891                fig_dir / "第二问3-20动态非对称风险权重.png",
892                colors=[C_ALPHA],
893                xticks=XTICKS_DAY,
894                xlim=(0, 24),
895                ylim=(0, max(cfg.alpha_max + 0.5, 5)))
896 _save_line_fig(tt,[reserve_store[sample_idx]],
897                ["风险备用量"],
898                f"{dates[sample_idx]} 风险备用量",
899                "时刻 / h",

```

```

896         "功率 / kW",
897         fig_dir / "第二问3-20风险备用量.png",
898         colors=[C_RES],
899         xticks=XTICKS_DAY,
900         xlim=(0, 24))
901
902 soc_curve=np.empty(145); soc_curve[0]=soc_start[sample_idx]
903 cur=soc_curve[0]
904 for t in range(144):
905     cur=cur+(cfg.eta_c*ch[sample_idx,t]-dis[sample_idx,t]/cfg.eta_d)*cfg.dt
906     soc_curve[t+1]=cur
907 #代表日SOC轨迹
908 fig, ax = plt.subplots(figsize=(11, 5.5), dpi=300)
909 t_soc = np.arange(145)/6.0
910 ax.fill_between(t_soc, soc_curve, cfg.e_min, where=soc_curve >= cfg.e_min,
911                alpha=0.10, color=C_DIS, linewidth=0)
912 ax.plot(t_soc, soc_curve, color=C_SOC, linewidth=2.8, marker='o', markersize=2.5,
913        label='储电量', zorder=10)
914 _limit_annot(ax, cfg.e_min, C_MIN, f'下限 {cfg.e_min:.0f}')
915 _limit_annot(ax, cfg.e_max, C_MAX, f'上限 {cfg.e_max:.0f}')
916 if not NO_TITLE:
917     ax.set_title(f"{dates[sample_idx]} 储能设备电量变化", fontsize=13, fontweight='bold',
918                pad=28)
919 ax.set_xlabel('时刻 / h', fontsize=AXIS_FS, color='black')
920 ax.set_ylabel('储电量 / kWh', fontsize=AXIS_FS, color='black')
921 _style_axes(ax)
922 ax.spines['left'].set_linewidth(1.5)
923 h, l = ax.get_legend_handles_labels()
924 ax.legend(h, l, loc='upper right', frameon=False, fontsize=15,
925        bbox_to_anchor=(1.0, 0.90))
926 ax.set_xticks(XTICKS_DAY); ax.set_xlim(0, 24)
927 ax.set_ylim(max(0, cfg.e_min-500), cfg.e_max+500)
928 fig.tight_layout()
929 fig.savefig(fig_dir/"第二问3-20储能设备电量变化.png", dpi=300, bbox_inches='tight')
930 plt.close(fig)
931 a_ld,p_ld = np.mean(load[vd],axis=1), np.mean(load_fc_store[vd,:144],axis=1)
932 a_pv,p_pv = np.mean(pv[vd],axis=1), np.mean(pv_fc_store[vd,:144],axis=1)
933 for ys,lbis,t1l,fname,cols in (
934     ([a_ld,p_ld],["实际日均负荷","预测日均负荷"],"2月起全年负荷预测结果对比",
935     "第二问全年负荷预测结果对比.png",[C_LOAD,C_DIS]),
936     ([a_pv,p_pv],["实际日均光伏","预测日均光伏"],"2月起全年光伏预测结果对比",
937     "第二问全年光伏预测结果对比.png",[C_PV,C_DIS])):
938     _save_line_fig(xs,ys,lbis,t1l,"日期","日均功率 / kW",fig_dir/fname,
939                 date_axis=True,colors=cols,line_styles=['-','-'],
940                 linewidths=[2.0,1.8])
941 #风险强度系数
942 rho_list = np.array([0.70, 0.75, 0.80, 0.85, 0.90])
943 total_wan = np.array([1366.92, 1365.86, 1366.63, 1368.51, 1371.41]) # 万元
944 best_i = int(np.argmin(total_wan))
945 fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
946 ax.plot(rho_list, total_wan * 1e4, color=C_BUY, linewidth=2.4,
947        marker='o', markersize=9, markerfacecolor=C_BUY,
948        markeredgewidth=1.5, label='全年总购电费', zorder=5)
949 ax.plot(rho_list[best_i], total_wan[best_i] * 1e4, marker='*', markersize=22,
950        color=C_CH, markeredgewidth=1.5, linestyle='none',
951        label=f'最优  $\rho$ ={rho_list[best_i]:.2f}', zorder=6)
952 ax.axvline(rho_list[best_i], linestyle='--', linewidth=1.2, color=C_CH, alpha=0.45)
953 for x0, y0 in zip(rho_list, total_wan):
954     ax.annotate(f'{y0:.2f}万', (x0, y0 * 1e4),
955                textcoords='offset points', xytext=(0, 14),
956                ha='center', fontsize=10.5,
957                color=C_CH if x0 == rho_list[best_i] else '#444444')
958 if not NO_TITLE:
959     ax.set_title(r'风险强度系数  $\rho$  灵敏度分析 (全年总购电费)',
960                fontsize=13, fontweight='bold', pad=28)
961 ax.set_xlabel(r'风险强度系数  $\rho$  (reserve_scale)', fontsize=AXIS_FS, color='black')
962 ax.set_ylabel('全年总购电费用 / 元', fontsize=AXIS_FS, color='black')
963 ax.set_xticks(rho_list)
964 ax.tick_params(labelsize=TICK_FS)
965 _style_axes(ax)
966 hd, lb = ax.get_legend_handles_labels()
967 ax.legend(hd, lb, loc='lower center', frameon=False, ncol=2,
968        fontsize=LEGEND_FS, bbox_to_anchor=(0.5, 1.02), columnspacing=1.8)
969 fig.tight_layout()
970 fig.savefig(fig_dir/"第二问全年总购电费用随风险强度系数的变化.png",
971        dpi=300, bbox_inches='tight')
972 plt.close(fig)
973 # 缓存绘图数据
974 try:
975     np.savez_compressed(
976         Path(cfg.out_dir)/"q2_plot_data.npz",
977         date_str=np.array([str(dates[i]) for i in range(run_D)]),
978         price=price, load=load[:run_D], pv=pv[:run_D],

```

```

978         load_fc=load_fc_store[:run_D], pv_fc=pv_fc_store[:run_D],
979         plan_buy=plan_buy[:run_D], ch=ch[:run_D], dis=dis[:run_D],
980         emg=emg[:run_D], sur=sur[:run_D],
981         soc_start=soc_start[:run_D], soc_end=soc_end[:run_D],
982         alpha=alpha_store[:run_D], reserve=reserve_store[:run_D],
983         e_min=cfg.e_min, e_max=cfg.e_max, e0=cfg.e0,
984         eta_c=cfg.eta_c, eta_d=cfg.eta_d, dt=cfg.dt,
985     )
986 except Exception as exc:
987     logger.warning("绘图数据缓存失败（不影响图片）：%s", exc)
988 def write_result_summary(dates, price, plan_buy, emg, sur, ch, dis, soc_end,
989                         alpha_store, run_D, cfg:Config, logger):
990     #从2月1日开始计算
991     submit_start=date(2025,2,1)
992     valid=[i for i in range(run_D)
993           if dates[i] >= submit_start and np.isfinite(plan_buy[i]).all()]
994     if not valid:
995         return
996     plan_cost=sum(float(np.sum(plan_buy[i]*price)*cfg.dt for i in valid)
997                  emg_cost=sum(float(np.sum(emg[i]*5*price)*cfg.dt for i in valid)
998                  emg_energy=sum(float(np.sum(emg[i])*cfg.dt for i in valid)
999                  sur_energy=sum(float(np.sum(sur[i])*cfg.dt for i in valid)
1000                  charge_energy=sum(float(np.sum(ch[i])*cfg.dt for i in valid)
1001                  discharge_energy=sum(float(np.sum(dis[i])*cfg.dt for i in valid)
1002                  alpha_mean=float(np.nanmean(alpha_store[valid]))
1003                  zero_emg_days=sum(1 for i in valid if np.sum(emg[i])*cfg.dt <= 1e-8)
1004     lines=["问题2正式结果摘要（1月仅作为预热期，不纳入统计）",
1005           f"正式统计日期范围：{dates[valid[0]]} 至 {dates[valid[-1]]}",
1006           f"累计计划购电费：{plan_cost:.2f} 元",
1007           f"累计紧急购电费：{emg_cost:.2f} 元",
1008           f"累计总购电费：{plan_cost+emg_cost:.2f} 元",
1009           f"累计紧急购电量：{emg_energy:.2f} kWh",
1010           f"累计富余电量：{sur_energy:.2f} kWh",
1011           f"累计充电量：{charge_energy:.2f} kWh",
1012           f"累计放电量：{discharge_energy:.2f} kWh",
1013           f"正式期末储电量：{soc_end[valid[-1]]:.2f} kWh",
1014           f"动态alpha平均值：{alpha_mean:.4f}",
1015           f"无紧急购电天数：{zero_emg_days}/{len(valid)}",
1016           f"正式提交文件：{cfg.out_xlsx}",
1017           f"可视化目录：{cfg.fig_dir}"],
1018     path=Path(cfg.out_dir)/"Q2_结果摘要.txt"
1019     path.write_text("\n".join(lines),encoding="utf-8")
1020 # 主程序
1021 def main():
1022     cfg = CFG
1023     out_dir = Path(cfg.out_dir)
1024     out_dir.mkdir(parents=True, exist_ok=True)
1025     logger = setup_logger(out_dir)
1026     dates, price, load_prior, pv_prior, load, pv = load_inputs(cfg, logger)
1027     D = len(dates)
1028     nT = cfg.slots_per_day
1029     run_D = min(D, cfg.max_days) if cfg.max_days > 0 else D
1030     plan_buy, soc_hist = (np.full((D,nT), np.nan) for _ in range(2))
1031     plan_ch, plan_dis, ch, dis, emg, sur, reserve_store = (np.zeros((D,nT)) for _ in range(7))
1032     soc_start, soc_end = np.full(D, np.nan), np.full(D, np.nan)
1033     load_fc_store, pv_fc_store = (np.full((D,cfg.forecast_slots), np.nan) for _ in range(2))
1034     alpha_store, net_errors, cplus_cache, cminus_cache = (np.full((D,nT), np.nan) for _ in range(4))
1035     lp_templates = build_lp_templates(price, cfg)
1036     daily_rows = []
1037     soc = cfg.e0
1038     for d in range(run_D):
1039         soc_start[d] = soc
1040         # 0:00预测：只使用已经发生的历史数据
1041         lf = load_forecast(d, load, load_prior, cfg, logger)
1042         pf, pvinfo = pv_forecast(d, pv, pv_prior, cfg, logger)
1043         load_fc_store[d] = lf
1044         pv_fc_store[d] = pf
1045         #动态非对称风险备用量
1046         update_risk_cost_cache(d, cplus_cache, cminus_cache, net_errors, price,
1047                               soc_hist, plan_buy, load, pv, cfg)
1048         alpha, beta, reserve = dynamic_risk_weights(d, net_errors, cplus_cache, cminus_cache,
1049             cfg)
1049         alpha_store[d] = alpha
1050         reserve_store[d] = reserve
1051         #当天只求解一次日前LP
1052         da = solve_day_ahead(price, lf, pf, reserve, soc, cfg, lp_templates)
1053         plan_buy[d] = da["buy"]; plan_ch[d] = da["ch"]
1054         plan_dis[d] = da["dis"]
1055         #实际运行
1056         for t in range(nT):
1057             soc_hist[d,t] = soc
1058             sol = operate_one_slot(soc, plan_buy[d,t], load[d,t], pv[d,t],

```

```

1059         plan_ch[d,t], plan_dis[d,t], cfg)
1060         ch[d,t] = sol["ch"]; dis[d,t] = sol["dis"]; emg[d,t] = sol["emg"]
1061         sur[d,t] = sol["sur"]
1062         soc = sol["E"]
1063         soc_end[d] = soc
1064         #当日预测误差只在当天结束后进入历史, 用于次日风险更新
1065         net_errors[d] = (load[d]-pv[d]) - (lf[:nT]-pf[:nT])
1066         plan_cost = float(np.sum(plan_buy[d]*price)*cfg.dt)
1067         emg_cost = float(np.sum(emg[d]*5*price)*cfg.dt)
1068         daily_rows.append([plan_cost, emg_cost, plan_cost+emg_cost,
1069                             float(np.sum(emg[d])*cfg.dt), float(np.sum(sur[d])*cfg.dt)])
1070     if run_D >= 32:
1071         fill_result2(cfg.template, cfg.out_xlsx, dates, price, plan_buy, ch, dis, soc_start, soc_end, emg
, cfg, logger)
1072     else:
1073         logger.warning("本次尚未覆盖2月1日, 不生成正式 result2。")
1074         # 题目指定日期若已计算, 则单独导出
1075         try:
1076             write_specified_date_results(dates, price, plan_buy, ch, dis, emg, soc_start, soc_end, cfg,
logger)
1077         except Exception as exc:
1078             logger.warning("指定日期结果单独导出失败: %s", exc)
1079         try:
1080             generate_visualizations(dates, price, load, pv, load_fc_store, pv_fc_store,
plan_buy, ch, dis, emg, sur, soc_start, soc_end,
1081                                     alpha_store, reserve_store, run_D, cfg, logger)
1082             write_result_summary(dates, price, plan_buy, emg, sur, ch, dis, soc_end,
alpha_store, run_D, cfg, logger)
1083         except Exception as exc:
1084             logger.error("可视化或结果摘要生成失败, 但不影响result2: %s", exc)
1085     if daily_rows:
1086         arr = np.asarray(daily_rows, dtype=float)
1087         print("累计计划购电费 %.2f 元; 累计紧急购电费 %.2f 元; 累计总费 %.2f 元; "
1088               "紧急购电量 %.2f kWh; 富余电量 %.2f kWh"
1089               % (arr[:, 0].sum(), arr[:, 1].sum(), arr[:, 2].sum(),
1090                  arr[:, 3].sum(), arr[:, 4].sum()))
1091         print("最终 result2: %s" % (cfg.out_xlsx if run_D >= 32 else "本次未生成"))
1092 if __name__ == "__main__":
1093     main()
1094

```

code/Q2.py

Q3.py

```

1  from __future__ import annotations
2  import logging
3  from dataclasses import dataclass
4  from pathlib import Path
5  from datetime import datetime, timedelta, date
6  from typing import Tuple, Dict, Optional
7  import numpy as np
8  from scipy.optimize import linprog
9  from scipy import sparse
10 from statsmodels.tsa.ar_model import AutoReg
11 try:
12     from statsmodels.tsa.seasonal import MSTL
13     HAS_MSTL = True
14 except Exception:
15     HAS_MSTL = False
16 try:
17     from openpyxl import load_workbook
18 except ImportError as e:
19     raise ImportError(
20         "缺少 openpyxl。请在当前 Python 环境执行: pip install openpyxl"
21     ) from e
22 import matplotlib
23 matplotlib.use("Agg")
24 import matplotlib.pyplot as plt
25 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial
Unicode MS", "DejaVu Sans"]
26 plt.rcParams["axes.unicode_minus"] = False
27 plt.rcParams["font.family"] = "sans-serif"
28 plt.rcParams["font.size"] = 12
29 plt.rcParams["axes.spines.top"] = False
30 plt.rcParams["axes.linewidth"] = 1.5
31 plt.rcParams["xtick.major.width"] = 1.5
32 plt.rcParams["ytick.major.width"] = 1.5
33 plt.rcParams["xtick.major.size"] = 6
34 plt.rcParams["ytick.major.size"] = 6
35 plt.rcParams["xtick.direction"] = "out"
36 plt.rcParams["ytick.direction"] = "out"
37 C_BUY, C_PV, C_DIS = "#2E74B5", "#27AE60", "#E67E22"

```

```

38 C_LOAD, C_CH, C_W = "#34495e", "#E74C3C", "#95A5A6"
39 C_SOC, C_PRICE, C_MIN = "#7F8C8D", "#E74C3C", "#2980B9"
40 #参数区
41 class Config:
42     base_dir = str(Path(__file__).resolve().parent)
43     att1 = str(Path(base_dir) / "附件1.xlsx")
44     att2 = str(Path(base_dir) / "附件2.xlsx")
45     att3 = str(Path(base_dir) / "附件3.xlsx")
46     _tpl_candidates = [
47         Path(base_dir) / "附件5" / "result3.xlsx",
48         Path(base_dir) / "result3.xlsx",
49     ]
50     template = str(next(p for p in _tpl_candidates if p.exists()))
51     out_dir = str(Path(base_dir) / "Q3结果")
52     out_xlsx = str(Path(out_dir) / "result3_filled_forecast_driven.xlsx")
53     fig_dir = str(Path(out_dir) / "Q3图片输出")
54     slots_pday = 144; dt = 1.0/6.0; hori_slots = 144; fore_slots = 288
55     eta_c = eta_d = 0.90; e_min, e_max = 1200.0, 10800.0; p_max = 5000.0; e0 = 6000.0
56     l_days = 42; pv_days = 60; l_min_days = 21; m_days = 7
57     ar_candi = (1, 2, 3, 6, 12, 18, 36); pv_candi = (1, 2, 3, 6, 12)
58     k_candi = (1, 2, 3, 5, 7); clear_quant = 0.93
59     pv_smooth = 2; rk_hisdays = 28; rk_decay = 0.90; rk_sam = 5
60     alpha_cold = 4.0; alpha_min = 1.0; alpha_max = 8.0; beta_fixed = 1.0; c_eps = 1e-4
61     tau_min = 0.55; tau_max = 0.88; reserve_scale = 0.75
62     roll_slots = 144; use_bias = False
63     stage_quant = 0.80; stage_scale = 0.75
64     stage_mindays = 7; stage_rhisdays = 35; stage_decay = 0.94
65     eps_cycle = 1e-5; eps_surplus = 1e-8; lp_method = "highs"
66     update_hours = (6, 12, 18); run_comparison = True
67     compare_schemes = (((), (6,)), (12,)), (18,)), (6, 12, 18))
68     compare_names = ("仅0:00", "0:00+6:00", "0:00+12:00", "0:00+18:00", "0:00+6:00+12:00+18:00")
69     represent_dates = (date(2025,3,20), date(2025,6,21), date(2025,9,23), date(2025,12,21))
70     max_days = 365
71 CFG = Config()
72 def setup_logger(out_dir: Path) -> logging.Logger:
73     out_dir.mkdir(parents=True, exist_ok=True)
74     logger = logging.getLogger("Q3")
75     logger.setLevel(logging.INFO)
76     logger.handlers.clear()
77     return logger
78 def excel_serial_to_date(x) -> datetime:
79     # Excel 1900日期系统 (含1900闰年历史兼容)
80     return datetime(1899, 12, 30) + timedelta(days=float(x))
81 def cell_to_date(x):
82     """兼容 openpyxl 返回的 date/datetime、Excel 序列号和常见日期字符串。"""
83     if x is None:
84         return None
85     if isinstance(x, datetime):
86         return x.date()
87     if isinstance(x, date):
88         return x
89     if isinstance(x, (int, float, np.integer, np.floating)):
90         return excel_serial_to_date(x).date()
91     if isinstance(x, str):
92         txt=x.strip()
93         for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%Y.%m.%d"):
94             try:
95                 return datetime.strptime(txt, fmt).date()
96             except ValueError:
97                 pass
98         raise ValueError(f"无法识别日期单元格: {x!r}")
99 def safe_float_array(x) -> np.ndarray:
100     return np.asarray([[float(v) if v is not None else np.nan for v in row] for row in x], dtype=float)
101 def weighted_quantile(values: np.ndarray, weights: np.ndarray, q: float) -> float:
102     values = np.asarray(values, dtype=float)
103     weights = np.asarray(weights, dtype=float)
104     m = np.isfinite(values) & np.isfinite(weights) & (weights > 0)
105     if m.sum() == 0:
106         return 0.0
107     v, w = values[m], weights[m]
108     idx = np.argsort(v)
109     v, w = v[idx], w[idx]
110     cw = np.cumsum(w) / np.sum(w)
111     return float(np.interp(q, cw, v))
112 def moving_average_circular(x: np.ndarray, half: int) -> np.ndarray:
113     if half <= 0:
114         return x.copy()
115     n = len(x)
116     out = np.zeros(n)
117     for i in range(n):
118         lo = max(0, i-half)
119         hi = min(n, i+half+1)

```

```

120         out[i] = np.mean(x[lo:hi])
121     return out
122 def choose_ar_forecast(series: np.ndarray, horizon: int, candidates: Tuple[int, ...], fallback:
123     float = 0.0) -> np.ndarray:
124     y = np.asarray(series, dtype=float)
125     y = y[np.isfinite(y)]
126     if len(y) < 8 or np.std(y) < 1e-8:
127         return np.full(horizon, float(np.mean(y) if len(y) else fallback))
128     best = None
129     for p in candidates:
130         if len(y) <= p + 5:
131             continue
132         try:
133             model = AutoReg(y, lags=p, trend="ct", old_names=False).fit()
134             if np.isfinite(model.aic) and (best is None or model.aic < best[0]):
135                 best = (model.aic, model)
136         except Exception:
137             continue
138     if best is None:
139         return np.full(horizon, float(np.mean(y[-min(7, len(y)):]))) )
140     try:
141         pred = np.asarray(best[1].predict(start=len(y), end=len(y)+horizon-1, dynamic=False),
142             dtype=float)
143         if np.any(~np.isfinite(pred)):
144             raise ValueError("AR forecast nonfinite")
145         return pred
146     except Exception:
147         return np.full(horizon, float(np.mean(y[-min(7, len(y)):]))) )
148 def load_inputs(cfg: Config, logger):
149     for p in (cfg.att1, cfg.att2, cfg.template):
150         if not Path(p).exists():
151             raise FileNotFoundError(f"未找到文件: {p}")
152     # 附件1
153     wb1 = load_workbook(cfg.att1, data_only=True, read_only=True)
154     s1 = wb1["Sheet1"]
155     rows1 = list(s1.iter_rows(min_row=2, max_row=145, min_col=2, max_col=4, values_only=True))
156     price = np.array([float(r[0]) for r in rows1], dtype=float)
157     load_prior = np.array([float(r[1]) for r in rows1], dtype=float)
158     pv_prior = np.array([float(r[2]) for r in rows1], dtype=float)
159     wb1.close()
160     # 附件2
161     wb2 = load_workbook(cfg.att2, data_only=True, read_only=True)
162     s1 = wb2["小区负载"]
163     sp = wb2["光伏发电实际功率"]
164     load_all = list(s1.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=
165         True))
166     pv_all = list(sp.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=True)
167         )
168     wb2.close()
169     dates = [cell_to_date(r[0]) for r in load_all]
170     load_rows = [r[1:] for r in load_all]
171     pv_rows = [r[1:] for r in pv_all]
172     load = safe_float_array(load_rows)
173     pv_actual = safe_float_array(pv_rows)
174     assert load.shape == (365, 144), load.shape
175     assert pv_actual.shape == (365, 144), pv_actual.shape
176     assert len(price) == 144
177     return dates, price, load_prior, pv_prior, load, pv_actual
178 _LOAD_MSTL_CACHE = {}
179 def load_forecast(day_idx: int, load: np.ndarray, prior: np.ndarray, cfg: Config, logger=None)
180     -> np.ndarray:
181     H = cfg.fore_slots
182     if day_idx == 0:
183         return np.tile(prior, 2)
184     hist_days = min(day_idx, cfg.l_days)
185     hist_mat = load[day_idx-hist_days:day_idx]
186     # 冷启动: 同一时刻近日期加权 + 附件1先验
187     if hist_days < cfg.l_min_days or not HAS_MSTL:
188         k = hist_mat.shape[0]
189         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
190         weights /= weights.sum()
191         shape = np.sum(hist_mat * weights[:, None], axis=0)
192         if hist_days >= 7:
193             shape = 0.55*shape + 0.45*load[day_idx-7]
194         w_prior = max(0.0, 1.0 - hist_days / cfg.l_min_days)
195         shape = (1-w_prior)*shape + w_prior*prior
196         return np.tile(shape, 2)
197     # 仅每隔若干天完整重估一次 MSTL; 数据已确认无明显异常, 因此关闭 robust 迭代
198     need_refit = (
199         not _LOAD_MSTL_CACHE
200         or day_idx - _LOAD_MSTL_CACHE.get("refit_day", -10**9) >= cfg.m_days
201     )
202     if need_refit:

```

```

198 y = hist_mat.reshape(-1)
199 try:
200     mstl = MSTL(y, periods=(144, 1008), stl_kwargs={"robust": False}).fit()
201     trend = np.asarray(mstl.trend)
202     seasonal = np.asarray(mstl.seasonal)
203     if seasonal.ndim == 1:
204         s_daily = seasonal
205         s_weekly = np.zeros_like(seasonal)
206     else:
207         s_daily = seasonal[:, 0]
208         s_weekly = seasonal[:, 1]
209     resid = np.asarray(mstl.resid)
210     # 保存稳定的日周期与周周期结构, 供未来几天复用
211     daily_tail = s_daily[-min(len(s_daily), 28*144):]
212     weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
213     daily_pattern = np.array([
214         np.nanmean(daily_tail[i::144]) for i in range(144)
215     ])
216     weekly_pattern = np.array([
217         np.nanmean(weekly_tail[i::1008]) for i in range(1008)
218     ])
219     _LOAD_MSTL_CACHE.clear()
220     _LOAD_MSTL_CACHE.update({
221         "refit_day": day_idx,
222         "origin_day": day_idx - hist_days,
223         "daily_pattern": daily_pattern,
224         "weekly_pattern": weekly_pattern,
225         "trend_last": float(trend[-1]),
226         "trend_slope": float(np.clip(
227             np.polyfit(
228                 np.arange(min(432, len(trend))), dtype=float,
229                 trend[-min(432, len(trend))]:],
230                 1
231             )[0], -5.0, 5.0
232         )),
233         "resid_tail": resid[-min(len(resid), 14*144):].copy(),
234     })
235 except Exception as e:
236     if logger:
237         logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
238     k = hist_mat.shape[0]
239     weights = np.exp(-0.10*np.arange(k-1, -1, -1))
240     weights /= weights.sum()
241     shape = np.sum(hist_mat * weights[:, None], axis=0)
242     if hist_days >= 7:
243         shape = 0.5*shape + 0.5*load[day_idx-7]
244     return np.tile(shape, 2)
245 # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
246 daily_pattern = _LOAD_MSTL_CACHE["daily_pattern"]
247 weekly_pattern = _LOAD_MSTL_CACHE["weekly_pattern"]
248 origin_day = _LOAD_MSTL_CACHE["origin_day"]
249 recent_days = min(hist_days, 3)
250 recent = load[day_idx-recent_days:day_idx].reshape(-1)
251 global_start = (day_idx - recent_days) * 144
252 gidx = global_start + np.arange(len(recent))
253 sd_hist = daily_pattern[gidx % 144]
254 sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
255 deseason = recent - sd_hist - sw_hist
256 m = min(432, len(deseason))
257 x = np.arange(m, dtype=float)
258 coef = np.polyfit(x, deseason[-m:], 1)
259 slope = float(np.clip(coef[0], -5.0, 5.0))
260 level = float(deseason[-1])
261 trend_future = level + slope*np.arange(1, H+1)
262 future_gidx = day_idx*144 + np.arange(H)
263 sd_future = daily_pattern[future_gidx % 144]
264 sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
265 # 残差 AR 仍每天轻量更新, 不需要重新做 MSTL
266 hist_axis = np.arange(len(deseason), dtype=float)
267 fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
268 resid_recent = deseason - fitted_trend_hist
269 ar_future = choose_ar_forecast(
270     resid_recent[-min(len(resid_recent), 3*144):],
271     H,
272     cfg.ar_candi,
273     0.0,
274 )
275 pred = trend_future + sd_future + sw_future + ar_future
276 return np.maximum(pred, 0.0)
277 def build_clear_envelope(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config) -> np.
278     ndarray:
279     if day_idx == 0:
280         base = prior.copy()
281         return np.maximum(base, 0.0)

```

```

281 w = min(day_idx, cfg.pv_days)
282 hist = pv[day_idx-w:day_idx]
283 q = np.quantile(hist, cfg.clear_quant, axis=0)
284 q = moving_average_circular(q, cfg.pv_smooth)
285 # 样本短时与附件1先验混合
286 prior_w = max(0.0, 1.0 - w/21.0)
287 base = (1-prior_w)*q + prior_w*prior
288 # 夜间稳定置0: 历史90%以上为0即置0
289 zero_ratio = np.mean(hist <= 1e-8, axis=0)
290 base[zero_ratio > 0.90] = 0.0
291 return np.maximum(base, 0.0)
292 def pv_forecast(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config, logger=None) ->
    Tuple[np.ndarray, Dict]:
293     H = cfg.fore_slots
294     if day_idx == 0:
295         return np.tile(prior, 2), {"kappa_d": 1.0, "kappa_next": 1.0, "envelope": prior.copy()}
296     B_d = build_clear_envelope(day_idx, pv, prior, cfg)
297     # 下一日包络: 年周期变化慢, 先用当前包络; 用近期日能量变化率做轻微尺度外推
298     recent_energy = np.sum(pv[max(0, day_idx-14):day_idx], axis=1)*cfg.dt
299     scale_next = 1.0
300     if len(recent_energy) >= 7:
301         x = np.arange(len(recent_energy))
302         slope = np.polyfit(x, recent_energy, 1)[0]
303         denom = max(np.mean(recent_energy), 1.0)
304         scale_next = float(np.clip(1.0 + slope/denom, 0.95, 1.05))
305     B_next = B_d * scale_next
306     # 用当前滚动包络近似反演近期日透射率; 只使用过去日期
307     w = min(day_idx, cfg.pv_days)
308     hist = pv[day_idx-w:day_idx]
309     denom = max(np.sum(B_d)*cfg.dt, 1e-6)
310     kappas = np.clip(np.sum(hist, axis=1)*cfg.dt/denom, 0.0, 1.0)
311     kpred = choose_ar_forecast(kappas, 2, cfg.k_candi, fallback=float(np.mean(kappas)))
312     kpred = np.clip(kpred, 0.0, 1.0)
313     base_d = B_d*kpred[0]
314     base_next = B_next*kpred[1]
315     # 残差构造: 每个历史日用其日能量对应kappa相对于当前包络重构
316     residual_days = hist - kappas[:, None]*B_d[None, :]
317     resid_series = residual_days.reshape(-1)
318     rpred = choose_ar_forecast(resid_series[-min(len(resid_series), 21*144):], H, cfg.pv_candi,
        0.0)
319     pred = np.concatenate([base_d, base_next]) + rpred
320     pred = np.maximum(pred, 0.0)
321     # 夜间稳定无发电时段置零
322     zero_mask = B_d <= 1e-8
323     for h in range(H):
324         if zero_mask[h % 144]:
325             pred[h] = 0.0
326     return pred, {"kappa_d": float(kpred[0]), "kappa_next": float(kpred[1]), "envelope": B_d}
327 def compute_historical_marginal_cost_day(j: int,
    available_day_exclusive: int,
    net_errors: np.ndarray,
    price: np.ndarray,
    soc_hist: np.ndarray,
    planned_hist: np.ndarray,
    load: np.ndarray,
    pv: np.ndarray,
    cfg: Config) -> Tuple[np.ndarray, np.ndarray]:
328
329     nT = cfg.slots_pday
330     cplus = np.full(nT, np.nan, dtype=float)
331     cminus = np.full(nT, np.nan, dtype=float)
332     if j < 0 or j >= available_day_exclusive:
333         return cplus, cminus
334     # 只把当前允许访问的历史展开; 后续窗口均是连续切片
335     n_avail = available_day_exclusive * nT
336     plan_f = planned_hist[:available_day_exclusive].reshape(-1)
337     load_f = load[:available_day_exclusive].reshape(-1)
338     pv_f = pv[:available_day_exclusive].reshape(-1)
339     soc_f = soc_hist[:available_day_exclusive].reshape(-1)
340     err_f = net_errors[:available_day_exclusive].reshape(-1)
341     slot_price = np.tile(price, available_day_exclusive)
342     base = j * nT
343     eff_rt = cfg.eta_c * cfg.eta_d
344     for t in range(nT):
345         e = net_errors[j, t]
346         if not np.isfinite(e):
347             continue
348         lam_t = price[t]
349         s0 = soc_hist[j, t] if np.isfinite(soc_hist[j, t]) else cfg.e0
350         g0 = base + t
351         g1 = g0 + 1
352         g2 = min(g0 + cfg.hori_slots, n_avail - 1)
353         if e > 0:
354             c_em = 5.0 * lam_t

```

```

362     c_bat = np.inf
363     if s0 > cfg.e_min + 1e-6 and g1 <= g2:
364         s1 = slice(g1, g2 + 1)
365         margin = plan_f[s1] + pv_f[s1] - load_f[s1]
366         sf = soc_f[s1]
367         pf = slot_price[s1]
368         soc_ok = (~np.isfinite(sf)) | (sf < cfg.e_max - 1e-6)
369         feasible = soc_ok & ((margin > 0.0) | (pf < lam_t))
370         if np.any(feasible):
371             c_bat = float(np.min(pf[feasible]) / eff_rt)
372     cplus[t] = min(c_em, c_bat) if np.isfinite(c_bat) else c_em
373 elif e < 0:
374     recovery = 0.0
375     if s0 < cfg.e_max - 1e-6 and g1 <= g2:
376         s1 = slice(g1, g2 + 1)
377         ef = err_f[s1]
378         pf = slot_price[s1]
379         m = np.isfinite(ef) & (ef > 0.0)
380         if np.any(m):
381             recovery = float(np.max(5.0 * pf[m] * eff_rt))
382     cminus[t] = max(0.0, lam_t - recovery)
383     return cplus, cminus
384 def update_risk_cost_cache(day_idx: int,
385                             cplus_cache: np.ndarray,
386                             cminus_cache: np.ndarray,
387                             net_errors: np.ndarray,
388                             price: np.ndarray,
389                             soc_hist: np.ndarray,
390                             planned_hist: np.ndarray,
391                             load: np.ndarray,
392                             pv: np.ndarray,
393                             cfg: Config) -> None:
394     if day_idx <= 0:
395         return
396     # 昨天: 只能使用到昨天24:00, 保持严格无未来信息泄露
397     j = day_idx - 1
398     cp, cm = compute_historical_marginal_cost_day(
399         j, day_idx, net_errors, price, soc_hist, planned_hist, load, pv, cfg)
400     cplus_cache[j] = cp
401     cminus_cache[j] = cm
402     # 前天: 此时“前天时刻 + 未来24h”已全部落在已发生历史内, 覆盖为最终缓存
403     if day_idx >= 2:
404         j = day_idx - 2
405         cp, cm = compute_historical_marginal_cost_day(
406             j, day_idx, net_errors, price, soc_hist, planned_hist, load, pv, cfg)
407         cplus_cache[j] = cp
408         cminus_cache[j] = cm
409 def dynamic_risk_weights(day_idx: int,
410                           net_errors: np.ndarray,
411                           cplus_cache: np.ndarray,
412                           cminus_cache: np.ndarray,
413                           cfg: Config) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
414     nT = cfg.slots_pday
415     alpha = np.full(nT, cfg.alpha_cold, dtype=float)
416     beta = np.full(nT, cfg.beta_fixed, dtype=float)
417     reserve = np.zeros(nT, dtype=float)
418     if day_idx <= 0:
419         return alpha, beta, reserve
420     j0 = max(0, day_idx - cfg.rk_hisdays)
421     hist_days = np.arange(j0, day_idx)
422     if hist_days.size == 0:
423         return alpha, beta, reserve
424     ages = day_idx - hist_days
425     base_w = cfg.rk_decay ** (ages - 1)
426     errs = net_errors[hist_days] # (HIST, 144)
427     cp_all = cplus_cache[hist_days]
428     cm_all = cminus_cache[hist_days]
429     for t in range(nT):
430         et = errs[:, t]
431         # 正向误差历史
432         mp = (et > 0) & np.isfinite(et) & np.isfinite(cp_all[:, t])
433         # 负向误差历史
434         mn = (et < 0) & np.isfinite(et) & np.isfinite(cm_all[:, t])
435         if np.count_nonzero(mp) >= cfg.rk_sam and np.count_nonzero(mn) >= cfg.rk_sam:
436             cp = float(np.average(cp_all[mp, t], weights=base_w[mp]))
437             cm = float(np.average(cm_all[mn, t], weights=base_w[mn]))
438             raw = cp / max(cm, cfg.c_eps)
439             alpha[t] = min(cfg.alpha_max, max(cfg.alpha_min, raw))
440             tau = alpha[t] / (alpha[t] + beta[t])
441             tau = min(cfg.tau_max, max(cfg.tau_min, tau))
442             if np.count_nonzero(mp) >= cfg.rk_sam:
443                 reserve[t] = cfg.reserve_scale * weighted_quantile(et[mp], base_w[mp], tau)
444     return alpha, beta, reserve
445 @dataclass

```

```

446 class LPTemplates:
447     da_Aeq: sparse.csr_matrix
448     da_c: np.ndarray
449     da_bounds: list
450     mu_terminal: float
451 def build_lp_templates(price: np.ndarray, cfg: Config) -> LPTemplates:
452     n = cfg.slots_pday
453     N = 5 * n
454     ib, ic, idis, isu, ie = 0, n, 2*n, 3*n, 4*n
455     rows, cols, data = [], [], []
456     r = 0
457     # 功率平衡: buy + dis - ch - sur = 风险校准后的预测净负荷
458     for t in range(n):
459         for col, val in ((ib+t,1.0),(idis+t,1.0),(ic+t,-1.0),(isu+t,-1.0)):
460             rows.append(r); cols.append(col); data.append(val)
461         r += 1
462     # SOC递推
463     for t in range(n):
464         rows.append(r); cols.append(ie+t); data.append(1.0)
465         if t > 0:
466             rows.append(r); cols.append(ie+t-1); data.append(-1.0)
467             rows.append(r); cols.append(ic+t); data.append(-cfg.eta_c*cfg.dt)
468             rows.append(r); cols.append(idis+t); data.append(cfg.dt/cfg.eta_d)
469         r += 1
470     Aeq = sparse.csr_matrix((data,(rows,cols)), shape=(r,N))
471     mu = cfg.eta_d * float(np.min(price))
472     c = np.zeros(N)
473     c[ib:ib+n] = price * cfg.dt
474     c[ic:ic+n] = cfg.eps_cycle * cfg.dt
475     c[idis:idis+n] = cfg.eps_cycle * cfg.dt
476     c[isu:isu+n] = cfg.eps_surplus * cfg.dt
477     c[ie+n-1] = -mu
478     bounds = ((0,None))*n + [(0,cfg.p_max))*n + [(0,cfg.p_max))*n +
479               [(0,None))*n + [(cfg.e_min,cfg.e_max))*n
480     return LPTemplates(Aeq, c, bounds, mu)
481 def solve_day_ahead(price: np.ndarray,
482                     load_fc: np.ndarray,
483                     pv_fc: np.ndarray,
484                     reserve: np.ndarray,
485                     soc0: float,
486                     cfg: Config,
487                     lp: LPTemplates) -> Dict[str,np.ndarray]:
488     """每天0:00求解一次全天计划购电及计划储能LP."""
489     n = cfg.slots_pday
490     net_req = load_fc[:n] - pv_fc[:n] + reserve
491     b = np.empty(2*n, dtype=float)
492     b[:n] = net_req
493     b[n:] = 0.0
494     b[n] = soc0
495     res = linprog(lp.da_c, A_eq=lp.da_Aeq, b_eq=b, bounds=lp.da_bounds, method=cfg.lp_method)
496     if not res.success:
497         raise RuntimeError(f"日前LP失败: {res.message}")
498     x = res.x
499     ib, ic, idis, isu, ie = 0,n,2*n,3*n,4*n
500     # 加回常数 mu*soc0, 得到论文中完整目标函数值
501     paper_obj = float(res.fun + lp.mu_terminal * soc0)
502     return {"buy":x[ib:ib+n], "ch":x[ic:ic+n], "dis":x[idis:idis+n],
503            "sur":x[isu:isu+n], "E":x[ie:ie+n], "obj":paper_obj}
504 def operate_one_slot(soc0: float, buy: float, load_actual: float, pv_actual: float,
505                     plan_ch: float, plan_dis: float, cfg: Config) -> Dict[str,float]:
506     dt = cfg.dt
507     # 先把计划动作投影到当前SOC可行范围
508     ch = max(0.0, min(float(plan_ch), cfg.p_max))
509     dis = max(0.0, min(float(plan_dis), cfg.p_max))
510     max_ch_soc = max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt))
511     max_dis_soc = max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt)
512     ch = min(ch, max_ch_soc)
513     dis = min(dis, max_dis_soc)
514     # 基于真实负荷/光伏检查功率平衡
515     gap = float(load_actual - (buy + pv_actual + dis - ch))
516     emg = 0.0
517     sur = 0.0
518     if gap > 1e-10:
519         # 1. 先取消充电, 相当于立即释放可用供电
520         cut_ch = min(ch, gap)
521         ch -= cut_ch
522         gap -= cut_ch
523         # 2. 再增加放电; 紧急购电价格为5倍, 故在可行范围内优先用储能
524         max_dis_total = min(cfg.p_max, max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt))
525         extra_dis = min(gap, max(0.0, max_dis_total - dis))
526         dis += extra_dis
527         gap -= extra_dis
528         # 3. 储能仍不足时才紧急购电

```

```

529     emg = max(0.0, gap)
530 elif gap < -1e-10:
531     excess = -gap
532     # 1. 先取消不必要的计划放电
533     cut_dis = min(dis, excess)
534     dis -= cut_dis
535     excess -= cut_dis
536     # 2. 再增加充电
537     max_ch_total = min(cfg.p_max, max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt)))
538     extra_ch = min(excess, max(0.0, max_ch_total - ch))
539     ch += extra_ch
540     excess -= extra_ch
541     # 3. 电池无法继续吸收的部分作为富余功率
542     sur = max(0.0, excess)
543     soc1 = soc0 + (cfg.eta_c * ch - dis / cfg.eta_d) * dt
544     soc1 = min(cfg.e_max, max(cfg.e_min, soc1))
545     return {"ch": ch, "dis": dis, "emg": emg, "sur": sur, "E": soc1}
546 def load_attachment3(cfg: Config, dates, logger):
547     if not Path(cfg.att3).exists():
548         # 兼容用户下载后的文件名
549         alt = Path(cfg.base_dir) / "附件3(1).xlsx"
550         if alt.exists():
551             cfg.att3 = str(alt)
552         else:
553             raise FileNotFoundError(f"未找到附件3: {cfg.att3}")
554     wb = load_workbook(cfg.att3, data_only=True, read_only=True)
555     ws = wb[wb.sheetnames[0]]
556     rows = list(ws.iter_rows(min_row=2, max_col=26, values_only=True))
557     wb.close()
558     date_to_idx = {d: i for i, d in enumerate(dates)}
559     out = np.full((len(dates), 4, 24), np.nan, dtype=float)
560     stage_map = {"0:00": 0, "6:00": 1, "12:00": 2, "18:00": 3}
561     current_date = None
562     for r in rows:
563         raw_date, raw_stage = r[0], r[1]
564         if raw_date not in (None, ""):
565             current_date = cell_to_date(raw_date)
566         if current_date is None:
567             continue
568         stage_txt = str(raw_stage).strip() if raw_stage is not None else ""
569         if stage_txt not in stage_map or current_date not in date_to_idx:
570             continue
571         vals = np.array([0.0 if v is None else float(v) for v in r[2:26]], dtype=float)
572         out[date_to_idx[current_date], stage_map[stage_txt], :] = np.maximum(vals, 0.0)
573     if np.isnan(out).any():
574         bad = int(np.isnan(out).sum())
575         raise ValueError(f"附件3读取不完整, 仍有 {bad} 个缺失预报值。")
576     return out
577 def pv_stage_forecast_horizon_10min(day_idx: int, stage_hour: int, pv_hourly: np.ndarray,
578                                     pv_actual: np.ndarray, cfg: Config) -> np.ndarray:
579     stage_idx = {0: 0, 6: 1, 12: 2, 18: 3}[stage_hour]
580     hourly = pv_hourly[day_idx, stage_idx]
581     start_slot = stage_hour * 6
582     if stage_hour == 0:
583         # 0点附近光伏通常为0; 用0作为当前时刻锚点, 不泄漏未来信息。
584         anchor = 0.0
585     else:
586         # 当前时刻可用的信息仅到发布时刻之前一个10分钟区间。
587         anchor = float(pv_actual[day_idx, start_slot - 1])
588     xp = np.arange(0, 25, dtype=float)
589     fp = np.concatenate([anchor, hourly])
590     target_rel = (np.arange(cfg.roll_slots) + 1) / 6.0
591     pred = np.interp(target_rel, xp, fp)
592     return np.maximum(pred, 0.0)
593 def load_stage_forecast_horizon(base_fc_full: np.ndarray, actual_today: np.ndarray,
594                                 stage_hour: int, cfg: Config) -> np.ndarray:
595     start = stage_hour * 6
596     H = cfg.roll_slots
597     seg = np.asarray(base_fc_full[start: start+H], dtype=float).copy()
598     if len(seg) != H:
599         raise ValueError("负荷48小时预测长度不足, 无法构造24小时MPC窗口。")
600     if cfg.use_bias and stage_hour > 0:
601         m = min(start, 18)
602         # 注意: 偏差估计只使用发布时刻以前已经实现的负荷。
603         base_day = np.asarray(base_fc_full[:cfg.slots_pday], dtype=float)
604         resid = actual_today[start-m: start] - base_day[start-m: start]
605         w = np.exp(np.linspace(-1.5, 0.0, m))
606         w /= w.sum()
607         bias = float(np.sum(w * resid))
608         decay = np.exp(-np.arange(H) / 36.0)
609         seg += bias * decay
610     return np.maximum(seg, 0.0)
611 def actual_horizon_from_arrays(day_idx: int, stage_hour: int, arr: np.ndarray,

```

```

612                                     cfg: Config) -> Optional[np.ndarray]:
613     start = stage_hour * 6
614     H = cfg.roll_slots
615     flat = arr.reshape(-1)
616     g0 = day_idx * cfg.slots_pday + start
617     if g0 + H > len(flat):
618         return None
619     return np.asarray(flat[g0:g0+H], dtype=float)
620 def stage_reserve_from_history(day_idx: int, stage_idx: int,
621                               stage_net_error_hist: np.ndarray,
622                               fallback_reserve_day: np.ndarray,
623                               stage_hour: int, cfg: Config) -> np.ndarray:
624     H = cfg.roll_slots
625     # 为确保任意18:00发布的24h预报已经完全实现, 统一只使用 d-2 及更早记录。
626     end = max(0, day_idx - 1)
627     start = max(0, end - cfg.stage_rhisdays)
628     hist = stage_net_error_hist[start:end, stage_idx, :]
629     # 问题二备用仅有当天144点。对跨日窗口按时钟位置循环展开作为冷启动基线。
630     fb = np.asarray(fallback_reserve_day, dtype=float)
631     clock = (stage_hour * 6 + np.arange(H)) % cfg.slots_pday
632     fallback = fb[clock]
633     if hist.shape[0] < cfg.stage_mindays:
634         return fallback.copy()
635     out = np.zeros(H, dtype=float)
636     age = np.arange(hist.shape[0]-1, -1, -1)
637     day_w = cfg.stage_decay ** age
638     for k in range(H):
639         vals = hist[:, k]
640         mask = np.isfinite(vals)
641         if int(np.sum(mask)) < cfg.stage_mindays:
642             out[k] = fallback[k]
643             continue
644         q = weighted_quantile(vals[mask], day_w[mask], cfg.stage_quant)
645         out[k] = cfg.stage_scale * max(0.0, float(q))
646     # 防止极端小样本产生不合理备用; 上限取储能/购电系统量级内的保守值。
647     return np.clip(out, 0.0, 5000.0)
648 def archive_stage_forecast_errors(forecast_day: int,
649                                   load_fc_full_store: np.ndarray,
650                                   pv_hourly: np.ndarray,
651                                   load: np.ndarray, pv: np.ndarray,
652                                   stage_net_error_hist: np.ndarray,
653                                   cfg: Config) -> None:
654     """在某日24小时预报窗口完全实现后, 归档0/6/12/18四个时点的净负荷误差。"""
655     if forecast_day < 0 or forecast_day >= len(load_fc_full_store):
656         return
657     base_full = load_fc_full_store[forecast_day]
658     if not np.all(np.isfinite(base_full)):
659         return
660     for s, h in enumerate((0, 6, 12, 18)):
661         lf = load_stage_forecast_horizon(base_full, load[forecast_day], h, cfg)
662         pf = pv_stage_forecast_horizon_10min(forecast_day, h, pv_hourly, pv, cfg)
663         la = actual_horizon_from_arrays(forecast_day, h, load, cfg)
664         pa = actual_horizon_from_arrays(forecast_day, h, pv, cfg)
665         if la is None or pa is None:
666             continue
667         stage_net_error_hist[forecast_day, s, :] = (la - pa) - (lf - pf)
668 def solve_rolling_mpc_24h(price: np.ndarray,
669                            load_fc_horizon: np.ndarray,
670                            pv_fc_horizon: np.ndarray,
671                            reserve_horizon: np.ndarray,
672                            soc0: float,
673                            prev_buy_day: np.ndarray,
674                            start_slot: int,
675                            cfg: Config) -> Dict[str, np.ndarray]:
676     H = cfg.roll_slots
677     if not (len(load_fc_horizon) == len(pv_fc_horizon) == len(reserve_horizon) == H):
678         raise ValueError("24小时MPC输入必须全部为144个10分钟时段。")
679     current_remaining = cfg.slots_pday - start_slot
680     # 变量: [buy, ch, dis, sur, E, up, down], 每组H个变量。
681     N = 7 * H
682     ib, ic, idis, isu, ie, iup, idn = 0, H, 2*H, 3*H, 4*H, 5*H, 6*H
683     rows, cols, data, b = [], [], [], []
684     r = 0
685     net = np.asarray(load_fc_horizon) - np.asarray(pv_fc_horizon) + np.asarray(reserve_horizon)
686     # 功率平衡
687     for j in range(H):
688         for col, val in ((ib+j,1.0),(idis+j,1.0),(ic+j,-1.0),(isu+j,-1.0)):
689             rows.append(r); cols.append(col); data.append(val)
690             b.append(float(net[j])); r += 1
691     # SOC递推
692     for j in range(H):
693         rows.append(r); cols.append(ie+j); data.append(1.0)
694         if j > 0:

```

```

695         rows.append(r); cols.append(ie+j-1); data.append(-1.0)
696         rhs = 0.0
697     else:
698         rhs = float(soc0)
699         rows.append(r); cols.append(ic+j); data.append(-cfg.eta_c*cfg.dt)
700         rows.append(r); cols.append(idis+j); data.append(cfg.dt/cfg.eta_d)
701         b.append(rhs); r += 1
702 # 仅当天剩余区间存在“与上一版已确认计划的调整关系”
703 for j in range(current_remaining):
704     rows.append(r); cols.append(ib+j); data.append(1.0)
705     rows.append(r); cols.append(iup+j); data.append(-1.0)
706     rows.append(r); cols.append(idn+j); data.append(1.0)
707     b.append(float(prev_buy_day[start_slot+j])); r += 1
708 Aeq = sparse.csr_matrix((data,(rows,cols)), shape=(r,N))
709 clock = (start_slot + np.arange(H)) % cfg.slots_pday
710 ph = np.asarray(price)[clock]
711 c = np.zeros(N)
712 # 当前日剩余区间只计相对既有合同的调整净费用。
713 c[iup:iup+current_remaining] = 1.5 * ph[:current_remaining] * cfg.dt
714 c[idn:idn+current_remaining] = -0.5 * ph[:current_remaining] * cfg.dt
715 # 下一日区间尚未签订，以正常电价形成“影子购电成本”，用于跨日SOC前瞻。
716 if current_remaining < H:
717     c[ib+current_remaining:ib+H] = ph[current_remaining:] * cfg.dt
718 c[ic:ic+H] = cfg.eps_cycle * cfg.dt
719 c[idis:idis+H] = cfg.eps_cycle * cfg.dt
720 c[isu:isu+H] = cfg.eps_surplus * cfg.dt
721 # 24h窗口末端仍赋予剩余电量机会价值，降低滚动窗口末端效应。
722 mu = cfg.eta_d * float(np.min(price))
723 c[ie+H-1] = -mu
724 bounds = []
725 bounds += [(0,None)] * H
726 bounds += [(0,cfg.p_max)] * H
727 bounds += [(0,cfg.p_max)] * H
728 bounds += [(0,None)] * H
729 bounds += [(cfg.e_min,cfg.e_max)] * H
730 # up/down只允许当前日剩余时段为正，跨日影子区间固定为0。
731 bounds += [(0,None) if j < current_remaining else (0,0) for j in range(H)]
732 bounds += [(0,None) if j < current_remaining else (0,0) for j in range(H)]
733 res = linprog(c, A_eq=Aeq, b_eq=np.asarray(b), bounds=bounds, method=cfg.lp_method)
734 if not res.success:
735     raise RuntimeError(f"{start_slot//6}:00 24小时MPC失败: {res.message}")
736 x = res.x
737 up = x[iup:iup+H]
738 dn = x[idn:idn+H]
739 adj_net = float(np.sum(
740     (1.5*ph[:current_remaining]*up[:current_remaining]
741     - 0.5*ph[:current_remaining]*dn[:current_remaining]) * cfg.dt
742 ))
743 return {
744     "buy": x[ib:ib+H],
745     "ch": x[ic:ic+H],
746     "dis": x[idis:idis+H],
747     "sur": x[isu:isu+H],
748     "E": x[ie:ie+H],
749     "up": up,
750     "down": dn,
751     "adj_net_cost": adj_net,
752     "up_energy": float(np.sum(up[:current_remaining]) * cfg.dt),
753     "down_energy": float(np.sum(dn[:current_remaining]) * cfg.dt),
754     "current_remaining": current_remaining,
755 }
756 def fill_result3(template_path, out_path, dates, price, initial_buy, effective_buy,
757     buy_cost_after_adjust, ch, dis, soc_start, soc_end, emg, cfg, logger):
758     from copy import copy
759     wb=load_workbook(template_path)
760     sp=wb["计划购电量"]; sa=wb["调整购电量"]; sc=wb["充放电量"]; se=wb["紧急购电量"]
761     date_to_idx={d:i for i,d in enumerate(dates)}
762     computed=[d for d in dates if d>=date(2025,2,1) and np.isfinite(initial_buy[date_to_idx[d]])
763     .all()]
764     def fill_buy_sheet(ws, arr, cost_arr):
765         for r in range(2, ws.max_row+1):
766             raw=ws.cell(r,1).value
767             if raw is None: continue
768             try: d=cell_to_date(raw)
769             except Exception: continue
770             if d not in date_to_idx: continue
771             i=date_to_idx[d]
772             if not np.isfinite(arr[i]).all(): continue
773             vals=arr[i]*cfg.dt
774             for c,v in enumerate(vals,start=2): ws.cell(r,c).value=float(v)
775             ws.cell(r,146).value=float(np.sum(vals))
776             ws.cell(r,147).value=float(cost_arr[i])
777     initial_cost=np.nansum(initial_buy*price[None,:],axis=1)*cfg.dt

```

```

777 fill_buy_sheet(sp, initial_buy, initial_cost)
778 fill_buy_sheet(sa, effective_buy, buy_cost_after_adjust)
779 charge_proto=[{"height":sc.row_dimensions[rr].height,
780               "cells":[sc.cell(rr,c) for c in range(1,7)]} for rr in range(2,8)]
781 emg_proto=[{"height":se.row_dimensions[rr].height,
782            "cells":[se.cell(rr,c) for c in range(1,4)]} for rr in range(2,min(5,se.max_row
+1))]
783 def cpstyle(src,dst):
784     if src.has_style: dst._style=copy(src._style)
785     dst.font=copy(src.font); dst.fill=copy(src.fill); dst.border=copy(src.border)
786     dst.alignment=copy(src.alignment); dst.protection=copy(src.protection)
787     dst.number_format=src.number_format
788     if sc.max_row>=2: sc.delete_rows(2,sc.max_row-1)
789     intervals=[(0,4,"0:00-4:00"),(4,8,"4:00-8:00"),(8,12,"8:00-12:00"),
790               (12,16,"12:00-16:00"),(16,20,"16:00-20:00"),(20,24,"20:00-24:00")]
791     rr=2
792     for d in computed:
793         i=date_to_idx[d]
794         for k,(h0,h1,label) in enumerate(intervals):
795             proto=charge_proto[k]
796             sc.row_dimensions[rr].height=proto["height"]
797             for c in range(1,7): cpstyle(proto["cells"][c-1],sc.cell(rr,c))
798             sc.cell(rr,1).value=d if k==0 else None; sc.cell(rr,2).value=label
799             a,b=h0*6,h1*6
800             sc.cell(rr,3).value=float(np.sum(ch[i,a:b])*cfg.dt)
801             sc.cell(rr,4).value=float(np.sum(dis[i,a:b])*cfg.dt)
802             if k==0: sc.cell(rr,5).value="0:00"; sc.cell(rr,6).value=float(soc_start[i])
803             elif k==1: sc.cell(rr,5).value="24:00"; sc.cell(rr,6).value=float(soc_end[i])
804             rr+=1
805     if se.max_row>=2: se.delete_rows(2,se.max_row-1)
806     rr=2
807     for d in computed:
808         i=date_to_idx[d]
809         events=_emergency_events(emg[i],cfg.dt) or [(None,None)]
810         for k,(period,energy) in enumerate(events):
811             proto=emg_proto[min(k,len(emg_proto)-1)]
812             se.row_dimensions[rr].height=proto["height"]
813             for c in range(1,4):cpstyle(proto["cells"][c-1],se.cell(rr,c))
814             se.cell(rr,1).value=d if k==0 else None; se.cell(rr,2).value=period; se.cell(rr,3).
value=energy
815             rr+=1
816     for ws in (sc,se):
817         for r in range(2,ws.max_row+1):
818             if ws.cell(r,1).value is not None: ws.cell(r,1).number_format='yyyy/m/d'
819     Path(out_path).parent.mkdir(parents=True,exist_ok=True)
820     wb.save(out_path); wb.close()
821 PAPER_TABLE1_INTERVALS = ("10:00-10:10", "12:00-12:10", "14:00-14:10",
822                           "16:00-16:10", "18:00-18:10", "20:00-20:10")
823 PAPER_TABLE2_INTERVALS = ("0:00-4:00", "4:00-8:00", "8:00-12:00",
824                           "12:00-16:00", "16:00-20:00", "20:00-24:00")
825 def _interval_start_slot(label: str) -> int:
826     hh, mm = label.split("-")[0].split(":")
827     return (int(hh) * 60 + int(mm)) // 10 - 1
828 def _slot_label(k: int) -> str:
829     mins = k * 10
830     if mins == 1440:
831         return "24:00"
832     return f"({mins // 60}) % 24}:{mins % 60:02d}"
833 def _emergency_events(emg_row: np.ndarray, dt: float) -> list:
834     """把逐时段紧急购电功率聚合成[(时间段, 购电量kWh), ...]。"""
835     events = []
836     t = 0
837     n = len(emg_row)
838     while t < n:
839         if emg_row[t] > 1e-6:
840             st = t
841             ee = 0.0
842             while t < n and emg_row[t] > 1e-6:
843                 ee += float(emg_row[t]) * dt
844                 t += 1
845             events.append((f"_{slot_label(st)}-{_slot_label(t)}", ee))
846         else:
847             t += 1
848     return events
849 def export_paper_tables(dates, price, initial_buy, effective_buy, ch, dis, emg,
850                        soc_start, soc_end, plan_cost, adj_cost, emg_cost,
851                        run_D, cfg, logger):
852     from openpyxl import Workbook
853     from openpyxl.styles import Font, Alignment, Border, Side
854     out_path = Path(cfg.out_dir) / "Q3指定日期结果.xlsx"
855     d2i = {d: i for i, d in enumerate(dates)}
856     targets = [d for d in cfg.represent_dates if d in d2i and d2i[d] < run_D]
857     if not targets:
858         logger.warning("【论文表格】表3指定日期不在本次运行范围内, 跳过导出。")

```

```

859     return None
860 dt = cfg.dt
861 thin = Side(style="thin", color="000000")
862 border = Border(left=thin, right=thin, top=thin, bottom=thin)
863 bold = Font(bold=True)
864 title_font = Font(bold=True, size=12)
865 center = Alignment(horizontal="center", vertical="center")
866 num = "#,##0.00"
867 def put(ws, r, c, v, font=None, align=None, bd=True, fmt=None):
868     cell = ws.cell(row=r, column=c, value=v)
869     if font is not None:
870         cell.font = font
871     if align is not None:
872         cell.alignment = align
873     if bd:
874         cell.border = border
875     if fmt is not None:
876         cell.number_format = fmt
877     return cell
878 wb = Workbook()
879 wb.remove(wb.active)
880 # 列宽
881 colw = {"A": 15, "B": 14, "C": 14, "D": 14, "E": 14, "F": 14, "G": 14, "H": 14}
882 # ----- 表1 -----
883 ws = wb.create_sheet("表1 购电量")
884 for col, w in colw.items():
885     ws.column_dimensions[col].width = w
886 r = 1
887 for d in targets:
888     i = d2i[d]
889     ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=6)
890     put(ws, r, 1, f"表1 微网在指定时间段的购电量及全天的购电量和购电费 ({d.strftime('%Y-%m-%d')})",
891         title_font, center, bd=False)
892     r += 1
893     for c, v in enumerate(["时间段", "购电量(kWh)", "时间段", "购电量(kWh)", "时间段", "购电
894 量(kWh)"], start=1):
895         put(ws, r, c, v, bold, center)
896     r += 1
897     vals = [float(effective_buy[i, _interval_start_slot(lb)]) * dt for lb in
898 PAPER_TABLE1_INTERVALS]
899     grid = [(0, 1, 2), (3, 4, 5)]
900     for tri in grid:
901         for col_off, idx in enumerate(tri):
902             put(ws, r, 1 + 2 * col_off, PAPER_TABLE1_INTERVALS[idx], None, center)
903             put(ws, r, 2 + 2 * col_off, vals[idx], None, center, fmt=num)
904         r += 1
905     day_buy = float(np.nansum(effective_buy[i]) * dt)
906     day_cost = float(plan_cost[i] + adj_cost[i] + emg_cost[i])
907     put(ws, r, 1, "全天购电量", bold, center)
908     put(ws, r, 2, day_buy, None, center, fmt=num)
909     put(ws, r, 3, "全天购电费(元)", bold, center)
910     put(ws, r, 4, day_cost, None, center, fmt=num)
911     put(ws, r, 5, None, None, center)
912     put(ws, r, 6, None, None, center)
913     r += 2
914 # ----- 表2 -----
915 ws = wb.create_sheet("表2 充放电电量")
916 for col, w in colw.items():
917     ws.column_dimensions[col].width = w
918 r = 1
919 ivs = [(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]
920 for d in targets:
921     i = d2i[d]
922     ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=6)
923     put(ws, r, 1, f"表2 储能设备在指定时间段的充放电电量及0:00和24:00的储电量 ({d.strftime('%Y
924 -%m-%d')})",
925         title_font, center, bd=False)
926     r += 1
927     for c, v in enumerate(["时间段", "充电量(kWh)", "放电量(kWh)", "时间段", "充电量(kWh)",
928 "放电量(kWh)"], start=1):
929         put(ws, r, c, v, bold, center)
930     r += 1
931     ch_e = [float(np.sum(ch[i, h0 * 6:h1 * 6]) * dt) for h0, h1 in ivs]
932     dis_e = [float(np.sum(dis[i, h0 * 6:h1 * 6]) * dt) for h0, h1 in ivs]
933     for a, b in ((0, 1), (2, 3), (4, 5)):
934         put(ws, r, 1, PAPER_TABLE2_INTERVALS[a], None, center)
935         put(ws, r, 2, ch_e[a], None, center, fmt=num)
936         put(ws, r, 3, dis_e[a], None, center, fmt=num)
937         put(ws, r, 4, PAPER_TABLE2_INTERVALS[b], None, center)
938         put(ws, r, 5, ch_e[b], None, center, fmt=num)
939         put(ws, r, 6, dis_e[b], None, center, fmt=num)
940     r += 1

```

```

937     put(ws, r, 1, "0:00 储电量", bold, center)
938     put(ws, r, 2, float(soc_start[i]), None, center, fmt=num)
939     put(ws, r, 3, "24:00 储电量", bold, center)
940     put(ws, r, 4, float(soc_end[i]), None, center, fmt=num)
941     put(ws, r, 5, None, None, center)
942     put(ws, r, 6, None, None, center)
943     r += 2
944 # ----- 表3 -----
945 ws = wb.create_sheet("表3_紧急购电量")
946 for col, w in colw.items():
947     ws.column_dimensions[col].width = w
948 events_per_date = [_emergency_events(emg[d2i[d]], dt) for d in targets]
949 ncol = 2 * len(targets)
950 ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=ncol)
951 put(ws, 1, 1, "表3 微网在指定日期的紧急购电量", title_font, center, bd=False)
952 for j, d in enumerate(targets):
953     c0 = 1 + 2 * j
954     ws.merge_cells(start_row=2, start_column=c0, end_row=2, end_column=c0 + 1)
955     put(ws, 2, c0, d.strftime("%Y-%m-%d"), bold, center)
956     put(ws, 2, c0 + 1, None, bold, center)
957     for j in range(len(targets)):
958         put(ws, 3, 1 + 2 * j, "时间段", bold, center)
959         put(ws, 3, 2 + 2 * j, "购电量(kWh)", bold, center)
960 nrow = max(3, max((len(e) for e in events_per_date), default=0))
961 r = 4
962 for k in range(nrow):
963     for j, evs in enumerate(events_per_date):
964         c0 = 1 + 2 * j
965         if k < len(evs):
966             put(ws, r, c0, evs[k][0], None, center)
967             put(ws, r, c0 + 1, evs[k][1], None, center, fmt=num)
968         else:
969             put(ws, r, c0, None, None, center)
970             put(ws, r, c0 + 1, None, None, center)
971     r += 1
972 # ----- 附表：费用与“0点计划/最终”对照 -----
973 ws = wb.create_sheet("附_费用与计划对照")
974 for col in "ABCDEFGH":
975     ws.column_dimensions[col].width = 18
976 heads = ["日期", "计划全天购电量(kWh)", "最终全天购电量(kWh)", "计划购电费(元)",
977          "调整净费用(元)", "紧急购电费(元)", "全天总费用(元)", "紧急购电量(kWh)"]
978 for c, v in enumerate(heads, start=1):
979     put(ws, 1, c, v, bold, center)
980 r = 2
981 for d in targets:
982     i = d2i[d]
983     row = [d.strftime("%Y-%m-%d"),
984            float(np.nansum(initial_buy[i]) * dt),
985            float(np.nansum(effective_buy[i]) * dt),
986            float(plan_cost[i]), float(adj_cost[i]), float(emg_cost[i]),
987            float(plan_cost[i] + adj_cost[i] + emg_cost[i]),
988            float(np.sum(emg[i]) * dt)]
989     for c, v in enumerate(row, start=1):
990         put(ws, r, c, v, None, center if c > 1 else None, fmt=(None if c == 1 else num))
991     r += 1
992 r += 2
993 put(ws, r, 1, "指定时段：0:00计划购电量与最终购电量对照", bold)
994 r += 1
995 for c, v in enumerate(["日期", "时间段", "计划购电量(kWh)", "最终购电量(kWh)"], start=1):
996     put(ws, r, c, v, bold, center)
997 r += 1
998 for d in targets:
999     i = d2i[d]
1000     for lb in PAPER_TABLE1_INTERVALS:
1001         s = _interval_start_slot(lb)
1002         put(ws, r, 1, d.strftime("%Y-%m-%d"), None, center)
1003         put(ws, r, 2, lb, None, center)
1004         put(ws, r, 3, float(initial_buy[i, s]) * dt, None, center, fmt=num)
1005         put(ws, r, 4, float(effective_buy[i, s]) * dt, None, center, fmt=num)
1006     r += 1
1007 # ----- 说明 -----
1008 ws = wb.create_sheet("说明")
1009 ws.column_dimensions["A"].width = 110
1010 notes = [
1011     "问题3 论文表格（表1/表2/表3）—— 指定日期：2025.3.20、2025.6.21、2025.9.23、2025.12.21",
1012     "，",
1013     "数据来源：Q3结果/result3_filled_forecast_driven.xlsx 同源的计算结果（调整后的最终策略）。",
1014     "表1 购电量：取“调整后的最终购电量”（0:00计划经6:00/12:00/18:00预报调整后的结果）。",
1015     "表1 全天购电费：= 计划购电费 + 调整净费用 + 紧急购电费（问题3总购电费口径）。",
1016     "表2 充放电电量：为指定4小时时间段的最终充电量与放电量；0:00/24:00 储电量为当日首末储电量。",
1017     "表3 紧急购电量：为最终紧急购电的时间段与电量；留空表示该日期没有发生紧急购电。",

```

```

1017     "附表：给出每日0:00计划与最终购电量的对照及费用拆分，便于论文中解释“调整”的作用。",
1018 ]
1019 for i, t in enumerate(notes, start=1):
1020     put(ws, i, 1, t, bold if i == 1 else None, bd=False)
1021 Path(out_path).parent.mkdir(parents=True, exist_ok=True)
1022 wb.save(out_path)
1023 wb.close()
1024 return str(out_path)
1025 NO_TITLE = True # 图内不带标题
1026 AXIS_FS = 15 # 坐标轴标签字号（黑色）
1027 TICK_FS = 12 # 刻度字号
1028 LEGEND_FS = 17 # 图例字号
1029 SOFT_RED, SOFT_GREEN, SOFT_BLUE = '#E9A6A1', '#8BCF8B', '#7097CA' # figures4papers 柔和色
1030 BAR_EDGE = dict(edgecolor='black', linewidth=1.2)
1031 def _style_y(ax):
1032     ax.grid(True, axis='y', linestyle='--', linewidth=0.7, alpha=0.3)
1033     ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
1034     ax.tick_params(labelsize=TICK_FS)
1035 def _set_labels(ax, x, y):
1036     ax.set_xlabel(x, fontsize=AXIS_FS, color='black')
1037     ax.set_ylabel(y, fontsize=AXIS_FS, color='black')
1038 def _legend_top(ax, ncol=None):
1039     h, l = ax.get_legend_handles_labels()
1040     if h:
1041         ax.legend(h, l, loc='lower center', frameon=False, ncol=ncol or len(h),
1042                 fontsize=LEGEND_FS, bbox_to_anchor=(0.5, 1.02), columnspacing=1.8)
1043 def save_q3_outputs(dates, price, initial_buy, effective_buy, ch, dis, emg, sur,
1044                   soc_start, soc_end, plan_cost, adj_cost, emg_cost,
1045                   up_energy, down_energy,
1046                   run_D, cfg, logger):
1047     out = Path(cfg.out_dir); fig = Path(cfg.fig_dir); out.mkdir(parents=True, exist_ok=True); fig.
1048     mkdir(parents=True, exist_ok=True)
1049     if run_D < 32: return
1050     idx = [i for i in range(31, run_D)]
1051     x = [dates[i] for i in idx]
1052     # 01 SOC日末轨迹图已按需删除（月度聚合仍需 idx/x）
1053     months = range(2, 13); pc = []; ac = []; ec = []; tc = []
1054     for m in months:
1055         ids = [i for i in idx if dates[i].month == m]
1056         p0 = sum(plan_cost[i] for i in ids); aa = sum(adj_cost[i] for i in ids); ee = sum(emg_cost[i]
1057         for i in ids)
1058         pc.append(p0); ac.append(aa); ec.append(ee); tc.append(p0+aa+ee)
1059     xx = np.arange(11)
1060     # 02 月度总费用：不再把数量级差异很大的三个分量硬堆叠在一起。
1061     f, a = plt.subplots(figsize=(12, 5), dpi=300)
1062     a.bar(xx, np.array(tc)/1e4, width=0.55, color=SOFT_BLUE, alpha=0.95, zorder=3, **BAR_EDGE)
1063     a.set_xticks(xx, [f'{m}月' for m in months])
1064     if not NO_TITLE:
1065         a.set_title('完整滚动方案的月度总购电费用', fontsize=13, fontweight='bold')
1066     _set_labels(a, '月份', '总费用 / 万元')
1067     _style_y(a)
1068     f.tight_layout(); f.savefig(fig/'第三问月度总购电费用.png', dpi=300, bbox_inches='tight'); plt
1069     .close(f)
1070     # 03 调整净费用与紧急购电费用单独展示，避免被计划购电费淹没。
1071     f, a = plt.subplots(figsize=(12, 5), dpi=300)
1072     a.bar(xx-0.19, np.array(ac)/1e4, width=.38, color=SOFT_GREEN, alpha=0.95, label='调整净费用',
1073           zorder=3, **BAR_EDGE)
1074     a.bar(xx+0.19, np.array(ec)/1e4, width=.38, color=SOFT_RED, alpha=0.95, label='紧急购电费', zorder
1075           =3, **BAR_EDGE)
1076     a.axhline(0, lw=.9, color='black')
1077     a.set_xticks(xx, [f'{m}月' for m in months])
1078     if not NO_TITLE:
1079         a.set_title('月度调整净费用与紧急购电费用', fontsize=13, fontweight='bold')
1080     _set_labels(a, '月份', '费用 / 万元')
1081     _style_y(a); _legend_top(a, ncol=2)
1082     f.tight_layout(); f.savefig(fig/'第三问月度调整净费用与紧急购电费用.png', dpi=300, bbox_inches
1083     = 'tight'); plt.close(f)
1084     formal = idx
1085     total_plan = sum(plan_cost[i] for i in formal); total_adj = sum(adj_cost[i] for i in formal);
1086     total_emg = sum(emg_cost[i] for i in formal)
1087     with open(out/'Q3_结果摘要.txt', 'w', encoding='utf-8') as f:
1088         f.write(f'问题3正式结果摘要（1月仅作为预热期）\n'+ '*55+'\n')
1089         f.write(f'正式统计日期范围：{dates[31]} 至 {dates[run_D-1]}\n')
1090         f.write(f'累计初始计划购电费：{total_plan:.2f} 元\n')
1091         f.write(f'累计调整净费用：{total_adj:.2f} 元\n')
1092         f.write(f'累计紧急购电费：{total_emg:.2f} 元\n')
1093         f.write(f'累计总费用：{total_plan+total_adj+total_emg:.2f} 元\n')
1094         f.write(f'累计紧急购电量：{sum(np.sum(emg[i])*cfg.dt for i in formal):.2f} kWh\n')
1095         f.write(f'累计富余电量：{sum(np.sum(sur[i])*cfg.dt for i in formal):.2f} kWh\n')
1096         f.write(f'累计向上调整电量：{sum(up_energy[i] for i in formal):.2f} kWh\n')
1097         f.write(f'累计向下调整电量：{sum(down_energy[i] for i in formal):.2f} kWh\n')
1098         f.write(f'期末储电量：{soc_end[run_D-1]:.2f} kWh\n')

```

```

1092 def simulate_update_scheme(update_hours, dates, price, load, pv,
1093                             load_fc_full_store, pv_hourly,
1094                             fallback_reserve_store, stage_net_error_hist,
1095                             run_D, cfg, lp):
1096     n = cfg.slots_pday
1097     update_hours = tuple(sorted(update_hours))
1098     plan_cost=np.zeros(run_D); adj_cost=np.zeros(run_D); emg_cost=np.zeros(run_D)
1099     emg_energy=np.zeros(run_D); surplus_energy=np.zeros(run_D)
1100     up_energy=np.zeros(run_D); down_energy=np.zeros(run_D)
1101     soc_start=np.zeros(run_D); soc_end=np.zeros(run_D)
1102     emg_days=np.zeros(run_D, dtype=int)
1103     daily_emg=np.zeros(run_D)
1104     soc=cfg.e0
1105     stage_idx_map={0:0,6:1,12:2,18:3}
1106     for d in range(run_D):
1107         soc_start[d]=soc
1108         lf_full=load_fc_full_store[d]
1109         lf0=lf_full[:n]
1110         pf0=pv_stage_forecast_horizon_10min(d,0,pv_hourly,pv,cfg)[:n]
1111         fallback_reserve=fallback_reserve_store[d]
1112         # 0:00计划仍按当天合同生成; 0点风险备用沿用问题二动态模块,
1113         # 保证与第二问的基准逻辑连续。
1114         da=solve_day_ahead(price,lf0,pf0,fallback_reserve,soc,cfg,lp)
1115         initial_buy=da['buy'].copy()
1116         current_buy=da['buy'].copy()
1117         current_ch=da['ch'].copy()
1118         current_dis=da['dis'].copy()
1119         plan_cost[d]=float(np.sum(initial_buy*price)*cfg.dt)
1120         stages=(0,)+update_hours
1121         for si,h in enumerate(stages):
1122             start=h*6
1123             if h>0:
1124                 lf_h=load_stage_forecast_horizon(lf_full,load[d],h,cfg)
1125                 pf_h=pv_stage_forecast_horizon_10min(d,h,pv_hourly,pv,cfg)
1126                 reserve_h=stage_reserve_from_history(
1127                     d,stage_idx_map[h],stage_net_error_hist,
1128                     fallback_reserve,h,cfg
1129                 )
1130                 m=n-start
1131                 adj=solve_rolling_mpc_24h(
1132                     price,lf_h,pf_h,reserve_h,soc,current_buy,start,cfg
1133                 )
1134                 current_buy[start:]=adj['buy'][:m]
1135                 current_ch[start:]=adj['ch'][:m]
1136                 current_dis[start:]=adj['dis'][:m]
1137                 adj_cost[d]+=adj['adj_net_cost']
1138                 up_energy[d]+=adj['up_energy']
1139                 down_energy[d]+=adj['down_energy']
1140             end=(stages[si+1]*6) if si+1<len(stages) else n
1141             for t in range(start,end):
1142                 sol=operate_one_slot(
1143                     soc,current_buy[t],load[d,t],pv[d,t],
1144                     current_ch[t],current_dis[t],cfg
1145                 )
1146                 soc=sol['E']
1147                 emg_energy[d]+=sol['emg']*cfg.dt
1148                 surplus_energy[d]+=sol['sur']*cfg.dt
1149                 emg_cost[d]+=sol['emg']*5*price[t]*cfg.dt
1150             soc_end[d]=soc
1151             daily_emg[d]=emg_energy[d]
1152             emg_days[d]=1 if emg_energy[d]>1e-6 else 0
1153     return {
1154         'plan_cost':plan_cost,'adj_cost':adj_cost,'emg_cost':emg_cost,
1155         'emg_energy':emg_energy,'surplus_energy':surplus_energy,
1156         'up_energy':up_energy,'down_energy':down_energy,
1157         'soc_start':soc_start,'soc_end':soc_end,'emg_days':emg_days,
1158         'daily_emg':daily_emg,
1159     }
1160 def run_update_time_comparison(dates, price, load, pv,
1161                                 load_fc_full_store, pv_hourly,
1162                                 fallback_reserve_store, stage_net_error_hist,
1163                                 run_D, cfg, lp, logger):
1164     """比较仅0点、单独增加6/12/18点以及完整滚动方案。重点突出0点与12点。"""
1165     if run_D<32:
1166         return None
1167     out=Path(cfg.out_dir); fig=Path(cfg.fig_dir)
1168     out.mkdir(parents=True,exist_ok=True); fig.mkdir(parents=True,exist_ok=True)
1169     formal=slice(31,run_D)
1170     results=[]; raw={}
1171     for name, hours in zip(cfg.compare_names, cfg.compare_schemes):
1172         r=simulate_update_scheme(
1173             hours, dates, price, load, pv, load_fc_full_store, pv_hourly,
1174             fallback_reserve_store, stage_net_error_hist, run_D, cfg, lp
1175         )

```

```

1176 raw[name]=r
1177 plan=float(np.sum(r['plan_cost'][formal]))
1178 adj=float(np.sum(r['adj_cost'][formal]))
1179 emgc=float(np.sum(r['emg_cost'][formal]))
1180 total=plan+adj+emgc
1181 daily=np.asarray(r['emg_energy'][31:run_D],dtype=float)
1182 results.append({
1183     '方案':name,
1184     '更新时点':','.join(map(str,hours)) if hours else '无日内更新',
1185     '初始计划购电费':plan, '调整净费用':adj,
1186     '紧急购电费':emgc, '总费用':total,
1187     '紧急购电量kWh':float(np.sum(daily)),
1188     '富余电量kWh':float(np.sum(r['surplus_energy'][formal])),
1189     '向上调整电量kWh':float(np.sum(r['up_energy'][formal])),
1190     '向下调整电量kWh':float(np.sum(r['down_energy'][formal])),
1191     '紧急购电天数':int(np.sum(daily>1e-6)),
1192     '单日最大紧急购电量kWh':float(np.max(daily)) if len(daily) else 0.0,
1193     '日紧急购电量P95kWh':float(np.quantile(daily,0.95)) if len(daily) else 0.0,
1194     '期末SOC':float(r['soc_end'][run_D-1]),
1195 })
1196 names=[x['方案'] for x in results]
1197 x=np.arange(len(names))
1198 # 04: 相对0点方案总费用变化, 比从0开始画总费用更有辨识度。
1199 base_total=results[0]['总费用']
1200 vals=np.array([r['总费用']-base_total for r in results])/1e4
1201 f,a=plt.subplots(figsize=(10.8,5.4),dpi=300)
1202 a.bar(x,vals,width=0.40,color=SOFT_GREEN,alpha=0.95,zorder=3,**BAR_EDGE); a.axhline(0,lw=.9,
1203     color='black')
1204 a.set_xticks(x,names); a.tick_params(labelsize=TICK_FS)
1205 if not NO_TITLE:
1206     a.set_title('不同预报更新时点的经济性增益', fontsize=13, fontweight='bold')
1207 a.set_xlabel('预报更新时点方案', fontsize=AXIS_FS, color='black')
1208 a.set_ylabel('相对仅0:00方案的总费用变化 / 万元', fontsize=AXIS_FS, color='black')
1209 for i,v in enumerate(vals):
1210     a.text(i,v+(0.15 if v>=0 else -0.15),f'{v:+.2f}',ha='center',
1211         va='bottom' if v>=0 else 'top',fontsize=11)
1212 _style_y(a)
1213 f.tight_layout(); f.savefig(fig/'第三问不同预报更新时点的经济性增益.png',dpi=300,bbox_inches
1214     ='tight'); plt.close(f)
1215 # 05-08 预报时点方案对比图已按需删除(数据仍随 CSV 输出)
1216 # 09 月度调整净费用图已按需删除; 月度聚合保留供图10使用
1217 full=raw[cfg.compare_names[-1]]
1218 months=range(2,13); xx=np.arange(11)
1219 adjm=[]; upm=[]; dnm=[]
1220 for m in months:
1221     ids=[i for i in range(31,run_D) if dates[i].month==m]
1222     adjm.append(float(np.sum(full['adj_cost'][ids])))
1223     upm.append(float(np.sum(full['up_energy'][ids])))
1224     dnm.append(float(np.sum(full['down_energy'][ids])))
1225 # 10: 完整方案月度向上/向下调整量
1226 f,a=plt.subplots(figsize=(12,5),dpi=300)
1227 a.bar(xx-.19,np.array(upm)/1000,width=.38,color=SOFT_BLUE,alpha=0.95,label='向上调整',zorder
1228     =3,**BAR_EDGE)
1229 a.bar(xx+.19,np.array(dnm)/1000,width=.38,color=SOFT_RED,alpha=0.95,label='向下调整',zorder
1230     =3,**BAR_EDGE)
1231 a.set_xticks(xx,[f'{m}月' for m in months])
1232 if not NO_TITLE:
1233     a.set_title('完整滚动方案的月度向上与向下调整电量', fontsize=13, fontweight='bold')
1234 _set_labels(a,'月份','调整电量 / MWh')
1235 _style_y(a); _legend_top(a,ncol=2)
1236 f.tight_layout(); f.savefig(fig/'第三问向上与向下调整电量.png',dpi=300,bbox_inches='tight');
1237 plt.close(f)
1238 return results
1239 def save_representative_rolling_figures(dates, price, load, pv,
1240     load_fc_full_store, pv_hourly,
1241     fallback_reserve_store, stage_net_error_hist,
1242     soc_start, run_D, cfg, lp, logger):
1243     """指定日期可视化: 只展示有解释力的第三问新增信息。"""
1244     fig=Path(cfg.fig_dir); fig.mkdir(parents=True,exist_ok=True)
1245     date_to_idx={d:i for i,d in enumerate(dates)}
1246     n=cfg.slots_pday
1247     hours_x=(np.arange(n)+0.5)/6.0
1248     stage_idx_map={0:0,6:1,12:2,18:3}
1249     for target in cfg.represent_dates:
1250         if target not in date_to_idx:
1251             continue
1252         d=date_to_idx[target]
1253         if d>run_D or not np.isfinite(soc_start[d]):
1254             continue
1255         lf_full=load_fc_full_store[d]
1256         lf0=lf_full[:n]
1257         pf0=pv_stage_forecast_horizon_10min(d,0,pv_hourly,pv,cfg)[:n]

```

```

1253 fallback=fallback_reserve_store[d]
1254 soc=float(soc_start[d])
1255 da=solve_day_ahead(price,lf0,pf0,fallback,soc,cfg,lp)
1256 initial_buy=da['buy'].copy(); current_buy=da['buy'].copy()
1257 current_ch=da['ch'].copy(); current_dis=da['dis'].copy()
1258 executed=np.zeros(n)
1259 adjustments={6:np.zeros(n),12:np.zeros(n),18:np.zeros(n)}
1260 stages=(0,6,12,18)
1261 for si,h in enumerate(stages):
1262     start=h*6
1263     if h>0:
1264         before=current_buy.copy()
1265         lf_h=load_stage_forecast_horizon(lf_full,load[d],h,cfg)
1266         pf_h=pv_stage_forecast_horizon_10min(d,h,pv_hourly,pv,cfg)
1267         reserve_h=stage_reserve_from_history(
1268             d,stage_idx_map[h],stage_net_error_hist,
1269             fallback,h,cfg
1270         )
1271         adj=solve_rolling_mpc_24h(price,lf_h,pf_h,reserve_h,soc,current_buy,start,cfg)
1272         m=n-start
1273         current_buy[start:]=adj['buy'][:m]; current_ch[start:]=adj['ch'][:m]
1274         current_dis[start:]=adj['dis'][:m]
1275         adjustments[h][start:]=current_buy[start:]-before[start:]
1276     end=(stages[si+1]*6) if si+1<len(stages) else n
1277     executed[start:end]=current_buy[start:end]
1278     for t in range(start,end):
1279         sol=operate_one_slot(
1280             soc,current_buy[t],load[d,t],pv[d,t],
1281             current_ch[t],current_dis[t],cfg
1282         )
1283         soc=sol['E']
1284     tag=f'{target.month}-{target.day}'
1285     # 11A: 0点计划与最终执行, 避免5条高度重合曲线。
1286     f,a=plt.subplots(figsize=(12,5.4),dpi=300)
1287     a.plot(hours_x,initial_buy,lw=2.0,color=C_BUY,label='0:00初始计划')
1288     a.plot(hours_x,executed,lw=1.9,color=C_DIS,label='最终执行购电策略')
1289     a.fill_between(hours_x,initial_buy,executed,alpha=.14,color=C_BUY,label='滚动调整区间')
1290     for h in (6,12,18): a.axvline(h,ls=':',lw=.9,color='#888888',zorder=1)
1291     a.set_xlim(0,24); a.set_xticks(np.arange(0,25,2))
1292     if not NO_TITLE:
1293         a.set_title(f'{target} 初始计划与最终执行购电策略', fontsize=13, fontweight='bold')
1294     _set_labels(a,'时刻 / h','购电功率 / kW')
1295     _style_y(a)
1296     hd,lb=a.get_legend_handles_labels()
1297     a.legend([hd[1],hd[2],hd[0]], [lb[1],lb[2],lb[0]],loc='lower center',
1298             frameon=False,ncol=3,fontsize=LEGEND_FS,bbox_to_anchor=(0.5,1.02),
1299             columnspacing=1.8)
1300     f.tight_layout(); f.savefig(fig/f'第三问{tag}初始计划与最终执行购电策略.png',dpi=300,
1301     bbox_inches='tight'); plt.close(f)
1302     # 11B/12 图已按需删除
1303 def main():
1304     cfg=CFG
1305     out=Path(cfg.out_dir); out.mkdir(parents=True,exist_ok=True)
1306     logger=setup_logger(out)
1307     # 兼容本次上传文件名: 默认文件缺失时回退到带(3)后缀的副本
1308     for attr,alt in (('att1','附件1(3).xlsx'),('att2','附件2(3).xlsx'),('template','result3.xlsx')):
1309         cur=getattr(cfg,attr)
1310         if (not cur or not Path(cur).exists()) and (Path(cfg.base_dir)/alt).exists():
1311             setattr(cfg,attr,Path(cfg.base_dir)/alt)
1312     dates,price,load_prior,pv_prior,load,pv=load_inputs(cfg,logger)
1313     pv_hourly=load_attachment3(cfg,dates,logger)
1314     D=len(dates); n=cfg.slots_pday
1315     run_D=min(D,cfg.max_days) if cfg.max_days>0 else D
1316     initial_buy, effective_buy, soc_hist = (np.full((D,n),np.nan) for _ in range(3))
1317     ch, dis, emg, sur = (np.zeros((D,n)) for _ in range(4))
1318     soc_start, soc_end = np.full(D,np.nan), np.full(D,np.nan)
1319     net_errors, cplus_cache, cminus_cache = (np.full((D,n),np.nan) for _ in range(3))
1320     fallback_reserve_store = np.zeros((D,n))
1321     # 0点负荷预测保留未来48小时, 用于6/12/18点跨日24小时MPC。
1322     load_fc_full_store = np.full((D,cfg.fore_slots),np.nan)
1323     pv_fc0_store = np.full((D,n),np.nan)
1324     # [日期, 发布时点0/6/12/18, 未来144个10分钟时段]。
1325     stage_net_error_hist = np.full((D,4,cfg.roll_slots),np.nan)
1326     plan_cost=np.zeros(D); adj_cost=np.zeros(D); emg_cost=np.zeros(D)
1327     buy_cost_after_adjust=np.zeros(D)
1328     up_energy=np.zeros(D); down_energy=np.zeros(D)
1329     lp=build_lp_templates(price,cfg)
1330     soc=cfg.e0
1331     stage_idx_map={0:0,6:1,12:2,18:3}
1332     for d in range(run_D):
1333         soc_start[d]=soc
1334         # 到第d天0:00时, d-2日各发布时点的24小时预报已经完全实现, 可安全归档。

```

```

1334     if d>=2:
1335         archive_stage_forecast_errors(
1336             d-2,load_fc_full_store,pv_hourly,load,pv,
1337             stage_net_error_hist,cfg
1338         )
1339     # 0:00负荷预测：保留完整48小时。
1340     lf_full=load_forecast(d,load,load_prior,cfg,logger)
1341     if len(lf_full)<cfg.fore_slots:
1342         raise ValueError('负荷预测未返回完整48小时序列。')
1343     lf_full=np.asarray(lf_full[:cfg.fore_slots],dtype=float)
1344     load_fc_full_store[d]=lf_full
1345     lf0=lf_full[:n].copy()
1346     # 附件3 0:00未来24小时光伏预报。
1347     pf0_full=pv_stage_forecast_horizon_10min(d,0,pv_hourly,pv,cfg)
1348     pf0=pf0_full[:n].copy()
1349     pv_fc0_store[d]=pf0
1350     # 0:00计划沿用问题二动态风险模块，保证问题二→问题三模型连续。
1351     update_risk_cost_cache(
1352         d,cplus_cache,cminus_cache,net_errors,
1353         price,soc_hist,initial_buy,load,pv,cfg
1354     )
1355     fallback_reserve=dynamic_risk_weights(
1356         d,net_errors,cplus_cache,cminus_cache,cfg
1357     )[2]
1358     fallback_reserve_store[d]=fallback_reserve
1359     da=solve_day_ahead(price,lf0,pf0,fallback_reserve,soc,cfg,lp)
1360     initial_buy[d]=da['buy']; current_buy=da['buy'].copy()
1361     current_ch=da['ch'].copy(); current_dis=da['dis'].copy()
1362     effective_buy[d]=current_buy
1363     plan_cost[d]=float(np.sum(initial_buy[d]*price)*cfg.dt)
1364     # 0-6、6-12、12-18、18-24四段执行。
1365     stage_hours=(0,6,12,18)
1366     for si,h in enumerate(stage_hours):
1367         start=h*6
1368         if h>0:
1369             # 未来24小时预测窗口，允许跨越午夜。
1370             lf_h=load_stage_forecast_horizon(lf_full,load[d],h,cfg)
1371             pf_h=pv_stage_forecast_horizon_10min(d,h,pv_hourly,pv,cfg)
1372             reserve_h=stage_reserve_from_history(
1373                 d,stage_idx_map[h],stage_net_error_hist,
1374                 fallback_reserve,h,cfg
1375             )
1376             adj=solve_rolling_mpc_24h(
1377                 price,lf_h,pf_h,reserve_h,
1378                 soc,current_buy,start,cfg
1379             )
1380             # result3只允许修改当天剩余计划；跨日部分仅用于MPC前瞻，不落入当天合同。
1381             m=n-start
1382             current_buy[start:]=adj['buy'][:m]; current_ch[start:]=adj['ch'][:m]
1383             current_dis[start:]=adj['dis'][:m]
1384             adj_cost[d]+=adj['adj_net_cost']
1385             up_energy[d]+=adj['up_energy']
1386             down_energy[d]+=adj['down_energy']
1387             end=(stage_hours[si+1]*6) if si+1<len(stage_hours) else n
1388             effective_buy[d,start:end]=current_buy[start:end]
1389             for t in range(start,end):
1390                 soc_hist[d,t]=soc
1391                 sol=operate_one_slot(
1392                     soc,current_buy[t],load[d,t],pv[d,t],
1393                     current_ch[t],current_dis[t],cfg
1394                 )
1395                 ch[d,t]=sol['ch']; dis[d,t]=sol['dis']; emg[d,t]=sol['emg']
1396                 sur[d,t]=sol['sur']; soc=sol['E']
1397             soc_end[d]=soc
1398             emg_cost[d]=float(np.sum(emg[d]*5*price)*cfg.dt)
1399             buy_cost_after_adjust[d]=plan_cost[d]+adj_cost[d]
1400             # 0点预测误差仅用于问题二继承的风险模块与预测评价。
1401             net_errors[d]=(load[d]-pv[d])-(lf0-pf0)
1402             # 保存checkpoint，供Q3_2.py等下游脚本复用基线轨迹与阶段风险备用。
1403             np.savez(
1404                 Path(cfg.out_dir)/'q3_checkpoint_forecast_driven.npz',
1405                 soc_start=soc_start[:run_D], soc_end=soc_end[:run_D], last_day=run_D-1,
1406                 ch=ch[:run_D], dis=dis[:run_D], emg=emg[:run_D], sur=sur[:run_D],
1407                 initial_buy=initial_buy[:run_D], effective_buy=effective_buy[:run_D],
1408                 adj_cost=adj_cost[:run_D], up_energy=up_energy[:run_D],
1409                 down_energy=down_energy[:run_D],
1410                 fallback_reserve=fallback_reserve_store[:run_D],
1411                 stage_net_error_hist=stage_net_error_hist[:run_D]
1412             )
1413         if run_D>=32:
1414             fill_result3(
1415                 cfg.template,cfg.out_xlsx,dates,price,
1416                 initial_buy,effective_buy,buy_cost_after_adjust,

```

```

1417         ch,dis,soc_start,soc_end,emg,cfg,logger
1418     )
1419     save_q3_outputs(
1420         dates,price,initial_buy,effective_buy,ch,dis,emg,sur,
1421         soc_start,soc_end,plan_cost,adj_cost,emg_cost,
1422         up_energy,down_energy,
1423         run_D,cfg,logger
1424     )
1425     # 论文表格：按表1/表2/表3格式给出表3指定日期的结果，导出到Excel。
1426     if run_D>=32:
1427         export_paper_tables(
1428             dates,price,initial_buy,effective_buy,ch,dis,emg,
1429             soc_start,soc_end,plan_cost,adj_cost,emg_cost,
1430             run_D,cfg,logger
1431         )
1432     if run_D>=32:
1433         save_representative_rolling_figures(
1434             dates,price,load,pv,
1435             load_fc_full_store,pv_hourly,
1436             fallback_reserve_store,stage_net_error_hist,
1437             soc_start,run_D,cfg,lp,logger
1438         )
1439     if cfg.run_comparison and run_D>=32:
1440         run_update_time_comparison(
1441             dates,price,load,pv,
1442             load_fc_full_store,pv_hourly,
1443             fallback_reserve_store,stage_net_error_hist,
1444             run_D,cfg,lp,logger
1445         )
1446     print('问题三线性规划版运行结束。')
1447     if run_D>=32:
1448         formal=slice(31,run_D)
1449         tp=float(np.sum(plan_cost[formal]))
1450         ta=float(np.sum(adj_cost[formal]))
1451         te=float(np.sum(emg_cost[formal]))
1452         print(f'正式期（{dates[31]} 至 {dates[run_D-1]}）累计：初始计划购电费 {tp:.2f} 元，'
1453               f'调整净费用 {ta:.2f} 元，紧急购电费 {te:.2f} 元，总费用 {tp+ta+te:.2f} 元')
1454         print(f'result3: {cfg.out_xlsx}')
1455 if __name__ == "__main__":
1456     main()

```

code/Q3.py

Q3-2.py

```

1 from __future__ import annotations
2 import csv
3 import math
4 import importlib.util
5 import sys
6 from pathlib import Path
7 from datetime import datetime, date, timedelta
8 from typing import Dict, List, Tuple
9 import numpy as np
10 from openpyxl import load_workbook, Workbook
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"]
15 plt.rcParams["axes.unicode_minus"] = False; plt.rcParams["font.size"] = 12
16 plt.rcParams["axes.spines.top"] = False; plt.rcParams["axes.linewidth"] = 1.5
17 plt.rcParams["xtick.major.width"] = 1.5; plt.rcParams["ytick.major.width"] = 1.5
18 plt.rcParams["xtick.major.size"] = 6; plt.rcParams["ytick.major.size"] = 6
19 C_BUY, C_PV, C_DIS, C_LOAD = "#2E74B5", "#27AE60", "#E67E22", "#34495E"
20 C_CH, C_SOC = "#E74C3C", "#7F8C8D"
21 class RunConfig:
22     BASE = Path(__file__).resolve().parent if "__file__" in globals() else Path(".")
23     Q3_MAIN_CANDIDATES = [BASE / "Q3.py"]
24     ATT1_CANDIDATES = [BASE / "附件1.xlsx", BASE / "附件1(3).xlsx", BASE / "附件1(1).xlsx"]
25     ATT2_CANDIDATES = [BASE / "附件2.xlsx", BASE / "附件2(3).xlsx", BASE / "附件2(2).xlsx"]
26     ATT3_CANDIDATES = [BASE / "附件3.xlsx", BASE / "附件3(1).xlsx"]
27     CHECKPOINT_CANDIDATES = [
28         BASE / "Q3结果" / "q3_checkpoint_forecast_driven.npz",
29         BASE / "q3_outputs_forecast_driven" / "q3_checkpoint_forecast_driven.npz",
30         BASE / "q3_checkpoint_forecast_driven.npz", BASE / "q3_checkpoint_forecast_driven(1).npz"
31     ]
32     OUT = BASE / "Q3_2结果"
33     FIG = OUT / "Q3_2图片输出"
34     CACHE = OUT / "Q3_02_preprocess_cache.npz"
35     CANDIDATE_INTERVALS = ((0, 6), (6, 12), (12, 18))

```

```

36     MAX_EXTRA_HOURS, MIN_INTERVAL_RISK_SHARE = 2, 0.05
37     MANUAL_EXTRA_HOURS = None # 如需固定, 可改为 (10, 14)
38     RECENT_ERROR_HOURS, BIAS_DECAY_HOURS, HISTORY_DAYS_FOR_TAIL = 3, 4.0, 30
39     MAX_BIAS_ABS_KW, MAX_DAYS = 1500.0, 365
40     REPRESENTATIVE_DATES = (date(2025, 3, 20), date(2025, 6, 21),
41                             date(2025, 9, 23), date(2025, 12, 21))
42 def first_existing(paths: List[Path]) -> Path:
43     for p in paths:
44         if p.exists():
45             return p
46     raise FileNotFoundError("未找到文件: \n" + "\n".join(str(p) for p in paths))
47 def ensure_dir(p: Path):
48     p.mkdir(parents=True, exist_ok=True)
49 def import_py(path: Path, module_name: str):
50     spec = importlib.util.spec_from_file_location(module_name, str(path))
51     if spec is None or spec.loader is None:
52         raise ImportError(f"无法加载Python脚本: {path}")
53     mod = importlib.util.module_from_spec(spec)
54     sys.modules[module_name] = mod
55     spec.loader.exec_module(mod)
56     return mod
57 def write_csv(path: Path, rows: List[dict]):
58     if not rows:
59         return
60     ensure_dir(path.parent)
61     with open(path, "w", newline="", encoding="utf-8-sig") as f:
62         w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
63         w.writeheader(); w.writerows(rows)
64 def style(ax):
65     ax.grid(True, axis="y", linestyle="--", linewidth=0.7, alpha=0.3)
66     ax.spines["top"].set_visible(False)
67     ax.spines["right"].set_visible(False)
68     ax.tick_params(labelsize=TICK_FS)
69 NO_TITLE, AXIS_FS, TICK_FS, LEGEND_FS = True, 15, 12, 17
70 SOFT_RED, SOFT_GREEN, SOFT_BLUE, SOFT_GRAY = '#E9A6A1', '#8BCF8B', '#7097CA', '#CFCECE'
71 BAR_EDGE = dict(edgecolor='black', linewidth=1.2)
72 def _set_labels(ax, x, y):
73     ax.set_xlabel(x, fontsize=AXIS_FS, color='black')
74     ax.set_ylabel(y, fontsize=AXIS_FS, color='black')
75 def _legend_top(ax, ncol=None):
76     h, l = ax.get_legend_handles_labels()
77     if h:
78         ax.legend(h, l, loc='lower center', frameon=False, ncol=ncol or len(h),
79                 fontsize=LEGEND_FS, bbox_to_anchor=(0.5, 1.02), columnspacing=1.8)
80 def load_core_inputs(att1: Path, att2: Path, q3):
81     wb1 = load_workbook(att1, data_only=True, read_only=True)
82     ws1 = wb1[wb1.sheetnames[0]]
83     rows1 = list(ws1.iter_rows(min_row=2, max_row=145, min_col=2, max_col=4, values_only=True))
84     wb1.close()
85     price = np.asarray([float(r[0]) for r in rows1], dtype=float)
86     load_prior = np.asarray([float(r[1]) for r in rows1], dtype=float)
87     pv_prior = np.asarray([float(r[2]) for r in rows1], dtype=float)
88     wb2 = load_workbook(att2, data_only=True, read_only=True)
89     sl, sp = wb2["小区负载"], wb2["光伏发电实际功率"]
90     load_rows = list(sl.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=True))
91     pv_rows = list(sp.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=True))
92     wb2.close()
93     dates = [q3.cell_to_date(r[0]) for r in load_rows]
94     load = np.asarray([float(v) for v in r[1:145]] for r in load_rows, dtype=float)
95     pv = np.asarray([float(v) for v in r[1:145]] for r in pv_rows, dtype=float)
96     if price.shape != (144,) or load.shape != (365, 144) or pv.shape != (365, 144):
97         raise ValueError(f"输入尺寸异常: price={price.shape}, load={load.shape}, pv={pv.shape}")
98     return dates, price, load_prior, pv_prior, load, pv
99 def load_checkpoint(cp_path: Path, run_D: int):
100     d = np.load(cp_path, allow_pickle=True)
101     required = ["soc_start", "ch", "dis", "fallback_reserve", "stage_net_error_hist"]
102     missing = [k for k in required if k not in d.files]
103     if missing:
104         raise ValueError(
105             "Q3_01 checkpoint缺少字段: " + ", ".join(missing) +
106             "。请先用当前正式Q3主程序完整运行一次。"
107         )
108     out = {k: np.asarray(d[k]) for k in d.files}
109     if len(out["soc_start"]) < run_D:
110         raise ValueError(
111             f"checkpoint只运行到{len(out['soc_start'])}天, 但本程序要求{run_D}天。"
112             "请先把Q3_01完整运行到相同天数。"
113         )
114     return out
115 def reconstruct_soc_hourly(checkpoint: dict, run_D: int, cfg) -> np.ndarray:
116     """用Q3_01真实10分钟充放电轨迹重构每小时起点SOC, 供风险带使用。"""

```

```

117 soc_start = np.asarray(checkpoint["soc_start"][:run_D], dtype=float)
118 ch = np.asarray(checkpoint["ch"][:run_D], dtype=float)
119 dis = np.asarray(checkpoint["dis"][:run_D], dtype=float)
120 out = np.zeros((run_D, 24), dtype=float)
121 for d in range(run_D):
122     soc = float(soc_start[d])
123     for h in range(24):
124         out[d, h] = soc
125         for k in range(h * 6, h * 6 + 6):
126             soc += cfg.eta_c * ch[d, k] * cfg.dt - dis[d, k] * cfg.dt / cfg.eta_d
127             soc = min(max(soc, cfg.e_min), cfg.e_max)
128     return out
129 class RiskBandConfig:
130     DAYLIGHT_THRESHOLD, BAND_QUANTILE, BAND_MIN_KW, BAND_FLOOR_RATIO = 100.0, 0.85, 120.0, 0.08
131     USE_HOUR_SMOOTH, C_ES, C_SUR, K_SHORT = True, 0.03, 0.06, 5.0
132     K_DIS, K_CH, EPS = 1.5, 0.5, 1e-9
133 def load_forecast_records_xlsx(path: Path):
134     wb = load_workbook(path, data_only=True, read_only=True)
135     ws, current_date = wb[wb.sheetnames[0]], None
136     recs = []
137     for r in ws.iter_rows(min_row=2, max_col=26, values_only=True):
138         if r[0] not in (None, ""):
139             if isinstance(r[0], datetime):
140                 current_date = r[0].date()
141             elif isinstance(r[0], date):
142                 current_date = r[0]
143             else:
144                 current_date = datetime.strptime(str(r[0]).replace('/', '-'))[:10], '%Y-%m-%d').
145         date()
146         if current_date is None:
147             continue
148         txt = str(r[1]).strip()
149         issue_hour = int(txt.split(':')[0]) if ':' in txt else int(float(txt))
150         fc = np.asarray([0.0 if v is None else float(v) for v in r[2:26]], dtype=float)
151         issue_dt = datetime.combine(current_date, datetime.min.time()) + timedelta(hours=
152         issue_hour)
153         recs.append({
154             'issue_dt': issue_dt,
155             'issue_hour': issue_hour,
156             'fc': np.maximum(fc, 0.0),
157         })
158     wb.close()
159     recs.sort(key=lambda z: z['issue_dt'])
160     return recs
161 def build_target_times(dates: List[date]):
162     return [datetime.combine(dd, datetime.min.time()) + timedelta(hours=h)
163             for dd in dates for h in range(1, 25)]
164 def effective_forecast_records(records: List[dict], target_times: List[datetime]):
165     values = np.full(len(target_times), np.nan)
166     for i, t in enumerate(target_times):
167         candidates = []
168         for r in records:
169             if r['issue_dt'] >= t:
170                 break
171             val, _ = record_value_at_target(r, t)
172             if val is not None:
173                 candidates.append((r['issue_dt'], val))
174         if candidates:
175             values[i] = max(candidates, key=lambda z: z[0])[1]
176     if np.isnan(values).any():
177         bad = np.where(np.isnan(values))[0]
178         raise ValueError(f'有{len(bad)}个目标时刻缺少生效预报，首个索引={bad[0]}')
179     return values
180 def rb_qsafe(arr, q):
181     x = np.asarray(arr, dtype=float); x = x[np.isfinite(x)]
182     return float(np.quantile(x, q)) if x.size else np.nan
183 def rb_mean(arr):
184     x = np.asarray(arr, dtype=float); x = x[np.isfinite(x)]
185     return float(np.mean(x)) if x.size else np.nan
186 def rb_rmse(arr):
187     x = np.asarray(arr, dtype=float); x = x[np.isfinite(x)]
188     return float(np.sqrt(np.mean(x*x))) if x.size else np.nan
189 def rb_mae(arr):
190     x = np.asarray(arr, dtype=float); x = x[np.isfinite(x)]
191     return float(np.mean(np.abs(x))) if x.size else np.nan
192 def risk_economic_costs(price: float, load: float, fc_pv: float, soc: float, cfg):
193     a_dis = max(0.0, min(cfg.p_max, cfg.eta_d * max(soc - cfg.e_min, 0.0)))
194     a_ch = max(0.0, min(cfg.p_max, max(cfg.e_max - soc, 0.0) / cfg.eta_c))
195     net = load - fc_pv; shortage = max(net, 0.0); surplus = max(-net, 0.0)
196     pi_plus = a_dis / (a_dis + shortage + RiskBandConfig.EPS)
197     pi_minus = a_ch / (a_ch + surplus + RiskBandConfig.EPS)
198     c_plus = RiskBandConfig.C_ES + price * (
199         RiskBandConfig.K_DIS * pi_plus + RiskBandConfig.K_SHORT * (1.0 - pi_plus)
200     )

```

```

199     c_minus = RiskBandConfig.C_ES + RiskBandConfig.K_CH * price * pi_minus + RiskBandConfig.
200         C_SUR * (1.0 - pi_minus)
201     return c_plus, c_minus
202 def build_hour_scale(actual_flat, fc_flat, clock):
203     err = fc_flat - actual_flat
204     daylight = (actual_flat >= RiskBandConfig.DAYLIGHT_THRESHOLD) | (fc_flat >= RiskBandConfig.
205         DAYLIGHT_THRESHOLD)
206     base = {}
207     for h in range(1, 25):
208         m = (clock == h) & daylight
209         if np.any(m):
210             qv = rb_qsafe(np.abs(err[m]), RiskBandConfig.BAND_QUANTILE)
211             med = float(np.median(actual_flat[m]))
212             floor = max(RiskBandConfig.BAND_FLOOR_RATIO * med, RiskBandConfig.BAND_MIN_KW)
213             base[h] = max(0.0 if np.isnan(qv) else float(qv), floor)
214         else:
215             base[h] = 0.0
216     if RiskBandConfig.USE_HOUR_SMOOTH:
217         vals = np.array([base[h] for h in range(1, 25)], dtype=float); sm = vals.copy()
218         for i in range(24):
219             sm[i] = 0.25*vals[max(i-1,0)] + 0.50*vals[i] + 0.25*vals[min(i+1,23)]
220         return {h: float(sm[h-1]) for h in range(1,25)}
221     return dict(base)
222 def analyze_risk_scheme(name: str, dates, load_hourly, pv_actual_hourly,
223     price_hourly, soc_hourly, fc_flat, cfg):
224     target_times = build_target_times(dates)
225     actual = pv_actual_hourly.reshape(-1)
226     load = load_hourly.reshape(-1)
227     price = np.tile(price_hourly, len(dates))
228     soc = soc_hourly.reshape(-1)
229     clock = np.asarray([24 if t.hour == 0 else t.hour for t in target_times], dtype=int)
230     error = fc_flat - actual
231     smooth_scale = build_hour_scale(actual, fc_flat, clock)
232     n = len(target_times)
233     lower, upper, bp, bm, danger, safe = (np.zeros(n) for _ in range(6))
234     for i in range(n):
235         cp, cm = risk_economic_costs(price[i], load[i], fc_flat[i], soc[i], cfg)
236         w = smooth_scale[int(clock[i])]
237         den = max(cp + cm, RiskBandConfig.EPS)
238         bplus, bminus = 2.0 * cm / den * w, 2.0 * cp / den * w
239         lo, hi = actual[i] - bminus, actual[i] + bplus
240         d_ex = max(fc_flat[i] - hi, 0.0)
241         s_ex = max(lo - fc_flat[i], 0.0)
242         lower[i]=lo; upper[i]=hi; bp[i]=bplus; bm[i]=bminus
243         danger[i]=d_ex; safe[i]=s_ex
244     hourly=[]
245     for h in range(1,25):
246         m=(clock==h)
247         hourly.append({
248             '目标小时':h, '样本数':int(np.sum(m)),
249             '危险高估次数':int(np.sum(danger[m]>0)),
250             '危险高估总量kW':float(np.sum(danger[m])),
251             '保守低估次数':int(np.sum(safe[m]>0)),
252             '保守低估总量kW':float(np.sum(safe[m])),
253             '总带外次数':int(np.sum((danger[m]+safe[m])>0)),
254             '总带外量kW':float(np.sum(danger[m]+safe[m])),
255             'RMSE':rb_rmse(error[m]), 'MAE':rb_mae(error[m]),
256             '平均b_plus':rb_mean(bp[m]), '平均b_minus':rb_mean(bm[m]),
257         })
258     overall={
259         '方案':name,
260         '危险高估次数':int(np.sum(danger>0)),
261         '危险高估总量kW':float(np.sum(danger)),
262         '保守低估次数':int(np.sum(safe>0)),
263         '保守低估总量kW':float(np.sum(safe)),
264         '总带外次数':int(np.sum((danger+safe)>0)),
265         '总带外量kW':float(np.sum(danger+safe)),
266         'RMSE':rb_rmse(error), 'MAE':rb_mae(error),
267         '平均b_plus':rb_mean(bp), '平均b_minus':rb_mean(bm),
268     }
269     return {
270         'target_times':target_times,'actual':actual,'forecast':fc_flat,
271         'lower':lower,'upper':upper,'danger':danger,'safe':safe,
272         'hourly':hourly,'overall':overall,
273     }
274 def risk_compare_table(base, enhanced):
275     keys=['危险高估次数','危险高估总量kW','保守低估次数','保守低估总量kW',
276         '总带外次数','总带外量kW','RMSE','MAE','平均b_plus','平均b_minus']
277     out=[]
278     for k in keys:
279         b=float(base['overall'][k]); e=float(enhanced['overall'][k])
280         out.append({'指标':k,'原方案':b,'增强方案':e,
281             '变化率%':((e-b)/b*100.0 if abs(b)>1e-12 else np.nan)})

```

```

280     return out
281 def risk_day_slice(analysis, day_str):
282     dd=datetime.strptime(day_str,'%Y-%m-%d').date()
283     idx=[]
284     for i,t in enumerate(analysis['target_times']):
285         td=t.date() if t.hour!=0 else t.date()-timedelta(days=1)
286         if td==dd: idx.append(i)
287     if len(idx)!=24: return None
288     return slice(idx[0],idx[-1]+1)
289 def plot_risk_day_compare(base, enhanced, day_str, path):
290     sl1=risk_day_slice(base,day_str); sl2=risk_day_slice(enhanced,day_str)
291     if sl1 is None or sl2 is None: return
292     fig,axes=plt.subplots(2,1,figsize=(12.5,9.0),sharex=True,dpi=300)
293     for ax,ana,sl,ptitle in [(axes[0],base,sl1,'原四时点预报'),(axes[1],enhanced,sl2,'增加预报后')]:
294         x=np.arange(1,25); ac=ana['actual'][sl]; fc=ana['forecast'][sl]
295         lo=ana['lower'][sl]; hi=ana['upper'][sl]
296         md=ana['danger'][sl]>0; ms=ana['safe'][sl]>0
297         ax.fill_between(x,lo,hi,alpha=.12,color=C_BUY,label='动态非对称风险带')
298         ax.plot(x,ac,lw=2.2,color=C_PV,label='实际光伏')
299         ax.plot(x,fc,lw=2.2,color=C_BUY,label='生效光伏预报')
300         ax.plot(x,lo,lw=1.0,ls='--',alpha=.75,color=C_SOC,label='下界')
301         ax.plot(x,hi,lw=1.0,ls='--',alpha=.75,color=C_SOC,label='上界')
302         if np.any(md): ax.scatter(x[md],fc[md],s=46,marker='^',color=C_CH,zorder=6,label='危险高估点')
303         if np.any(ms): ax.scatter(x[ms],fc[ms],s=46,marker='v',color=C_DIS,zorder=6,label='保守低估点')
304         ax.text(0.012,0.97,ptitle,transform=ax.transAxes,fontsize=13,
305               color='#444444',va='top',ha='left')
306         _set_labels(ax,'','光伏功率 / kW')
307         style(ax)
308         ax.legend(frameon=False,ncol=4, fontsize=14, loc='lower center',
309               bbox_to_anchor=(0.5,1.02), columnspacing=1.4)
310     axes[-1].set_xlim(1,24); axes[-1].set_xticks(range(1,25))
311     _set_labels(axes[-1],'目标小时 / h','')
312     fig.tight_layout(); fig.savefig(path,dpi=300,bbox_inches='tight'); plt.close(fig)
313 def plot_risk_overall(base, enhanced, path):
314     labels=['危险高估总量kW','保守低估总量kW','总带外量kW','RMSE','MAE']
315     b=[base['overall'][k] for k in labels]; e=[enhanced['overall'][k] for k in labels]
316     rel=[100*ee/bb if abs(bb)>1e-12 else np.nan for ee,bb in zip(e,b)]
317     x=np.arange(len(labels)); width=.36
318     fig,ax=plt.subplots(figsize=(10.8,5.4),dpi=300)
319     ax.bar(x-width/2,[100]*len(labels),width,color=SOFT_GRAY,alpha=0.95,label='原方案',zorder=3,**BAR_EDGE)
320     ax.bar(x+width/2,rel,width,color=SOFT_GREEN,alpha=0.95,label='增强方案',zorder=3,**BAR_EDGE)
321     ax.axhline(100,ls='--',lw=1.0,alpha=0.55,color=C_LOAD)
322     ax.set_xticks(x,labels)
323     if not NO_TITLE:
324         ax.set_title('增强方案相对原方案的风险指标变化', fontsize=13, fontweight='bold')
325     _set_labels(ax,'风险指标','相对原方案 / %')
326     _legend_top(ax,ncol=2)
327     style(ax)
328     fig.tight_layout(); fig.savefig(path,dpi=300,bbox_inches='tight'); plt.close(fig)
329 def interval_candidate(hourly_rows: List[dict], start: int, end: int):
330     stats = {int(r["目标小时"]): r for r in hourly_rows}
331     hours = list(range(start + 1, end + 1))
332     total_energy = sum(float(stats[h]["危险高估总量kW"]) for h in hours)
333     total_count = sum(int(stats[h]["危险高估次数"]) for h in hours)
334     if total_energy <= 0:
335         return None
336     weights = []
337     for h in hours:
338         # 以危险高估总量为主，次数为辅。
339         w = float(stats[h]["危险高估总量kW"]) + 5.0 * int(stats[h]["危险高估次数"])
340         weights.append(max(w, 0.0))
341     centroid = sum(h * w for h, w in zip(hours, weights)) / max(sum(weights), 1e-12)
342     candidate = int(round(centroid - 1.0))
343     candidate = max(start + 1, min(candidate, end - 1 if end - start > 1 else end))
344     return {
345         "区间": f"{start}:00-{end}:00", "风险重心": centroid, "候选时点": candidate,
346         "危险高估次数": total_count, "危险高估总量kW": total_energy,
347     }
348 def select_extra_hours(base_analysis: dict):
349     if RunConfig.MANUAL_EXTRA_HOURS is not None:
350         return sorted(set(int(x) for x in RunConfig.MANUAL_EXTRA_HOURS), [])
351     candidates = []
352     for a, b in RunConfig.CANDIDATE_INTERVALS:
353         r = interval_candidate(base_analysis["hourly"], a, b)
354         if r is not None:
355             candidates.append(r)
356     total = sum(r["危险高估总量kW"] for r in candidates)
357     for r in candidates:
358         r["风险占比"] = r["危险高估总量kW"] / max(total, 1e-12)

```

```

359 kept = [r for r in candidates if r["风险占比"] >= RunConfig.MIN_INTERVAL_RISK_SHARE]
360 kept.sort(key=lambda z: z["危险高估总量kw"], reverse=True)
361 kept = kept[:RunConfig.MAX_EXTRA_HOURS]
362 hours = sorted({int(r["候选时点"]) for r in kept})
363 return hours, candidates
364 def actual_hour_value(target_dt: datetime, dates: List[date], actual_hourly: np.ndarray):
365     if target_dt.hour == 0:
366         dd = target_dt.date() - timedelta(days=1)
367         h = 24
368     else:
369         dd, h = target_dt.date(), target_dt.hour
370     try:
371         di = dates.index(dd)
372     except ValueError:
373         return None
374     return float(actual_hourly[di, h - 1])
375 def past_climatology(target_dt: datetime, dates: List[date], actual_hourly: np.ndarray,
376                     issue_dt: datetime):
377     if target_dt.hour == 0:
378         target_date = target_dt.date() - timedelta(days=1)
379         h = 24
380     else:
381         target_date, h = target_dt.date(), target_dt.hour
382     vals = []
383     for back in range(1, RunConfig.HISTORY_DAYS_FOR_TAIL + 1):
384         dd = target_date - timedelta(days=back)
385         if dd >= issue_dt.date():
386             continue
387         try:
388             di = dates.index(dd)
389         except ValueError:
390             continue
391         vals.append(float(actual_hourly[di, h - 1]))
392     return max(float(np.median(vals)), 0.0) if vals else 0.0
393 def latest_record_before(records: List[dict], issue_dt: datetime):
394     c = [r for r in records if r["issue_dt"] < issue_dt]
395     return max(c, key=lambda z: z["issue_dt"]) if c else None
396 def record_value_at_target(rec: dict, target_dt: datetime):
397     lead = int(round((target_dt - rec["issue_dt"]).total_seconds() / 3600.0))
398     if 1 <= lead <= 24:
399         return float(rec["fc"][lead - 1]), lead
400     return None, None
401 def build_extra_record(extra_issue_dt: datetime, official_records: List[dict],
402                     dates: List[date], actual_hourly: np.ndarray):
403     old = latest_record_before(official_records, extra_issue_dt)
404     if old is None:
405         return None
406     residuals, weights = [], []
407     for back in range(RunConfig.RECENT_ERROR_HOURS - 1, -1, -1):
408         target_dt = extra_issue_dt - timedelta(hours=back)
409         old_val, _ = record_value_at_target(old, target_dt)
410         actual_val = actual_hour_value(target_dt, dates, actual_hourly)
411         if old_val is None or actual_val is None:
412             continue
413         residuals.append(actual_val - old_val)
414         weights.append(math.exp(-back / 1.5))
415     if residuals:
416         w = np.asarray(weights, dtype=float); w /= w.sum()
417         bias = float(np.dot(w, np.asarray(residuals, dtype=float)))
418     else:
419         bias = 0.0
420     bias = float(np.clip(bias, -RunConfig.MAX_BIAS_ABS_KW, RunConfig.MAX_BIAS_ABS_KW))
421     fc_new = np.zeros(24, dtype=float)
422     for lead in range(1, 25):
423         target_dt = extra_issue_dt + timedelta(hours=lead)
424         base_val, _ = record_value_at_target(old, target_dt)
425         if base_val is None:
426             base_val = past_climatology(target_dt, dates, actual_hourly, extra_issue_dt)
427         correction = bias * math.exp(-(lead - 1) / RunConfig.BIAS_DECAY_HOURS)
428         if base_val < 100.0:
429             correction *= 0.25
430         fc_new[lead - 1] = max(base_val + correction, 0.0)
431     return {"issue_dt": extra_issue_dt, "issue_hour": extra_issue_dt.hour, "fc": fc_new}
432 def build_augmented_records(official_records: List[dict], extra_hours: List[int],
433                             dates: List[date], actual_hourly: np.ndarray):
434     out = [dict(r) for r in official_records]
435     official_dates = sorted({r["issue_dt"].date() for r in official_records})
436     for dd in official_dates:
437         for h in extra_hours:
438             issue_dt = datetime.combine(dd, datetime.min.time()) + timedelta(hours=h)
439             rec = build_extra_record(issue_dt, official_records, dates, actual_hourly)
440             if rec is not None:
441                 out.append(rec)
442     out.sort(key=lambda z: z["issue_dt"])

```

```

443     return out
444 def export_records_xlsx(path: Path, records: List[dict]):
445     wb = Workbook(); ws = wb.active
446     ws.title = "Sheet1"
447     ws.append(["日期", "预报时刻"] + [f"预报{i}小时" for i in range(1, 25)])
448     by_date: Dict[date, List[dict]] = {}
449     for r in records:
450         by_date.setdefault(r["issue_dt"].date(), []).append(r)
451     for dd in sorted(by_date):
452         rr = sorted(by_date[dd], key=lambda z: z["issue_dt"])
453         for j, r in enumerate(rr):
454             date_cell = dd.isoformat() if j == 0 else None
455             ws.append([date_cell, f"{r['issue_hour']}:00"] + [float(v) for v in r["fc"]])
456     ensure_dir(path.parent)
457     wb.save(path)
458     wb.close()
459 def build_or_load_preprocess_cache(q3, cfg, dates, load, load_prior, run_D: int):
460     if RunConfig.CACHE.exists():
461         z = np.load(RunConfig.CACHE, allow_pickle=True)
462         if "load_fc_full_store" in z.files:
463             arr = np.asarray(z["load_fc_full_store"], dtype=float)
464             if arr.shape[0] >= run_D and arr.shape[1] == cfg.fore_slots:
465                 return arr[:run_D].copy()
466     if hasattr(q3, "_LOAD_MSTL_CACHE"):
467         q3._LOAD_MSTL_CACHE.clear()
468     store = np.full((run_D, cfg.fore_slots), np.nan, dtype=float)
469     for d in range(run_D):
470         fc = q3.load_forecast(d, load, load_prior, cfg, logger=None)
471         store[d] = np.asarray(fc[:cfg.fore_slots], dtype=float)
472     np.savez_compressed(RunConfig.CACHE, load_fc_full_store=store)
473     return store
474 def records_by_key(records: List[dict]):
475     return {(r["issue_dt"].date(), int(r["issue_hour"])): r for r in records}
476 def pv_record_horizon_10min(day_idx: int, stage_hour: int, rec_map: dict,
477                             dates: List[date], pv_actual: np.ndarray, cfg):
478     key = (dates[day_idx], int(stage_hour))
479     if key not in rec_map:
480         raise KeyError(f"缺少{dates[day_idx]} {stage_hour}:00的光伏预报记录")
481     hourly = np.asarray(rec_map[key]["fc"], dtype=float)
482     start_slot = int(stage_hour) * 6
483     anchor = 0.0 if stage_hour == 0 else float(pv_actual[day_idx, start_slot - 1])
484     xp = np.arange(0, 25, dtype=float)
485     fp = np.concatenate([[anchor], hourly])
486     target_rel = (np.arange(cfg.hori_slots) + 1) / 6.0
487     pred = np.interp(target_rel, xp, fp)
488     return np.maximum(pred, 0.0)
489 def actual_horizon(day_idx: int, stage_hour: int, arr: np.ndarray, cfg):
490     start = stage_hour * 6
491     H = cfg.hori_slots
492     flat = arr.reshape(-1)
493     g0 = day_idx * cfg.slots_pday + start
494     if g0 + H > len(flat):
495         return None
496     return np.asarray(flat[g0:g0 + H], dtype=float)
497 def build_extra_stage_error_history(extra_hours: List[int], rec_map: dict,
498                                     dates: List[date], load_fc_full_store: np.ndarray,
499                                     load: np.ndarray, pv: np.ndarray, cfg, run_D: int):
500     """事后预计算历史误差数组；仿真调用时仍严格只截取d-2及更早历史。"""
501     out = {h: np.full((run_D, cfg.hori_slots), np.nan, dtype=float) for h in extra_hours}
502     for d in range(run_D):
503         for h in extra_hours:
504             la = actual_horizon(d, h, load, cfg)
505             pa = actual_horizon(d, h, pv, cfg)
506             if la is None or pa is None:
507                 continue
508             lf = q3_global.load_stage_forecast_horizon(load_fc_full_store[d], load[d], h, cfg)
509             pf = pv_record_horizon_10min(d, h, rec_map, dates, pv, cfg)
510             out[h][d] = (la - pa) - (lf - pf)
511     return out
512 def extra_stage_reserve(day_idx: int, stage_hour: int, hist_by_hour: dict,
513                         fallback_reserve_day: np.ndarray, cfg):
514     H = cfg.hori_slots
515     fb = np.asarray(fallback_reserve_day, dtype=float)
516     clock = (stage_hour * 6 + np.arange(H)) % cfg.slots_pday
517     fallback = fb[clock]
518     # 与Q3_01官方阶段备用相同：仅使用d-2及更早已完整实现的24h预报误差。
519     end = max(0, day_idx - 1)
520     start = max(0, end - cfg.stage_rhisdays)
521     hist = hist_by_hour[stage_hour][start:end]
522     if hist.shape[0] < cfg.stage_mindays:
523         return fallback.copy()
524     out = np.zeros(H, dtype=float)
525     age = np.arange(hist.shape[0] - 1, -1, -1)

```

```

527 day_w = cfg.stage_decay ** age
528 for k in range(H):
529     vals = hist[:, k]
530     mask = np.isfinite(vals)
531     if int(np.sum(mask)) < cfg.stage_mindays:
532         out[k] = fallback[k]
533         continue
534     q = q3_global.weighted_quantile(vals[mask], day_w[mask], cfg.stage_quant)
535     out[k] = cfg.stage_scale * max(0.0, float(q))
536 return np.clip(out, 0.0, 5000.0)
537 def simulate_scheme(name: str,
538                    update_hours: Tuple[int, ...],
539                    records: List[dict],
540                    dates, price, load, pv,
541                    load_fc_full_store,
542                    fallback_reserve_store,
543                    official_stage_error_hist,
544                    extra_stage_error_hist,
545                    run_D: int, cfg, lp):
546 n = cfg.slots_pday
547 official_idx = {0: 0, 6: 1, 12: 2, 18: 3}
548 update_hours = tuple(sorted(set(int(h) for h in update_hours)))
549 rec_map = records_by_key(records)
550 plan_cost = np.zeros(run_D); adj_cost = np.zeros(run_D); emg_cost = np.zeros(run_D)
551 emg_energy = np.zeros(run_D); surplus_energy = np.zeros(run_D); up_energy = np.zeros(run_D)
552 down_energy = np.zeros(run_D); soc_start = np.zeros(run_D); soc_end = np.zeros(run_D)
553 soc = cfg.e0
554 for d in range(run_D):
555     soc_start[d] = soc
556     lf_full = load_fc_full_store[d]
557     lf0 = lf_full[:n]
558     pf0 = pv_record_horizon_10min(d, 0, rec_map, dates, pv, cfg)[:n]
559     fallback_reserve = fallback_reserve_store[d]
560     da = q3_global.solve_day_ahead(price, lf0, pf0, fallback_reserve, soc, cfg, lp)
561     initial_buy = da["buy"].copy(); current_buy = da["buy"].copy()
562     current_ch = da["ch"].copy(); current_dis = da["dis"].copy()
563     plan_cost[d] = float(np.sum(initial_buy * price) * cfg.dt)
564     stages = (0,) + update_hours
565     for si, h in enumerate(stages):
566         start = h * 6
567         if h > 0:
568             lf_h = q3_global.load_stage_forecast_horizon(lf_full, load[d], h, cfg)
569             pf_h = pv_record_horizon_10min(d, h, rec_map, dates, pv, cfg)
570             if h in official_idx:
571                 reserve_h = q3_global.stage_reserve_from_history(
572                     d, official_idx[h], official_stage_error_hist,
573                     fallback_reserve, h, cfg
574                 )
575             else:
576                 reserve_h = extra_stage_reserve(
577                     d, h, extra_stage_error_hist,
578                     fallback_reserve, cfg
579                 )
580             adj = q3_global.solve_rolling_mpc_24h(
581                 price, lf_h, pf_h, reserve_h,
582                 soc, current_buy, start, cfg
583             )
584             m = n - start
585             current_buy[start:] = adj["buy"][:m]; current_ch[start:] = adj["ch"][:m]
586             current_dis[start:] = adj["dis"][:m]; adj_cost[d] += adj["adj_net_cost"]
587             up_energy[d] += adj["up_energy"]; down_energy[d] += adj["down_energy"]
588         end = stages[si + 1] * 6 if si + 1 < len(stages) else n
589         for t in range(start, end):
590             sol = q3_global.operate_one_slot(
591                 soc, current_buy[t], load[d, t], pv[d, t],
592                 current_ch[t], current_dis[t], cfg
593             )
594             soc = sol["E"]; emg_energy[d] += sol["emg"] * cfg.dt
595             surplus_energy[d] += sol["sur"] * cfg.dt
596             emg_cost[d] += sol["emg"] * 5.0 * price[t] * cfg.dt
597         soc_end[d] = soc
598     return {
599         "plan_cost": plan_cost, "adj_cost": adj_cost, "emg_cost": emg_cost,
600         "emg_energy": emg_energy, "surplus_energy": surplus_energy, "up_energy": up_energy,
601         "down_energy": down_energy, "soc_end": soc_end,
602     }
603 def summarize_scheme(name: str, hours: Tuple[int, ...], result: dict, run_D: int):
604     start = 31 if run_D >= 32 else 0
605     sl = slice(start, run_D)
606     daily = np.asarray(result["emg_energy"][sl], dtype=float)
607     plan = float(np.sum(result["plan_cost"][sl]))
608     adj = float(np.sum(result["adj_cost"][sl]))
609     emgc = float(np.sum(result["emg_cost"][sl]))
610     return {

```

```

611     "方案": name, "全部预报时点": ", ".join(["0"] + [str(h) for h in hours]),
612     "初始计划购电费": plan, "调整净费用": adj, "紧急购电费": emgc,
613     "总费用": plan + adj + emgc, "紧急购电量kWh": float(np.sum(daily)),
614     "日紧急购电量P95kWh": float(np.quantile(daily, 0.95)) if len(daily) else 0.0,
615     "单日最大紧急购电量kWh": float(np.max(daily)) if len(daily) else 0.0,
616     "紧急购电天数": int(np.sum(daily > 1e-6)),
617     "富余电量kWh": float(np.sum(result["surplus_energy"][s1])),
618     "向上调整电量kWh": float(np.sum(result["up_energy"][s1])),
619     "向下调整电量kWh": float(np.sum(result["down_energy"][s1])),
620     "期末SOCKWh": float(result["soc_end"][run_D - 1]),
621 }
622 def plot_lp_comparison(rows: List[dict], out_dir: Path):
623     ensure_dir(out_dir)
624     names = [r["方案"] for r in rows]; x = np.arange(len(rows))
625     # 01/02: 单指标柱状图 (01为相对基线变化), 柔色+黑描边、无标题、大字号
626     for key, scale, color, ylabel, title, fname, fmt, diff in (
627         ("总费用", 1e4, SOFT_BLUE, "相对基线总费用变化 / 万元",
628         "新增预报时点对总费用的影响", "第三问新增预报时点对总费用的影响.png", "{:.2f}",
629         True),
630         ("紧急购电量kWh", 1e3, SOFT_RED, "紧急购电量 / MWh",
631         "新增预报时点对紧急购电量的影响", "第三问新增预报时点对紧急购电量的影响.png", "{:.2
632         f}", False)):
633         vals = np.asarray([r[key] - rows[0][key] if diff else r[key] for r in rows]) / scale
634         fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
635         ax.bar(x, vals, width=0.40, color=color, alpha=0.95, zorder=3, **BAR_EDGE)
636         if diff:
637             ax.axhline(0, lw=1, color='black')
638             ax.set_xticks(x, names)
639             if not NO_TITLE:
640                 ax.set_title(title, fontsize=13, fontweight="bold")
641             _set_labels(ax, '预报时点方案', ylabel)
642             style(ax)
643             for i, v in enumerate(vals):
644                 dy = (0.15 if v >= 0 else -0.15) if diff else 0.0
645                 ax.text(i, v + dy, fmt.format(v), ha="center",
646                 va="bottom" if (not diff or v >= 0) else "top", fontsize=11)
647             fig.tight_layout()
648             fig.savefig(out_dir / fname, dpi=300, bbox_inches="tight")
649             plt.close(fig)
650     # 03 P95与最大单日紧急购电图已按需删除 (数据仍随 CSV 输出)
651 q3_global = None
652 def main():
653     global q3_global
654     ensure_dir(RunConfig.OUT); ensure_dir(RunConfig.FIG)
655     q3_path = first_existing(RunConfig.Q3_MAIN_CANDIDATES)
656     att1 = first_existing(RunConfig.ATT1_CANDIDATES)
657     att2 = first_existing(RunConfig.ATT2_CANDIDATES)
658     att3 = first_existing(RunConfig.ATT3_CANDIDATES)
659     cp_path = first_existing(RunConfig.CHECKPOINT_CANDIDATES)
660     q3 = import_py(q3_path, "q3_main_model"); q3_global = q3; cfg = q3.CFG
661     cfg.base_dir = str(RunConfig.BASE); cfg.att1 = str(att1); cfg.att2 = str(att2); cfg.att3 =
662     str(att3)
663     cfg.max_days = RunConfig.MAX_DAYS; cfg.use_bias = False
664     dates, price, load_prior, pv_prior, load, pv = load_core_inputs(att1, att2, q3)
665     run_D = min(len(dates), RunConfig.MAX_DAYS)
666     dates_run = dates[:run_D]; load_run = load[:run_D]; pv_run = pv[:run_D]
667     checkpoint = load_checkpoint(cp_path, run_D)
668     fallback_reserve_store = np.asarray(checkpoint["fallback_reserve"][:run_D], dtype=float)
669     official_stage_error_hist = np.asarray(checkpoint["stage_net_error_hist"][:run_D], dtype=
670     float)
671     price_hourly = price.reshape(24, 6).mean(axis=1)
672     hour_idx = [6 * h - 1 for h in range(1, 25)]
673     load_hourly = load_run[:, hour_idx]; pv_hourly_actual = pv_run[:, hour_idx]
674     soc_hourly = reconstruct_soc_hourly(checkpoint, run_D, cfg)
675     official_records_all = load_forecast_records_xlsx(att3)
676     official_records = [r for r in official_records_all if r["issue_dt"].date() in set(dates_run
677     )]
678     target_times = build_target_times(dates_run)
679     fc0 = effective_forecast_records(official_records, target_times)
680     base_risk = analyze_risk_scheme("原四时点预报", dates_run, load_hourly, pv_hourly_actual,
681     price_hourly, soc_hourly, fc0, cfg)
682     write_csv(RunConfig.OUT / "风险带_原方案按目标小时汇总.csv", base_risk["hourly"])
683     extra_hours, _ = select_extra_hours(base_risk)
684     print("\n风险带筛选")
685     if not extra_hours:
686         raise RuntimeError("未筛选出新增预报时点, 无法继续增强方案验证。")
687     print("最终新增时点: ", ", ".join(f"{h}:00" for h in extra_hours))
688     augmented_records = build_augmented_records(
689         official_records, extra_hours, dates_run, pv_hourly_actual
690     )
691     extra_tag = "_".join(str(h) for h in extra_hours)
692     enhanced_xlsx = RunConfig.OUT / f"附件3新增预报_{extra_tag}_结果.xlsx"
693     export_records_xlsx(enhanced_xlsx, augmented_records)

```

```

689 fc1 = effective_forecast_records(augmented_records, target_times)
690 enhanced_risk = analyze_risk_scheme("增加预报后", dates_run, load_hourly, pv_hourly_actual,
691                                   price_hourly, soc_hourly, fc1, cfg)
692 write_csv(RunConfig.OUT / "风险带_增强方案按目标小时汇总.csv", enhanced_risk["hourly"])
693 risk_cmp = risk_compare_table(base_risk, enhanced_risk)
694 write_csv(RunConfig.OUT / "风险带_原方案vs增强方案.csv", risk_cmp)
695 for dd in RunConfig.REPRESENTATIVE_DATES:
696     if dd in dates_run:
697         plot_risk_day_compare(
698             base_risk, enhanced_risk, dd.isoformat(),
699             RunConfig.FIG / f"第三问{dd.month}-{dd.day}动态非对称风险带前后对比.png"
700         )
701 # 各目标小时带外量对比图已按需删除
702 plot_risk_overall(
703     base_risk, enhanced_risk,
704     RunConfig.FIG / "第三问增强方案相对原方案的风险指标变化.png"
705 )
706 load_fc_full_store = build_or_load_preprocess_cache(
707     q3, cfg, dates_run, load_run, load_prior, run_D)
708 aug_map = records_by_key(augmented_records)
709 extra_stage_hist = build_extra_stage_error_history(
710     extra_hours, aug_map, dates_run, load_fc_full_store, load_run, pv_run, cfg, run_D)
711 lp = q3.build_lp_templates(price, cfg)
712 base_updates = (6, 12, 18)
713 schemes = [("原0/6/12/18", base_updates)]
714 for h in extra_hours:
715     hrs = tuple(sorted(set(base_updates + (h,))))
716     schemes.append((f"原方案+{h}:00", hrs))
717 if len(extra_hours) >= 2:
718     all_hrs = tuple(sorted(set(base_updates + tuple(extra_hours))))
719     schemes.append(("原方案+全部新增时点", all_hrs))
720 schemes = list(dict.fromkeys(schemes))
721 summaries = []
722 print("\n完整24h滚动LP验证")
723 for name, hrs in schemes:
724     records = official_records if hrs == base_updates else augmented_records
725     r = simulate_scheme(
726         name, hrs, records, dates_run, price, load_run, pv_run,
727         load_fc_full_store, fallback_reserve_store, official_stage_error_hist,
728         extra_stage_hist, run_D, cfg, lp)
729     summaries.append(summarize_scheme(name, hrs, r, run_D))
730 write_csv(RunConfig.OUT / "新增预报时点_完整滚动LP方案对比.csv", summaries)
731 plot_lp_comparison(summaries, RunConfig.FIG)
732 base_row = summaries[0]
733 delta_rows = []
734 metrics = ["总费用", "紧急购电量kWh", "日紧急购电量P95kWh",
735            "单日最大紧急购电量kWh", "富余电量kWh", "向上调整电量kWh", "向下调整电量kWh"]
736 for row in summaries[1:]:
737     item = {"方案": row["方案"]}
738     for m in metrics:
739         b = float(base_row[m])
740         v = float(row[m])
741         item[f"{m}_变化量"] = v - b
742         item[f"{m}_变化率%"] = (v - b) / b * 100.0 if abs(b) > 1e-12 else np.nan
743     delta_rows.append(item)
744 write_csv(RunConfig.OUT / "新增预报方案_相对基线改善.csv", delta_rows)
745 print("\n最终方案比较")
746 for r in summaries:
747     print(f"{r['方案']}: 总费用={r['总费用']:.2f} 元 | "
748           f"紧急购电={r['紧急购电量kWh']:.2f} kWh | "
749           f"P95={r['日紧急购电量P95kWh']:.2f} kWh | "
750           f"最大单日={r['单日最大紧急购电量kWh']:.2f} kWh | "
751           f"富余={r['富余电量kWh']:.2f} kWh")
752 min_cost = min(summaries, key=lambda r: r["总费用"])
753 min_emg = min(summaries, key=lambda r: r["紧急购电量kWh"])
754 conclusion = ["问题3新增预报时点结果整合", "=" * 70,
755              "风险带筛选新增时点: " + ", ".join(f"{h}:00" for h in extra_hours),
756              f"最低总费用方案: {min_cost['方案']}, {min_cost['总费用']:.2f} 元",
757              f"最低紧急购电方案: {min_emg['方案']}, {min_emg['紧急购电量kWh']:.2f} kWh" ]
758 (RunConfig.OUT / "结果整合.txt").write_text("\n".join(conclusion), encoding="utf-8")
759 print("\n输出目录: ", RunConfig.OUT)
760 print("方案对比: ", RunConfig.OUT / "新增预报时点_完整滚动LP方案对比.csv")
761 if __name__ == "__main__":
762     main()

```

code/Q3-2.py

Q4-2.py

```

1 from __future__ import annotations
2 import sys

```

```

3 import logging
4 from dataclasses import dataclass
5 from pathlib import Path
6 from datetime import datetime, timedelta, date
7 from typing import Tuple, Dict
8 import numpy as np
9 from scipy.optimize import linprog
10 from scipy import sparse
11 from statsmodels.tsa.ar_model import AutoReg
12 try:
13     from statsmodels.tsa.seasonal import MSTL
14     HAS_MSTL = True
15 except Exception:
16     HAS_MSTL = False
17 try:
18     from openpyxl import load_workbook, Workbook
19 except ImportError as e:
20     raise ImportError(
21         "缺少 openpyxl。请在当前 Python 环境执行: pip install openpyxl"
22     ) from e
23 import matplotlib
24 matplotlib.use("Agg")
25 import matplotlib.pyplot as plt
26 import matplotlib.dates as mdates
27 plt.rcParams["font.family"] = "sans-serif"
28 plt.rcParams["font.sans-serif"] = ["Noto Sans CJK SC", "Microsoft YaHei", "SimHei", "DejaVu Sans"]
29 plt.rcParams["axes.unicode_minus"] = False
30 plt.rcParams["font.size"] = 12
31 plt.rcParams["axes.spines.top"] = False
32 plt.rcParams["axes.linewidth"] = 1.5
33 plt.rcParams["xtick.major.width"] = 1.5
34 plt.rcParams["ytick.major.width"] = 1.5
35 plt.rcParams["xtick.major.size"] = 6
36 plt.rcParams["ytick.major.size"] = 6
37 C_BUY, C_PV, C_DIS, C_LOAD, C_CH = "#2E74B5", "#27AE60", "#E67E22", "#34495E", "#E74C3C"
38 C_W, C_SOC, C_MIN, C_SOCB, C_RES = "#95A5A6", "#7F8C8D", "#2980B9", "#1F77B4", "#16A085"
39 C_ALPHA, C_MAX, C_PRICE = "#8E44AD", "#C0392B", "#E74C3C"
40 class Config:
41     b_dir = str(Path(__file__).resolve().parent)
42     att1 = str(Path(b_dir) / "附件1.xlsx")
43     att2 = str(Path(b_dir) / "附件2.xlsx")
44     att4 = str(Path(b_dir) / "附件4.xlsx")
45     if not Path(att4).exists() and Path(b_dir, "附件4(2).xlsx").exists():
46         att4 = str(Path(b_dir) / "附件4(2).xlsx")
47     temp = str(Path(b_dir) / "附件5" / "result4-2.xlsx")
48     if not Path(temp).exists() and Path(b_dir, "result4-2.xlsx").exists():
49         temp = str(Path(b_dir) / "result4-2.xlsx")
50     if not Path(temp).exists() and Path(b_dir, "result4-2(1).xlsx").exists():
51         temp = str(Path(b_dir) / "result4-2(1).xlsx")
52     o_dir = str(Path(b_dir) / "Q4-2结果")
53     o_xlsx = str(Path(o_dir) / "result4-2_filled.xlsx")
54     fig_dir = str(Path(o_dir) / "Q4-2图片输出")
55     slots_pday = 144; dt = 1.0 / 6.0
56     hori_slots = 144; fore_slots = 288
57     eta_c = eta_d = 0.90
58     e_min, e_max, p_max, e0 = 1200.0, 10800.0, 5000.0, 6000.0
59     l_wdays = pri_wdays = 42; pv_wdays = 60
60     l_mindays = pri_mindays = 21; mstl_days = 7
61     ar_candi = pri_ar_candi = (1, 2, 3, 6, 12, 18, 36)
62     pv_ar_candi = (1, 2, 3, 6, 12)
63     kar_candi = (1, 2, 3, 5, 7)
64     clear_quant = 0.93; pv_smo_win = 2
65     rk_hisdays = 28; rk_decay = 0.90; rk_minsamp = 5
66     alpha_cold, alpha_min, alpha_max = 4.0, 1.0, 8.0
67     beta_fixed = 1.0; c_eps = 1e-4
68     tau_min, tau_max = 0.55, 0.88; reserve_scale = 0.65
69     eps_cyc, eps_surplus = 1e-5, 1e-8; lp_method = "highs"
70     max_days = 365; checkpoint_every_days = 30
71 CFG = Config()
72 def setup_logger(out_dir: Path) -> logging.Logger:
73     out_dir.mkdir(parents=True, exist_ok=True)
74     logger = logging.getLogger("Q2")
75     logger.setLevel(logging.INFO)
76     logger.handlers.clear()
77     fmt = logging.Formatter("%(message)s")
78     sh = logging.StreamHandler(sys.stdout)
79     sh.setFormatter(fmt)
80     logger.addHandler(sh)
81     return logger
82 def excel_serial_to_date(x) -> datetime:
83     return datetime(1899, 12, 30) + timedelta(days=float(x))
84 def cell_to_date(x):

```

```

85     """兼容 openpyxl 返回的 date/datetime、Excel 序列号和常见日期字符串。"""
86     if x is None:
87         return None
88     if isinstance(x, datetime):
89         return x.date()
90     try:
91         from datetime import date as _date
92         if isinstance(x, _date):
93             return x
94     except Exception:
95         pass
96     if isinstance(x, (int, float, np.integer, np.floating)):
97         return excel_serial_to_date(x).date()
98     if isinstance(x, str):
99         txt=x.strip()
100         for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%Y.%m.%d"):
101             try:
102                 return datetime.strptime(txt, fmt).date()
103             except ValueError:
104                 pass
105     raise ValueError(f"无法识别日期单元格: {x!r}")
106 def safe_float_array(x) -> np.ndarray:
107     return np.asarray([[float(v) if v is not None else np.nan for v in row] for row in x], dtype
                        =float)
108 def weighted_quantile(values: np.ndarray, weights: np.ndarray, q: float) -> float:
109     values = np.asarray(values, dtype=float)
110     weights = np.asarray(weights, dtype=float)
111     m = np.isfinite(values) & np.isfinite(weights) & (weights > 0)
112     if m.sum() == 0:
113         return 0.0
114     o=np.argsort(v:=values[m]);w=weights[m][o];v=v[o];cw=np.cumsum(w)/w.sum()
115     return float(np.interp(q, cw, v))
116 def moving_average_circular(x: np.ndarray, half: int) -> np.ndarray:
117     if half <= 0:
118         return x.copy()
119     n = len(x); out = np.zeros(n)
120     for i in range(n):
121         lo = max(0, i-half); hi = min(n, i+half+1)
122         out[i] = np.mean(x[lo:hi])
123     return out
124 def choose_ar_forecast(series: np.ndarray, horizon: int, candidates: Tuple[int, ...], fallback:
                        float = 0.0) -> np.ndarray:
125     """用AIC在少量AutoReg候选中选阶; 失败时返回fallback/近期均值。"""
126     y = np.asarray(series, dtype=float)
127     y = y[np.isfinite(y)]
128     if len(y) < 8 or np.std(y) < 1e-8:
129         return np.full(horizon, float(np.mean(y) if len(y) else fallback))
130     best = None
131     for p in candidates:
132         if len(y) <= p + 5:
133             continue
134         try:
135             model = AutoReg(y, lags=p, trend="ct", old_names=False).fit()
136             if np.isfinite(model.aic) and (best is None or model.aic < best[0]):
137                 best = (model.aic, model)
138         except Exception:
139             continue
140     if best is None:
141         return np.full(horizon, float(np.mean(y[-min(7, len(y)):])))
142     try:
143         pred = np.asarray(best[1].predict(start=len(y), end=len(y)+horizon-1, dynamic=False),
                            dtype=float)
144         if np.any(~np.isfinite(pred)):
145             raise ValueError("AR forecast nonfinite")
146         return pred
147     except Exception:
148         return np.full(horizon, float(np.mean(y[-min(7, len(y)):])))
149 def load_inputs(cfg: Config, logger):
150     logger.info("【数据读取】正在读取附件1、附件2、附件4……")
151     wb1 = load_workbook(cfg.att1, data_only=True, read_only=True)
152     ws1 = wb1["Sheet1"] if "Sheet1" in wb1.sheetnames else wb1[wb1.sheetnames[0]]
153     rows1 = list(ws1.iter_rows(min_row=2, max_row=145, min_col=2, max_col=4, values_only=True))
154     wb1.close()
155     price_prior = np.asarray([float(r[0]) for r in rows1], dtype=float)
156     load_prior = np.asarray([float(r[1]) for r in rows1], dtype=float)
157     pv_prior = np.asarray([float(r[2]) for r in rows1], dtype=float)
158     wb2 = load_workbook(cfg.att2, data_only=True, read_only=True)
159     sl = wb2["小区负载"]
160     sp = wb2["光伏发电实际功率"]
161     load_all = list(sl.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=
                            True))
162     pv_all = list(sp.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=True)
                  )
163     wb2.close()

```

```

164 dates = [cell_to_date(r[0]) for r in load_all]
165 load = safe_float_array([r[1:] for r in load_all])
166 pv_actual = safe_float_array([r[1:] for r in pv_all])
167 wb4 = load_workbook(cfg.att4, data_only=True, read_only=True)
168 ws4 = wb4["Sheet1"] if "Sheet1" in wb4.sheetnames else wb4[wb4.sheetnames[0]]
169 price_rows = list(ws4.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145, values_only=
170 True))
171 wb4.close()
172 price_dates = [cell_to_date(r[0]) for r in price_rows]
173 price_actual = safe_float_array([r[1:] for r in price_rows])
174 if load.shape != (365,144) or pv_actual.shape != (365,144) or price_actual.shape !=
(365,144):
175     raise ValueError(f"数据维度异常: load={load.shape}, pv={pv_actual.shape}, price={
price_actual.shape}")
176 if dates != price_dates:
177     raise ValueError("附件2与附件4日期未完全对齐。")
178 if np.any(~np.isfinite(price_actual)) or np.any(price_actual < 0):
179     raise ValueError("附件4存在缺失或负电价; 当前模型按非负实时电价处理。")
180 logger.info("【数据读取】完成: 365天×144个10分钟时段。")
181 logger.info("电价范围 %.4f~%.4f 元/kWh, 全年均价 %.4f 元/kWh。",
float(np.min(price_actual)), float(np.max(price_actual)), float(np.mean(
price_actual)))
182 return dates, price_prior, load_prior, pv_prior, load, pv_actual, price_actual
183 _LOAD_MSTL_CACHE = {}
184 def load_forecast(day_idx: int, load: np.ndarray, prior: np.ndarray, cfg: Config, logger=None)
-> np.ndarray:
185     H = cfg.fore_slots
186     if day_idx == 0:
187         return np.tile(prior, 2)
188     hist_days = min(day_idx, cfg.l_wdays)
189     hist_mat = load[day_idx-hist_days:day_idx]
190     # 冷启动: 同一时刻近期日加权 + 附件1先验
191     if hist_days < cfg.l_mindays or not HAS_MSTL:
192         k = hist_mat.shape[0]
193         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
194         weights /= weights.sum()
195         shape = np.sum(hist_mat * weights[:, None], axis=0)
196         if hist_days >= 7:
197             shape = 0.55*shape + 0.45*load[day_idx-7]
198         w_prior = max(0.0, 1.0 - hist_days / cfg.l_mindays)
199         shape = (1-w_prior)*shape + w_prior*prior
200         return np.tile(shape, 2)
201     # 仅每隔若干天完整重估一次 MSTL; 数据已确认无明显异常, 因此关闭 robust 迭代
202     need_refit = (
203         not _LOAD_MSTL_CACHE
204         or day_idx - _LOAD_MSTL_CACHE.get("refit_day", -10**9) >= cfg.mstl_days
205     )
206     if need_refit:
207         y = hist_mat.reshape(-1)
208         try:
209             mstl = MSTL(y, periods=(144, 1008), stl_kwargs={"robust": False}).fit()
210             trend = np.asarray(mstl.trend)
211             seasonal = np.asarray(mstl.seasonal)
212             if seasonal.ndim == 1:
213                 s_daily = seasonal; s_weekly = np.zeros_like(seasonal)
214             else:
215                 s_daily, s_weekly = seasonal[:, 0], seasonal[:, 1]
216             resid = np.asarray(mstl.resid)
217             # 保存稳定的日周期与周周期结构, 供未来几天复用
218             daily_tail = s_daily[-min(len(s_daily), 28*144):]
219             weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
220             daily_pattern = np.array([
221                 np.nanmean(daily_tail[i::144]) for i in range(144)
222             ])
223             weekly_pattern = np.array([
224                 np.nanmean(weekly_tail[i::1008]) for i in range(1008)
225             ])
226             _LOAD_MSTL_CACHE.clear()
227             _LOAD_MSTL_CACHE.update({
228                 "refit_day": day_idx, "origin_day": day_idx - hist_days,
229                 "daily_pattern": daily_pattern, "weekly_pattern": weekly_pattern,
230                 "trend_last": float(trend[-1]),
231                 "trend_slope": float(np.clip(
232                     np.polyfit(
233                         np.arange(min(432, len(trend)), dtype=float),
234                         trend[-min(432, len(trend)):],
235                         1
236                     )[0], -5.0, 5.0
237                 )),
238                 "resid_tail": resid[-min(len(resid), 14*144):].copy(),
239             })
240         if logger:
241             logger.info("负荷预测: 第 %d 天重新拟合 MSTL (日/周周期)。", day_idx + 1)

```

```

242     except Exception as e:
243         if logger:
244             logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
245             k = hist_mat.shape[0]
246             weights = np.exp(-0.10*np.arange(k-1, -1, -1))
247             weights /= weights.sum()
248             shape = np.sum(hist_mat * weights[:, None], axis=0)
249             if hist_days >= 7:
250                 shape = 0.5*shape + 0.5*load[day_idx-7]
251             return np.tile(shape, 2)
252 # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
253 daily_pattern = _LOAD_MSTL_CACHE["daily_pattern"]
254 weekly_pattern = _LOAD_MSTL_CACHE["weekly_pattern"]
255 origin_day = _LOAD_MSTL_CACHE["origin_day"]
256 recent_days = min(hist_days, 3)
257 recent = load[day_idx-recent_days:day_idx].reshape(-1)
258 global_start = (day_idx - recent_days) * 144
259 gidx = global_start + np.arange(len(recent))
260 sd_hist = daily_pattern[gidx % 144]
261 sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
262 deseason = recent - sd_hist - sw_hist
263 m = min(432, len(deseason))
264 x = np.arange(m, dtype=float)
265 coef = np.polyfit(x, deseason[-m:], 1)
266 slope = float(np.clip(coef[0], -5.0, 5.0))
267 level = float(deseason[-1])
268 trend_future = level + slope*np.arange(1, H+1)
269 future_gidx = day_idx*144 + np.arange(H)
270 sd_future = daily_pattern[future_gidx % 144]
271 sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
272 # 残差 AR 仍每天轻量更新, 不需要重新做 MSTL
273 hist_axis = np.arange(len(deseason), dtype=float)
274 fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
275 resid_recent = deseason - fitted_trend_hist
276 ar_future = choose_ar_forecast(
277     resid_recent[-min(len(resid_recent), 3*144):],
278     H,
279     cfg.ar_candi,
280     0.0,
281 )
282 pred = trend_future + sd_future + sw_future + ar_future
283 return np.maximum(pred, 0.0)
284 _PRICE_MSTL_CACHE = {}
285 def price_forecast(day_idx: int, price_actual: np.ndarray, prior: np.ndarray, cfg: Config,
286     logger=None) -> np.ndarray:
287     H = cfg.fore_slots
288     if day_idx == 0:
289         return np.tile(prior, 2)
290     hist_days = min(day_idx, cfg.pri_wdays)
291     hist_mat = price_actual[day_idx-hist_days:day_idx]
292 # 冷启动: 同一时刻近期日加权 + 附件1先验
293 if hist_days < cfg.pri_mindays or not HAS_MSTL:
294     k = hist_mat.shape[0]
295     weights = np.exp(-0.10*np.arange(k-1, -1, -1))
296     weights /= weights.sum()
297     shape = np.sum(hist_mat * weights[:, None], axis=0)
298     if hist_days >= 7:
299         shape = 0.55*shape + 0.45*price_actual[day_idx-7]
300     w_prior = max(0.0, 1.0 - hist_days / cfg.pri_mindays)
301     shape = (1-w_prior)*shape + w_prior*prior
302     return np.tile(shape, 2)
303 # 仅每隔若干天完整重估一次 MSTL; 数据已确认无明显异常, 因此关闭 robust 迭代
304 need_refit = (
305     not _PRICE_MSTL_CACHE
306     or day_idx - _PRICE_MSTL_CACHE.get("refit_day", -10**9) >= cfg.mstl_days
307 )
308 if need_refit:
309     y = hist_mat.reshape(-1)
310     try:
311         mstl = MSTL(y, periods=(144, 1008), stl_kwargs={"robust": False}).fit()
312         trend = np.asarray(mstl.trend)
313         seasonal = np.asarray(mstl.seasonal)
314         if seasonal.ndim == 1:
315             s_daily = seasonal; s_weekly = np.zeros_like(seasonal)
316         else:
317             s_daily, s_weekly = seasonal[:, 0], seasonal[:, 1]
318         resid = np.asarray(mstl.resid)
319 # 保存稳定的日周期与周周期结构, 供未来几天复用
320 daily_tail = s_daily[-min(len(s_daily), 28*144):]
321 weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
322 daily_pattern = np.array([
323     np.nanmean(daily_tail[i::144]) for i in range(144)
324 ])
325 weekly_pattern = np.array([

```

```

325         np.nanmean(weekly_tail[i::1008]) for i in range(1008)
326     ])
327     _PRICE_MSTL_CACHE.clear()
328     _PRICE_MSTL_CACHE.update({
329         "refit_day": day_idx, "origin_day": day_idx - hist_days,
330         "daily_pattern": daily_pattern, "weekly_pattern": weekly_pattern,
331         "trend_last": float(trend[-1]),
332         "trend_slope": float(np.clip(
333             np.polyfit(
334                 np.arange(min(432, len(trend)), dtype=float),
335                 trend[-min(432, len(trend)):],
336                 1
337             )[0], -5.0, 5.0
338         )),
339         "resid_tail": resid[-min(len(resid), 14*144):].copy(),
340     })
341     if logger:
342         logger.info("电价预测: 第 %d 天重新拟合 MSTL (日/周周期)。", day_idx + 1)
343     except Exception as e:
344         if logger:
345             logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
346         k = hist_mat.shape[0]
347         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
348         weights /= weights.sum()
349         shape = np.sum(hist_mat * weights[:, None], axis=0)
350         if hist_days >= 7:
351             shape = 0.5*shape + 0.5*price_actual[day_idx-7]
352         return np.tile(shape, 2)
353     # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
354     daily_pattern = _PRICE_MSTL_CACHE["daily_pattern"]
355     weekly_pattern = _PRICE_MSTL_CACHE["weekly_pattern"]
356     origin_day = _PRICE_MSTL_CACHE["origin_day"]
357     recent_days = min(hist_days, 3)
358     recent = price_actual[day_idx-recent_days:day_idx].reshape(-1)
359     global_start = (day_idx - recent_days) * 144
360     gidx = global_start + np.arange(len(recent))
361     sd_hist = daily_pattern[gidx % 144]
362     sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
363     deseason = recent - sd_hist - sw_hist
364     m = min(432, len(deseason))
365     x = np.arange(m, dtype=float)
366     coef = np.polyfit(x, deseason[-m:], 1)
367     slope = float(np.clip(coef[0], -5.0, 5.0))
368     level = float(deseason[-1])
369     trend_future = level + slope*np.arange(1, H+1)
370     future_gidx = day_idx*144 + np.arange(H)
371     sd_future = daily_pattern[future_gidx % 144]
372     sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
373     # 残差 AR 仍每天轻量更新, 不需要重新做 MSTL
374     hist_axis = np.arange(len(deseason), dtype=float)
375     fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
376     resid_recent = deseason - fitted_trend_hist
377     ar_future = choose_ar_forecast(resid_recent[-min(len(resid_recent), 3*144):],
378         H, cfg.pri_ar_candi, 0.0,)
379     pred = trend_future + sd_future + sw_future + ar_future
380     return np.maximum(pred, 0.0)
381 def build_clear_envelope(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config) -> np.
    ndarray:
382     if day_idx == 0:
383         base = prior.copy()
384         return np.maximum(base, 0.0)
385     w = min(day_idx, cfg.pv_wdays)
386     hist = pv[day_idx-w:day_idx]
387     q = np.quantile(hist, cfg.clear_quant, axis=0)
388     q = moving_average_circular(q, cfg.pv_smo_win)
389     # 样本短时与附件1先验混合
390     prior_w = max(0.0, 1.0 - w/21.0)
391     base = (1-prior_w)*q + prior_w*prior
392     # 夜间稳定置0: 历史90%以上为0即置0
393     zero_ratio = np.mean(hist <= 1e-8, axis=0)
394     base[zero_ratio > 0.90] = 0.0
395     return np.maximum(base, 0.0)
396 def pv_forecast(day_idx: int, pv: np.ndarray, prior: np.ndarray, cfg: Config, logger=None) ->
    Tuple[np.ndarray, Dict]:
397     H = cfg.fore_slots
398     if day_idx == 0:
399         return np.tile(prior, 2), {"kappa_d": 1.0, "kappa_next": 1.0, "envelope": prior.copy()}
400     B_d = build_clear_envelope(day_idx, pv, prior, cfg)
401     # 下一日包络: 年周期变化慢, 先用当前包络; 用近期日能量变化率做轻微尺度外推
402     recent_energy = np.sum(pv[max(0, day_idx-14):day_idx], axis=1)*cfg.dt
403     scale_next = 1.0
404     if len(recent_energy) >= 7:
405         x = np.arange(len(recent_energy))

```

```

406     slope = np.polyfit(x, recent_energy, 1)[0]
407     denom = max(np.mean(recent_energy), 1.0)
408     scale_next = float(np.clip(1.0 + slope/denom, 0.95, 1.05))
409     B_next = B_d * scale_next
410     # 用当前滚动包络近似反演近期日透射率; 只使用过去日期
411     w = min(day_idx, cfg.pv_wdays)
412     hist = pv[day_idx-w:day_idx]
413     denom = max(np.sum(B_d)*cfg.dt, 1e-6)
414     kappas = np.clip(np.sum(hist, axis=1)*cfg.dt/denom, 0.0, 1.0)
415     kpred = choose_ar_forecast(kappas, 2, cfg.kar_candi, fallback=float(np.mean(kappas)))
416     kpred = np.clip(kpred, 0.0, 1.0)
417     base_d = B_d*kpred[0]
418     base_next = B_next*kpred[1]
419     # 残差构造: 每个历史日用其日能量对应kappa相对于当前包络重构
420     residual_days = hist - kappas[:, None]*B_d[None, :]
421     resid_series = residual_days.reshape(-1)
422     rpred = choose_ar_forecast(resid_series[-min(len(resid_series), 21*144):], H, cfg.
423     pv_ar_candi, 0.0)
424     pred = np.concatenate([base_d, base_next]) + rpred
425     pred = np.maximum(pred, 0.0)
426     # 夜间稳定无发电时段置零
427     zero_mask = B_d <= 1e-8
428     for h in range(H):
429         if zero_mask[h % 144]:
430             pred[h] = 0.0
431     return pred, {"kappa_d": float(kpred[0]), "kappa_next": float(kpred[1]), "envelope": B_d}
432 def compute_historical_marginal_cost_day(j: int,
433     available_day_exclusive: int,
434     net_errors: np.ndarray,
435     price_actual: np.ndarray,
436     soc_hist: np.ndarray,
437     planned_hist: np.ndarray,
438     load: np.ndarray,
439     pv: np.ndarray,
440     cfg: Config) -> Tuple[np.ndarray, np.ndarray]:
441     nT = cfg.slots_pday
442     cplus = np.full(nT, np.nan, dtype=float)
443     cminus = np.full(nT, np.nan, dtype=float)
444     if j < 0 or j >= available_day_exclusive:
445         return cplus, cminus
446     # 只把当前允许访问的历史展开; 后续窗口均是连续切片
447     n_avail = available_day_exclusive * nT
448     plan_f = planned_hist[:available_day_exclusive].reshape(-1)
449     load_f = load[:available_day_exclusive].reshape(-1)
450     pv_f = pv[:available_day_exclusive].reshape(-1)
451     soc_f = soc_hist[:available_day_exclusive].reshape(-1)
452     err_f = net_errors[:available_day_exclusive].reshape(-1)
453     slot_price = price_actual[:available_day_exclusive].reshape(-1)
454     base = j * nT
455     eff_rt = cfg.eta_c * cfg.eta_d
456     for t in range(nT):
457         e = net_errors[j, t]
458         if not np.isfinite(e):
459             continue
460         lam_t = price_actual[j, t]
461         s0 = soc_hist[j, t] if np.isfinite(soc_hist[j, t]) else cfg.e0
462         g0 = base + t
463         g1 = g0 + 1
464         g2 = min(g0 + cfg.hori_slots, n_avail - 1)
465         if e > 0:
466             c_em = 5.0 * lam_t
467             c_bat = np.inf
468             if s0 > cfg.e_min + 1e-6 and g1 <= g2:
469                 s1 = slice(g1, g2 + 1)
470                 margin = plan_f[s1] + pv_f[s1] - load_f[s1]
471                 sf = soc_f[s1]
472                 pf = slot_price[s1]
473                 soc_ok = (~np.isfinite(sf)) | (sf < cfg.e_max - 1e-6)
474                 feasible = soc_ok & ((margin > 0.0) | (pf < lam_t))
475                 if np.any(feasible):
476                     c_bat = float(np.min(pf[feasible]) / eff_rt)
477             cplus[t] = min(c_em, c_bat) if np.isfinite(c_bat) else c_em
478         elif e < 0:
479             recovery = 0.0
480             if s0 < cfg.e_max - 1e-6 and g1 <= g2:
481                 s1 = slice(g1, g2 + 1)
482                 ef = err_f[s1]
483                 pf = slot_price[s1]
484                 m = np.isfinite(ef) & (ef > 0.0)
485                 if np.any(m):
486                     recovery = float(np.max(5.0 * pf[m] * eff_rt))
487             cminus[t] = max(0.0, lam_t - recovery)
488     return cplus, cminus
489 def update_risk_cost_cache(day_idx: int,

```

```

489         cplus_cache: np.ndarray,
490         cminus_cache: np.ndarray,
491         net_errors: np.ndarray,
492         price_actual: np.ndarray,
493         soc_hist: np.ndarray,
494         planned_hist: np.ndarray,
495         load: np.ndarray,
496         pv: np.ndarray,
497         cfg: Config) -> None:
498     if day_idx <= 0:
499         return
500     # 昨天: 只能使用到昨天24:00, 保持严格无未来信息泄露
501     j = day_idx - 1
502     cp, cm = compute_historical_marginal_cost_day(
503         j, day_idx, net_errors, price_actual, soc_hist, planned_hist, load, pv, cfg)
504     cplus_cache[j] = cp
505     cminus_cache[j] = cm
506     if day_idx >= 2:
507         j = day_idx - 2
508         cp, cm = compute_historical_marginal_cost_day(
509             j, day_idx, net_errors, price_actual, soc_hist, planned_hist, load, pv, cfg)
510         cplus_cache[j] = cp
511         cminus_cache[j] = cm
512 def dynamic_risk_weights(day_idx: int,
513     net_errors: np.ndarray,
514     cplus_cache: np.ndarray,
515     cminus_cache: np.ndarray,
516     cfg: Config) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
517     nT = cfg.slots_pday
518     alpha = np.full(nT, cfg.alpha_cold, dtype=float)
519     beta = np.full(nT, cfg.beta_fixed, dtype=float)
520     reserve = np.zeros(nT, dtype=float)
521     if day_idx <= 0:
522         return alpha, beta, reserve
523     j0 = max(0, day_idx - cfg.rk_hisdays)
524     hist_days = np.arange(j0, day_idx)
525     if hist_days.size == 0:
526         return alpha, beta, reserve
527     ages = day_idx - hist_days
528     base_w = cfg.rk_decay ** (ages - 1)
529     errs = net_errors[hist_days] # (HIST, 144)
530     cp_all = cplus_cache[hist_days]
531     cm_all = cminus_cache[hist_days]
532     for t in range(nT):
533         et = errs[:, t]
534         # 正向误差历史
535         mp = (et > 0) & np.isfinite(et) & np.isfinite(cp_all[:, t])
536         # 负向误差历史
537         mn = (et < 0) & np.isfinite(et) & np.isfinite(cm_all[:, t])
538         if np.count_nonzero(mp) >= cfg.rk_minsamp and np.count_nonzero(mn) >= cfg.rk_minsamp:
539             cp = float(np.average(cp_all[mp, t], weights=base_w[mp]))
540             cm = float(np.average(cm_all[mn, t], weights=base_w[mn]))
541             raw = cp / max(cm, cfg.c_eps)
542             alpha[t] = min(cfg.alpha_max, max(cfg.alpha_min, raw))
543             tau = alpha[t] / (alpha[t] + beta[t])
544             tau = min(cfg.tau_max, max(cfg.tau_min, tau))
545             if np.count_nonzero(mp) >= cfg.rk_minsamp:
546                 reserve[t] = cfg.reserve_scale * weighted_quantile(et[mp], base_w[mp], tau)
547     return alpha, beta, reserve
548 @dataclass
549 class LPStructure:
550     Aeq: sparse.csr_matrix
551     bounds: list
552 def build_lp_structure(cfg: Config) -> LPStructure:
553     """全年只构造一次等式矩阵; 每天仅更新价格目标与净负荷右端项。"""
554     n = cfg.slots_pday
555     N = 5*n
556     ib, ic, idis, isu, ie = 0, n, 2*n, 3*n, 4*n
557     rows, cols, data = [], [], []
558     r = 0
559     for t in range(n):
560         for col, val in ((ib+t, 1.0), (idis+t, 1.0), (ic+t, -1.0), (isu+t, -1.0)):
561             rows.append(r); cols.append(col); data.append(val)
562             r += 1
563     for t in range(n):
564         rows.append(r); cols.append(ie+t); data.append(1.0)
565         if t>0:
566             rows.append(r); cols.append(ie+t-1); data.append(-1.0)
567             rows.append(r); cols.append(ic+t); data.append(-cfg.eta_c*cfg.dt)
568             rows.append(r); cols.append(idis+t); data.append(cfg.dt/cfg.eta_d)
569             r += 1
570     Aeq = sparse.csr_matrix((data, (rows, cols)), shape=(r, N))
571     bounds = ([ (0, None)]*n + [(0, cfg.p_max)]*n + [(0, cfg.p_max)]*n +
572         [(0, None)]*n + [(cfg.e_min, cfg.e_max)]*n)

```

```

573     return LPStructure(Aeq,bounds)
574 def solve_day_ahead(price_fc_48: np.ndarray, load_fc: np.ndarray,
575                    pv_fc: np.ndarray, reserve: np.ndarray, soc0: float,
576                    cfg: Config, lp: LPStructure) -> Dict[str,np.ndarray]:
577     """每天0:00求解一次日前LP; 当日价格用于目标, 次日预测最低价定义日末SOC机会价值。"""
578     n=cfg.slots_pday
579     net_req = load_fc[:n] - pv_fc[:n] + reserve
580     b=np.empty(2*n,dtype=float); b[:n]=net_req; b[n:]=0.0; b[n]=soc0
581     price_today=np.asarray(price_fc_48[:n],dtype=float)
582     if len(price_fc_48)>=2*n:
583         next_day=np.asarray(price_fc_48[n:2*n],dtype=float)
584     else:
585         next_day=price_today
586     mu_terminal=cfg.eta_d*float(np.min(next_day))
587     c=np.zeros(5*n,dtype=float)
588     c[:n]=price_today*cfg.dt; c[n:3*n]=cfg.eps_cyc*cfg.dt
589     c[3*n:4*n]=cfg.eps_surplus*cfg.dt; c[5*n-1]=-mu_terminal
590     res=linprog(c,A_eq=lp.Aeq,b_eq=b,bounds=lp.bounds,method=cfg.lp_method)
591     if not res.success:
592         raise RuntimeError(f"日前LP失败: {res.message}")
593     x=res.x
594     return {"buy":x[:n],"ch":x[n:2*n],"dis":x[2*n:3*n],"sur":x[3*n:4*n],
595            "E":x[4*n:5*n],"obj":float(res.fun+mu_terminal*soc0),"mu_terminal":mu_terminal}
596 def operate_one_slot(soc0: float, buy: float, load_actual: float, pv_actual: float,
597                    plan_ch: float, plan_dis: float, cfg: Config) -> Dict[str,float]:
598     dt = cfg.dt
599     # 先把计划动作投影到当前SOC可行范围
600     ch = max(0.0, min(float(plan_ch), cfg.p_max))
601     dis = max(0.0, min(float(plan_dis), cfg.p_max))
602     max_ch_soc = max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt))
603     max_dis_soc = max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt)
604     ch = min(ch, max_ch_soc)
605     dis = min(dis, max_dis_soc)
606     # 基于真实负荷/光伏检查功率平衡
607     gap = float(load_actual - (buy + pv_actual + dis - ch))
608     emg = sur = 0.0
609     if gap > 1e-10:
610         cut_ch = min(ch, gap); ch -= cut_ch; gap -= cut_ch
611         max_dis_total = min(cfg.p_max, max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt))
612         extra_dis = min(gap, max(0.0, max_dis_total - dis))
613         dis += extra_dis; gap -= extra_dis
614         emg = max(0.0, gap)
615     elif gap < -1e-10:
616         excess = -gap
617         # 1. 先取消不必要的计划放电
618         cut_dis = min(dis, excess); dis -= cut_dis; excess -= cut_dis
619         # 2. 再增加充电
620         max_ch_total = min(cfg.p_max, max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt)))
621         extra_ch = min(excess, max(0.0, max_ch_total - ch))
622         ch += extra_ch; excess -= extra_ch
623         # 3. 电池无法继续吸收的部分作为富余功率
624         sur = max(0.0, excess)
625     soc1 = soc0 + (cfg.eta_c*ch - dis/cfg.eta_d)*dt
626     soc1 = min(cfg.e_max, max(cfg.e_min, soc1))
627     return {"ch":ch, "dis":dis, "emg":emg, "sur":sur, "E":soc1}
628 def fill_result4_2(template_path:str, out_path:str, dates, price_actual,
629                  plan_buy, ch, dis, soc_start, soc_end, emg, cfg:Config, logger):
630     from copy import copy
631     wb = load_workbook(template_path)
632     sp, sc, se = wb["计划购电量"], wb["充放电量"], wb["紧急购电量"]
633     date_to_idx={d:i for i,d in enumerate(dates)}
634     first_submit = date(2025, 2, 1)
635     computed_dates=[d for d in dates if d >= first_submit and d in date_to_idx
636                    and np.isfinite(plan_buy[date_to_idx[d]]).all()]
637     # 计划购电量: 模板本身已经逐日展开
638     for r in range(2, sp.max_row + 1):
639         raw_date = sp.cell(r,1).value
640         if raw_date is None:
641             continue
642         try:
643             d=date_to_date(raw_date)
644         except Exception:
645             continue
646         if d not in date_to_idx:
647             continue
648         i=date_to_idx[d]
649         if not np.isfinite(plan_buy[i]).all():
650             continue
651         vals=(plan_buy[i]*cfg.dt).tolist() # kWh/10min
652         for c, v in enumerate(vals, start=2):
653             sp.cell(r,c).value = None if not np.isfinite(v) else float(v)
654         sp.cell(r,146).value=float(np.nansum(plan_buy[i])*cfg.dt) # EP 全天购电量
655         sp.cell(r,147).value=float(np.nansum(plan_buy[i]*price_actual[i])*cfg.dt) # EQ 全天

```

```

656 购电费
657 charge_proto=[]
658 for rr in range(2,8):
659     charge_proto.append({
660         "height": sc.row_dimensions[rr].height,
661         "cells": [sc.cell(rr,cc) for cc in range(1,7)]
662     })
663 emg_proto=[]
664 for rr in range(2,5):
665     emg_proto.append({"height": se.row_dimensions[rr].height,
666         "cells": [se.cell(rr,cc) for cc in range(1,4)]})
667 def copy_cell_style(src, dst):
668     if src.has_style:
669         dst._style = copy(src._style)
670     if src.number_format:
671         dst.number_format = src.number_format
672     dst.font = copy(src.font); dst.fill = copy(src.fill)
673     dst.border = copy(src.border); dst.alignment = copy(src.alignment)
674     dst.protection = copy(src.protection)
675 # 充放电量：彻底展开 2/1--已计算末日
676 if sc.max_row >= 2:
677     sc.delete_rows(2, sc.max_row-1)
678 intervals=[(0,4,"0:00-4:00"),(4,8,"4:00-8:00"),(8,12,"8:00-12:00"),
679     (12,16,"12:00-16:00"),(16,20,"16:00-20:00"),(20,24,"20:00-24:00")]
680 r=2
681 for d in computed_dates:
682     i=date_to_idx[d]
683     for k,(h0,h1,label) in enumerate(intervals):
684         proto=charge_proto[k]
685         sc.row_dimensions[r].height = proto["height"]
686         for cc in range(1,7):
687             copy_cell_style(proto["cells"][cc-1], sc.cell(r,cc))
688         sc.cell(r,1).value = d if k==0 else None
689         sc.cell(r,2).value = label
690         s0=h0*6; s1=h1*6
691         sc.cell(r,3).value=float(np.nansum(ch[i,s0:s1])*cfg.dt)
692         sc.cell(r,4).value=float(np.nansum(dis[i,s0:s1])*cfg.dt)
693         if k==0:
694             sc.cell(r,5).value="0:00"
695             sc.cell(r,6).value=float(soc_start[i])
696         elif k==1:
697             sc.cell(r,5).value="24:00"
698             sc.cell(r,6).value=float(soc_end[i])
699         else:
700             sc.cell(r,5).value=None
701             sc.cell(r,6).value=None
702         r += 1
703 # 紧急购电：逐日展开，不保留省略号
704 def fmt_slot(k):
705     mins=k*10
706     if mins == 1440:
707         return "24:00"
708     hh=(mins//60)%24; mm=mins%60
709     return f"{hh}:{mm:02d}"
710 if se.max_row >= 2:
711     se.delete_rows(2, se.max_row-1)
712 r=2
713 for d in computed_dates:
714     i=date_to_idx[d]
715     events=[]; t=0
716     while t<144:
717         if emg[i,t] > 1e-6:
718             st=t; energy=0.0
719             while t<144 and emg[i,t] > 1e-6:
720                 energy += emg[i,t]*cfg.dt
721                 t += 1
722             events.append((f"{fmt_slot(st)}-{fmt_slot(t)}", float(energy)))
723         else:
724             t += 1
725     if not events:
726         events=[(None,None)]
727     for k,(period,energy) in enumerate(events):
728         proto=emg_proto[min(k, len(emg_proto)-1)]
729         se.row_dimensions[r].height = proto["height"]
730         for cc in range(1,4):
731             copy_cell_style(proto["cells"][cc-1], se.cell(r,cc))
732         se.cell(r,1).value = d if k==0 else None
733         se.cell(r,2).value = period; se.cell(r,3).value = energy
734         r += 1
735 # 日期与数值格式统一
736 sc.column_dimensions['A'].width = max(sc.column_dimensions['A'].width or 10, 12)
737 sc.column_dimensions['B'].width = max(sc.column_dimensions['B'].width or 12, 15)
738 se.column_dimensions['A'].width = max(se.column_dimensions['A'].width or 10, 12)
739 se.column_dimensions['B'].width = max(se.column_dimensions['B'].width or 12, 18)

```

```

739 for ws in (sc,se):
740     for rr in range(2, ws.max_row+1):
741         if ws.cell(rr,1).value is not None:
742             ws.cell(rr,1).number_format='yyyy/m/d'
743 for rr in range(2, sc.max_row+1):
744     sc.cell(rr,3).number_format='0.00'
745     sc.cell(rr,4).number_format='0.00'
746     sc.cell(rr,6).number_format='0.00'
747 for rr in range(2, se.max_row+1):
748     se.cell(rr,3).number_format='0.00'
749 Path(out_path).parent.mkdir(parents=True, exist_ok=True)
750 wb.save(out_path)
751 wb.close()
752 logger.info("result4-2 已完整展开并保存: %s", out_path)
753 def write_specified_date_results(dates, price_actual, plan_buy, ch, dis, emg,
754                                 soc_start, soc_end, cfg:Config, logger):
755     targets=[date(2025,3,20), date(2025,6,21), date(2025,9,23), date(2025,12,21)]
756     date_to_idx={d:i for i,d in enumerate(dates)}
757     out_dir=Path(cfg.o_dir)
758     out_dir.mkdir(parents=True, exist_ok=True)
759     # 表1: 指定六个10min时段 + 全天购电量/购电费
760     slot_hours=[10,12,14,16,18,20]
761     table1=[]
762     for d in targets:
763         if d not in date_to_idx:
764             continue
765         i=date_to_idx[d]
766         if not np.isfinite(plan_buy[i]).all():
767             continue
768         vals=[]
769         for h in slot_hours:
770             t=h*6
771             vals.append(float(plan_buy[i,t]*cfg.dt))
772         total_energy=float(np.nansum(plan_buy[i])*cfg.dt)
773         total_cost=float(np.nansum(plan_buy[i]*price_actual[i])*cfg.dt)
774         table1.append([str(d),*vals,total_energy,total_cost])
775     hdr1=['日期', '10:00-10:10/kWh', '12:00-12:10/kWh', '14:00-14:10/kWh',
776          '16:00-16:10/kWh', '18:00-18:10/kWh', '20:00-20:10/kWh',
777          '全天购电量/kWh', '全天购电费/元']
778     # 表2: 每个指定日期6个四小时区间 + 日初/日末SOC
779     intervals=[(0,4,'0:00-4:00'),(4,8,'4:00-8:00'),(8,12,'8:00-12:00'),
780               (12,16,'12:00-16:00'),(16,20,'16:00-20:00'),(20,24,'20:00-24:00')]
781     table2=[]
782     for d in targets:
783         if d not in date_to_idx:
784             continue
785         i=date_to_idx[d]
786         if not np.isfinite(plan_buy[i]).all():
787             continue
788         for k,(h0,h1,label) in enumerate(intervals):
789             s0=h0*6; s1=h1*6
790             table2.append([str(d) if k==0 else '', label,
791                           float(np.nansum(ch[i,s0:s1])*cfg.dt),
792                           float(np.nansum(dis[i,s0:s1])*cfg.dt),
793                           float(soc_start[i]) if k==0 else '',
794                           float(soc_end[i]) if k==0 else ''])
795     hdr2=['日期', '时间段', '充电量/kWh', '放电量/kWh', '0:00储电量/kWh', '24:00储电量/kWh']
796     # 表3: 紧急购电事件, 连续10min时段自动合并
797     def fmt_slot(k):
798         mins=k*10
799         if mins==1440:
800             return '24:00'
801         return f'{mins//60}:{mins%60:02d}'
802     table3=[]
803     for d in targets:
804         if d not in date_to_idx:
805             continue
806         i=date_to_idx[d]
807         if not np.isfinite(plan_buy[i]).all():
808             continue
809         events=[]; t=0
810         while t<144:
811             if emg[i,t] > 1e-6:
812                 st=t; energy=0.0
813                 while t<144 and emg[i,t] > 1e-6:
814                     energy += emg[i,t]*cfg.dt
815                     t += 1
816                 events.append((f'{fmt_slot(st)}-{fmt_slot(t)}', float(energy)))
817             else:
818                 t += 1
819         if not events:
820             table3.append([str(d), '无紧急购电', 0.0])
821     else:

```

```

822         for k,(period,energy) in enumerate(events):
823             table3.append([str(d) if k==0 else '',period,energy])
824     hdr3=['日期','紧急购电时间段','紧急购电量/kWh']
825     # 三个表合并写入一个Excel (各占一个sheet)
826     xlsx=out_dir/'Q4-2_指定日期数据.xlsx'
827     book=Workbook(); book.remove(book.active)
828     for name,hdr,rows in (('表1 计划购电',hdr1,table1),
829                          ('表2 充放电',hdr2,table2),
830                          ('表3 紧急购电',hdr3,table3)):
831         ws=book.create_sheet(name)
832         ws.append(hdr)
833         for row in rows:
834             ws.append(row)
835     try:
836         book.save(xlsx)
837     except PermissionError:
838         book.save(str(xlsx).replace('.xlsx','_new.xlsx')) # 文件被占用时改存副本
839     book.close()
840     logger.info("指定日期表1/表2/表3 已合并保存: %s", xlsx)
841 def _style_axes(ax):
842     """统一论文风格: 减少边框、轻虚线网格、字号一致。"""
843     ax.spines['top'].set_visible(False)
844     ax.spines['right'].set_visible(False)
845     ax.grid(True, axis='y', linestyle='--', linewidth=0.7, alpha=0.3)
846     ax.tick_params(labelsize=TICK_FS)
847 NO_TITLE, AXIS_FS, TICK_FS, LEGEND_FS = True, 15, 12, 17
848 def _save_line_fig(x, ys, labels, title, xlabel, ylabel, path, date_axis=False,
849                  hlines=None, ylim=None, colors=None, line_styles=None,
850                  xticks=None, xlim=None, linewidths=None, legend_loc=None):
851     fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
852     for k, (y, lab) in enumerate(zip(ys, labels)):
853         c = colors[k] if colors else None
854         ls = line_styles[k] if line_styles else '-'
855         lw = linewidths[k] if linewidths else 2.2
856         ax.plot(x, y, linewidth=lw, label=lab, color=c, linestyle=ls)
857     if hlines:
858         for item in hlines:
859             yv, lab = item[0], item[1]
860             hc = item[2] if len(item) > 2 else C_SOC
861             ax.axhline(yv, linestyle='--', linewidth=1.0, color=hc, alpha=0.55, label=lab)
862     if not NO_TITLE:
863         ax.set_title(title, fontsize=13, fontweight='bold', pad=28)
864     ax.set_xlabel(xlabel, fontsize=AXIS_FS, color='black')
865     ax.set_ylabel(ylabel, fontsize=AXIS_FS, color='black')
866     _style_axes(ax)
867     if labels or hlines:
868         if legend_loc: # 例如 (1.0, 0.90): 右上、限值线下方
869             ax.legend(loc='upper right', frameon=False, ncol=1,
870                     fontsize=LEGEND_FS-2, bbox_to_anchor=legend_loc)
871         else: # 默认: 图外顶部居中
872             ax.legend(loc='lower center', frameon=False,
873                     ncol=max(1, len(labels)), fontsize=LEGEND_FS,
874                     bbox_to_anchor=(0.5, 1.02), columnspacing=1.8)
875     if date_axis:
876         ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=5, maxticks=9))
877         ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
878         fig.autofmt_xdate(rotation=25)
879     if xticks is not None:
880         ax.set_xticks(xticks)
881     if xlim is not None:
882         ax.set_xlim(*xlim)
883     if ylim is not None:
884         ax.set_ylim(*ylim)
885     fig.tight_layout()
886     fig.savefig(path, dpi=300, bbox_inches='tight')
887     plt.close(fig)
888 def _save_bar_fig(xlabels, values, title, xlabel, ylabel, path, color=C_BUY):
889     fig, ax = plt.subplots(figsize=(10.8, 5.4), dpi=300)
890     xx=np.arange(len(values))
891     ax.bar(xx, values, width=0.55, color=color, alpha=0.85)
892     ax.set_xticks(xx)
893     ax.set_xticklabels(xlabels, fontsize=TICK_FS)
894     if not NO_TITLE:
895         ax.set_title(title, fontsize=13, fontweight='bold', pad=12)
896     ax.set_xlabel(xlabel, fontsize=AXIS_FS, color='black')
897     ax.set_ylabel(ylabel, fontsize=AXIS_FS, color='black')
898     _style_axes(ax)
899     fig.tight_layout()
900     fig.savefig(path, dpi=300, bbox_inches='tight')
901     plt.close(fig)
902 def generate_visualizations(dates, price_actual, price_fc_store, plan_buy, run_D, cfg:Config,
903                             logger):
904     fig_dir=Path(cfg.fig_dir); fig_dir.mkdir(parents=True,exist_ok=True)

```

```

904 valid=[i for i in range(run_D) if dates[i]>=date(2025,2,1) and np.isfinite(plan_buy[i]).all
905 ()]
906 if not valid: return
907 tt=np.arange(144)/6
908 # 指定日波动电价预测对比 (3-20 / 6-21 / 9-23 / 12-21)
909 for td in [date(2025,3,20),date(2025,6,21),date(2025,9,23),date(2025,12,21)]:
910     i=next((j for j in valid if dates[j]==td),None)
911     if i is not None:
912         _save_line_fig(tt,[price_actual[i],price_fc_store[i,:144]],
913             ['实际电价','0:00预测电价'],f'{td} 波动电价预测对比',
914             '时刻 / h','电价 / (元/kWh)',
915             fig_dir/f'第四问{td.month}-{td.day}波动电价预测对比.png',
916             colors=[C_BUY,C_DIS],line_styles=['-','--'],
917             linewidths=[2,0,1.8],
918             xticks=list(range(0,25,2)),xlim=(0,24))
919 # 全年10分钟级电价预测一致性散点图
920 a=price_actual[valid].reshape(-1); p=price_fc_store[valid,:144].reshape(-1)
921 fig,ax=plt.subplots(figsize=(6.2,6.2),dpi=300)
922 ax.scatter(a,p,s=6,alpha=.06,color=C_BUY)
923 lo=float(min(a.min(),p.min())); hi=float(max(a.max(),p.max()))
924 ax.plot([lo,hi],[lo,hi], '--',lw=1.4,color=C_CH,label='y=x')
925 if not NO_TITLE:
926     ax.set_title('全年10分钟级电价预测一致性', fontsize=13, fontweight='bold')
927 ax.set_xlabel('实际电价 / (元/kWh)', fontsize=AXIS_FS, color='black')
928 ax.set_ylabel('预测电价 / (元/kWh)', fontsize=AXIS_FS, color='black')
929 ax.tick_params(labelsize=TICK_FS)
930 _style_axes(ax)
931 ax.legend(loc='upper left', frameon=False, fontsize=13)
932 fig.tight_layout()
933 fig.savefig(fig_dir/'第四问全年10分钟级电价预测一致性.png',dpi=300,bbbox_inches='tight')
934 plt.close(fig)
935 def write_result_summary(dates, price_actual, price_fc_store, plan_buy, emg, sur, ch, dis,
936 soc_end,
937 alpha_store, run_D, cfg:Config, logger):
938 valid=[i for i in range(run_D) if dates[i]>=date(2025,2,1) and np.isfinite(plan_buy[i]).all
939 ()]
940 if not valid: return
941 plan_cost=sum(float(np.sum(plan_buy[i]*price_actual[i])*cfg.dt) for i in valid)
942 emg_cost=sum(float(np.sum(emg[i]*5*price_actual[i])*cfg.dt) for i in valid)
943 emg_energy=sum(float(np.sum(emg[i])*cfg.dt) for i in valid)
944 sur_energy=sum(float(np.sum(sur[i])*cfg.dt) for i in valid)
945 charge_energy=sum(float(np.sum(ch[i])*cfg.dt) for i in valid)
946 discharge_energy=sum(float(np.sum(dis[i])*cfg.dt) for i in valid)
947 ae=np.concatenate([(price_actual[i]-price_fc_store[i,:144]) for i in valid])
948 aa=np.concatenate([price_actual[i] for i in valid])
949 price_mae=float(np.mean(np.abs(ae))); price_rmse=float(np.sqrt(np.mean(ae**2)))
950 denom=np.maximum(np.abs(aa),1e-3); price_mape=float(np.mean(np.abs(ae)/denom)*100)
951 zero=sum(1 for i in valid if np.sum(emg[i])*cfg.dt<=1e-8)
952 lines=[f'问题4-2结果整合 (电价预测驱动, 1月仅预热)',
953     f' 正式统计日期范围: {dates[valid[0]]} 至 {dates[valid[-1]]}',
954     f' 累计计划购电费 (真实电价结算): {plan_cost:.2f} 元',
955     f' 累计紧急购电费 (5倍真实电价): {emg_cost:.2f} 元',
956     f' 累计总费用: {plan_cost+emg_cost:.2f} 元',
957     f' 累计紧急购电量: {emg_energy:.2f} kWh',
958     f' 累计富余电量: {sur_energy:.2f} kWh',
959     f' 累计充电量: {charge_energy:.2f} kWh',
960     f' 累计放电量: {discharge_energy:.2f} kWh',
961     f' 期末储电量: {soc_end[valid[-1]]:.2f} kWh',
962     f' 无紧急购电天数: {zero}/{len(valid)}',
963     f' 电价预测MAE: {price_mae:.6f} 元/kWh',
964     f' 电价预测RMSE: {price_rmse:.6f} 元/kWh',
965     f' 电价预测MAPE: {price_mape:.3f}%',
966     f' 动态alpha平均值: {float(np.nanmean(alpha_store[valid])):.4f}',
967     f' 正式结果文件: {cfg.o_xlsx}',f' 可视化目录: {cfg.fig_dir}']
968 p=Path(cfg.o_dir)/'Q4-2_结果整合.txt'; p.write_text('\n'.join(lines),encoding='utf-8')
969 def main():
970     cfg=CFG; out_dir=Path(cfg.o_dir); out_dir.mkdir(parents=True,exist_ok=True); logger=
971     setup_logger(out_dir)
972     logger.info('Q4-2')
973     logger.info(' 负荷与电价共享MSTL重估周期=%d天, 但模型缓存完全独立.',cfg.mstl_days)
974     dates, price_prior, pv_prior, load, pv, price_actual = load_inputs(cfg,logger)
975     D=len(dates); n=cfg.slots_pday; run_D=min(D,cfg.max_days) if cfg.max_days>0 else D
976     plan_buy=np.full((D,n),np.nan); plan_ch=np.zeros((D,n)); plan_dis=np.zeros((D,n))
977     ch=np.zeros((D,n)); dis=np.zeros((D,n)); emg=np.zeros((D,n)); sur=np.zeros((D,n))
978     soc_hist=np.full((D,n),np.nan); soc_start=np.full(D,np.nan); soc_end=np.full(D,np.nan)
979     load_fc_store=np.full((D,cfg.fore_slots),np.nan); pv_fc_store=np.full((D,cfg.fore_slots),np.
980     nan)
981     price_fc_store=np.full((D,cfg.fore_slots),np.nan)
982     alpha_store=np.full((D,n),np.nan); reserve_store=np.zeros((D,n)); net_errors=np.full((D,n),
983     np.nan)
984     cplus_cache=np.full((D,n),np.nan); cminus_cache=np.full((D,n),np.nan); mu_store=np.full(D,np
985     .nan)

```

```

979 lp=build_lp_structure(cfg); soc=cfg.e0
980 for d in range(run_D):
981     soc_start[d]=soc
982     lf=load_forecast(d,load,load_prior,cfg,logger)
983     pf,_=pv_forecast(d,pv,pv_prior,cfg,logger)
984     prf=price_forecast(d,price_actual,price_prior,cfg,logger)
985     load_fc_store[d]=lf; pv_fc_store[d]=pf; price_fc_store[d]=prf
986     update_risk_cost_cache(d,cplus_cache,cminus_cache,net_errors,price_actual,soc_hist,
plan_buy,load,pv,cfg)
987     alpha,_=reserve=dynamic_risk_weights(d,net_errors,cplus_cache,cminus_cache,cfg)
988     alpha_store[d]=alpha; reserve_store[d]=reserve
989     da=solve_day_ahead(prf,lf,pf,reserve,soc,cfg,lp); plan_buy[d]=da['buy']; plan_ch[d]=da['
ch']; plan_dis[d]=da['dis']; mu_store[d]=da['mu_terminal']
990     for t in range(n):
991         soc_hist[d,t]=soc
992         sol=operate_one_slot(soc,plan_buy[d,t],load[d,t],pv[d,t],plan_ch[d,t],plan_dis[d,t],
cfg)
993         ch[d,t]=sol['ch']; dis[d,t]=sol['dis']; emg[d,t]=sol['emg']; sur[d,t]=sol['sur'];
soc=sol['E']
994         soc_end[d]=soc
995         net_errors[d]=(load[d]-pv[d])-(lf[:n]-pf[:n])
996         if cfg.checkpoint_every_days>0 and ((d+1)%cfg.checkpoint_every_days==0 or d==run_D-1):
997             np.savez_compressed(out_dir/'q4_2_checkpoint.npz',last_day=d,plan_buy=plan_buy[:d
+1],ch=ch[:d+1],dis=dis[:d+1],emg=emg[:d+1],sur=sur[:d+1],soc_start=soc_start[:d+1],soc_end
=soc_end[:d+1],load_fc=load_fc_store[:d+1],pv_fc=pv_fc_store[:d+1],price_fc=price_fc_store
[:d+1],alpha=alpha_store[:d+1],reserve=reserve_store[:d+1],mu=mu_store[:d+1])
998         if run_D>=32 and Path(cfg.temp).exists():
999             fill_result4_2(cfg.temp,cfg.o_xlsx,dates,price_actual,plan_buy,ch,dis,soc_start,soc_end,
emg,cfg,logger)
1000     elif run_D>=32:
1001         logger.warning('未找到 %s: 图和其余结果仍正常输出。',cfg.temp)
1002     write_specified_date_results(dates,price_actual,plan_buy,ch,dis,emg,soc_start,soc_end,cfg,
logger)
1003     generate_visualizations(dates,price_actual,price_fc_store,plan_buy,run_D,cfg,logger)
1004     write_result_summary(dates,price_actual,price_fc_store,plan_buy,emg,sur,ch,dis,soc_end,
alpha_store,run_D,cfg,logger)
1005     logger.info('Q4-2全部完成, 输出目录: %s',out_dir)
1006 if __name__ == '__main__':
1007     main()

```

code/Q4-2.py

Q4-3.py

```

1 from __future__ import annotations; import os, sys, csv, math, time, logging
2 from dataclasses import dataclass; from pathlib import Path
3 from datetime import datetime, timedelta, date
4 from typing import List, Tuple, Dict, Optional; import numpy as np
5 from scipy.optimize import linprog; from scipy import sparse
6 try:
7     from tqdm import tqdm
8 except Exception:
9     def tqdm(it, **kwargs): return it
10 from statsmodels.tsa.ar_model import AutoReg
11 try:
12     from statsmodels.tsa.seasonal import MSTL; HAS_MSTL=True
13 except Exception:
14     HAS_MSTL=False
15 try:
16     from openpyxl import load_workbook
17 except ImportError as e:
18     raise ImportError("缺少 openpyxl. 请执行: pip install openpyxl") from e
19 class Config:
20     base_dir=str(Path(__file__).resolve().parent)
21     att1=str(Path(base_dir)/"附件1.xlsx"); att2=str(Path(base_dir)/"附件2.xlsx")
22     att3=str(Path(base_dir)/"附件3.xlsx"); att4=str(Path(base_dir)/"附件4.xlsx")
23     template=str(Path(base_dir)/"附件5"/"result4-3.xlsx")
24     if not Path(template).exists():
25         for _name in ("result4-3.xlsx","result4-3(1).xlsx"):
26             _cand=Path(base_dir)/_name
27             if _cand.exists(): template=str(_cand); break
28     out_xlsx=str(Path(base_dir)/"Q4-3结果"/"result4-3_filled.xlsx")
29     out_dir=str(Path(base_dir)/"Q4-3结果")
30     slots_per_day=144; dt=1.0/6.0; horizon_slots=144; forecast_slots=288
31     rolling_horizon_slots=144
32     eta_c=0.90; eta_d=0.90; e_min=1200.0; e_max=10800.0; p_max=5000.0; e0=6000.0
33     load_window_days=42; price_window_days=42
34     load_min_mstl_days=21; price_min_mstl_days=21; mstl_refit_days=7
35     ar_lags_candidates=(1,2,3,6,12,18,36)
36     price_ar_lags_candidates=(1,2,3,6,12,18,36)
37     # Q3正式模型风险参数保持不变
38     risk_history_days=28; risk_decay=0.90; risk_min_samples=5

```

```

39 alpha_cold=4.0; alpha_min=1.0; alpha_max=8.0; beta_fixed=1.0
40 c_eps=1e-4; tau_min=0.55; tau_max=0.88; reserve_scale=0.75
41 use_intraday_load_bias=False
42 stage_reserve_quantile=0.80; stage_reserve_scale=0.75
43 stage_reserve_min_days=7; stage_reserve_history_days=35
44 stage_reserve_decay=0.94
45 # 日内电价滚动修正：仅用发布时刻之前最近3h真实电价误差
46 price_intraday_recent_slots=18; price_intraday_decay_hours=4.0
47 price_intraday_bias_clip=0.35
48 eps_cycle=1e-5; eps_surplus=1e-8; lp_method="highs"
49 run_comparison=True; comparison_schemes=((),(6,),(12,),(18,),(6,12,18))
50 comparison_names=( "仅0:00", "0:00+6:00", "0:00+12:00", "0:00+18:00",
51 "0:00+6:00+12:00+18:00")
52 focus_scheme_hours=(12,); focus_scheme_name="0:00+12:00"
53 representative_dates=( date(2025, 3, 20), date(2025, 6, 21), date(2025, 9,
54 23), date(2025, 12, 21))
55 max_days=365; checkpoint_every_days=30; verbose_each_day=False
56 reuse_q42_forecast_cache=True
57 q42_cache=str(Path(base_dir)/"Q4-2结果"/"q4_2_checkpoint.npz")
58 CFG=Config()
59 def setup_logger(out_dir:Path->logging.Logger:
60 out_dir.mkdir(parents=True,exist_ok=True); logger=logging.getLogger("Q4-3")
61 logger.setLevel(logging.INFO); logger.handlers.clear()
62 fmt=logging.Formatter("%(message)s")
63 sh=logging.StreamHandler(sys.stdout); sh.setFormatter(fmt)
64 fh=logging.FileHandler(out_dir/"q4_3_run.log",mode="w",encoding="utf-8")
65 fh.setFormatter(fmt)
66 logger.addHandler(sh); logger.addHandler(fh); return logger
67 def excel_serial_to_date(x) -> datetime:
68 return datetime(1899, 12, 30) + timedelta(days=float(x))
69 def cell_to_date(x):
70 """兼容 openpyxl 返回的 date/datetime、Excel 序列号和常见日期字符串。"""
71 if x is None:
72 return None
73 if isinstance(x, datetime):
74 return x.date()
75 try:
76 from datetime import date as _date
77 if isinstance(x, _date):
78 return x
79 except Exception:
80 pass
81 if isinstance(x, (int, float, np.integer, np.floating)):
82 return excel_serial_to_date(x).date()
83 if isinstance(x, str):
84 txt=x.strip()
85 for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%Y.%m.%d"):
86 try:
87 return datetime.strptime(txt, fmt).date()
88 except ValueError:
89 pass
90 raise ValueError(f"无法识别日期单元格: {x!r}")
91 def safe_float_array(x) -> np.ndarray:
92 return np.asarray( [[float(v) if v is not None else np.nan for v in row] for
93 row in x], dtype=float)
94 def weighted_quantile( values: np.ndarray, weights: np.ndarray,
95 q: float) -> float:
96 values = np.asarray(values, dtype=float)
97 weights = np.asarray(weights, dtype=float)
98 m = np.isfinite(values) & np.isfinite(weights) & (weights > 0)
99 if m.sum() == 0:
100 return 0.0
101 v, w = values[m], weights[m]; idx = np.argsort(v); v, w = v[idx], w[idx]
102 cw = np.cumsum(w) / np.sum(w); return float(np.interp(q, cw, v))
103 def moving_average_circular(x: np.ndarray, half: int) -> np.ndarray:
104 if half <= 0:
105 return x.copy()
106 n = len(x); out = np.zeros(n)
107 for i in range(n):
108 lo = max(0, i-half); hi = min(n, i+half+1); out[i] = np.mean(x[lo:hi])
109 return out
110 def choose_ar_forecast( series: np.ndarray, horizon: int, candidates: Tuple[int,
111 ...], fallback: float = 0.0) -> np.ndarray:
112 """用AIC在少量AutoReg候选中选阶；失败时返回fallback/近期均值。"""
113 y = np.asarray(series, dtype=float); y = y[np.isfinite(y)]
114 if len(y) < 8 or np.std(y) < 1e-8:
115 return np.full(horizon, float(np.mean(y) if len(y) else fallback))
116 best = None
117 for p in candidates:
118 if len(y) <= p + 5:
119 continue
120 try:
121 model = AutoReg(y, lags=p, trend="ct", old_names=False).fit()
122 if np.isfinite(model.aic) and (best is None or model.aic < best[0]):

```

```

123         best = (model.aic, model)
124     except Exception:
125         continue
126 if best is None:
127     return np.full(horizon, float(np.mean(y[-min(7, len(y)):]))) )
128 try:
129     pred = np.asarray( best[1].predict(start=len(y), end=len(y)+ horizon- 1,
130         dynamic=False), dtype=float)
131     if np.any(~np.isfinite(pred)):
132         raise ValueError("AR forecast nonfinite")
133     return pred
134 except Exception:
135     return np.full(horizon, float(np.mean(y[-min(7, len(y)):]))) )
136 def load_inputs(cfg:Config,logger):
137     logger.info("【数据读取】正在读取附件1、附件2、附件3、附件4……")
138     for attr, names in [ ('att1', ('附件1.xlsx', '附件1(3).xlsx', '附件1(2).xlsx')),
139         ('att2', ('附件2.xlsx', '附件2(3).xlsx', '附件2(1).xlsx')), ('att3',
140         ('附件3.xlsx', '附件3(1).xlsx')), ('att4', ('附件4.xlsx', '附件4(2).xlsx'))]:
141         if not Path(getattr(cfg, attr)).exists():
142             for name in names:
143                 cand=Path(cfg.base_dir)/name
144                 if cand.exists(): setattr(cfg, attr, str(cand)); break
145     for p in (cfg.att1, cfg.att2, cfg.att3, cfg.att4):
146         if not Path(p).exists(): raise FileNotFoundError(f"未找到文件: {p}")
147     wb1=load_workbook(cfg.att1, data_only=True, read_only=True)
148     ws1=wb1["Sheet1"] if "Sheet1" in wb1.sheetnames else wb1[wb1.sheetnames[0]]
149     rows1=list( ws1.iter_rows(min_row=2, max_row=145, min_col=2, max_col=4,
150         values_only=True))
151     wb1.close()
152     price_prior=np.asarray([float(r[0]) for r in rows1], dtype=float)
153     load_prior=np.asarray([float(r[1]) for r in rows1], dtype=float)
154     wb2=load_workbook(cfg.att2, data_only=True, read_only=True)
155     sl=wb2["小区负载"]; sp=wb2["光伏发电实际功率"]
156     load_all=list( sl.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145,
157         values_only=True))
158     pv_all=list( sp.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145,
159         values_only=True))
160     wb2.close(); dates=[cell_to_date(r[0]) for r in load_all]
161     load=safe_float_array([r[1:] for r in load_all])
162     pv=safe_float_array([r[1:] for r in pv_all])
163     wb4=load_workbook(cfg.att4, data_only=True, read_only=True)
164     ws4=wb4["Sheet1"] if "Sheet1" in wb4.sheetnames else wb4[wb4.sheetnames[0]]
165     pr_rows=list( ws4.iter_rows(min_row=2, max_row=366, min_col=1, max_col=145,
166         values_only=True))
167     wb4.close(); price_dates=[cell_to_date(r[0]) for r in pr_rows]
168     price_actual=safe_float_array([r[1:] for r in pr_rows])
169     if load.shape!=( 365, 144) or pv.shape!=( 365, 144) or price_actual.shape!=(
170         365, 144):
171         raise ValueError(
172             f"数据维度异常: load={load.shape}, pv={pv.shape}, price={price_actual.shape}")
173     if dates!=price_dates: raise ValueError("附件2与附件4日期未完全对齐。")
174     if np.any( ~np.isfinite(price_actual)) or np.any( price_actual< 0):
175         raise ValueError("附件4存在缺失或负电价。")
176     logger.info( "【数据读取】完成: 365天×144个10分钟时段; 电价范围 %.4f~%.4f 元/kWh。",
177         float(np.min(price_actual)), float(np.max(price_actual)))
178     return dates, price_prior, load_prior, load, pv, price_actual
179 _LOAD_MSTL_CACHE = {}
180 def load_forecast( day_idx: int, load: np.ndarray, prior: np.ndarray,
181     cfg: Config, logger=None) -> np.ndarray:
182     H = cfg.forecast_slots
183     if day_idx == 0:
184         return np.tile(prior, 2)
185     hist_days = min(day_idx, cfg.load_window_days)
186     hist_mat = load[day_idx-hist_days:day_idx]
187     # 冷启动: 同一时刻近期日加权 + 附件1先验
188     if hist_days < cfg.load_min_mstl_days or not HAS_MSTL:
189         k = hist_mat.shape[0]; weights = np.exp(-0.10*np.arange(k-1, -1, -1))
190         weights /= weights.sum()
191         shape = np.sum(hist_mat * weights[:, None], axis=0)
192         if hist_days >= 7:
193             shape = 0.55*shape + 0.45*load[day_idx-7]
194         w_prior = max(0.0, 1.0 - hist_days / cfg.load_min_mstl_days)
195         shape = (1-w_prior)*shape + w_prior*prior; return np.tile(shape, 2)
196     # 仅每隔若干天完整重估一次 MSTL; 数据已确认无明显异常, 因此关闭 robust 迭代
197     need_refit = ( not _LOAD_MSTL_CACHE or day_idx -
198         _LOAD_MSTL_CACHE.get("refit_day", - 10**9) >= cfg.mstl_refit_days )
199     if need_refit:
200         y = hist_mat.reshape(-1)
201         try:
202             mstl = MSTL( y, periods=(144, 1008),
203                 stl_kwargs={"robust": False}).fit( )
204             trend = np.asarray(mstl.trend); seasonal = np.asarray(mstl.seasonal)
205             if seasonal.ndim == 1:

```

```

206         s_daily = seasonal; s_weekly = np.zeros_like(seasonal)
207     else:
208         s_daily = seasonal[:, 0]; s_weekly = seasonal[:, 1]
209     resid = np.asarray(mstl.resid)
210     # 保存稳定的日周期与周周期结构, 供未来几天复用
211     daily_tail = s_daily[-min(len(s_daily), 28*144):]
212     weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
213     daily_pattern = np.array( [ np.nanmean(daily_tail[i::144]) for
214                               i in range(144) ])
215     weekly_pattern = np.array( [ np.nanmean(weekly_tail[i::1008]) for
216                                i in range(1008) ])
217     _LOAD_MSTL_CACHE.clear()
218     _LOAD_MSTL_CACHE.update( { "refit_day": day_idx,
219                               "origin_day": day_idx - hist_days,
220                               "daily_pattern": daily_pattern, "weekly_pattern": weekly_pattern,
221                               "trend_last": float(trend[-1]),
222                               "trend_slope": float(np.clip( np.polyfit( np.arange(min(432,
223                               len(trend))), dtype=float), trend[- min(432, len(trend)):],
224                               1 )[0], - 5.0, 5.0 )), "resid_tail": resid[- min(len(resid), 14*
225                               144):].copy(), })
226     if logger:
227         logger.info("负荷预测: 第 %d 天重新拟合 MSTL (日/周周期)。", day_idx + 1)
228     except Exception as e:
229         if logger:
230             logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
231         k = hist_mat.shape[0]
232         weights = np.exp(-0.10*np.arange(k-1, -1, -1))
233         weights /= weights.sum()
234         shape = np.sum(hist_mat * weights[:, None], axis=0)
235         if hist_days >= 7:
236             shape = 0.5*shape + 0.5*load[day_idx-7]
237         return np.tile(shape, 2)
238     # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
239     daily_pattern = _LOAD_MSTL_CACHE["daily_pattern"]
240     weekly_pattern = _LOAD_MSTL_CACHE["weekly_pattern"]
241     origin_day = _LOAD_MSTL_CACHE["origin_day"]; recent_days = min(hist_days, 3)
242     recent = load[day_idx-recent_days:day_idx].reshape(-1)
243     global_start = (day_idx - recent_days) * 144
244     gidx = global_start + np.arange(len(recent))
245     sd_hist = daily_pattern[gidx % 144]
246     sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
247     deseason = recent - sd_hist - sw_hist; m = min(432, len(deseason))
248     x = np.arange(m, dtype=float); coef = np.polyfit(x, deseason[-m:], 1)
249     slope = float(np.clip(coef[0], -5.0, 5.0)); level = float(deseason[-1])
250     trend_future = level + slope*np.arange(1, H+1)
251     future_gidx = day_idx*144 + np.arange(H)
252     sd_future = daily_pattern[future_gidx % 144]
253     sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
254     # 残差 AR 仍每天轻量更新, 不需要重新做 MSTL
255     hist_axis = np.arange(len(deseason), dtype=float)
256     fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
257     resid_recent = deseason - fitted_trend_hist
258     ar_future = choose_ar_forecast( resid_recent[- min(len(resid_recent), 3*
259     144):], H, cfg.ar_lags_candidates, 0.0, )
260     pred = trend_future + sd_future + sw_future + ar_future
261     return np.maximum(pred, 0.0)
262 _PRICE_MSTL_CACHE = {}
263 def price_forecast( day_idx: int, price_actual: np.ndarray, prior: np.ndarray,
264                   cfg: Config, logger=None) -> np.ndarray:
265     H = cfg.forecast_slots
266     if day_idx == 0:
267         return np.tile(prior, 2)
268     hist_days = min(day_idx, cfg.price_window_days)
269     hist_mat = price_actual[day_idx-hist_days:day_idx]
270     # 冷启动: 同一时刻近期日加权 + 附件1先验
271     if hist_days < cfg.price_min_mstl_days or not HAS_MSTL:
272         k = hist_mat.shape[0]; weights = np.exp(-0.10*np.arange(k-1, -1, -1))
273         weights /= weights.sum()
274         shape = np.sum(hist_mat * weights[:, None], axis=0)
275         if hist_days >= 7:
276             shape = 0.55*shape + 0.45*price_actual[day_idx-7]
277         w_prior = max(0.0, 1.0 - hist_days / cfg.price_min_mstl_days)
278         shape = (1-w_prior)*shape + w_prior*prior; return np.tile(shape, 2)
279     # 仅每隔若干天完整重估一次 MSTL; 数据已确认无明显异常, 因此关闭 robust 迭代
280     need_refit = ( not _PRICE_MSTL_CACHE.get("refit_day", - 10**9) >= cfg.mstl_refit_days )
281     if need_refit:
282         y = hist_mat.reshape(-1)
283         try:
284             mstl = MSTL( y, periods=(144, 1008),
285                         stl_kwargs={"robust": False}).fit( )
286             trend = np.asarray(mstl.trend); seasonal = np.asarray(mstl.seasonal)
287             if seasonal.ndim == 1:

```

```

289         s_daily = seasonal; s_weekly = np.zeros_like(seasonal)
290     else:
291         s_daily = seasonal[:, 0]; s_weekly = seasonal[:, 1]
292     resid = np.asarray(mstl.resid)
293     # 保存稳定的日周期与周周期结构, 供未来几天复用
294     daily_tail = s_daily[-min(len(s_daily), 28*144):]
295     weekly_tail = s_weekly[-min(len(s_weekly), 6*1008):]
296     daily_pattern = np.array( [ np.nanmean(daily_tail[i::144]) for
297                               i in range(144) ])
298     weekly_pattern = np.array( [ np.nanmean(weekly_tail[i::1008]) for
299                               i in range(1008) ])
300     _PRICE_MSTL_CACHE.clear()
301     _PRICE_MSTL_CACHE.update( { "refit_day": day_idx,
302                               "origin_day": day_idx - hist_days,
303                               "daily_pattern": daily_pattern, "weekly_pattern": weekly_pattern,
304                               "trend_last": float(trend[-1]),
305                               "trend_slope": float(np.clip( np.polyfit( np.arange(min(432,
306                               len(trend))), dtype=float), trend[- min(432, len(trend)):],
307                               1 )[0], - 5.0, 5.0 )), "resid_tail": resid[- min(len(resid), 14*
308                               144):].copy(), })
309     if logger:
310         logger.info("电价预测: 第 %d 天重新拟合 MSTL (日/周周期)。", day_idx + 1)
311 except Exception as e:
312     if logger:
313         logger.warning("MSTL失败(day=%d), 回退季节加权预测: %s", day_idx, e)
314     k = hist_mat.shape[0]
315     weights = np.exp(-0.10*np.arange(k-1, -1, -1))
316     weights /= weights.sum()
317     shape = np.sum(hist_mat * weights[:, None], axis=0)
318     if hist_days >= 7:
319         shape = 0.5*shape + 0.5*price_actual[day_idx-7]
320     return np.tile(shape, 2)
321 # 使用最近一次 MSTL 的季节结构, 并用最新 3 天数据快速校正趋势和残差
322 daily_pattern = _PRICE_MSTL_CACHE["daily_pattern"]
323 weekly_pattern = _PRICE_MSTL_CACHE["weekly_pattern"]
324 origin_day = _PRICE_MSTL_CACHE["origin_day"]
325 recent_days = min(hist_days, 3)
326 recent = price_actual[day_idx-recent_days:day_idx].reshape(-1)
327 global_start = (day_idx - recent_days) * 144
328 gidx = global_start + np.arange(len(recent))
329 sd_hist = daily_pattern[gidx % 144]
330 sw_hist = weekly_pattern[(gidx - origin_day*144) % 1008]
331 deseason = recent - sd_hist - sw_hist; m = min(432, len(deseason))
332 x = np.arange(m, dtype=float); coef = np.polyfit(x, deseason[-m:], 1)
333 slope = float(np.clip(coef[0], -5.0, 5.0)); level = float(deseason[-1])
334 trend_future = level + slope*np.arange(1, H+1)
335 future_gidx = day_idx*144 + np.arange(H)
336 sd_future = daily_pattern[future_gidx % 144]
337 sw_future = weekly_pattern[(future_gidx - origin_day*144) % 1008]
338 # 残差 AR 仍每天轻量更新, 不需要重新做 MSTL
339 hist_axis = np.arange(len(deseason), dtype=float)
340 fitted_trend_hist = level + slope*(hist_axis - (len(deseason)-1))
341 resid_recent = deseason - fitted_trend_hist
342 ar_future = choose_ar_forecast( resid_recent[- min(len(resid_recent), 3*
343         144):], H, cfg.price_ar_lags_candidates, 0.0, )
344 pred = trend_future + sd_future + sw_future + ar_future
345 return np.maximum(pred, 0.0)
346 def compute_historical_marginal_cost_day( j: int, available_day_exclusive: int,
347     net_errors: np.ndarray, price_actual: np.ndarray, soc_hist: np.ndarray,
348     planned_hist: np.ndarray, load: np.ndarray, pv: np.ndarray,
349     cfg: Config) -> Tuple[ np.ndarray, np.ndarray]:
350     nT = cfg.slots_per_day; cplus = np.full(nT, np.nan, dtype=float)
351     cminus = np.full(nT, np.nan, dtype=float)
352     if j < 0 or j >= available_day_exclusive:
353         return cplus, cminus
354     # 只把当前允许访问的历史展开; 后续窗口均是连续切片
355     n_avail = available_day_exclusive * nT
356     plan_f = planned_hist[:available_day_exclusive].reshape(-1)
357     load_f = load[:available_day_exclusive].reshape(-1)
358     pv_f = pv[:available_day_exclusive].reshape(-1)
359     soc_f = soc_hist[:available_day_exclusive].reshape(-1)
360     err_f = net_errors[:available_day_exclusive].reshape(-1)
361     slot_price = price_actual[:available_day_exclusive].reshape(-1)
362     base = j * nT; eff_rt = cfg.eta_c * cfg.eta_d
363     for t in range(nT):
364         e = net_errors[j, t]
365         if not np.isfinite(e):
366             continue
367         lam_t = price_actual[j, t]
368         s0 = soc_hist[j, t] if np.isfinite(soc_hist[j, t]) else cfg.e0
369         g0 = base + t; g1 = g0 + 1
370         g2 = min(g0 + cfg.horizon_slots, n_avail - 1)
371         if e > 0:
372             c_em = 5.0 * lam_t; c_bat = np.inf

```

```

373         if s0 > cfg.e_min + 1e-6 and g1 <= g2:
374             s1 = slice(g1, g2 + 1)
375             margin = plan_f[s1] + pv_f[s1] - load_f[s1]; sf = soc_f[s1]
376             pf = slot_price[s1]
377             soc_ok = (~np.isfinite(sf)) | (sf < cfg.e_max - 1e-6)
378             feasible = soc_ok & ((margin > 0.0) | (pf < lam_t))
379             if np.any(feasible):
380                 c_bat = float(np.min(pf[feasible]) / eff_rt)
381             cplus[t] = min(c_em, c_bat) if np.isfinite(c_bat) else c_em
382         elif e < 0:
383             recovery = 0.0
384             if s0 < cfg.e_max - 1e-6 and g1 <= g2:
385                 s1 = slice(g1, g2 + 1); ef = err_f[s1]; pf = slot_price[s1]
386                 m = np.isfinite(ef) & (ef > 0.0)
387                 if np.any(m):
388                     recovery = float(np.max(5.0 * pf[m] * eff_rt))
389             cminus[t] = max(0.0, lam_t - recovery)
390     return cplus, cminus
391 def update_risk_cost_cache( day_idx: int, cplus_cache: np.ndarray,
392                             cminus_cache: np.ndarray, net_errors: np.ndarray, price_actual: np.ndarray,
393                             soc_hist: np.ndarray, planned_hist: np.ndarray, load: np.ndarray,
394                             pv: np.ndarray, cfg: Config) -> None:
395     if day_idx <= 0:
396         return
397     # 昨天: 只能使用到昨天24:00, 保持严格无未来信息泄露
398     j = day_idx - 1
399     cp, cm = compute_historical_marginal_cost_day( j, day_idx, net_errors,
400                                                    price_actual, soc_hist, planned_hist, load, pv, cfg)
401     cplus_cache[j] = cp; cminus_cache[j] = cm
402     # 前天: 此时“前天时刻 + 未来24h”已全部落在已发生历史内, 覆盖为最终缓存
403     if day_idx >= 2:
404         j = day_idx - 2
405         cp, cm = compute_historical_marginal_cost_day( j, day_idx, net_errors,
406                                                    price_actual, soc_hist, planned_hist, load, pv, cfg)
407         cplus_cache[j] = cp; cminus_cache[j] = cm
408 def dynamic_risk_weights( day_idx: int, net_errors: np.ndarray,
409                           cplus_cache: np.ndarray, cminus_cache: np.ndarray, cfg: Config) -> Tuple[
410     np.ndarray, np.ndarray, np.ndarray]:
411     nT = cfg.slots_per_day; alpha = np.full(nT, cfg.alpha_cold, dtype=float)
412     beta = np.full(nT, cfg.beta_fixed, dtype=float)
413     reserve = np.zeros(nT, dtype=float)
414     if day_idx <= 0:
415         return alpha, beta, reserve
416     j0 = max(0, day_idx - cfg.risk_history_days)
417     hist_days = np.arange(j0, day_idx)
418     if hist_days.size == 0:
419         return alpha, beta, reserve
420     ages = day_idx - hist_days; base_w = cfg.risk_decay ** (ages - 1)
421     errs = net_errors[hist_days] # (HIST, 144)
422     cp_all = cplus_cache[hist_days]; cm_all = cminus_cache[hist_days]
423     for t in range(nT):
424         et = errs[:, t]
425         # 正向误差历史
426         mp = (et > 0) & np.isfinite(et) & np.isfinite(cp_all[:, t])
427         # 负向误差历史
428         mn = (et < 0) & np.isfinite(et) & np.isfinite(cm_all[:, t])
429         if np.count_nonzero( mp) >= cfg.risk_min_samples and np.count_nonzero(
430             mn) >= cfg.risk_min_samples:
431             cp = float(np.average(cp_all[mp, t], weights=base_w[mp]))
432             cm = float(np.average(cm_all[mn, t], weights=base_w[mn]))
433             raw = cp / max(cm, cfg.c_eps)
434             alpha[t] = min(cfg.alpha_max, max(cfg.alpha_min, raw))
435             tau = alpha[t] / (alpha[t] + beta[t])
436             tau = min(cfg.tau_max, max(cfg.tau_min, tau))
437             if np.count_nonzero(mp) >= cfg.risk_min_samples:
438                 reserve[ t] = cfg.reserve_scale * weighted_quantile( et[mp],
439                                     base_w[mp], tau)
440     return alpha, beta, reserve
441 def operate_one_slot( soc0: float, buy: float, load_actual: float,
442                      pv_actual: float, plan_ch: float, plan_dis: float, cfg: Config) -> Dict[str,
443                      float]:
444     dt = cfg.dt
445     # 先把计划动作投影到当前SOC可行范围
446     ch = max(0.0, min(float(plan_ch), cfg.p_max))
447     dis = max(0.0, min(float(plan_dis), cfg.p_max))
448     max_ch_soc = max(0.0, (cfg.e_max - soc0) / (cfg.eta_c * dt))
449     max_dis_soc = max(0.0, (soc0 - cfg.e_min) * cfg.eta_d / dt)
450     ch = min(ch, max_ch_soc); dis = min(dis, max_dis_soc)
451     # 基于真实负荷/光伏检查功率平衡
452     gap = float(load_actual - (buy + pv_actual + dis - ch)); emg = 0.0
453     sur = 0.0
454     if gap > 1e-10:
455         cut_ch = min(ch, gap); ch -= cut_ch; gap -= cut_ch

```

```

456     max_dis_total = min( cfg.p_max, max(0.0, (soc0 - cfg.e_min) *cfg.eta_d /
457 dt))
458     extra_dis = min(gap, max(0.0, max_dis_total - dis)); dis += extra_dis
459     gap -= extra_dis
460
461     emg = max(0.0, gap)
462 elif gap < -1e-10:
463     excess = -gap
464     cut_dis = min(dis, excess); dis -= cut_dis; excess -= cut_dis
465     max_ch_total = min( cfg.p_max, max(0.0, (cfg.e_max - soc0) / (cfg.eta_c *
466 dt)))
467     extra_ch = min(excess, max(0.0, max_ch_total - ch)); ch += extra_ch
468     excess -= extra_ch
469     sur = max(0.0, excess)
470     soc1 = soc0 + (cfg.eta_c*ch - dis/cfg.eta_d)*dt
471     soc1 = min(cfg.e_max, max(cfg.e_min, soc1))
472     return {"ch":ch, "dis":dis, "emg":emg, "sur":sur, "E":soc1}
473 def load_attachment3(cfg: Config, dates, logger):
474     if not Path(cfg.att3).exists():
475         alt = Path(cfg.base_dir) / "附件3(1).xlsx"
476         if alt.exists():
477             cfg.att3 = str(alt)
478         else:
479             raise FileNotFoundError(f"未找到附件3: {cfg.att3}")
480     wb = load_workbook(cfg.att3, data_only=True, read_only=True)
481     ws = wb[wb.sheetnames[0]]
482     rows = list(ws.iter_rows(min_row=2, max_col=26, values_only=True))
483     wb.close(); date_to_idx = {d:i for i,d in enumerate(dates)}
484     out = np.full((len(dates), 4, 24), np.nan, dtype=float)
485     stage_map = {"0:00":0, "6:00":1, "12:00":2, "18:00":3}; current_date = None
486     for r in rows:
487         raw_date, raw_stage = r[0], r[1]
488         if raw_date not in (None, ""):
489             current_date = cell_to_date(raw_date)
490             if current_date is None:
491                 continue
492             stage_txt = str(raw_stage).strip() if raw_stage is not None else ""
493             if stage_txt not in stage_map or current_date not in date_to_idx:
494                 continue
495             vals = np.array( [0.0 if v is None else float(v) for v in r[2:26]],
496 dtype=float)
497             out[ date_to_idx[current_date], stage_map[stage_txt], :] = np.maximum(
498 vals, 0.0)
499             if np.isnan(out).any():
500                 bad = int(np.isnan(out).sum())
501                 raise ValueError(f"附件3读取不完整, 仍有 {bad} 个缺失预报值。")
502     logger.info("【附件3】读取完成: %d天 × 4个预报时刻 × 24小时。", len(dates)); return out
503 def pv_stage_forecast_horizon_10min( day_idx: int, stage_hour: int,
504 pv_hourly: np.ndarray, pv_actual: np.ndarray, cfg: Config) -> np.ndarray:
505     stage_idx = {0:0, 6:1, 12:2, 18:3}[stage_hour]
506     hourly = pv_hourly[day_idx, stage_idx]; start_slot = stage_hour * 6
507     if stage_hour == 0:
508         anchor = 0.0
509     else:
510         anchor = float(pv_actual[day_idx, start_slot - 1])
511     xp = np.arange(0, 25, dtype=float); fp = np.concatenate([[anchor], hourly])
512     target_rel = (np.arange(cfg.rolling_horizon_slots) + 1) / 6.0
513     pred = np.interp(target_rel, xp, fp); return np.maximum(pred, 0.0)
514 def load_stage_forecast_horizon( base_fc_full: np.ndarray,
515 actual_today: np.ndarray, stage_hour: int, cfg: Config) -> np.ndarray:
516     start = stage_hour * 6; H = cfg.rolling_horizon_slots
517     seg = np.asarray(base_fc_full[start:start+H], dtype=float).copy()
518     if len(seg) != H:
519         raise ValueError("负荷48小时预测长度不足, 无法构造24小时MPC窗口。")
520     if cfg.use_intraday_load_bias and stage_hour > 0:
521         m = min(start, 18)
522         # 注意: 偏差估计只使用发布时刻以前已经实现的负荷。
523         base_day = np.asarray(base_fc_full[:cfg.slots_per_day], dtype=float)
524         resid = actual_today[start-m:start] - base_day[start-m:start]
525         w = np.exp(np.linspace(-1.5, 0.0, m)); w /= w.sum()
526         bias = float(np.sum(w * resid)); decay = np.exp(-np.arange(H) / 36.0)
527         seg += bias * decay
528     return np.maximum(seg, 0.0)
529 def actual_horizon_from_arrays( day_idx: int, stage_hour: int, arr: np.ndarray,
530 cfg: Config) -> Optional[ np.ndarray]:
531     start = stage_hour * 6; H = cfg.rolling_horizon_slots
532     flat = arr.reshape(-1); g0 = day_idx * cfg.slots_per_day + start
533     if g0 + H > len(flat):
534         return None
535     return np.asarray(flat[g0:g0+H], dtype=float)
536 def stage_reserve_from_history( day_idx: int, stage_idx: int,
537 stage_net_error_hist: np.ndarray, fallback_reserve_day: np.ndarray,
538 stage_hour: int, cfg: Config) -> np.ndarray:
539     H = cfg.rolling_horizon_slots

```

```

540 # 为确保任意18:00发布的24h预报已经完全实现, 统一只使用 d-2 及更早记录。
541 end = max(0, day_idx - 1)
542 start = max(0, end - cfg.stage_reserve_history_days)
543 hist = stage_net_error_hist[start:end, stage_idx, :]
544 # 问题二备用仅有当天144点。对跨日窗口按时钟位置循环展开作为冷启动基线。
545 fb = np.asarray(fallback_reserve_day, dtype=float)
546 clock = (stage_hour * 6 + np.arange(H)) % cfg.slots_per_day
547 fallback = fb[clock]
548 if hist.shape[0] < cfg.stage_reserve_min_days:
549     return fallback.copy()
550 out = np.zeros(H, dtype=float); age = np.arange(hist.shape[0]-1, -1, -1)
551 day_w = cfg.stage_reserve_decay ** age
552 for k in range(H):
553     vals = hist[:, k]; mask = np.isfinite(vals)
554     if int(np.sum(mask)) < cfg.stage_reserve_min_days:
555         out[k] = fallback[k]; continue
556     q = weighted_quantile(vals[mask], day_w[mask],
557         cfg.stage_reserve_quantile)
558     out[k] = cfg.stage_reserve_scale * max(0.0, float(q))
559 # 防止极端小样本产生不合理备用; 上限取储能/购电系统量级内的保守值。
560 return np.clip(out, 0.0, 5000.0)
561 def archive_stage_forecast_errors( forecast_day: int,
562     load_fc_full_store: np.ndarray, pv_hourly: np.ndarray, load: np.ndarray,
563     pv: np.ndarray, stage_net_error_hist: np.ndarray, cfg: Config) -> None:
564     """在某日24小时预报窗口完全实现后, 归档0/6/12/18四个时点的净负荷误差。"""
565     if forecast_day < 0 or forecast_day >= len(load_fc_full_store):
566         return
567     base_full = load_fc_full_store[forecast_day]
568     if not np.all(np.isfinite(base_full)):
569         return
570     for s, h in enumerate((0, 6, 12, 18)):
571         lf = load_stage_forecast_horizon(base_full, load[forecast_day], h, cfg)
572         pf = pv_stage_forecast_horizon_10min( forecast_day, h, pv_hourly, pv,
573             cfg)
574         la = actual_horizon_from_arrays(forecast_day, h, load, cfg)
575         pa = actual_horizon_from_arrays(forecast_day, h, pv, cfg)
576         if la is None or pa is None:
577             continue
578         stage_net_error_hist[forecast_day, s, :] = (la - pa) - (lf - pf)
579 def price_stage_forecast_horizon( price_fc_full: np.ndarray,
580     price_actual_today: np.ndarray, stage_hour: int, cfg: Config) -> np.ndarray:
581     start = stage_hour * 6; H = cfg.rolling_horizon_slots
582     seg = np.asarray(price_fc_full[start:H], dtype=float).copy()
583     if len(seg) != H: raise ValueError("48h 电价预测长度不足, 无法构造24h滚动窗口。")
584     if stage_hour <= 0: return np.maximum(seg, 0.0)
585     m = min(start, cfg.price_intraday_recent_slots)
586     base_today = np.asarray(price_fc_full[:cfg.slots_per_day], dtype=float)
587     resid = np.asarray( price_actual_today[start- m:start],
588         dtype=float) - base_today[start- m:start]
589     w = np.exp(np.linspace(-1.5, 0.0, m)); w /= w.sum(); bias = float(np.sum(w*resid))
590     bias = float( np.clip(bias, - cfg.price_intraday_bias_clip,
591         cfg.price_intraday_bias_clip))
592     tau = max(cfg.price_intraday_decay_hours*6.0, 1.0)
593     decay = np.exp(-np.arange(H, dtype=float)/tau)
594     return np.maximum(seg + bias*decay, 0.0)
595 def try_load_q42_forecast_cache(cfg: Config, D: int, logger):
596     if not cfg.reuse_q42_forecast_cache: return None, None
597     p = Path(cfg.q42_cache)
598     if not p.exists():
599         logger.info("未找到Q4-2缓存, 将按同一方法重建负荷与电价预测。"); return None, None
600     try:
601         z = np.load(p, allow_pickle=False); lf = np.asarray(z['load_fc'], dtype=float)
602         pf = np.asarray(z['price_fc'], dtype=float)
603         if lf.shape[0] < D or pf.shape[0] < D or lf.shape[
604             1] < cfg.forecast_slots or pf.shape[1] < cfg.forecast_slots:
605             raise ValueError(f"缓存维度不足 load={lf.shape}, price={pf.shape}")
606         logger.info("【预测缓存】已复用Q4-2的0:00负荷/48h电价预测, 跳过全年MSTL重建。")
607         return lf[:D, :cfg.forecast_slots].copy(), pf[:D,
608             :cfg.forecast_slots].copy()
609     except Exception as e:
610         logger.warning("【预测缓存】读取失败: %s; 将重新预测。", e); return None, None
611 @dataclass
612 class DayAheadTemplates:
613     Aeq: sparse.csr_matrix; bounds: list
614     def build_day_ahead_templates(cfg: Config) -> DayAheadTemplates:
615         n = cfg.slots_per_day; N = 5*n; ib, ic, idis, isu, ie = 0, n, 2*n, 3*n, 4*n
616         rows = []; cols = []; data = []; r = 0
617         for t in range(n):
618             for col, val in ((ib+t, 1.0), (idis+t, 1.0), (ic+t, -1.0), (isu+t, -1.0)):
619                 rows.append(r); cols.append(col); data.append(val)
620             r += 1
621         for t in range(n):
622             rows.append(r); cols.append(ie+t); data.append(1.0)

```

```

623         if t>0: rows.append(r); cols.append(ie+t-1); data.append(-1.0)
624         rows.append(r); cols.append(ic+t); data.append(-cfg.eta_c*cfg.dt)
625         rows.append(r); cols.append(idis+t); data.append(cfg.dt/cfg.eta_d); r+=1
626     Aeq=sparse.csr_matrix((data,(rows,cols)),shape=(r,N))
627     bounds=[[(0, None)]* n+ [(0, cfg.p_max)]* n+ [(0, cfg.p_max)]* n+ [(0,
628         None)]* n+ [(cfg.e_min, cfg.e_max)]* n)
629     return DayAheadTemplates(Aeq,bounds)
630 def solve_day_ahead_price( price_fc_full:np.ndarray, load_fc:np.ndarray,
631     pv_fc:np.ndarray, reserve:np.ndarray, soc0:float, cfg:Config,
632     lp:DayAheadTemplates):
633     n=cfg.slots_per_day; pr=np.asarray(price_fc_full,dtype=float)
634     if len(pr)<2*n: raise ValueError("日前电价预测必须包含未来48h。")
635     price_today=pr[:n]; price_next=pr[n:2*n]; c=np.zeros(5*n)
636     c[:n]=price_today*cfg.dt; c[n:3*n]=cfg.eps_cycle*cfg.dt
637     c[3*n:4*n]=cfg.eps_surplus*cfg.dt
638     mu=cfg.eta_d*float(np.min(price_next)); c[5*n-1]=-mu
639     b=np.empty(2*n,dtype=float)
640     b[:n]=np.asarray(load_fc[:n])-np.asarray(pv_fc[:n])+np.asarray(reserve)
641     b[n:]=0.0; b[n]=soc0
642     res=linprog(c,A_eq=lp.Aeq,b_eq=b,bounds=lp.bounds,method=cfg.lp_method)
643     if not res.success: raise RuntimeError(f"0:00日前LP失败: {res.message}")
644     x=res.x
645     return { 'buy':x[:n], 'ch':x[n:2* n], 'dis':x[2* n:3* n], 'sur':x[3* n:4*n],
646         'E':x[4* n:5* n], 'mu':mu}
647 def solve_rolling_mpc_24h_price( price_h:np.ndarray, load_fc_horizon:np.ndarray,
648     pv_fc_horizon:np.ndarray, reserve_horizon:np.ndarray, soc0:float,
649     prev_buy_day:np.ndarray, start_slot:int, cfg:Config):
650     H=cfg.rolling_horizon_slots; price_h=np.asarray(price_h,dtype=float)
651     if not( len(price_h)== len(load_fc_horizon)== len(pv_fc_horizon)==
652         len(reserve_horizon)== H):
653         raise ValueError("24h滚动MPC全部输入必须为144点。")
654     current_remaining=cfg.slots_per_day-start_slot
655     N=7*H; ib,ic,idis,isu,ie,iup,idn=0,H,2*H,3*H,4*H,5*H,6*H
656     rows=[]; cols=[]; data=[]; b=[]; r=0
657     net=np.asarray( load_fc_horizon)-np.asarray( pv_fc_horizon)+np.asarray(
658         reserve_horizon)
659     for j in range(H):
660         for col,val in ((ib+j,1.0),(idis+j,1.0),(ic+j,-1.0),(isu+j,-1.0)):
661             rows.append(r); cols.append(col); data.append(val)
662             b.append(float(net[j])); r+=1
663     for j in range(H):
664         rows.append(r); cols.append(ie+j); data.append(1.0)
665         if j>0: rows.append(r); cols.append(ie+j-1); data.append(-1.0); rhs=0.0
666         else: rhs=float(soc0)
667         rows.append(r); cols.append(ic+j); data.append(-cfg.eta_c*cfg.dt)
668         rows.append(r); cols.append(idis+j); data.append(cfg.dt/cfg.eta_d)
669         b.append(rhs); r+=1
670     for j in range(current_remaining):
671         rows.append(r); cols.append(ib+j); data.append(1.0)
672         rows.append(r); cols.append(iup+j); data.append(-1.0)
673         rows.append(r); cols.append(idn+j); data.append(1.0)
674         b.append(float(prev_buy_day[start_slot+j])); r+=1
675     Aeq=sparse.csr_matrix((data,(rows,cols)),shape=(r,N)); c=np.zeros(N)
676     c[iup:iup+current_remaining]=1.5*price_h[:current_remaining]*cfg.dt
677     c[idn:idn+current_remaining]=-0.5*price_h[:current_remaining]*cfg.dt
678     if current_remaining<H: c[ ib+ current_remaining:ib+ H]=price_h[
679         current_remaining:]*cfg.dt
680     c[ic:ic+H]=cfg.eps_cycle*cfg.dt; c[idis:idis+H]=cfg.eps_cycle*cfg.dt
681     c[isu:isu+H]=cfg.eps_surplus*cfg.dt
682     # 窗口末端机会价值: 未来24h预测价格中最低可替代购电价
683     mu=cfg.eta_d*float(np.min(price_h)); c[ie+H-1]=-mu; bounds=[]
684     bounds += [(0,None)]*H; bounds += [(0,cfg.p_max)]*H
685     bounds += [(0,cfg.p_max)]*H
686     bounds += [(0,None)]*H; bounds += [(cfg.e_min,cfg.e_max)]*H
687     bounds += [(0,None)]*H if j<current_remaining else (0,0) for j in range(H)
688     bounds += [(0,None)]*H if j<current_remaining else (0,0) for j in range(H)
689     res=linprog( c, A_eq=Aeq, b_eq=np.asarray(b), bounds=bounds,
690         method=cfg.lp_method)
691     if not res.success: raise RuntimeError(
692         f"{start_slot//6}:00滚动LP失败: {res.message}")
693     x=res.x; up=x[iup:iup+H]; dn=x[idn:idn+H]
694     return { 'buy':x[ib:ib+ H], 'ch':x[ic:ic+ H], 'dis':x[idis:idis+ H],
695         'sur':x[isu:isu+ H], 'E':x[ie:ie+ H], 'up':up, 'down':dn,
696         'up_energy':float(np.sum(up[:current_remaining])* cfg.dt),
697         'down_energy':float(np.sum(dn[:current_remaining])* cfg.dt), 'mu':mu,
698         'current_remaining':current_remaining}
699 def _copy_style(src,dst):
700     from copy import copy
701     if src._style: dst._style=copy(src._style)
702     dst.font=copy(src.font); dst.fill=copy(src.fill)
703     dst.border=copy(src.border); dst.alignment=copy(src.alignment)
704     dst.protection=copy(src.protection); dst.number_format=src.number_format
705 def fill_result4_3( template_path, out_path, dates, price_actual, initial_buy,
706     effective_buy, buy_cost_after_adjust, ch, dis, soc_start, soc_end, emg, cfg,

```

```

707 logger):
708 if not Path(template_path).exists():
709     logger.warning("【模板缺失】未找到 %s; CSV/TXT/图仍正常输出。",template_path); return
710 wb=load_workbook(template_path); sp=wb['计划购电量']; sa=wb['调整购电量']
711 sc=wb['充放电电量']; se=wb['紧急购电量']; date_to_idx={d:i for i,d in enumerate(dates)}
712 computed=[ d for d in dates if d>= date(2025, 2, 1) and
713             np.isfinite(initial_buy[date_to_idx[d]]).all()]
714 init_cost=np.nansum(initial_buy*price_actual,axis=1)*cfg.dt
715 def fill_buy(ws,arr, costs):
716     for r in range(2,ws.max_row+1):
717         raw=ws.cell(r,1).value
718         if raw is None: continue
719         try:d=date_to_idx[raw]
720         except Exception:continue
721         if d not in date_to_idx: continue
722         i=date_to_idx[d]
723         if not np.isfinite(arr[i]).all(): continue
724         vals=arr[i]*cfg.dt
725         for c,v in enumerate(vals,start=2): ws.cell(r,c).value=float(v)
726         ws.cell(r,146).value=float(np.sum(vals))
727         ws.cell(r,147).value=float(costs[i])
728 fill_buy(sp,initial_buy,init_cost)
729 fill_buy(sa,effective_buy,buy_cost_after_adjust)
730 charge_proto=[ {'height':sc.row_dimensions[r].height, 'cells':[sc.cell(r,
731 c) for c in range(1, 7)]} for r in range(2, 8)]
732 emg_proto=[ {'height':se.row_dimensions[r].height, 'cells':[se.cell(r,c) for
733 c in range(1, 4)]} for r in range(2, min(5, se.max_row+ 1))]
734 if sc.max_row>=2: sc.delete_rows(2,sc.max_row-1)
735 intervals=[ (0, 4, '0:00-4:00'), (4, 8, '4:00-8:00'), (8, 12, '8:00-12:00'),
736             (12, 16, '12:00-16:00'), (16, 20, '16:00-20:00'), (20, 24,
737             '20:00-24:00')]
738 rr=2
739 for d in computed:
740     i=date_to_idx[d]
741     for k,(h0,h1,label) in enumerate(intervals):
742         proto=charge_proto[k]; sc.row_dimensions[rr].height=proto['height']
743         for c in range(1,7): _copy_style(proto['cells'][c-1],sc.cell(rr,c))
744         sc.cell(rr,1).value=d if k==0 else None; sc.cell(rr,2).value=label
745         a,b=h0*6,h1*6; sc.cell(rr,3).value=float(np.sum(ch[i,a:b])*cfg.dt)
746         sc.cell(rr,4).value=float(np.sum(dis[i,a:b])*cfg.dt)
747         if k==0: sc.cell( rr, 5).value='0:00'; sc.cell( rr, 6).value=float(
748             soc_start[i])
749         elif k==1: sc.cell( rr, 5).value='24:00'; sc.cell( rr,
750             6).value=float( soc_end[i])
751         rr+=1
752 def fmtslot(k):
753     mins=k*10
754     if mins==1440:return '24:00'
755     return f'{{(mins//60)%24}}:{{mins%60:02d}}'
756 if se.max_row>=2:se.delete_rows(2,se.max_row-1)
757 rr=2
758 for d in computed:
759     i=date_to_idx[d]; events=[]; t=0
760     while t<144:
761         if emg[i,t]>1e-6:
762             st=t; ee=0.0
763             while t<144 and emg[i,t]>1e-6: ee+=emg[i,t]*cfg.dt; t+=1
764             events.append((f'{{fmtslot(st)}}-{{fmtslot(t)}}',ee))
765         else:t+=1
766     if not events:events=[(None,None)]
767     for k,(period,energy) in enumerate(events):
768         proto=emg_proto[min(k,len(emg_proto)-1)]
769         se.row_dimensions[rr].height=proto['height']
770         for c in range(1,4):_copy_style(proto['cells'][c-1],se.cell(rr,c))
771         se.cell(rr,1).value=d if k==0 else None; se.cell(rr,2).value=period
772         se.cell(rr,3).value=energy; rr+=1
773 for ws in (sc,se):
774     for r in range(2,ws.max_row+1):
775         if ws.cell( r, 1).value is not None: ws.cell( r,
776             1).number_format='yyyy/m/d'
777 Path(out_path).parent.mkdir(parents=True,exist_ok=True); wb.save(out_path)
778 wb.close(); logger.info("【结果输出】result4-3已保存: %s",out_path)
779 # 题目表1要求给出的指定时间段 (10分钟)
780 PAPER_TABLE1_INTERVALS = ( "10:00-10:10", "12:00-12:10", "14:00-14:10",
781                             "16:00-16:10", "18:00-18:10", "20:00-20:10")
782 # 题目表2要求给出的指定时间段 (4小时)
783 PAPER_TABLE2_INTERVALS = ( "0:00-4:00", "4:00-8:00", "8:00-12:00","12:00-16:00",
784                             "16:00-20:00", "20:00-24:00")
785 def _interval_start_slot(label: str) -> int:
786     """把'10:00-10:10'这类标签映射到行内列序号(1-based列-2)。标签起点=(slot+1)*10分钟。"""
787     hh, mm = label.split("-")[0].split(":")
788     return (int(hh) * 60 + int(mm)) // 10 - 1
789 def _slot_label(k: int) -> str:

```

```

790 mins = k * 10
791 if mins == 1440:
792     return "24:00"
793 return f"({mins // 60} % 24):{mins % 60:02d}"
794 def _emergency_events(emg_row: np.ndarray, dt: float) -> list:
795     """把逐时段紧急购电功率聚合成为[(时间段, 购电量kWh), ...]。"""
796     events = []; t = 0
797     n = len(emg_row)
798     while t < n:
799         if emg_row[t] > 1e-6:
800             st = t; ee = 0.0
801             while t < n and emg_row[t] > 1e-6:
802                 ee += float(emg_row[t]) * dt; t += 1
803             events.append((f"_{slot_label(st)}-{_slot_label(t)}", ee))
804         else:
805             t += 1
806     return events
807 def export_paper_tables_q43( dates, price_actual, initial_buy, effective_buy, ch,
808     dis, emg, soc_start, soc_end, plan_cost, adj_cost, emg_cost, run_D, cfg,
809     logger):
810     from openpyxl import Workbook
811     from openpyxl.styles import Font, Alignment, Border, Side
812     out_path = Path(cfg.out_dir) / "Q4-3_论文表格_指定日期.xlsx"
813     d2i = {d: i for i, d in enumerate(dates)}
814     targets = [ d for d in cfg.representative_dates if d in d2i and d2i[d] <
815         run_D]
816     if not targets:
817         logger.warning("【论文表格】表3指定日期不在本次运行范围内, 跳过导出。"); return None
818     dt = cfg.dt; thin = Side(style="thin", color="000000")
819     border = Border(left=thin, right=thin, top=thin, bottom=thin)
820     bold = Font(bold=True); title_font = Font(bold=True, size=12)
821     center = Alignment(horizontal="center", vertical="center"); num = "#,##0.00"
822     def put(ws, r, c, v, font=None, align=None, bd=True, fmt=None):
823         cell = ws.cell(row=r, column=c, value=v)
824         if font is not None:
825             cell.font = font
826         if align is not None:
827             cell.alignment = align
828         if bd:
829             cell.border = border
830         if fmt is not None:
831             cell.number_format = fmt
832         return cell
833     wb = Workbook(); wb.remove(wb.active)
834     colw = { "A": 15, "B": 14, "C": 14, "D": 14, "E": 14, "F": 14, "G": 14,
835         "H": 14}
836     # 表1
837     ws = wb.create_sheet("表1_购电量")
838     for col, w in colw.items():
839         ws.column_dimensions[col].width = w
840     r = 1
841     for d in targets:
842         i = d2i[d]
843         ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=6)
844         put( ws, r, 1, f"表1 微网在指定时间段的购电量及全天的购电量和购电费 ({d.strftime('%Y-%m-%d')})",
845             title_font, center, bd=False)
846         r += 1
847         for c, v in enumerate( ["时间段", "购电量(kWh)", "时间段", "购电量(kWh)", "时间段",
848             "购电量(kWh)"], start=1):
849             put(ws, r, c, v, bold, center)
850         r += 1
851         vals = [ float(effective_buy[i, _interval_start_slot(lb)]) * dt for
852             lb in PAPER_TABLE1_INTERVALS]
853         for tri in ((0, 1, 2), (3, 4, 5)):
854             for col_off, idx in enumerate(tri):
855                 put( ws, r, 1 + 2 * col_off, PAPER_TABLE1_INTERVALS[idx], None,
856                     center)
857                 put(ws, r, 2 + 2 * col_off, vals[idx], None, center, fmt=num)
858             r += 1
859         day_buy = float(np.nansum(effective_buy[i]) * dt)
860         day_cost = float(plan_cost[i] + adj_cost[i] + emg_cost[i])
861         put(ws, r, 1, "全天购电量", bold, center)
862         put(ws, r, 2, day_buy, None, center, fmt=num)
863         put(ws, r, 3, "全天购电费(元)", bold, center)
864         put(ws, r, 4, day_cost, None, center, fmt=num)
865         put(ws, r, 5, None, None, center); put(ws, r, 6, None, None, center)
866         r += 2
867     # 表2
868     ws = wb.create_sheet("表2_充放电量")
869     for col, w in colw.items():
870         ws.column_dimensions[col].width = w
871     r = 1; ivs = [(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]
872     for d in targets:

```

```

872     i = d2i[d]
873     ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=6)
874     put( ws, r, 1,
875         f"表2 储能设备在指定时间段的充放电量及0:00和24:00的储电量 ({d.strftime('%Y-%m-%d')})
    ) ",
876         title_font, center, bd=False)
877     r += 1
878     for c, v in enumerate( ["时间段", "充电量(kWh)", "放电量(kWh)", "时间段", "充电量(kWh)",
879         "放电量(kWh)"], start=1):
880         put(ws, r, c, v, bold, center)
881     r += 1
882     ch_e = [float(np.sum(ch[i, h0 * 6:h1 * 6]) * dt) for h0, h1 in ivs]
883     dis_e = [float(np.sum(dis[i, h0 * 6:h1 * 6]) * dt) for h0, h1 in ivs]
884     for a, b in ((0, 1), (2, 3), (4, 5)):
885         put(ws, r, 1, PAPER_TABLE2_INTERVALS[a], None, center)
886         put(ws, r, 2, ch_e[a], None, center, fmt=num)
887         put(ws, r, 3, dis_e[a], None, center, fmt=num)
888         put(ws, r, 4, PAPER_TABLE2_INTERVALS[b], None, center)
889         put(ws, r, 5, ch_e[b], None, center, fmt=num)
890         put(ws, r, 6, dis_e[b], None, center, fmt=num); r += 1
891     put(ws, r, 1, "0:00 储电量", bold, center)
892     put(ws, r, 2, float(soc_start[i]), None, center, fmt=num)
893     put(ws, r, 3, "24:00 储电量", bold, center)
894     put(ws, r, 4, float(soc_end[i]), None, center, fmt=num)
895     put(ws, r, 5, None, None, center); put(ws, r, 6, None, None, center)
896     r += 2
897 # 表3
898 ws = wb.create_sheet("表3_紧急购电量")
899 for col, w in colw.items():
900     ws.column_dimensions[col].width = w
901 events_per_date = [_emergency_events(emg[d2i[d]], dt) for d in targets]
902 ncol = 2 * len(targets)
903 ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=ncol)
904 put(ws, 1, 1, "表3 微网在指定日期的紧急购电量", title_font, center, bd=False)
905 for j, d in enumerate(targets):
906     c0 = 1 + 2 * j
907     ws.merge_cells( start_row=2, start_column=c0, end_row=2, end_column=c0 +
908         1)
909     put(ws, 2, c0, d.strftime("%Y-%m-%d"), bold, center)
910     put(ws, 2, c0 + 1, None, bold, center)
911 for j in range(len(targets)):
912     put(ws, 3, 1 + 2 * j, "时间段", bold, center)
913     put(ws, 3, 2 + 2 * j, "购电量(kWh)", bold, center)
914 nrow = max(3, max((len(e) for e in events_per_date), default=0)); r = 4
915 for k in range(nrow):
916     for j, evs in enumerate(events_per_date):
917         c0 = 1 + 2 * j
918         if k < len(evs):
919             put(ws, r, c0, evs[k][0], None, center)
920             put(ws, r, c0 + 1, evs[k][1], None, center, fmt=num)
921         else:
922             put(ws, r, c0, None, None, center)
923             put(ws, r, c0 + 1, None, None, center)
924     r += 1
925 # 附表: 费用与“0点计划/最终”对照
926 ws = wb.create_sheet("附_费用与计划对照")
927 for col in "ABCDEFGH":
928     ws.column_dimensions[col].width = 18
929 heads = [ "日期", "计划全天购电量(kWh)", "最终全天购电量(kWh)", "计划购电费(元)", "调整净费
    用(元)",
930     "紧急购电费(元)", "全天总费用(元)", "紧急购电量(kWh)"]
931 for c, v in enumerate(heads, start=1):
932     put(ws, 1, c, v, bold, center)
933 r = 2
934 for d in targets:
935     i = d2i[d]
936     row = [ d.strftime("%Y-%m-%d"), float(np.nansum(initial_buy[i]) * dt),
937         float(np.nansum(effective_buy[i]) * dt), float(plan_cost[i]),
938         float(adj_cost[i]), float(emg_cost[i]), float(plan_cost[i] +
939         adj_cost[i] + emg_cost[i]), float(np.sum(emg[i]) * dt)]
940     for c, v in enumerate(row, start=1):
941         put( ws, r, c, v, None, center if c > 1 else None, fmt=(None if c ==
942             1 else num))
943     r += 1
944 r += 2; put(ws, r, 1, "指定时段: 0:00计划购电量与最终购电量对照", bold); r += 1
945 for c, v in enumerate(["日期", "时间段", "计划购电量(kWh)", "最终购电量(kWh)"], start=1):
946     put(ws, r, c, v, bold, center)
947 r += 1
948 for d in targets:
949     i = d2i[d]
950     for lb in PAPER_TABLE1_INTERVALS:
951         s = _interval_start_slot(lb)
952         put(ws, r, 1, d.strftime("%Y-%m-%d"), None, center)

```

```

953         put(ws, r, 2, lb, None, center)
954         put(ws, r, 3, float(initial_buy[i, s]) * dt, None, center, fmt=num)
955         put(ws, r, 4, float(effective_buy[i, s]) * dt, None, center,
956             fmt=num)
957         r += 1
958     # 说明
959     ws = wb.create_sheet("说明"); ws.column_dimensions["A"].width = 110
960     notes = ["问题4-3 论文表格（表1/表2/表3）—— 指定日期：2025.3.20、2025.6.21、2025.9.23、
961         2025.12.21",
962         "波动电价场景：计划购电按0:00预测电价制定，结算与紧急购电（5倍）均按真实电价计算。",
963         "表1 购电量：取“调整后的最终购电量”（0:00计划经6:00/12:00/18:00预报调整后的结果）。",
964         "表1 全天购电费：= 计划购电费 + 调整净费用 + 紧急购电费（问题4-3总购电费用口径）。",
965         "表2 充放电量：为指定4小时时间段的最终充电量与放电量；0:00/24:00 储电量为当日首末储电
966         量。",
967         "表3 紧急购电量：为最终紧急购电的时间段与电量；留空表示该日期没有发生紧急购电。",
968         "附表：给出每日0:00计划与最终购电量的对照及费用拆分，便于论文中解释“调整”的作用。", ]
969     for i, t in enumerate(notes, start=1):
970         put(ws, i, 1, t, bold if i == 1 else None, bd=False)
971     Path(out_path).parent.mkdir(parents=True, exist_ok=True); wb.save(out_path)
972     wb.close()
973     logger.info( "【论文表格】表1/表2/表3已导出： %s（日期：%s）", out_path,
974         ", ".join(d.strftime("%Y-%m-%d") for d in targets))
975     return str(out_path)
976 def save_outputs( dates, price_actual, price_fc0, price_stage_fc, initial_buy,
977     effective_buy, ch, dis, emg, sur, soc_start, soc_end, plan_cost, adj_cost,
978     emg_cost, up_energy, down_energy, load_rmse, pv_rmse0, run_D, cfg, logger):
979     out=Path(cfg.out_dir)
980     out.mkdir(parents=True, exist_ok=True)
981     rows=[]
982     for d in range(run_D):
983         rows.append( [str(dates[d]), soc_start[d], soc_end[d], plan_cost[d],
984             adj_cost[d], emg_cost[d], plan_cost[d]+ adj_cost[d]+ emg_cost[d],
985             float(np.sum(emg[d])* cfg.dt), float(np.sum(sur[d])* cfg.dt),
986             up_energy[d], down_energy[d], load_rmse[d], pv_rmse0[d]])
987     with open( out/ 'q4_3_daily_summary.csv', 'w', newline='',
988         encoding='utf-8-sig') as f:
989         w=csv.writer(f)
990         w.writerow( ['日期', '日初SOC', '日末SOC', '初始计划购电费', '调整净费用', '紧急购电费',
991             '总费用',
992             '紧急购电量kWh', '富余电量kWh', '向上调整电量kWh', '向下调整电量kWh', '负荷RMSE',
993             '0点光伏RMSE'])
994         w.writerows(rows)
995     if run_D<32: return
996     valid=[i for i in range(31,run_D)]; formal=np.array(valid,dtype=int)
997     total_plan=float(np.sum(plan_cost[formal]))
998     total_adj=float(np.sum(adj_cost[formal]))
999     total_emg=float(np.sum(emg_cost[formal]))
1000     emg_energy=float(np.sum(emg[formal])*cfg.dt)
1001     sur_energy=float(np.sum(sur[formal])*cfg.dt)
1002     zero=int(np.sum(np.sum(emg[formal],axis=1)*cfg.dt<1e-6))
1003     daily_emg=np.sum(emg[formal],axis=1)*cfg.dt
1004     p95=float(np.quantile(daily_emg,.95)); mx=float(np.max(daily_emg))
1005     a=price_actual[formal].reshape(-1); p=price_fc0[formal,:144].reshape(-1)
1006     mask=np.isfinite(a)&np.isfinite(p)
1007     mae=float(np.mean(np.abs(a[mask]-p[mask])))
1008     rmse=float(np.sqrt(np.mean((a[mask]-p[mask])**2)))
1009     mape=float( np.mean(np.abs((a[mask]- p[mask])/ np.maximum(np.abs(a[mask]),
1010         1e-3)))* 100)
1011     lines=[ '问题4-3正式结果摘要（电价预测驱动，1月仅预热）', '='* 68,
1012         f'正式统计日期范围： {dates[31]} 至 {dates[run_D- 1]}',
1013         f'累计初始计划购电费（真实电价结算）： {total_plan:.2f} 元',
1014         f'累计调整净费用（真实电价结算）： {total_adj:.2f} 元',
1015         f'累计紧急购电费（5倍真实电价）： {total_emg:.2f} 元', f'累计总费用： {total_plan+
1016         total_adj+total_emg:.2f} 元', f'累计紧急购电量： {emg_energy:.2f} kWh',
1017         f'累计富余电量： {sur_energy:.2f} kWh',
1018         f'累计向上调整电量： {float(np.sum(up_energy[formal])):.2f} kWh',
1019         f'累计向下调整电量： {float(np.sum(down_energy[formal])):.2f} kWh',
1020         f'日紧急购电P95： {p95:.2f} kWh', f'最大单日紧急购电： {mx:.2f} kWh',
1021         f'无紧急购电天数： {zero}/{len(formal)}', f'期末储电量： {soc_end[run_D- 1]:.2f} kWh',
1022         f'0:00电价预测MAE： {mae:.6f} 元/kWh', f'0:00电价预测RMSE： {rmse:.6f} 元/kWh',
1023         f'0:00电价预测MAPE： {mape:.3f}%', '', f'正式结果文件： {cfg.out_xlsx}']
1024     (out/'Q4-3_结果摘要.txt').write_text('\n'.join(lines),encoding='utf-8')
1025     # 各发布时点未来24h电价预测误差
1026     metrics=[]
1027     for si,h in enumerate((0,6,12,18)):
1028         aa=[]; pp=[]
1029         for d in valid:
1030             real=actual_horizon_from_arrays(d,h,price_actual,cfg)
1031             pred=price_stage_fc[d,si]
1032             if real is None or not np.all(np.isfinite(pred)):continue
1033             aa.append(real); pp.append(pred)
1034         if aa:

```

```

1031         aa=np.concatenate(aa); pp=np.concatenate(pp); er=pp-aa
1032         metrics.append( [h, float(np.mean(np.abs(er))),
1033                         float(np.sqrt(np.mean(er* er))), float(np.mean(np.abs(er)/
1034                             np.maximum(np.abs(aa), 1e-3))* 100)])
1035     with open( out/ 'Q4-3_电价分阶段预测指标.csv', 'w', newline='',
1036               encoding='utf-8-sig') as f:
1037         w=csv.writer(f); w.writerow(['发布时点/h','MAE元每kWh','RMSE元每kWh','MAPE%'])
1038         w.writerows(metrics)
1039     # 指定日期表1/2/3简表
1040     target_rows=[]; date_to_idx={d:i for i,d in enumerate(dates)}
1041     for dd in cfg.representative_dates:
1042         i=date_to_idx.get(dd)
1043         if i is None:continue
1044         target_rows.append( [str(dd), float(np.sum(initial_buy[i])* cfg.dt),
1045                             float(np.sum(effective_buy[i])* cfg.dt), plan_cost[i], adj_cost[i],
1046                             emg_cost[i], float(np.sum(emg[i])* cfg.dt), soc_start[i],
1047                             soc_end[i]])
1048     with open(out/'Q4-3_指定日期结果汇总.csv','w',newline='',encoding='utf-8-sig') as f:
1049         w=csv.writer(f)
1050         w.writerow( ['日期', '初始计划购电量kWh', '最终调整购电量kWh', '初始计划购电费元', '调整
1051                     净费用元',
1052                     '紧急购电费元', '紧急购电量kWh', '0:00SOC', '24:00SOC'])
1053         w.writerows(target_rows)
1054     logger.info("Q4-3摘要及CSV结果已保存: %s", out)
1055 def simulate_update_scheme_q43( update_hours, dates, price_actual, load, pv,
1056     load_fc_full_store, price_fc_full_store, pv_hourly, fallback_reserve_store,
1057     stage_net_error_hist, run_D, cfg, lp, collect_response=False):
1058     n=cfg.slots_per_day; update_hours=tuple(sorted(update_hours))
1059     plan_cost=np.zeros(run_D); adj_cost=np.zeros(run_D)
1060     emg_cost=np.zeros(run_D)
1061     emg_energy=np.zeros(run_D); surplus_energy=np.zeros(run_D)
1062     up_energy=np.zeros(run_D); down_energy=np.zeros(run_D)
1063     soc_start=np.zeros(run_D); soc_end=np.zeros(run_D)
1064     response_records=[]; soc=cfg.e0; stage_idx_map={0:0,6:1,12:2,18:3}
1065     for d in range(run_D):
1066         soc_start[d]=soc; lf_full=np.asarray(load_fc_full_store[d],dtype=float)
1067         pr_full=np.asarray(price_fc_full_store[d],dtype=float); lf0=lf_full[:n]
1068         pf0=pv_stage_forecast_horizon_10min(d,0,pv_hourly,pv,cfg)[:n]
1069         fallback=np.asarray(fallback_reserve_store[d],dtype=float)
1070         da=solve_day_ahead_price(pr_full,lf0,pf0,fallback,soc,cfg,lp)
1071         initial_buy=da['buy'].copy(); current_buy=da['buy'].copy()
1072         current_ch=da['ch'].copy(); current_dis=da['dis'].copy()
1073         plan_cost[d]=float(np.sum(initial_buy*price_actual[d])*cfg.dt)
1074         # 上一阶段“已知信息”基准: 用于12:00重点响应分析。
1075         prev_pv_day=pf0.copy(); prev_net_day=(lf0-pf0+fallback).copy()
1076         prev_price_day=pr_full[:n].copy(); stages=(0,)+update_hours
1077         for si,h in enumerate(stages):
1078             start=h*6
1079             if h>0:
1080                 lf_h=load_stage_forecast_horizon(lf_full,load[d],h,cfg)
1081                 pf_h=pv_stage_forecast_horizon_10min(d,h,pv_hourly,pv,cfg)
1082                 reserve_h=stage_reserve_from_history( d, stage_idx_map[h],
1083                     stage_net_error_hist, fallback, h, cfg)
1084                 pr_h=price_stage_forecast_horizon(pr_full,price_actual[d],h,cfg)
1085                 m=n-start; buy_before=current_buy[start:].copy()
1086                 ch_before=current_ch[start:].copy()
1087                 dis_before=current_dis[start:].copy()
1088                 pv_before=prev_pv_day[start:].copy()
1089                 net_before=prev_net_day[start:].copy()
1090                 price_before=prev_price_day[start:].copy()
1091                 net_after=(lf_h[:m]-pf_h[:m]+reserve_h[:m]).copy()
1092                 price_after=pr_h[:m].copy()
1093                 adj=solve_rolling_mpc_24h_price( pr_h, lf_h, pf_h, reserve_h,
1094                     soc, current_buy, start, cfg)
1095                 buy_after=adj['buy'][:m].copy(); ch_after=adj['ch'][:m].copy()
1096                 dis_after=adj['dis'][:m].copy(); current_buy[start:]=buy_after
1097                 current_ch[start:]=ch_after; current_dis[start:]=dis_after
1098                 # 与正式Q4-3一致: 真实价格只用于事后结算。
1099                 upv=adj['up'][:m]; dnv=adj['down'][:m]
1100                 real_today=price_actual[d,start:]
1101                 adj_cost[ d]+=float( np.sum((1.5* real_today* upv- 0.5*
1102                     real_today* dnv)* cfg.dt))
1103                 up_energy[d]+=adj['up_energy']
1104                 down_energy[d]+=adj['down_energy']
1105                 if collect_response:
1106                     for j in range(m):
1107                         delta_buy=float(buy_after[j]-buy_before[j])
1108                         response_records.append( { '日期':str(dates[d]), '更新时点':h,
1109                             '目标时刻':f"({start+ j})//6:02d":(((start+ j)%6)*10:02d)", '旧光伏预
1109                             报kW':float(pv_before[j]),
1110                             '新光伏预报kW':float(pf_h[j]), '光伏预报修正量kW':float(pf_h[j]-
1111                             pv_before[j]), '旧预测电价元每kWh':float(price_before[j]),
1112                             '新预测电价元每kWh':float(price_after[j]),

```

```

1112         '电价预测修正量元每kWh':float(price_after[j]-
1113         price_before[j]), '旧有效净负荷kW':float(net_before[j]),
1114         '新有效净负荷kW':float(net_after[j]),
1115         '有效净负荷修正量kW':float(net_after[j]- net_before[j]),
1116         '旧购电计划kW':float(buy_before[j]),
1117         '新购电计划kW':float(buy_after[j]), '购电调整量kW':delta_buy,
1118         '向上调整kW':max(delta_buy, 0.0), '向下调整kW':max(-
1119         delta_buy, 0.0), '原充电计划kW':float(ch_before[j]),
1120         '新充电计划kW':float(ch_after[j]),
1121         '原放电计划kW':float(dis_before[j]),
1122         '新放电计划kW':float(dis_after[j]), })
1123     prev_pv_day[start:]=pf_h[:m]; prev_net_day[start:]=net_after
1124     prev_price_day[start:]=price_after
1125     end=(stages[si+1]*6) if si+1<len(stages) else n
1126     for t in range(start,end):
1127         sol=operate_one_slot( soc, current_buy[t], load[d, t], pv[d, t],
1128         current_ch[t], current_dis[t], cfg)
1129         soc=sol['E']; emg_energy[d]+=sol['emg']*cfg.dt
1130         surplus_energy[d]+=sol['sur']*cfg.dt
1131         emg_cost[d]+=sol['emg']*5.0*price_actual[d,t]*cfg.dt
1132     soc_end[d]=soc
1133     return { 'plan_cost':plan_cost, 'adj_cost':adj_cost, 'emg_cost':emg_cost,
1134             'emg_energy':emg_energy, 'surplus_energy':surplus_energy,
1135             'up_energy':up_energy, 'down_energy':down_energy, 'soc_start':soc_start,
1136             'soc_end':soc_end, 'response_records':response_records, }
1137 def run_update_time_comparison_q43( dates, price_actual, load, pv,
1138     load_fc_full_store, price_fc_full_store, pv_hourly, fallback_reserve_store,
1139     stage_net_error_hist, run_D, cfg, lp, logger):
1140     """复现原Q3五方案对照，并扩展到“光伏+电价”同步滚动更新。"""
1141     if run_D<32:
1142         logger.info('测试天数不足32天，跳过正式期方案比较。'); return None
1143     out=Path(cfg.out_dir)
1144     out.mkdir(parents=True,exist_ok=True)
1145     formal=slice(31,run_D); results=[]; raw={}
1146     logger.info('开始五方案比较：负荷预测、价格基础预测、风险参数和储能参数保持一致。')
1147     for name, hours in zip(cfg.comparison_names, cfg.comparison_schemes):
1148         logger.info('正在计算方案：%s', name)
1149         r=simulate_update_scheme_q43( hours, dates, price_actual, load, pv,
1150         load_fc_full_store, price_fc_full_store, pv_hourly,
1151         fallback_reserve_store, stage_net_error_hist, run_D, cfg, lp,
1152         collect_response=(tuple(hours)== tuple(cfg.focus_scheme_hours)))
1153         raw[name]=r; plan=float(np.sum(r['plan_cost'][formal]))
1154         adj=float(np.sum(r['adj_cost'][formal]))
1155         emgc=float(np.sum(r['emg_cost'][formal]))
1156         daily=np.asarray(r['emg_energy'][31:run_D], dtype=float)
1157         results.append( { '方案':name, '更新时点':', '.join(map(str, hours)) if
1158         hours else '无日内更新', '初始计划购电费':plan, '调整净费用':adj, '紧急购电费':emgc,
1159         '总费用':plan+ adj+ emgc, '紧急购电量kWh':float(np.sum(daily)),
1160         '富余电量kWh':float(np.sum(r['surplus_energy'][formal])),
1161         '向上调整电量kWh':float(np.sum(r['up_energy'][formal])),
1162         '向下调整电量kWh':float(np.sum(r['down_energy'][formal])),
1163         '紧急购电天数':int(np.sum(daily> 1e-6)),
1164         '单日最大紧急购电量kWh':float(np.max(daily)) if len(daily) else 0.0,
1165         '日紧急购电量P95kWh':float(np.quantile(daily, 0.95)) if len(daily) else
1166         0.0, '期末SOC':float(r['soc_end'][run_D- 1]), })
1167     csv_path=out/'Q4-3_既有预报时点方案对照.csv'
1168     with open(csv_path, 'w', newline='', encoding='utf-8-sig') as f:
1169         w=csv.DictWriter(f, fieldnames=list(results[0].keys())); w.writeheader()
1170         w.writerows(results)
1171     base=results[0]; txt=out/'Q4-3_既有预报时点方案对照结论.txt'
1172     with open(txt, 'w', encoding='utf-8') as f:
1173         f.write('Q4-3既有预报时点价值对照——波动电价下24小时滚动线性规划\n'+ '='*68+'\n')
1174         f.write('说明：固定负荷预测、0:00基础电价预测、风险参数和储能参数；仅改变6/12/18哪些时点
1175         允许同步更新光伏预报与电价预报。\\n')
1176         for r in results:
1177             f.write(
1178                 f"{r['方案']}: 总费用 {r['总费用']:.2f} 元, 紧急购电 {r['紧急购电量kWh']:.2f}
1179                 kWh, P95 {r['日紧急购电量P95kWh']:.2f} kWh, "
1180                 f"最大单日 {r['单日最大紧急购电量kWh']:.2f} kWh, 富余 {r['富余电量kWh']:.2f} kWh
1181                 。\\n")
1182             f.write('\\n相对仅0:00方案: \\n')
1183             for r in results[1:]:
1184                 f.write( f"{r['方案']}: 总费用变化 {r['总费用']-base['总费用']:.2f} 元, 紧急购电量
1185                 减少 {base['紧急购电量kWh']-r['紧急购电量kWh']:.2f} kWh, 富余电量减少 {base['富余电量kWh']-
1186                 r['富余电量kWh']:.2f} kWh。\\n")
1187             focus=raw.get(cfg.focus_scheme_name)
1188             if focus is not None and focus.get('response_records'):
1189                 recs=[ r for r in focus['response_records'] if r['更新时点']== 12 and
1190                 str(r['日期'])>= str(dates[31])]
1191                 if recs:
1192                     detail_path=out/'Q4-3_0点与12点预报决策响应明细.csv'
1193                     with open(detail_path, 'w', newline='', encoding='utf-8-sig') as f:

```

```

1189         w=csv.DictWriter(f,fieldnames=list(recs[0].keys()))
1190         w.writeheader(); w.writerows(recs)
1191         dpv=np.array([r['光伏预报修正量kW'] for r in recs],dtype=float)
1192         dprice=np.array([r['电价预测修正量元每kWh'] for r in recs],dtype=float)
1193         dnet=np.array([r['有效净负荷修正量kW'] for r in recs],dtype=float)
1194         dbuy=np.array([r['购电调整量kW'] for r in recs],dtype=float)
1195         def corr( a, b): return float( np.corrcoef(a, b)[0, 1]) if np.std(
1196             a)>1e-9 and np.std( b)>1e-9 else np.nan
1197         cpv=corr(dpv,dbuy); cprice=corr(dprice,dbuy); cnet=corr(dnet,dbuy)
1198         up_kwh=float(np.sum(np.maximum(dbuy,0))*cfg.dt)
1199         down_kwh=float(np.sum(np.maximum(-dbuy,0))*cfg.dt)
1200         br=raw[cfg.comparison_names[0]]; fr=focus
1201         bt=float( np.sum(br['plan_cost'][formal]))+
1202             np.sum(br['adj_cost'][formal])+ np.sum(br['emg_cost'][formal]))
1203         ft=float( np.sum(fr['plan_cost'][formal])+
1204             np.sum(fr['adj_cost'][formal])+ np.sum(fr['emg_cost'][formal]))
1205         bemg=float(np.sum(br['emg_energy'][formal]))
1206         femg=float(np.sum(fr['emg_energy'][formal]))
1207         bsur=float(np.sum(br['surplus_energy'][formal]))
1208         fsur=float(np.sum(fr['surplus_energy'][formal]))
1209         with open(out/'Q4-3_0点与12点预报重点结论.txt','w',encoding='utf-8') as f:
1210             f.write('Q4-3: 0:00 与 12:00 预报的边际价值分析\n'+ '*62+\n')
1211             f.write('12:00同时更新附件3光伏预报与基于已实现实时价格的电价预测，再重解未来24
1212 小时LP.\n')
1213             f.write(f'corr(光伏预报修正量，购电调整量) = {cpv:.4f}\n')
1214             f.write(f'corr(电价预测修正量，购电调整量) = {cprice:.4f} (电价作用具有跨时段储
1215 能耦合，不要简单单调符号)\n')
1216             f.write(f'corr(有效净负荷修正量，购电调整量) = {cnet:.4f}\n')
1217             f.write(f'12:00累计向上调整 = {up_kwh:.2f} kWh\n12:00累计向下调整 = {down_kwh:.2
1218 f} kWh\n')
1219             f.write(f'仅0:00总费用 = {bt:.2f} 元\n0:00+12:00总费用 = {ft:.2f} 元，变化 {ft-
1220 bt:.2f} 元\n')
1221             f.write( f'紧急购电变化 = {femg- bemg:+.2f} kWh\n富余电量变化 = {fsur-bsur:+.2f}
1222 kWh\n')
1223             logger.info('已输出预报修正与购电调整响应分析结果。')
1224         logger.info('完成: %s',csv_path); return results
1225 def main():
1226     cfg=CFG; out=Path(cfg.out_dir); out.mkdir(parents=True,exist_ok=True)
1227     logger=setup_logger(out)
1228     logger.info('不同预报时点选择的结果影响')
1229     dates,price_prior,load_prior,load,pv,price_actual=load_inputs(cfg,logger)
1230     pv_hourly=load_attachment3(cfg,dates,logger); D=len(dates)
1231     n=cfg.slots_per_day; run_D=min(D,cfg.max_days) if cfg.max_days>0 else D
1232     cache_load,cache_price=try_load_q42_forecast_cache(cfg,D,logger)
1233     initial_buy=np.full((D,n),np.nan); effective_buy=np.full((D,n),np.nan)
1234     ch=np.zeros((D,n)); dis=np.zeros((D,n)); emg=np.zeros((D,n))
1235     sur=np.zeros((D,n)); soc_hist=np.full((D,n),np.nan)
1236     soc_start=np.full(D,np.nan); soc_end=np.full(D,np.nan)
1237     net_errors=np.full((D,n),np.nan); cplus=np.full((D,n),np.nan)
1238     cminus=np.full((D,n),np.nan)
1239     alpha_store=np.full((D,n),np.nan); fallback_reserve_store=np.zeros((D,n))
1240     load_fc_full_store=np.full((D,cfg.forecast_slots),np.nan)
1241     price_fc_full_store=np.full((D,cfg.forecast_slots),np.nan)
1242     stage_hist=np.full((D,4,cfg.rolling_horizon_slots),np.nan)
1243     price_stage_fc=np.full((D,4,cfg.rolling_horizon_slots),np.nan)
1244     plan_cost=np.zeros(D); adj_cost=np.zeros(D); emg_cost=np.zeros(D)
1245     up_energy=np.zeros(D); down_energy=np.zeros(D)
1246     load_rmse=np.zeros(D); pv_rmse0=np.zeros(D); mu0=np.zeros(D)
1247     lp=build_day_ahead_templates(cfg); soc=cfg.e0; stage_idx={0:0,6:1,12:2,18:3}
1248     hours=(0,6,12,18)
1249     bar=tqdm( range(run_D), desc='Q4-3全年计算', unit='day', dynamic_ncols=True,
1250         mininterval=.25)
1251     for d in bar:
1252         day=dates[d]; soc_start[d]=soc
1253         if d>=2: archive_stage_forecast_errors( d- 2, load_fc_full_store,
1254             pv_hourly, load, pv, stage_hist, cfg)
1255         if cache_load is not None: lf_full=np.asarray(cache_load[d],dtype=float)
1256         else: lf_full=np.asarray( load_forecast(d, load, load_prior, cfg,
1257             logger)[:cfg.forecast_slots], dtype=float)
1258         if cache_price is not None: pr_full=np.asarray( cache_price[d],
1259             dtype=float)
1260         else: pr_full=np.asarray( price_forecast(d, price_actual, price_prior,
1261             cfg, logger)[:cfg.forecast_slots], dtype=float)
1262         load_fc_full_store[d]=lf_full; price_fc_full_store[d]=pr_full
1263         lf0=lf_full[:n]
1264         pf0=pv_stage_forecast_horizon_10min(d,0,pv_hourly,pv,cfg)[:n]
1265         price_stage_fc[d,0]=pr_full[:n]
1266         update_risk_cost_cache( d, cplus, cminus, net_errors, price_actual,
1267             soc_hist, initial_buy, load, pv, cfg)
1268         alpha,_,fallback=dynamic_risk_weights(d,net_errors,cplus,cminus,cfg)
1269         alpha_store[d]=alpha; fallback_reserve_store[d]=fallback
1270         da=solve_day_ahead_price(pr_full,lf0,pf0,fallback,soc,cfg,lp)
1271         mu0[d]=da['mu']; initial_buy[d]=da['buy']; current_buy=da['buy'].copy()

```

```

1267 current_ch=da['ch'].copy(); current_dis=da['dis'].copy()
1268 effective_buy[d]=current_buy
1269 # 真实价格只用于事后结算, 不进入0:00决策
1270 plan_cost[d]=float(np.sum(initial_buy[d]*price_actual[d])*cfg.dt)
1271 for si,h in enumerate(hours):
1272     start=h*6
1273     if h>0:
1274         lf_h=load_stage_forecast_horizon(lf_full,load[d],h,cfg)
1275         pf_h=pv_stage_forecast_horizon_10min(d,h,pv_hourly,pv,cfg)
1276         reserve_h=stage_reserve_from_history( d, stage_idx[h],
1277             stage_hist, fallback, h, cfg)
1278         pr_h=price_stage_forecast_horizon(pr_full,price_actual[d],h,cfg)
1279         price_stage_fc[d,si]=pr_h
1280         adj=solve_rolling_mpc_24h_price( pr_h, lf_h, pf_h, reserve_h,
1281             soc, current_buy, start, cfg)
1282         m=n-start; current_buy[start:]=adj['buy'][:m]
1283         current_ch[start:]=adj['ch'][:m]
1284         current_dis[start:]=adj['dis'][:m]
1285         # 真实结算
1286         real_today=price_actual[d,start:]
1287         upv=adj['up'][:m]; dnv=adj['down'][:m]
1288         adj_cost[d]=float( np.sum((1.5* real_today* upv- 0.5*
1289             real_today* dnv)* cfg.dt))
1290         up_energy[d]+=adj['up_energy']
1291         down_energy[d]+=adj['down_energy']
1292     end=hours[si+1]*6 if si+1<len(hours) else n
1293     effective_buy[d,start:end]=current_buy[start:end]
1294     for t in range(start,end):
1295         soc_hist[d,t]=soc
1296         sol=operate_one_slot( soc, current_buy[t], load[d, t], pv[d, t],
1297             current_ch[t], current_dis[t], cfg)
1298         ch[d,t]=sol['ch']; dis[d,t]=sol['dis']; emg[d,t]=sol['emg']
1299         sur[d,t]=sol['sur']; soc=sol['E']
1300     soc_end[d]=soc
1301     emg_cost[d]=float(np.sum(emg[d]*5*price_actual[d])*cfg.dt)
1302     net_errors[d]=(load[d]-pv[d])-(lf0-pf0)
1303     load_rmse[d]=float(np.sqrt(np.mean((load[d]-lf0)**2)))
1304     pv_rmse0[d]=float(np.sqrt(np.mean((pv[d]-pf0)**2)))
1305     if cfg.verbose_each_day:
1306         logger.info(
1307             '【%3d/%3d】%s | SOC %.0f→%.0f | 初始 %.2f | 调整 %.2f | 紧急 %.2f | 紧急 %.1f
1308 kWh',
1309                 d+ 1, run_D, day, soc_start[d], soc_end[d], plan_cost[d],
1310                 adj_cost[d], emg_cost[d], np.sum(emg[d])* cfg.dt)
1311     if (d+1)%30==0 or d==run_D-1:
1312         bar.set_postfix_str( f'{day} SOC={soc:.0f} 紧急={np.sum(emg[d])*cfg.dt:.0f}kWh')
1313     if cfg.checkpoint_every_days>0 and ( (d+ 1)%cfg.checkpoint_every_days==
1314         0 or d== run_D- 1):
1315         np.savez_compressed( out/ 'q4_3_checkpoint.npz', last_day=d,
1316             initial_buy=initial_buy[:d+ 1], effective_buy=effective_buy[:d+
1317             1], ch=ch[:d+ 1], dis=dis[:d+ 1], emg=emg[:d+ 1], sur=sur[:d+1],
1318             soc_start=soc_start[:d+ 1], soc_end=soc_end[:d+ 1],
1319             load_fc=load_fc_full_store[:d+ 1],
1320             price_fc=price_fc_full_store[:d+ 1],
1321             price_stage_fc=price_stage_fc[:d+ 1],
1322             fallback_reserve=fallback_reserve_store[:d+ 1],
1323             stage_net_error_hist=stage_hist[:d+ 1])
1324     buy_cost_after_adjust=plan_cost+adj_cost
1325     if run_D>=32: fill_result4_3( cfg.template, cfg.out_xlsx, dates,
1326         price_actual, initial_buy, effective_buy, buy_cost_after_adjust, ch,dis,
1327         soc_start, soc_end, emg, cfg, logger)
1328     save_outputs( dates, price_actual, price_fc_full_store, price_stage_fc,
1329         initial_buy, effective_buy, ch, dis, emg, sur, soc_start, soc_end,
1330         plan_cost, adj_cost, emg_cost, up_energy, down_energy, load_rmse,
1331         pv_rmse0, run_D, cfg, logger)
1332     # 表格
1333     if run_D>=32:
1334         export_paper_tables_q43( dates, price_actual, initial_buy,effective_buy,
1335             ch, dis, emg, soc_start, soc_end, plan_cost, adj_cost, emg_cost,
1336             run_D, cfg, logger)
1337     # 直接复用主模型已生成的全年预测/风险缓存
1338     if cfg.run_comparison and run_D>=32:
1339         run_update_time_comparison_q43( dates, price_actual, load, pv,
1340             load_fc_full_store, price_fc_full_store, pv_hourly,
1341             fallback_reserve_store, stage_hist, run_D, cfg, lp, logger )
1342     logger.info('='*72)
1343     logger.info('Q4-3完成。输出目录: %s',cfg.out_dir)
1344 if __name__=='__main__': main()

```

code/Q4-3.py

Q4-3-2.py

```

1 from __future__ import annotations
2 import csv
3 import math
4 import os
5 import sys
6 import importlib.util
7 from pathlib import Path
8 from datetime import datetime, date, timedelta
9 from typing import List, Dict, Tuple, Optional
10 import numpy as np
11 from openpyxl import load_workbook, Workbook
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei",
16     "Noto Sans CJK SC", "DejaVu Sans"]
17 plt.rcParams["axes.unicode_minus"] = False
18 # 全局绘图样式
19 plt.rcParams["font.size"] = 12
20 plt.rcParams["axes.spines.top"] = False
21 plt.rcParams["axes.linewidth"] = 1.5
22 plt.rcParams["xtick.major.width"] = 1.5
23 plt.rcParams["ytick.major.width"] = 1.5
24 plt.rcParams["xtick.major.size"] = 6
25 plt.rcParams["ytick.major.size"] = 6
26 # 配色
27 C_BUY="#2E74B5";C_PV="#27AE60";C_DIS="#E67E22";C_LOAD="#34495E"
28 C_CH="#E74C3C";C_W="#95A5A6";C_SOC="#7F8C8D";C_MIN="#2980B9"
29 C_RES="#16A085";C_ALPHA="#8E44AD"
30 class RunConfig:
31     BASE=Path(__file__).resolve().parent if "__file__" in globals() else Path(".")
32     Q43_CANDIDATES = [BASE / "Q4-3.py"]
33     ATT3_CANDIDATES = [BASE / "附件3.xlsx", BASE / "附件3(1).xlsx"]
34     CHECKPOINT_CANDIDATES = [
35         BASE / "Q4-3结果" / "q4_3_checkpoint.npz",
36         BASE / "q4_3_outputs_price_forecast" / "q4_3_checkpoint.npz",
37         BASE / "q4_3_checkpoint.npz",
38         BASE / "_q43_test_outputs" / "q4_3_checkpoint.npz",
39     ]
40     OUT = BASE / "Q4-3-2结果"
41     FIG = OUT / "Q4-3-2图片输出"
42     EXISTING_HOURS = (0, 6, 12, 18)
43     POSSIBLE_EXTRA_HOURS=tuple(h for h in range(1,24) if h not in (0,6,12,18))
44     LOOKAHEAD_HOURS = 4;LOOKAHEAD_DECAY_TAU = 1.5
45     MAX_AUTO_CANDIDATES = 7;MIN_AUTO_CANDIDATES = 4
46     MIN_CANDIDATE_GAP_HOURS = 2;MANUAL_EXTRA_HOURS = None
47     RUN_ALL_CANDIDATES_TOGETHER = True
48     RECENT_ERROR_HOURS = 3;BIAS_DECAY_HOURS = 4.0
49     HISTORY_DAYS_FOR_TAIL = 30;MAX_BIAS_ABS_KW = 1500.0
50     MAX_DAYS = int(os.environ.get("Q43_EXTRA_MAX_DAYS", "365"))
51     REPRESENTATIVE_DATES = (
52         date(2025, 3, 20), date(2025, 6, 21),
53         date(2025, 9, 23), date(2025, 12, 21),
54     )
55 class RiskBandConfig:
56     DAYLIGHT_THRESHOLD = 100.0; BAND_QUANTILE = 0.85
57     BAND_MIN_KW = 120.0; BAND_FLOOR_RATIO = 0.08; USE_HOUR_SMOOTH = True
58     C_ES = 0.03; C_SUR = 0.06; K_SHORT = 5.0
59     K_DIS = 1.5; K_CH = 0.5; EPS = 1e-9
60 def first_existing(paths: List[Path]) -> Path:
61     for p in paths:
62         if p.exists():
63             return p
64     raise FileNotFoundError("未找到文件: \n" + "\n".join(str(p) for p in paths))
65 def ensure_dir(p: Path):
66     p.mkdir(parents=True, exist_ok=True)
67 def import_py(path: Path, name: str):
68     spec = importlib.util.spec_from_file_location(name, str(path))
69     if spec is None or spec.loader is None:
70         raise ImportError(f"无法加载: {path}")
71     mod = importlib.util.module_from_spec(spec)
72     sys.modules[name] = mod
73     spec.loader.exec_module(mod)
74     return mod
75 def write_csv(path: Path, rows: List[dict]):
76     if not rows:
77         return
78     ensure_dir(path.parent)
79     with open(path, "w", newline="", encoding="utf-8-sig") as f:
80         w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
81         w.writeheader(); w.writerows(rows)
82 def style(ax):
83     ax.grid(True, axis="y", linestyle="--", linewidth=0.7, alpha=0.3)

```

```

84     ax.spines["top"].set_visible(False)
85     ax.spines["right"].set_visible(False)
86     ax.tick_params(labelsiz=TICK_FS)
87 NO_TITLE = True
88 AXIS_FS = 15
89 TICK_FS = 12
90 LEGEND_FS = 17
91 SOFT_RED,SOFT_GREEN,SOFT_BLUE,SOFT_GRAY='#E9A6A1','#8BCF8B','#7097CA','#CFCECF'
92 BAR_EDGE = dict(edgecolor='black', linewidth=1.2)
93 def _set_labels(ax, x, y):
94     ax.set_xlabel(x, fontsize=AXIS_FS, color='black')
95     ax.set_ylabel(y, fontsize=AXIS_FS, color='black')
96 def _legend_top(ax, ncol=None, fs=LEGEND_FS):
97     h, l = ax.get_legend_handles_labels()
98     if h:
99         ax.legend(h, l, loc='lower center', frameon=False, ncol=ncol or len(h),
100                  fontsize=fs, bbox_to_anchor=(0.5, 1.02), columnspacing=1.6)
101 def load_checkpoint(path: Path, run_D: int):
102     z = np.load(path, allow_pickle=True)
103     need = ["soc_start", "ch", "dis", "effective_buy", "load_fc", "price_fc",
104            "fallback_reserve", "stage_net_error_hist"]
105     miss = [k for k in need if k not in z.files]
106     if miss:
107         raise ValueError("Q4-3 checkpoint缺少字段: " + ", ".join(miss))
108     out = {k: np.asarray(z[k]) for k in z.files}
109     if len(out["soc_start"]) < run_D:
110         raise ValueError(f"checkpoint只有{len(out['soc_start'])}天, "
111                          f"要求{run_D}天。请先完整运行Q4-3。")
112     return out
113 def reconstruct_soc_hourly(cp: dict, run_D: int, cfg):
114     soc_start = np.asarray(cp["soc_start"][:run_D], float)
115     ch = np.asarray(cp["ch"][:run_D], float)
116     dis = np.asarray(cp["dis"][:run_D], float)
117     out = np.zeros((run_D, 24), float)
118     for d in range(run_D):
119         soc = float(soc_start[d])
120         for h in range(24):
121             out[d, h] = soc
122             for k in range(h*6, h*6+6):
123                 soc += cfg.eta_c*ch[d,k]*cfg.dt - dis[d,k]*cfg.dt/cfg.eta_d
124                 soc = min(cfg.e_max, max(cfg.e_min, soc))
125     return out
126 def load_forecast_records_xlsx(path: Path, kind="official"):
127     wb = load_workbook(path, data_only=True, read_only=True)
128     ws = wb[wb.sheetnames[0]]
129     cur = None; recs=[]
130     for r in ws.iter_rows(min_row=2, max_col=26, values_only=True):
131         if r[0] not in (None, ""):
132             x=r[0]
133             if isinstance(x, datetime): cur=x.date()
134             elif isinstance(x, date): cur=x
135             else: cur=datetime.strptime(str(x).replace('/', '-')[:10], '%Y-%m-%d').date()
136             if cur is None: continue
137             txt=str(r[1]).strip(); h=int(txt.split(':')[0]) if ':' in txt else int(float(txt))
138             fc=np.asarray([0.0 if v is None else float(v) for v in r[2:26]],float)
139             recs.append({"issue_dt":datetime.combine(cur,datetime.min.time())+timedelta(hours=h),
140                        "issue_hour":h,"fc":np.maximum(fc,0.0),"kind":kind})
141     wb.close(); recs.sort(key=lambda z:z["issue_dt"])
142     return recs
143 def records_by_key(records):
144     return {(r["issue_dt"].date(), int(r["issue_hour"])):r for r in records}
145 def latest_record_before(records, issue_dt):
146     c=[r for r in records if r["issue_dt"] < issue_dt]
147     return max(c,key=lambda z:z["issue_dt"]) if c else None
148 def record_value_at_target(rec, target_dt):
149     lead=int(round((target_dt-rec["issue_dt"]).total_seconds()/3600.0))
150     if 1 <= lead <= 24:
151         return float(rec["fc"][lead-1]), lead
152     return None, None
153 def actual_hour_value(target_dt, dates, actual_hourly):
154     dmap={d:i for i,d in enumerate(dates)}
155     dd=target_dt.date(); h=target_dt.hour
156     if h==0:
157         dd=dd-timedelta(days=1); hh=23
158     else: hh=h-1
159     if dd not in dmap: return None
160     return float(actual_hourly[dmap[dd],hh])
161 def past_climatology(target_dt, dates, actual_hourly, issue_dt):
162     dmap={d:i for i,d in enumerate(dates)}; vals=[]
163     dd=target_dt.date(); h=target_dt.hour
164     if h==0: dd=dd-timedelta(days=1); hh=23
165     else: hh=h-1
166     if dd not in dmap: return 0.0
167     i=dmap[dd]

```

```

168     for j in range(max(0,i-RunConfig.HISTORY_DAYS_FOR_TAIL),i):
169         vals.append(actual_hourly[j, hh])
170     return float(np.median(vals)) if vals else 0.0
171 def build_extra_record(extra_issue_dt, official_records, dates, actual_hourly):
172     old=latest_record_before(official_records, extra_issue_dt)
173     if old is None: return None
174     residuals=[]; weights=[]
175     for back in range(RunConfig.RECENT_ERROR_HOURS-1, -1, -1):
176         t=extra_issue_dt-timedelta(hours=back)
177         ov,_=record_value_at_target(old,t); av=actual_hour_value(t, dates, actual_hourly)
178         if ov is None or av is None: continue
179         residuals.append(av-ov); weights.append(math.exp(-back/1.5))
180     if residuals:
181         w=np.asarray(weights, float); w/=w.sum()
182         bias=float(np.dot(w, np.asarray(residuals, float)))
183     else: bias=0.0
184     bias=float(np.clip(bias, -RunConfig.MAX_BIAS_ABS_KW, RunConfig.MAX_BIAS_ABS_KW))
185     fc=np.zeros(24, float)
186     for lead in range(1,25):
187         t=extra_issue_dt+timedelta(hours=lead)
188         base,_=record_value_at_target(old,t)
189         if base is None: base=past_climatology(t, dates, actual_hourly, extra_issue_dt)
190         corr=bias*math.exp(-(lead-1)/RunConfig.BIAS_DECAY_HOURS)
191         if base < 100.0: corr*=0.25
192         fc[lead-1]=max(base+corr, 0.0)
193     return {"issue_dt":extra_issue_dt, "issue_hour":extra_issue_dt.hour, "fc":fc,
194           "kind":"extra_proxy", "bias":bias, "source_issue_hour":old["issue_hour"]}
195 def build_augmented_records(official_records, extra_hours, dates, actual_hourly):
196     out=[dict(r) for r in official_records]; diag=[]
197     days=sorted({r["issue_dt"].date() for r in official_records})
198     for dd in days:
199         for h in extra_hours:
200             issue=datetime.combine(dd, datetime.min.time())+timedelta(hours=h)
201             rec=build_extra_record(issue, official_records, dates, actual_hourly)
202             if rec is None: continue
203             out.append(rec); diag.append({"日期":dd.isoformat(), "新增发布时间":h,
204             "上一官方发布时间":rec["source_issue_hour"],
205             "截至发布时间估计偏差bias_kw":round(rec["bias"], 4)})
206     out.sort(key=lambda z:z["issue_dt"])
207     return out, diag
208 def export_records_xlsx(path, records):
209     wb=Workbook(); ws=wb.active; ws.title="Sheet1"
210     ws.append(["日期", "预报时刻"]+[f"预报{i}小时" for i in range(1,25)])
211     by={}
212     for r in records: by.setdefault(r["issue_dt"].date(), []).append(r)
213     for dd in sorted(by):
214         rr=sorted(by[dd], key=lambda z:z["issue_dt"])
215         for j, r in enumerate(rr):
216             ws.append([dd.isoformat() if j==0 else None, f"{r['issue_hour']}:00"]
217                     +[float(v) for v in r["fc"]])
218     ensure_dir(path.parent); wb.save(path); wb.close()
219 def pv_record_horizon_10min(day_idx, stage_hour, rec_map, dates, pv_actual, cfg):
220     rec=rec_map[(dates[day_idx], int(stage_hour))]
221     hourly=np.asarray(rec["fc"], float); start=stage_hour*6
222     anchor=0.0 if stage_hour==0 else float(pv_actual[day_idx, start-1])
223     xp=np.arange(25, dtype=float); fp=np.concatenate([anchor, hourly])
224     target=(np.arange(cfg.rolling_horizon_slots)+1)/6.0
225     return np.maximum(np.interp(target, xp, fp), 0.0)
226 def effective_pv_hourly(records, dates):
227     """得到每天1..24点目标时刻的最后可用PV预报及其发布时间。"""
228     vals=np.full((len(dates), 24), np.nan); issue=np.full((len(dates), 24), -1, int)
229     for di, dd in enumerate(dates):
230         d0=datetime.combine(dd, datetime.min.time())
231         for hh in range(1,25):
232             t=d0+timedelta(hours=hh); best=None
233             for r in records:
234                 if r["issue_dt"] >= t: break
235                 v, lead=record_value_at_target(r, t)
236                 if v is not None and (best is None or r["issue_dt"]>best[0]):
237                     best=(r["issue_dt"], v, r["issue_hour"])
238             if best is not None:
239                 vals[di, hh-1]=best[1]; issue[di, hh-1]=best[2]
240     if np.isnan(vals).any(): raise ValueError("存在目标小时缺少PV预报")
241     return vals, issue
242 def risk_economic_costs(price, load, fc_pv, soc, cfg):
243     # 与Q3风险带一致，但直接使用Q4实际波动电价。
244     a_dis=max(0.0, min(cfg.p_max, cfg.eta_d*max(soc-cfg.e_min, 0.0)))
245     a_ch=max(0.0, min(cfg.p_max, max(cfg.e_max-soc, 0.0)/cfg.eta_c))
246     net=load-fc_pv; shortage=max(net, 0.0); surplus=max(-net, 0.0)
247     pi_plus=a_dis/(a_dis+shortage+RiskBandConfig.EPS)
248     pi_minus=a_ch/(a_ch+surplus+RiskBandConfig.EPS)
249     cp=RiskBandConfig.C_ES+price*(RiskBandConfig.K_DIS*pi_plus+RiskBandConfig.K_SHORT*(1-pi_plus))

```

```

250     cm=RiskBandConfig.C_ES+RiskBandConfig.K_CH*price*pi_minus+RiskBandConfig.C_SUR*(1-pi_minus)
251     return cp,cm
252 def build_hour_scale(actual,fc):
253     err=fc-actual; raw=np.zeros(24); base=np.zeros(24)
254     for h in range(24):
255         m=((actual[:,h]>=RiskBandConfig.DAYLIGHT_THRESHOLD)
256            |(fc[:,h]>=RiskBandConfig.DAYLIGHT_THRESHOLD))
257         if np.any(m):
258             q=float(np.quantile(np.abs(err[m,h]),RiskBandConfig.BAND_QUANTILE))
259             med=float(np.median(actual[m,h]))
260             floor=max(RiskBandConfig.BAND_FLOOR_RATIO*med,
261                      RiskBandConfig.BAND_MIN_KW)
262             raw[h]=q; base[h]=max(q,floor)
263     if RiskBandConfig.USE_HOUR_SMOOTH:
264         sm=base.copy()
265         for h in range(24): sm[h]=.25*base[max(h-1,0)]+.5*base[h]+.25*base[min(h+1,23)]
266         base=sm
267     return raw,base
268 def build_effective_price_hourly(q43, price_fc_full, price_actual, cfg, run_D):
269     """按原0/6/12/18方案，给每个目标整点配置当时最后一次可用的价格预测。"""
270     pred=np.full((run_D,24),np.nan); issue=np.full((run_D,24),-1,int)
271     stages=(0,6,12,18)
272     for d in range(run_D):
273         for h0 in stages:
274             ph=q43.price_stage_forecast_horizon(price_fc_full[d],price_actual[d],h0,cfg)
275             # ph[0]对应发布后第一个10分钟；整点目标 h= h0+1..., 取每小时末点位置 5,11,...
276             next_stage={0:6,6:12,12:18,18:24}[h0]
277             for target_h in range(h0+1,next_stage+1):
278                 j=(target_h-h0)*6-1
279                 pred[d,target_h-1]=ph[j]; issue[d,target_h-1]=h0
280     return pred,issue
281 def build_combined_hourly_risk(q43, dates, load, pv, price_actual, cp, cfg,
282                                run_D, official_records):
283     # 小时样本取每小时末10分钟点，与附件3整点目标保持一致。
284     idx=np.array([6*h-1 for h in range(1,25)])
285     load_h=load[:,run_D][:,idx]; pv_h=pv[:,run_D][:,idx]; price_h=price_actual[:,run_D][:,idx]
286     soc_h=reconstruct_soc_hourly(cp,run_D,cfg)
287     buy_h=np.zeros((run_D,24));
288     for h in range(24): buy_h[:,h]=np.sum(cp["effective_buy"][:,run_D,h*6:(h+1)*6],axis=1)*cfg.dt
289     pv_fc,pv_issue=effective_pv_hourly(official_records,dates[:,run_D])
290     _,scale=build_hour_scale(pv_h,pv_fc)
291     pfc_h,p_issue=build_effective_price_hourly(q43,cp["price_fc"][:,run_D],
292                                                price_actual[:,run_D],cfg,run_D)
293     rows=[]; agg=[]
294     for h in range(24):
295         pv_econ=[]; price_econ=[]; danger=[]
296         for d in range(run_D):
297             cpv,cmv=risk_economic_costs(price_h[d,h],load_h[d,h],pv_fc[d,h],soc_h[d,h],cfg)
298             den=max(cpv+cmv,RiskBandConfig.EPS)
299             bplus=2*cmv/den*scale[h]
300             d_ex=max(pv_fc[d,h]-(pv_h[d,h]+bplus),0.0)
301             # 危险高估近似造成的1小时缺口经济暴露（元）
302             pv_loss=d_ex*cpv
303             # 价格预测误差 × 该小时基线购电电量（元）
304             price_loss=abs(pfc_h[d,h]-price_h[d,h])*buy_h[d,h]
305             pv_econ.append(pv_loss); price_econ.append(price_loss); danger.append(d_ex)
306         sp=float(np.sum(pv_econ)); sl=float(np.sum(price_econ)); sc=sp+sl
307         agg.append({"目标小时":h+1,"PV危险高估经济风险元":sp,"价格预测经济暴露元":sl,
308                   "综合信息风险元":sc,"PV危险高估总量kW":float(np.sum(danger)),
309                   "平均实际电价":float(np.mean(price_h[:,h])),
310                   "平均绝对价格误差":float(np.mean(np.abs(pfc_h[:,h]-price_h[:,h]))),
311                   "平均基线购电电量kWh":float(np.mean(buy_h[:,h]))})
312     return agg,pv_fc,pfc_h
313 def _future_coverage_score(hourly_rows, issue_hour):
314     stats = {int(r["目标小时"]): float(r["综合信息风险元"]) for r in hourly_rows}
315     score = 0.0
316     covered = []
317     for k in range(1, RunConfig.LOOKAHEAD_HOURS + 1):
318         target = issue_hour + k
319         if target > 24:
320             break
321         w = math.exp(-(k - 1) / RunConfig.LOOKAHEAD_DECAY_TAU)
322         val = stats.get(target, 0.0); score += w * val
323         covered.append((target, w, val))
324     return float(score), covered
325 def build_candidate_score_rows(hourly_rows):
326     raw_risk = {int(r["目标小时"]): float(r["综合信息风险元"]) for r in hourly_rows}
327     rows = []
328     for h in RunConfig.POSSIBLE_EXTRA_HOURS:
329         score, covered = _future_coverage_score(hourly_rows, h)
330         rows.append({
331             "候选发布时间": int(h),
332             "未来风险覆盖得分元": float(score),

```

```

333         "覆盖目标小时": ", ".join(str(x[0]) for x in covered),
334         "该时点自身综合风险元": float(raw_risk.get(h, 0.0)),
335         "是否局部峰值": 0,
336         "是否最终候选": 0,
337         "候选排名": "",
338     })
339     smap = {int(r["候选发布时间"]): float(r["未来风险覆盖得分元"]) for r in rows}
340     possible = list(RunConfig.POSSIBLE_EXTRA_HOURS)
341     for r in rows:
342         h = int(r["候选发布时间"])
343         s = smap[h]
344         # 在“可选发布时间序列”上判断局部峰值，避免 6/12/18 的空缺导致误判。
345         pos = possible.index(h)
346         left = smap[possible[pos-1]] if pos > 0 else -np.inf
347         right = smap[possible[pos+1]] if pos < len(possible)-1 else -np.inf
348         if s >= left and s >= right and s > 0:
349             r["是否局部峰值"] = 1
350     return rows
351 def select_extra_hours(hourly_rows):
352     if RunConfig.MANUAL_EXTRA_HOURS is not None:
353         manual = sorted(set(int(x) for x in RunConfig.MANUAL_EXTRA_HOURS))
354         rows = build_candidate_score_rows(hourly_rows)
355         for r in rows:
356             if int(r["候选发布时间"]) in manual:
357                 r["是否最终候选"] = 1
358     return manual, rows
359 rows = build_candidate_score_rows(hourly_rows)
360 local = [r for r in rows if int(r["是否局部峰值"]) == 1]
361 local.sort(key=lambda z: float(z["未来风险覆盖得分元"]), reverse=True)
362 selected = []
363 def far_enough(h):
364     return all(abs(h - x) >= RunConfig.MIN_CANDIDATE_GAP_HOURS for x in selected)
365 # 第一轮：优先选择局部峰值。
366 for r in local:
367     h = int(r["候选发布时间"])
368     if far_enough(h):
369         selected.append(h)
370     if len(selected) >= RunConfig.MAX_AUTO_CANDIDATES:
371         break
372 # 第二轮：若局部峰值太少，则按全局评分补足，但仍保持最小时距。
373 if len(selected) < RunConfig.MIN_AUTO_CANDIDATES:
374     all_rows = sorted(rows, key=lambda z: float(z["未来风险覆盖得分元"]), reverse=True)
375     for r in all_rows:
376         h = int(r["候选发布时间"])
377         if h in selected:
378             continue
379         if far_enough(h):
380             selected.append(h)
381         if len(selected) >= RunConfig.MIN_AUTO_CANDIDATES:
382             break
383     selected = sorted(selected)
384     rank_order = sorted(selected, key=lambda h: next(
385         float(r["未来风险覆盖得分元"]) for r in rows if int(r["候选发布时间"]) == h
386     ), reverse=True)
387     rank_map = {h: i+1 for i, h in enumerate(rank_order)}
388     for r in rows:
389         h = int(r["候选发布时间"])
390         if h in selected:
391             r["是否最终候选"] = 1
392             r["候选排名"] = rank_map[h]
393     return selected, rows
394 def actual_horizon(day_idx, stage_hour, arr, cfg):
395     H=cfg.rolling_horizon_slots; flat=arr.reshape(-1); g0=day_idx*cfg.slots_per_day+stage_hour*6
396     if g0+H>len(flat): return None
397     return np.asarray(flat[g0:g0+H], float)
398 def build_extra_stage_error_history(q43, extra_hours, rec_map, dates, load_fc, load, pv, cfg, run_D):
399     out={h:np.full((run_D, cfg.rolling_horizon_slots), np.nan) for h in extra_hours}
400     for d in range(run_D):
401         for h in extra_hours:
402             la=actual_horizon(d, h, load, cfg); pa=actual_horizon(d, h, pv, cfg)
403             if la is None or pa is None: continue
404             lf=q43.load_stage_forecast_horizon(load_fc[d], load[d], h, cfg)
405             pf=pv.record_horizon_10min(d, h, rec_map, dates, pv, cfg)
406             out[h][d]=(la-pa)-(lf-pf)
407     return out
408 def extra_stage_reserve(q43, day_idx, h, hist_by_hour, fallback_day, cfg):
409     H=cfg.rolling_horizon_slots; fb=np.asarray(fallback_day, float)
410     clock=(h*6+np.arange(H))%cfg.slots_per_day
411     fallback=fb[clock]; end=max(0, day_idx-1)
412     start=max(0, end-cfg.stage_reserve_history_days)
413     hist=hist_by_hour[h][start:end]
414     if hist.shape[0] < cfg.stage_reserve_min_days: return fallback.copy()
415     out=np.zeros(H); age=np.arange(hist.shape[0]-1, -1, -1); w=cfg.stage_reserve_decay**age

```

```

416     for k in range(H):
417         vals=hist[:,k]; m=np.isfinite(vals)
418         if np.sum(m)<cfg.stage_reserve_min_days:out[k]=fallback[k]
419         else:out[k]=cfg.stage_reserve_scale*max(0.0,float(
420             q43.weighted_quantile(vals[m],w[m],cfg.stage_reserve_quantile)))
421     return np.clip(out,0,5000)
422 def simulate_scheme(q43,name,update_hours,records,dates,price_actual,load,pv,load_fc,price_fc,
423     fallback_store,official_hist,extra_hist,run_D,cfg,lp):
424     n=cfg.slots_per_day; official_idx={0:0,6:1,12:2,18:3}; rec_map=records_by_key(records)
425     update_hours=tuple(sorted(set(int(h) for h in update_hours)))
426     plan=np.zeros(run_D); adjc=np.zeros(run_D); emgc=np.zeros(run_D)
427     emge=np.zeros(run_D); sure=np.zeros(run_D)
428     upe=np.zeros(run_D); dne=np.zeros(run_D); s0a=np.zeros(run_D)
429     s1a=np.zeros(run_D); soc=cfg.e0
430     for d in range(run_D):
431         s0a[d]=soc; lf_full=load_fc[d]; pr_full=price_fc[d]; fallback=fallback_store[d]
432         pf0=pv_record_horizon_10min(d,0,rec_map,dates,pv,cfg)[:n]
433         da=q43.solve_day_ahead_price(pr_full,lf_full[:n],pf0,fallback,soc,cfg,lp)
434         init=da["buy"].copy(); cb=da["buy"].copy(); cc=da["ch"].copy(); cd=da["dis"].copy()
435         plan[d]=float(np.sum(init*price_actual[d])*cfg.dt)
436         stages=(0,)+update_hours
437         for si,h in enumerate(stages):
438             start=h*6
439             if h>0:
440                 lf=q43.load_stage_forecast_horizon(lf_full,load[d],h,cfg)
441                 pf=pv_record_horizon_10min(d,h,rec_map,dates,pv,cfg)
442                 if h in official_idx:
443                     reserve=q43.stage_reserve_from_history(d,official_idx[h],
444                         official_hist,fallback,h,cfg)
445                 else:
446                     reserve=extra_stage_reserve(q43,d,h,extra_hist,fallback,cfg)
447                 pr=q43.price_stage_forecast_horizon(pr_full,price_actual[d],h,cfg)
448                 sol=q43.solve_rolling_mpc_24h_price(pr,lf,pf,reserve,soc,cb,start,cfg)
449                 m=n-start; cb[start:]=sol["buy"][:m]
450                 cc[start:]=sol["ch"][:m]; cd[start:]=sol["dis"][:m]
451                 up=sol["up"][:m]; dn=sol["down"][:m]; real=price_actual[d,start:]
452                 adjc[d]+=float(np.sum((1.5*real*up-0.5*real*dn)*cfg.dt))
453                 upe[d]+=sol["up_energy"]; dne[d]+=sol["down_energy"]
454             end=stages[si+1]*6 if si+1<len(stages) else n
455             for t in range(start,end):
456                 o=q43.operate_one_slot(soc,cb[t],load[d,t],pv[d,t],cc[t],cd[t],cfg)
457                 soc=o["E"]; emge[d]+=o["emg"]*cfg.dt; sure[d]+=o["sur"]*cfg.dt
458                 emgc[d]+=o["emg"]*5*price_actual[d,t]*cfg.dt
459             s1a[d]=soc
460     return {"plan_cost":plan,"adj_cost":adjc,"emg_cost":emgc,"emg_energy":emge,"surplus_energy":
461         sure,
462         "up_energy":upe,"down_energy":dne,"soc_start":s0a,"soc_end":s1a}
463 def summarize(name,hours,r,run_D):
464     s=31 if run_D>=32 else 0; s1=slice(s,run_D); daily=np.asarray(r["emg_energy"][s1],float)
465     plan=float(np.sum(r["plan_cost"][s1])); adj=float(np.sum(r["adj_cost"][s1]))
466     emg=float(np.sum(r["emg_cost"][s1]))
467     return {"方案":name,"全部预报时点":",".join(["0"]+[str(h) for h in hours]),
468         "初始计划购电费":plan,"调整净费用":adj,
469         "紧急购电费":emg,"总费用":plan+adj+emg,"紧急购电量kWh":float(np.sum(daily)),
470         "日紧急购电量P95kWh":float(np.quantile(daily,.95)) if len(daily) else 0.0,
471         "单日最大紧急购电量kWh":float(np.max(daily)) if len(daily) else 0.0,
472         "紧急购电天数":int(np.sum(daily>1e-6)),
473         "富余电量kWh":float(np.sum(r["surplus_energy"][s1])),
474         "向上调整电量kWh":float(np.sum(r["up_energy"][s1])),
475         "向下调整电量kWh":float(np.sum(r["down_energy"][s1])),
476         "期末SOCkWh":float(r["soc_end"][run_D-1])}
477 def build_risk_band_diagnostics(q43,dates,load,pv,price_actual,cp,cfg,run_D,pv_fc_h):
478     idx=np.array([6*h-1 for h in range(1,25)])
479     load_h=load[:,run_D][:,idx]
480     pv_h=pv[:,run_D][:,idx]
481     price_h=price_actual[:,run_D][:,idx]
482     soc_h=reconstruct_soc_hourly(cp,run_D,cfg)
483     _,scale=build_hour_scale(pv_h,pv_fc_h)
484     rows=[]
485     by_day={}
486     for d in range(run_D):
487         lo=np.zeros(24); hi=np.zeros(24)
488         bp=np.zeros(24); bm=np.zeros(24)
489         danger=np.zeros(24,dtype=bool); safe=np.zeros(24,dtype=bool)
490         for h in range(24):
491             cplus,cminus=risk_economic_costs(
492                 price_h[d,h],load_h[d,h],pv_fc_h[d,h],soc_h[d,h],cfg
493             )
494             den=max(cplus+cminus,RiskBandConfig.EPS)
495             bplus=2*cminus/den*scale[h]
496             bminus=2*cplus/den*scale[h]
497             lower=max(0.0,pv_h[d,h]-bminus)
498             upper=pv_h[d,h]+bplus

```

```

498 bp[h]=bplus; bm[h]=bminus
499 lo[h]=lower; hi[h]=upper
500 danger[h]=pv_fc_h[d,h] > upper
501 safe[h]=pv_fc_h[d,h] < lower
502 rows.append({
503     "日期": str(dates[d]), "目标小时": h+1,
504     "实际光伏kW": float(pv_h[d,h]), "生效光伏预报kW": float(pv_fc_h[d,h]),
505     "风险带下界kW": float(lower), "风险带上界kW": float(upper),
506     "b_minus_kW": float(bminus), "b_plus_kW": float(bplus),
507     "危险高估": int(danger[h]), "保守低估": int(safe[h]),
508     "电价元每kWh": float(price_h[d,h]), "SOC近似kWh": float(soc_h[d,h]),
509     "c_plus_元每kWh": float(cplus), "c_minus_元每kWh": float(cminus),
510 })
511 by_day[dates[d]]={"actual":pv_h[d].copy(),"forecast":pv_fc_h[d].copy(),
512                  "lower":lo,"upper":hi,"danger":danger,
513                  "safe":safe,"b_plus":bp,"b_minus":bm}
514 return rows,by_day
515 def plot_risk_band_single(day, ana, path):
516     x=np.arange(1,25)
517     ac=ana["actual"]; fc=ana["forecast"]
518     lo=ana["lower"]; hi=ana["upper"]
519     danger=ana["danger"]; safe=ana["safe"]
520     fig,ax=plt.subplots(figsize=(12.6,5.8),dpi=200)
521     ax.fill_between(x,lo,hi,alpha=.12,color=C_BUY,label="动态非对称风险带")
522     ax.plot(x,ac,lw=2.2,color=C_PV,label="实际光伏")
523     ax.plot(x,fc,lw=2.0,color=C_BUY,label="生效光伏预报")
524     ax.plot(x,lo,lw=1.0,ls="--",alpha=.75,color=C_SOC,label="风险带下界")
525     ax.plot(x,hi,lw=1.0,ls="--",alpha=.75,color=C_SOC,label="风险带上界")
526     if np.any(danger):
527         ax.scatter(x[danger],fc[danger],s=56,marker="^",color=C_CH,zorder=6,label="危险高估")
528     if np.any(safe):
529         ax.scatter(x[safe],fc[safe],s=56,marker="v",color=C_DIS,zorder=6,label="保守低估")
530     ax.set_xlim(1,24)
531     ax.set_xticks(range(1,25))
532     ax.set_xlabel("目标小时 / h", fontsize=12)
533     ax.set_ylabel("光伏功率 / kW", fontsize=12)
534     ax.set_title(f"{day}: 动态非对称光伏经济风险带", fontsize=13, fontweight="bold", pad=12)
535     ax.legend(frameon=False,ncol=3, fontsize=10)
536     style(ax)
537     fig.tight_layout()
538     fig.savefig(path,bbox_inches="tight")
539     plt.close(fig)
540 def plot_risk_band_summary(rows, path):
541     danger=np.zeros(24); safe=np.zeros(24); total=np.zeros(24)
542     for r in rows:
543         h=int(r["目标小时"])-1
544         danger[h]+=int(r["危险高估"])
545         safe[h]+=int(r["保守低估"])
546         total[h]+=1
547     total=np.maximum(total,1); danger_rate=100*danger/total
548     safe_rate=100*safe/total; x=np.arange(1,25)
549     fig,ax=plt.subplots(figsize=(12.6,5.5),dpi=200)
550     ax.bar(x-.18,danger_rate,.36,color=C_CH,alpha=0.85,label="危险高估越界率")
551     ax.bar(x+.18,safe_rate,.36,color=C_DIS,alpha=0.85,label="保守低估越界率")
552     ax.set_xticks(range(1,25))
553     ax.set_xlabel("目标小时 / h", fontsize=12)
554     ax.set_ylabel("全年越界率 / %", fontsize=12)
555     ax.set_title("动态非对称光伏风险带: 各目标小时全年越界率", fontsize=13, fontweight="bold",
556                 pad=12)
557     ax.legend(frameon=False, fontsize=10)
558     style(ax)
559     fig.tight_layout()
560     fig.savefig(path,bbox_inches="tight")
561     plt.close(fig)
562 def plot_representative_risk_bands(q43, dates, load, pv, price_actual, cp,
563     cfg, run_D, pv_fc_h, out_dir):
564     rows,by_day=build_risk_band_diagnostics(
565         q43,dates,load,pv,price_actual,cp,cfg,run_D,pv_fc_h
566     )
567     write_csv(RunConfig.OUT/"动态非对称风险带逐时诊断.csv",rows)
568     # 09/10 风险带图已按需删除 (逐时诊断数据仍随 CSV 输出)
569     return rows
570 def plot_candidate_risk(hourly, candidate_rows, extra_hours, path):
571     x=np.arange(1,25)
572     pv=np.array([r["PV危险高估经济风险元"] for r in hourly])/1e4
573     pr=np.array([r["价格预测经济暴露元"] for r in hourly])/1e4
574     # 候选时点: 光伏风险与电价信息价值联合分布 (柔色+黑描边, 无标题)
575     fig,ax=plt.subplots(figsize=(12.5,5.8),dpi=300)
576     ax.bar(x,pv,width=0.55,color=SOFT_RED,alpha=0.95,label="光伏危险高估经济风险",zorder=3,**
577           BAR_EDGE)
578     ax.bar(x,pr,bottom=pv,width=0.55,color=SOFT_BLUE,alpha=0.95,
579           label="电价预测经济暴露",zorder=3,**BAR_EDGE)

```

```

578     for i,h in enumerate(extra_hours):
579         ax.axvline(h,ls="--",lw=1.1,color='#888888',alpha=0.8,
580                   label=f"自动候选 {h}:00" if i==0 else None)
581     ax.set_xticks(range(1,25))
582     if not NO_TITLE:
583         ax.set_title("Q4-3新增预报时点: 光伏风险与电价信息价值联合分布",
584                      fontsize=13,fontweight="bold",pad=12)
585     _set_labels(ax,"目标小时 / h","累计风险尺度 / 万元")
586     style(ax); _legend_top(ax, ncol=3)
587     fig.tight_layout()
588     fig.savefig(path,bbox_inches="tight"); plt.close(fig)
589     # 信息价值评分曲线 (无标题、大字号、图例图外上方)
590     xx=np.array([int(r["候选发布时间"]) for r in candidate_rows])
591     ss=np.array([float(r["未来风险覆盖得分元"]) for r in candidate_rows])/1e4
592     local=np.array([int(r["是否局部峰值"]) for r in candidate_rows],dtype=bool)
593     chosen=np.array([int(r["是否最终候选"]) for r in candidate_rows],dtype=bool)
594     fig,ax=plt.subplots(figsize=(12.5,5.5),dpi=300)
595     # 参考 figures4papers: LineCollection 渐变透明度折线 (左浅右深) + 曲线下浅填充
596     LINE_BLUE = '#3775BA'
597     from matplotlib.collections import LineCollection
598     pts = np.array([xx, ss]).T.reshape(-1, 1, 2)
599     segs = np.concatenate([pts[:-1], pts[1:]], axis=1)
600     alphas = np.linspace(0.30, 1.0, len(segs))
601     lc = LineCollection(segs, colors=[(0.216, 0.459, 0.729, a) for a in alphas],
602                          linewidth=3.2, capstyle='round', zorder=3)
603     ax.add_collection(lc)
604     ax.fill_between(xx, ss, 0, color=LINE_BLUE, alpha=0.05, linewidth=0, zorder=1)
605     ax.margins(x=0.02)
606     ax.plot(xx, ss, linestyle='none', marker="o", markersize=7.5,
607            markerfacecolor=LINE_BLUE, markeredgcolor='white',
608            markeredgewidth=1.4, label="未来风险覆盖得分", zorder=4)
609     if np.any(local):
610         ax.scatter(xx[local],ss[local],s=110,marker="^",color='#E9A6A1',
611                   edgcolor='#3775BA',linewidth=1.0,label="局部峰值",zorder=5)
612     if np.any(chosen):
613         # 星形沿纵轴上移半格, 避免覆盖下方的局部峰值三角
614         ax.scatter(xx[chosen],ss[chosen]+0.55,s=260,marker="*",color='#E67E22',
615                   edgcolor='white',linewidth=1.2,label="最终自动候选",zorder=6)
616     ax.set_xticks(range(1,24))
617     if not NO_TITLE:
618         ax.set_title("新增预报发布时间的未来风险覆盖得分", fontsize=13, fontweight="bold", pad
619                      =12)
620     _set_labels(ax,"新增预报发布时间 / h","可覆盖信息价值 / 万元")
621     style(ax); _legend_top(ax, ncol=3)
622     fig.tight_layout()
623     fig.savefig(path.parent/"第四问预报时间的信息价值.png",bbox_inches="tight"); plt.close(fig)
624 def plot_scheme(rows,out):
625     names=[r["方案"] for r in rows]; x=np.arange(len(rows)); base=rows[0]
626     # 相对基线总费用变化 (figures4papers 暖色柔粉红 + 黑描边, 无标题、大字号)
627     vals=np.array([r["总费用"]-base["总费用"] for r in rows])/1e4
628     f,a=plt.subplots(figsize=(max(10.5,1.15*len(rows)),5.2),dpi=300)
629     a.bar(x,vals,width=0.40,color='#E9A6A1',alpha=0.95,zorder=3,**BAR_EDGE)
630     a.axhline(0,lw=.9,color='black')
631     a.set_xticks(x,names,rotation=18,ha="right")
632     if not NO_TITLE:
633         a.set_title("新增预报时点对总费用的影响", fontsize=13, fontweight="bold", pad=12)
634     _set_labels(a,"预报时点方案","相对基线总费用变化 / 万元")
635     a.tick_params(labelsize=TICK_FS)
636     for i,v in enumerate(vals):
637         a.text(i,v+(0.12 if v>0 else -0.12),f"{v:+.2f}",ha="center",
638               va="bottom" if v>0 else "top",fontsize=11)
639     style(a); f.tight_layout()
640     f.savefig(out/"第四问新增预报时点对总费用的影响.png",bbox_inches="tight"); plt.close(f)
641     dc=np.array([r["总费用"]-base["总费用"] for r in rows])/1e4
642     de=np.array([r["紧急购电量kWh"]-base["紧急购电量kWh"] for r in rows])/1000
643     f,a=plt.subplots(figsize=(8.6,6.2),dpi=300)
644     a.axhline(0,lw=.8,color='black'); a.axvline(0,lw=.8,color='black')
645     cmap=plt.get_cmap('OrRd')
646     import matplotlib.colors as mcolors
647     norm=mcolors.Normalize(vmin=float(de.min()), vmax=float(de.max())+1e-9)
648     core=cmap(norm(de))
649     a.scatter(dc,de,s=430,color=core,alpha=0.10,zorder=4,linewidths=0)
650     a.scatter(dc,de,s=230,color=core,alpha=0.22,zorder=5,linewidths=0)
651     a.scatter(dc,de,s=130,color=core,edgcolor='black',linewidth=1.3,zorder=6)
652     offsets={
653         "原0/6/12/18": (10,-16,'left','center'),
654         "原方案+21:00": (12,0,'left','center'),
655         "原方案+16:00": (0,13,'center','bottom'),
656         "原方案+11:00": (12,10,'left','center'),
657         "原方案+1:00": (0,13,'center','bottom'),
658         "原方案+全部自动候选": (-14,0,'right','center'),
659     }

```

```

659 for i,n in enumerate(names):
660     dx,dy,ha,va=offsets.get(n,(0,13,'center','bottom'))
661     a.annotate(n,(dc[i],de[i]),xytext=(dx,dy),textcoords="offset points",
662               fontsize=13,ha=ha,va=va)
663 sm=plt.cm.ScalarMappable(cmap=cmap,norm=norm); sm.set_array([])
664 cbar=f.colorbar(sm,ax=a,pad=0.015,aspect=28)
665 cbar.set_label('相对基线紧急购电变化 / MWh',fontsize=12)
666 cbar.ax.tick_params(labelsize=10)
667 a.set_xlim(-0.8,19.6); a.set_ylim(-0.5,5.6)
668 if not NO_TITLE:
669     a.set_title("新增预报方案的经济性—可靠性 Pareto 分布", fontsize=13, fontweight="bold",
670               pad=12)
671 _set_labels(a,"相对基线总费用变化 / 万元","相对基线紧急购电变化 / MWh")
672 a.tick_params(labelsize=TICK_FS)
673 a.grid(True,axis="y",linestyle="--",linewidth=0.7,alpha=0.3)
674 a.spines["top"].set_visible(False); a.spines["right"].set_visible(False)
675 f.tight_layout()
676 f.savefig(out/"第四问可靠性Pareto分布.png",bbox_inches="tight"); plt.close(f)
677 def main():
678     ensure_dir(RunConfig.OUT); ensure_dir(RunConfig.FIG)
679     q43_path=first_existing(RunConfig.Q43_CANDIDATES)
680     att3=first_existing(RunConfig.ATT3_CANDIDATES)
681     cp_path=first_existing(RunConfig.CHECKPOINT_CANDIDATES)
682     q43=import_py(q43_path,"q43_main_for_extra"); cfg=q43.CFG; cfg.base_dir=str(RunConfig.BASE)
683     cfg.max_days=RunConfig.MAX_DAYS;cfg.verbose_each_day=False;cfg.use_intraday_load_bias=False
684     # 修正附件路径到当前目录
685     for attr,name in [("att1","附件1.xlsx"),("att2","附件2.xlsx"),
686                     ("att3","附件3.xlsx"),("att4","附件4.xlsx")]:
687         p=RunConfig.BASE/name
688         if p.exists(): setattr(cfg,attr,str(p))
689     class Silent:
690         def info(self,*a,**k): pass
691         def warning(self,*a,**k): pass
692     dates,price_prior,load_prior,load,pv,price_actual=q43.load_inputs(cfg,Silent())
693     run_D=min(len(dates),RunConfig.MAX_DAYS); dates=dates[:run_D]
694     load=load[:run_D]; pv=pv[:run_D]; price_actual=price_actual[:run_D]
695     cp=load_checkpoint(cp_path,run_D)
696     load_fc=np.asarray(cp["load_fc"][:run_D],float)
697     price_fc=np.asarray(cp["price_fc"][:run_D],float)
698     fallback=np.asarray(cp["fallback_reserve"][:run_D],float)
699     official_hist=np.asarray(cp["stage_net_error_hist"][:run_D],float)
700     official_all=load_forecast_records_xlsx(att3,"official"); dayset=set(dates)
701     official=[r for r in official_all if r["issue_dt"].date() in dayset]
702     hourly,pv_fc_h,price_fc_h=build_combined_hourly_risk(q43,dates,load,pv,
703                 price_actual,cp,cfg,run_D,official)
704     write_csv(RunConfig.OUT/"各小时_光伏风险与电价经济暴露.csv",hourly)
705     extra,crows=select_extra_hours(hourly)
706     write_csv(RunConfig.OUT/"自动候选时点筛选.csv",crows)
707     # 兼容旧文件名, 便于已有论文/脚本继续读取
708     write_csv(RunConfig.OUT/"综合风险候选时点筛选.csv",crows)
709     if not extra:
710         raise RuntimeError("未筛选出新增预报时点")
711     plot_candidate_risk(hourly,crows,extra,RunConfig.FIG/"第四问候选时点.png")
712     plot_representative_risk_bands(
713         q43,dates,load,pv,price_actual,cp,cfg,run_D,pv_fc_h,RunConfig.FIG
714     )
715     # 生成新增PV代理;
716     augmented,diag=build_augmented_records(official,extra,dates,pv[:,[6*h-1 for h in range(1,25)
717     ]])
718     write_csv(RunConfig.OUT/"新增光伏预报偏差修正明细.csv",diag)
719     tag="_".join(map(str,extra))
720     export_records_xlsx(RunConfig.OUT/f"附件3_新增预报_{tag}_在线代理.xlsx",augmented)
721     rec_map=records_by_key(augmented)
722     extra_hist=build_extra_stage_error_history(q43,extra,rec_map,dates,load_fc,
723         load,pv,cfg,run_D)
724     lp=q43.build_day_ahead_templates(cfg)
725     base=(6,12,18); schemes=[("原0/6/12/18",base)]
726     for h in extra:
727         schemes.append((f"原方案+{h}:00",tuple(sorted(set(base+(h,)))))
728     if RunConfig.RUN_ALL_CANDIDATES_TOGETHER and len(extra)>=2:
729         schemes.append(("原方案+全部自动候选",tuple(sorted(set(base+tuple(extra)))))
730     uniq=[]; seen=set()
731     for x in schemes:
732         if x[1] not in seen: uniq.append(x); seen.add(x[1])
733     rows=[]
734     for name,hrs in uniq:
735         records=official if hrs==base else augmented
736         r=simulate_scheme(q43,name,hrs,records,dates,price_actual,load,
737             pv,load_fc,price_fc,fallback,official_hist,extra_hist,run_D,cfg,lp)
738         rows.append(summarize(name,hrs,r,run_D))
739     write_csv(RunConfig.OUT/"新增预报时点_完整滚动LP方案对比.csv",rows)
740     plot_scheme(rows,RunConfig.FIG)
741     base_row=rows[0]; metrics=["总费用","紧急购电量kWh","日紧急购电量P95kWh",

```

```

740     "单日最大紧急购电量kWh","富余电量kWh","向上调整电量kWh","向下调整电量kWh"]
741 delta=[]
742 for r in rows[1:]:
743     z={"方案":r["方案"]}
744     for m in metrics:
745         b=float(base_row[m]); v=float(r[m]); z[m+"_变化量"]=v-b
746         z[m+"_变化率%"]=(v-b)/b*100 if abs(b)>1e-12 else np.nan
747     delta.append(z)
748 write_csv(RunConfig.OUT/"新增预报方案_相对基线改善.csv",delta)
749 minc=min(rows,key=lambda r:r["总费用"]); mine=min(rows,key=lambda r:r["紧急购电量kWh"])
750 lines=["Q4-3新增其他预报时点综合验证摘要","="*72,
751        "自动筛选候选时点: "+", ".join(f"{h}:00" for h in extra),
752        "筛选原则: 不预设具体时刻。先将PV危险高估经济风险与电价预测经济暴露统一为元, ",
753        "再计算每个可能发布时间对未来若干小时风险的指数衰减覆盖得分, 由局部峰值和最小时距自动
754        形成候选集。",
755        f"最低总费用方案: {minc['方案']}, {minc['总费用']:.2f} 元",
756        f"最低紧急购电方案: {mine['方案']}, {mine['紧急购电量kWh']:.2f} kWh","",
757        "最终是否增加时点, 应以完整LP的总费用、紧急购电、P95、最大单日风险和富余电量共同判
758        断: ",
759        "风险评分只负责生成候选, 不直接替代滚动LP的最终验证。",
760        "论文图形证据链: 动态非对称风险带->光伏/电价联合风险->自动候选评分->完整LP经济性与可
761        靠性比较。"]
762 (RunConfig.OUT/"综合验证摘要.txt").write_text("\n".join(lines),encoding="utf-8")
763 print("[完成] Q4-3-2 运行结束, 输出目录: ",RunConfig.OUT)
764 if __name__ == "__main__":
765     main()

```

code/Q4-3-2.py